

UC-Secure Star DKG for Non-Exportable Key Shares with VSS-Free Enforcement

Vipin Singh Sehrawat
{vipin.sehrawat.cs@gmail.com}

Circle Internet*

Abstract. Distributed Key Generation (DKG) lets parties derive a common public key while keeping the signing key secret-shared. In UC-secure DKG, the transcript must enforce (i) secrecy against unauthorized corruptions and (ii) uniqueness and affine consistency of the induced sharing. Classically, these obligations are satisfied by a Verifiable-Sharing Enforcement (VSE) layer—realized via Verifiable Secret Sharing (VSS) and/or commitment-and-proof mechanisms—that distributes and later manipulates shares. This paper addresses the Non-eXportable Key (NXK) setting enforced by hardware-backed key-isolation modules (e.g., secure enclaves/TEEs configured as restricted keystores, or HSM-like APIs), formalized via an ideal KeyBox (keystore) functionality $\mathcal{F}_{\text{KeyBox}}$ that keeps shares non-exportable, including caller-invertible affine images, and permits only attested KeyBox-to-KeyBox sealing. In this setting, confidentiality can be delegated to the NXK/KeyBox boundary; the remaining challenge in realizing VSE layer, without VSS-style mechanisms (opening/complaints/resharing), is to enforce transcript-defined affine consistency without exporting, opening, or resharing the secret shares. Assuming a key-opaque, state-continuous NXK/KeyBox boundary, classical rewinding/forking-lemma extraction arguments are inapplicable. Hence, straight-line extraction is required.

We present a Universally Composable (UC) DKG design for NXK by combining (i) NXK/KeyBox confidentiality; (ii) Unique Structure Verification (USV), which is a publicly verifiable certificate mechanism intended for tightly coupled NXK deployments where the certified scalar is non-exportable and never leaves the KeyBox, yet the corresponding public group element is deterministically derivable from the transcript. We combine these with (iii) UC-extractable non-interactive zero-knowledge arguments of knowledge via the Fischlin transform in our gRO-CRP-hybrid model, where gRO-CRP denotes a global Random Oracle with Context-Restricted Programmability, to enforce the affine constraints normally certified by VSS-style machinery. Using these tools, we construct a UC-secure Star DKG (SDKG) scheme that is tailored to multi-device wallets with a designated service that must co-sign but can never sign alone. SDKG realizes a 1+1-out-of- n star access structure wherein the center (mandatory) and any leaf of a star graph form a minimal authorized subset. SDKG implements a two-leaf star over roles (primary vs. recovery) and supports role-based registration. In the $\mathcal{F}_{\text{KeyBox}}$ -hybrid and gRO-CRP models, assuming authenticated confidential channels that leak only message lengths, under DL and DDH hardness assumptions, PRF security of the deterministic nonce-derivation function, and a seed integrity invariant (part of the state-continuity engineering budget), with adaptive corruptions and secure erasures, SDKG UC-realizes a transcript-driven refinement of the standard UC-DKG functionality. Over a prime-order group of size p , SDKG incurs $\tilde{O}(n \log p)$ communication overhead and $\tilde{O}(n \log^{2.585} p)$ bit-operation cost, while registering a recovery device incurs $\tilde{O}(\log p)$ communication and $\tilde{O}(\log^{2.585} p)$ bit-operation costs, respectively. For a 128-bit instantiation with fixed Fischlin parameters, the base transcript (for 1+1-out-of-3 SDKG) is ≈ 11 –13 KiB.

Keywords: DKG · UC security · Non-exportable keys · TEE · HSM · UC-NIZK-AoK · Verifiable-sharing enforcement · Star access structure · MPC wallets

* The views expressed in this paper are solely those of the author and do not necessarily reflect those of Circle Internet or any other affiliated organizations.

Table of Contents

List of Results	3
1 Introduction	4
1.1 Our Contributions	5
1.2 Related Work	6
1.3 Organization	6
2 The NXK and KeyBox Setting	7
2.1 Terminology and leakage model	8
2.2 Notation, conventions, and ideal channels	9
2.3 Admissible KeyBox profiles and key-opacity	9
2.4 State continuity and failure modes	13
2.5 Hybrid execution model and NXK-restricted material	15
2.6 Real-world instantiations of admissible KeyBox profiles	16
Implementation note (non-normative)	17
3 The Conflict: Verifiable Sharing vs. NXK	17
3.1 DKG subsumes dealerless (R)VSS	18
3.2 Exported-share enforcement vs. NXK	19
4 Cryptographic Primitives for State-Continuous KeyBoxes	20
5 Enforcing Public Structure without Export: USV Certificates	29
Relation to standard notions	30
5.1 Why USV is needed under hardened NXK profiles (overview)	31
5.2 An Instantiation	32
Properties of the USV certificate scheme	32
5.3 UC Security	35
6 Enforcing Consistency via Straight-Line Extraction	38
6.1 Affine DL relation	39
7 Star DKG (SDKG)	40
7.1 The Protocol	46
Hardened deployment: what runs where (KeyBox vs. host)	49
8 UC Security	55
8.1 From $\mathcal{F}_{\text{SDKG}}$ to the standard NXK-DKG interface	55
8.2 Formal necessity of USV for commit-only alternatives under hardened profiles	57
What must be transcript-defined (and why)	57
8.3 Main Theorem	60
9 UC Security of the 1+1-out-of- n SDKG Extension	67
10 Complexity and Overhead	71
11 Conclusion	71
A Programmable Secure Hardware Integration	74
B Candidate KeyBox Implementations and Profile-Capture Checklist	75
B.1 Candidate classes of implementations	75
C Illustrative application: NXK-compatible commit–reveal randomness beacons	76

List of Results

Lemma 1	(Additive degradation under approximate state continuity)	14
Lemma 2	(Additive degradation under approximate secure erasures)	14
Theorem 1	(UC-DKG implies UC-RVSS)	18
Lemma 3	(Uniqueness is necessary for UC-DKG)	18
Proposition 1	(External fresh-share enrollment and NXK)	19
Lemma 4	(Pre-query bound for gRO-CRP programming)	22
Lemma 5	(No cross-context influence in gRO-CRP)	22
Lemma 6	(Negligible Fischlin error for admissible parameters)	26
Proposition 2	(Strict GRO is insufficient for Fischlin universal simulation)	27
Lemma 7	(Fischlin-based UC-NIZK-AoKs for DL and DLEQ in gRO-CRP)	28
Lemma 8	(Opening-conditional tag simulatability)	32
Lemma 9	(Equivocation resistance of the USV instantiation)	34
Lemma 10	(Handle-bound non-malleability)	35
Theorem 2	(UC security of $\mathcal{F}_{\text{USV}}$)	36
Lemma 11	(Unique responses for Schnorr)	38
Lemma 12	(Unique responses for Chaum–Pedersen)	39
Lemma 13	(LinOS preserves key-opacity)	42
Lemma 14	(Rollback robustness of LinOS commitments)	42
Observation 1	(Lagrange weights)	51
Lemma 15	(Acc_{SDKG} need not call $\mathcal{F}_{\text{USV}}.\text{Verify}$)	53
Lemma 16	(Latent (marginal) uniformity of the SDKG key)	55
Lemma 17	(Closure under interface restriction)	56
Observation 2	(DKG properties captured by $\mathcal{F}_{\text{DKG}}^{*,\text{NXK}}$)	56
Lemma 18	(Commit-only transcripts cannot define M_2 under NXK/ $\mathcal{F}_{\text{KeyBox}}$)	58
Lemma 19	(Transcript uniquely determines the key)	60
Theorem 3	(UC realization of $\mathcal{F}_{\text{SDKG}}$ by $\Psi_{\text{SDKG}}^{(3)}$)	60
Lemma 20	(Fresh tagged statements for SDKG extraction)	62
Lemma 21	(Registration/RDR simulation)	65
Corollary 1	(Standard NXK-DKG interface)	67
Corollary 2	(Compiling out $\mathcal{F}_{\text{USV}}$)	67
Theorem 4	(UC security of $\mathcal{F}_{\text{SDKG}}^{(n)}$)	69

1 Introduction

The growing adoption of cryptocurrencies and decentralized applications has motivated the need for secure and versatile key management solutions. Multiparty Computation (MPC) [65, 35] wallets, implementing Distributed Key Generation (DKG) [54, 34] and threshold signing [23, 24], have become prevalent because they provide robust security without the need for a trusted party. In practice, some users may prefer managing multiple authorized devices, $\{P_i\}_{i=2}^n$, in collaboration with a service provider, P_1 , forming a star topology with P_1 as the center and $\{P_i\}_{i=2}^n$ as the leaves. Furthermore, some deployments require a designated service that must always be involved in signing, yet can never sign alone. Typical examples include: regulated custodians that must co-sign for compliance and auditing; “recovery-with-friction” consumer wallets, where a risk engine enforces spending limits and/or anomaly detection; and enterprise wallets, where all transactions must flow through a corporate signing service. In such settings, a uniform threshold structure is ill-suited because it either authorizes subsets $\subseteq \{P_i\}_{i=2}^n$, excluding P_1 , or forces P_1 to hold a threshold number of shares, effectively authorizing a singleton access structure. This work targets the star access structure in which the minimal authorized sets are $\{P_1, P_i\}$ for $i \geq 2$. Formally, the family of minimal authorized subsets (Γ_0) and the corresponding access structure (Γ) are defined as:

$$\Gamma_0 := \{\{P_1, P_i\}\}_{i=2}^n; \quad \Gamma := \{S \subseteq \{P_i\}_{i \in [n]} : \exists i \geq 2, \{P_1, P_i\} \subseteq S\}.$$

This mandatory-center co-signing pattern is reminiscent of mediated / server-supported signature systems [8, 7, 25] and more recent server-assisted key-use designs [47]. Our focus differs because we need dealerless DKG under secret non-exportability and Universally Composable (UC) composition [13]—natural requirements for MPC wallets deployed in complex environments.

Hardware-backed key-isolation modules—e.g., secure enclaves/TEEs when configured as restricted keystores, or HSM-like APIs [61]—can enforce a Non-eXportable Key (NXK) model which binds secrets to hardware and forbids exporting them, including any caller-invertible affine image, permitting only attested KeyBox-to-KeyBox sealing. Thus, in a DKG performed under NXK, each signing share is generated and stored inside the module in which it was created and can only be accessed via a restricted interface. This constraint rules out classical rewinding/forking-lemma extraction [55, 5] for Schnorr/Fiat–Shamir-style proofs [30, 58, 59] at the hardware boundary. Throughout, we use “KeyBox” to denote this kind of hardware-backed key-isolation module.

We introduce Unique Structure Verification (USV), which is meant to operate precisely at this NXK/KeyBox boundary. In particular, USV targets the tightly coupled setting in which the scalar being committed-to/certified is KeyBox-resident and non-exportable, so it cannot be revealed to the host or appear in the public transcript. The protocol exports only a publicly verifiable certificate, from which any verifier can deterministically derive the associated public group element needed by transcript-defined checks, without ever exporting the scalar or its invertible affine image.

A UC-secure DKG must realize the secrecy and uniqueness guarantees of dealerless (random) Verifiable Secret Sharing (VSS) [21, 29, 54] as a subtask (formalized in Section 3 (p. 17)). Therefore, existing UC-secure DKGs include some Verifiable-Sharing Enforcement (VSE) layer that prevents equivocation and enforces affine consistency of parties’ contributions. Classically, this layer is instantiated using VSS and its variants (e.g., [64, 10]) via commitments [6] and complaint/opening logic [29, 34]. In other UC(-style) threshold key-generation components (e.g., [29, 54, 34]), VSE layer is realized via commitment-and-proof / (Non)Interactive Zero-Knowledge ((N)IZK)-based mechanisms that enforce the required VSS-style obligations. Either way, traditional UC-secure DKG requires that (linear combinations of) shares can be computed and transmitted outside the device that holds them, an assumption that is incompatible with NXK.

Roles vs. devices. We distinguish between cryptographic roles and the physical devices / secure hardware instances that host those roles. Our base construction realizes a two-leaf star over roles: the mandatory service P_1 (center) must co-sign with either a primary-role share (held by a designated primary device, say P_2) or a recovery-role share. Our n -device extension supports Role-based Device Registration (RDR) under NXK by enrolling additional recovery devices as redundant front-ends for the same recovery role. Concretely, each recovery device contains an independent KeyBox instance that ultimately holds the same recovery-role share k_{rec} , but this replication is not a share export:

enrollment is performed via attested KeyBox-to-KeyBox sealing and a one-shot installation procedure that never places *share-deriving* plaintext in the public transcript and preserves the KeyBox profile assumptions, namely key-opacity and state continuity. Thus, the access structure is star-shaped over devices, while being implemented as a two-leaf star over unique role shares (primary vs. recovery); and hence we also call it 1+1-out-of- n (star) access structure.

1.1 Our Contributions

We model the NXK/KeyBox boundary as an ideal functionality: the KeyBox provides confidentiality for non-exportable shares via a restricted API. We leverage that, together with other tools summarized in Table 1 (p. 5), to realize VSE layer under NXK without VSS-style exported-share machinery. Our contributions are multifold and can be summarized as follows:

Hardware constraint	Standard approaches' shortcomings	Our mechanism
State continuity (no rollback)	Rewinding/forking-lemma extraction requires rolling back a prover to reuse the same commitment under two challenges.	Fischlin-based UC-NIZK-AoK with straight-line simulation/extraction formalized in gRO-CRP.
NXK (non-exportable secrets)	VSS/resharing/opening typically assumes exportable shares or share-derived witnesses to enforce polynomial relations and reconfigure trust; NXK blocks exporting the needed witnesses.	USV certificates, whose witness remains KeyBox-resident: certify transcript-defined public structure verifiable outside the KeyBox, while the witness never leaves.
No caller-decryptable "wrap"	Exporting encrypted shares under caller-known keys breaks key-opacity [9].	KeyBox-to-KeyBox sealing + RDR: ciphertexts only unwrap inside a KeyBox; the caller learns no decryption handle.

Table 1: Mapping KeyBox/NXK constraints to why UC-DKG enforcement via exportable shares fails, and the tools we use instead.

1. We introduce Unique Structure Verification (USV) as a non-interactive, publicly verifiable certificate designed for NXK-coupled deployments, where the scalar witness is non-exportable and confined to a KeyBox, and only the certificate is released outside the KeyBox boundary. Conceptually, USV is a publicly extractable commitment-to-group-element abstraction: any party can deterministically derive the canonical public group element, while the committed scalar remains hidden (and non-exportable). Moreover, tags are efficiently simulatable conditioned on the derived opening. We formalize USV both as a primitive and as a handle-bound ideal functionality, and prove its UC security in our global Random Oracle with Context-Restricted Programmability (gRO-CRP)-hybrid model (Definition 10 (p. 21)) with adaptive corruptions and secure erasures, under the Discrete Logarithm (DL) and Decision Diffie-Hellman (DDH) hardness assumptions.
2. We enforce transcript-defined affine relations directly using Fischlin-style [31] UC-NIZK Arguments of Knowledge (AoKs) with straight-line extraction in the gRO-CRP model.
3. We identify and formalize the specific enforcement functionality needed by UC-DKGs in the NXK setting: enforcing transcript-defined linear/affine relations between parties' hidden scalars while keeping designated signing shares non-exportable and resilient to adaptive corruptions, without invoking resharing. We show how to enforce these relations by:
 - (a) routing sensitive state through a KeyBox interface (confidentiality),
 - (b) using USV to obtain canonical, handle-bound public openings to the group elements corresponding to KeyBox-resident scalars, and
 - (c) using UC-NIZK-AoKs to certify the cross-party affine constraints that an exported-share enforcement layer would normally check via commitments, and VSS(-like) verification and dispute logic.

We also formalize the incompatibility between classical VSE and the NXK/KeyBox boundary: VSS-style complain-t/opening/resharing (and related reconfiguration mechanisms) inherently rely on exporting shares or share-derived values, which our NXK model rules out (Sections 3 (p. 17) and 8.2 (p. 57)).

4. We construct a constant-round UC-secure Star DKG (SDKG) scheme, supporting a 1+1-out-of-3 star access structure as the base case: P_1 must co-sign with either a primary device P_2 or a recovery role P_3 . Our n -device extension supports Role-based Device Registration (RDR) by enrolling additional recovery devices as redundant front-ends with independent KeyBoxes. Our constant-round UC-secure SDKG for the 1+1-out-of- n star access structure incurs $\tilde{O}(n \log p)$ communication cost and $\tilde{O}(n \log^{2.585} p)$ bit-ops computation overhead. The RDR extension adds per device $O(\log p)$ communication bits and $\tilde{O}(\log^{2.585} p)$ work.

1.2 Related Work

Most closely aligned in application model with our “mandatory service participates” setting, Snetkov et al. [63] give a UC treatment of server-supported signatures for smartphones. Their focus is two-party server-assisted signing, whereas we target dealerless DKG for star access structure under NXK with post-DKG RDR. For a recent systematization of DL-based DKG protocols, see [3]. When reviewing the rest of the related literature, we restrict attention to UC-secure DKGs, and evaluate them over two central axes: (i) NXK compatibility and (ii) RDR support. Lindell–Nof [46] give a practical full-threshold ECDSA protocol. Instantiating their ideal functionalities with UC-secure commitments and Zero-Knowledge (ZK) yields a protocol that UC-realizes an ideal ECDSA functionality. However, their scheme targets a fixed party set and manipulates shares via generic MPC over plaintext shares and ciphertexts (no RDR; not NXK). Canetti et al. [15] provide UC threshold ECDSA with proactive key refresh over a fixed party set; dynamic joins are not modeled and several sub-protocols compute or send non-trivial functions of shares (not NXK). Doerner et al. [26] give a three-round threshold ECDSA signing protocol and show that shared keys can be generated via a simple commit-release-and-complain procedure (without proofs of knowledge). However, their setting still assumes exportable shares (not NXK) and does not address RDR. Lindell [45] gives a three-round, straight-line-simulatable DKG for Schnorr signatures [58, 59]; again the party set is fixed and the protocol performs linear operations over shares outside the NXK model. Friedman et al. [33] support reconfiguration in a two-tier 2PC–MPC framework [32] via Publicly VSS [64] and threshold additively homomorphic encryption; this achieves RDR for validators but relies on public ciphertexts and homomorphic operations that lie outside the NXK model. Outside the UC framework, Katz [41] studies the round complexity of fully secure synchronous DKG in the DL setting.

Unlike [15] wherein the security proof is carried out in the strict Global Random Oracle (GRO) setting, we operate under a gRO-CRP model that provides a single global oracle but with local-call semantics and restricted programmability tailored to NXK/KeyBox environment. In contrast to VSS-based (and VSS-like) UC-DKGs, which require all-to-all distribution of share material and therefore incur $\Theta(n^2)$ aggregate communication in the party count even before accounting for complaint/opening traffic, SDKG incurs only a constant number of proof objects in the base run (1+1-out-of-3 setting) and one additional proof per registered device for the extension to the generic 1+1-out-of- n setting. Asymptotically, we treat the Fischlin parameters as functions of the security parameter and choose them to satisfy the standard Fischlin conditions so that the transform’s soundness/knowledge-extraction error is negligible in the security parameter. For concrete 128-bit security, we instantiate with fixed parameters and report explicit concrete bounds and sizes. Table 2 (p. 7) summarizes the resulting asymptotic costs.

Note 1. SDKG is specialized to a star access structure (1+1-out-of- n) in the NXK/KeyBox setting. Most prior UC-secure DKGs are analyzed for t -out-of- n threshold access structures, whose $\Theta(n^2)$ costs largely reflect the all-to-all communication pattern inherent to threshold DKG/VSS-style protocols. For this reason, Table 2 (p. 7) is intended as a qualitative comparison of models/techniques and reported asymptotics, not a like-for-like complexity comparison across identical access structures.

1.3 Organization

The rest of this paper is organized as follows: Section 2 (p. 7) formalizes the NXK/KeyBox setting and our UC execution model, including ideal secure channels and the KeyBox functionality capturing non-exportable long-term

Work	Support		DKG		RDR	
	NXK	RDR	Bit-ops	Comm. (bits)	Bit-ops	Comm. (bits)
SDKG	✓	✓	$\tilde{O}(n\kappa^{2.585})$	$\tilde{O}(n\kappa)$	$\tilde{O}(\kappa^{2.585})$	$\tilde{O}(\kappa)$
CGGMP21 [15]	×	×	$\tilde{O}(n^2\kappa^{2.585})$	$\Theta(n^2\kappa)$	N/A	N/A
Lindell–Nof [46]	×	×	$\tilde{O}(n^2\kappa^{2.585} + n^2\eta^{2.585})$	$\Theta(n^2(\kappa + \eta))$	N/A	N/A
Lindell [45]	×	×	$\tilde{O}(n\kappa^{2.585}(t + 1 + n))$	$O(n(t + 1 + n)\kappa)$	N/A	N/A
Friedman [33]	×	✓	$\tilde{O}(n^2\kappa^{2.585} + n\eta^{2.585})$	$O(n^2(\kappa + \eta))$	$\tilde{O}(n^2\kappa^{2.585} + n^2\eta^{2.585})$	$\Theta(n^2(\kappa + \eta))$

Table 2: Comparing UC-secure DKG schemes on dominant costs (Karatsuba model [40]). Notations: $\kappa := \log p$ for a prime p ; N is a Paillier/class-group modulus; $\eta := \log N$; and t is the Shamir polynomial degree.

secrets and state continuity. Section 3 (p. 17) isolates the VSE obligations that any UC-secure DKG must satisfy, and explains why standard exported-share enforcement and resharing mechanisms clash with NXK. It also recalls relevant cryptographic primitives, namely UC, and the DL and DDH hardness assumptions (along with Decisional DL Equivalence (DDLEQ) for the equivalent DDH game in the “same-exponent across $\mathcal{G}, \mathcal{H}$ ” form aligned with DLEQ/Chaum–Pedersen statements). Section 4 (p. 20) collects cryptographic and modeling preliminaries: the gRO-CRP model and its local-call semantics, and the UC/NIZK(-AoK) notions we use, including the optimized Fischlin transform enabling straight-line extraction; in particular, Proposition 2 (p. 27) explains why strict GRO is not enough for the required simulation interface. Section 5 (p. 29) introduces Unique Structure Verification (USV), gives a concrete instantiation, and proves UC security of the corresponding handle-bound functionality. Section 6 (p. 38) develops the UC-extractable NIZK-AoKs used throughout, including DL, DLEQ, and the affine-DL relations enforced by the transcript. Section 7 (p. 40) presents the SDKG protocol: the base 1+1-out-of-3 run and the one-shot role-based device registration (RDR) mechanism used to enroll additional recovery devices under NXK (Algorithm 1 (p. 51)). Section 8 (p. 55) proves UC security for the base protocol via a transcript-driven ideal functionality and derives the corresponding standard NXK-star DKG interface, including the following waypoints:

- Formal necessity of USV for commit-only alternatives under hardened profiles: Section 8.2 (p. 57).
- Main theorem: Theorem 3 (p. 60).
- Compilation (eliminating $\mathcal{F}_{\text{USV}}$ via UC composition): Corollary 2 (p. 67).

Section 9 (p. 67) formalizes the 1+1-out-of- n extension and establishes UC security of the scalable RDR mechanism. Section 10 (p. 71) provides a focused complexity and overhead discussion, including concrete parameterization and transcript sizes. Section 11 (p. 71) concludes. Appendix A (p. 74) discusses an optional tighter integration with programmable KeyBox implementations (e.g., TEEs and certain HSM/KMS-backed designs). Appendix B (p. 75) collects concrete candidate implementation classes and a profile-capture checklist for enforcing a KeyBox API profile in practice.

2 The NXK and KeyBox Setting

We work in an NXK/KeyBox model wherein long-term shares remain inside state-continuous KeyBoxes (no rewind/fork) and are *API-non-exportable* in the sense of *Reader Note* 2.1 (p. 8). The only permitted cross-KeyBox transfer of share-dependent data is attested KeyBox-to-KeyBox sealing, which returns only ciphertexts to the caller. Any *caller-recoverable* export of a resident share—whether as raw bytes, a caller-invertible affine image, or via any other API-visible behavior that is not simulatable from the corresponding public information—is disallowed. Formally, we assume admissible KeyBox profiles satisfy key-opacity. Informally, key-opacity means that the KeyBox’s external outputs are simulatable from the public key alone; the formal statement appears as Assumption 1 (p. 11) below. We model (i) authenticated confidential point-to-point channels with adversary-controlled scheduling via an ideal functionality $\mathcal{F}_{\text{channel}}$ (Fig. 1 (p. 9)), (ii) an authenticated public dissemination mechanism for transcript-public values via an ideal functionality $\mathcal{F}_{\text{pub}}$ (Fig. 2 (p. 9)), and (iii) a per-party NXK hardware boundary via a KeyBox functionality $\mathcal{F}_{\text{KeyBox}}$ (Fig. 3 (p. 11)) that generates and stores long-term shares internally and exposes only a restricted API.

2.1 Terminology and leakage model

Reader Note 2.1: Terminology: exportability vs. visibility

We distinguish four places where a value may reside or be observed:

- KeyBox internal state: data stored inside the trusted KeyBox boundary.
- Host RAM: volatile party state outside the KeyBox (subject to erasure and adaptive corruption).
- Adversary-visible transcript: everything observable to $\mathcal{A}$ outside honest KeyBoxes, including all messages sent over adversary-visible channels, all explicit leakage outputs of ideal functionalities (e.g., $\mathcal{F}_{\text{channel}}$ ’s length leakage and scheduling metadata), and all outputs returned to adversary-controlled ITMs.
- Persistent storage (outside the KeyBox): disk/logs/swap outside the KeyBox; we conservatively treat any such data as part of the adversary-visible transcript.

We use the following terms throughout:

- (API-)export / (API-)non-exportable (KeyBox-resident shares): A KeyBox-resident share is *API-non-exportable* if no KeyBox API call enables the caller to recover the share (or any caller-invertible affine image of it), even when combined with caller-held secrets. This is captured formally by *key-opacity* (Assumption 1 (p. 11)).
- Transcript-visible vs. transcript-private: A value is *transcript-visible* if it appears in the adversary-visible transcript; otherwise it is *transcript-private*. Specifically, payloads delivered over $\mathcal{F}_{\text{channel}}$ are transcript-private unless an endpoint is corrupted; only $\mathcal{F}_{\text{channel}}$ ’s explicit leakage (e.g., lengths) is transcript-visible.
- Protocol transcript / local views (unqualified “transcript”): When we refer to “the transcript” without the qualifier *adversary-visible*, we mean the parties’ local protocol views: the tuple of messages delivered to the parties (including plaintexts carried over $\mathcal{F}_{\text{channel}}$) together with the public values. This full transcript is not necessarily adversary-visible.
- NKK-restricted material (share-deriving material): NKK-restricted material must be transcript-private and must never be written to persistent storage outside a KeyBox. It may be handled transiently in host RAM during an atomic local step and must then be securely erased; under adaptive corruptions with secure erasures, such RAM values leak only if corruption occurs before erasure.

In this paper “non-exportable” refers to API-non-exportability of KeyBox-resident shares and does not mean that related ephemeral/share-deriving material can never appear transiently in host RAM. Adversarial access to honest host RAM is modeled only via the UC corruption interface with secure erasures (Definition 1 (p. 8)), instantiated in our $\mathcal{F}_{\text{KeyBox}}$ -hybrid model below (Definition 6 (p. 15)): until corruption, an honest party executes its local steps atomically and may hold NKK-restricted material transiently, after which it is securely erased; after corruption, the adversary controls the host ITM and learns its current (non-erased) state. We do not model an “always-on” adversary that can continuously scrape the RAM of an honest host without triggering a corruption event.

The secure-channel functionality $\mathcal{F}_{\text{channel}}$ abstracts both authenticated key establishment and the subsequent symmetric protection of payloads; its internal session-key material is not part of any party’s KeyBox state and is not modeled explicitly. In our NKK setting, the only role of $\mathcal{F}_{\text{channel}}$ is to keep NKK-restricted / share-deriving material transcript-private unless an endpoint is corrupted (*Reader Note 2.1* (p. 8)). When mapping to a concrete implementation, these channel keys can be realized as ordinary ephemeral host state (e.g., via an AKE or ephemeral IND-CCA KEM establishing per-session AEAD keys) and are subject to the same adaptive-corruption-with-secure-erasures discipline as other transient values: they may reside in host RAM during an atomic step and must be erased once no longer needed. This forward-secure/erasure discipline prevents later corruptions from retroactively decrypting previously recorded ciphertexts, matching the semantics of $\mathcal{F}_{\text{channel}}$ (Fig. 1 (p. 9)). Alternatively, one may place channel cryptography inside the KeyBox/profile adapter, at the cost of extending the admissible profile with the required symmetric primitives, but our model and proofs do not require this stronger placement.

Definition 1 (Adaptive corruptions with secure erasures). We work in the UC framework with adaptive corruptions and an explicit erasure discipline. Each party P_i maintains a local (host) state st_i outside any KeyBox boundary (cf. *Reader note 2.1* (p. 8)). A protocol may explicitly erase designated local variables/buffers from st_i once they are no longer needed. Upon corruption of P_i , the adversary learns only P_i ’s current local state st_i at the moment of corruption; any values explicitly erased by the protocol prior to corruption are not revealed. Thereafter, the

- ◆ **Parameters:** session identifier sid ; endpoints (P_s, P_r) ; leakage $\Phi(c) = |c|$.
- ◆ **State:** $\text{active} \in \{0, 1\}$ (init 0); multiset $\mathbf{Q}$ of (ρ, c, ϕ) ; set $\mathbf{D}$ of delivered tickets.
- ◆ Upon receiving $(\text{Init}, \text{sid}, P_s, P_r)$ from both P_s and P_r : set $\text{active} \leftarrow 1$; send $(\text{ChReady}, \text{sid}, P_s, P_r)$ to $\mathcal{A}$.
- ◆ Upon receiving $(\text{Send}, \text{sid}, c)$ from P_s with $\text{active} = 1$: sample $\rho \leftarrow_{\mathbb{S}} \{0, 1\}^\lambda$, set $\phi \leftarrow \Phi(c)$, insert (ρ, c, ϕ) into $\mathbf{Q}$, send ρ to P_s , and send $(\text{Leak}, \text{sid}, P_s, P_r, \rho, \phi)$ to $\mathcal{A}$. If P_s is corrupted, additionally reveal c to $\mathcal{A}$.
- ◆ Upon receiving $(\text{Deliver}, \text{sid}, \rho)$ from $\mathcal{A}$: if $(\rho, c, \phi) \in \mathbf{Q}$ and $\rho \notin \mathbf{D}$, delete it from $\mathbf{Q}$, add ρ to $\mathbf{D}$, and deliver $(\text{Recv}, \text{sid}, P_s, c)$ to P_r . If P_r is corrupted, reveal c to $\mathcal{A}$ at delivery time.
- ◆ Upon receiving $(\text{Inspect}, \text{sid})$ from $\mathcal{A}$: if both P_s and P_r are corrupted, reveal c for all $(\rho, c, \phi) \in \mathbf{Q}$.

Fig. 1: Ideal authenticated, length-leaking secure channel $\mathcal{F}_{\text{channel}}$

- ◆ **Parameters:** session identifier sid ; sender $P_s \in \mathbb{P}$; leakage $\Phi(c) = |c|$.
- ◆ **State:** multiset $\mathbf{Q}$ of (ρ, P_s, c, ϕ) ; set $\mathbf{D}$ of delivered tickets.
- ◆ Upon receiving $(\text{Publish}, \text{sid}, c)$ from P_s :
 sample $\rho \leftarrow_{\mathbb{S}} \{0, 1\}^\lambda$, set $\phi \leftarrow \Phi(c)$, insert (ρ, P_s, c, ϕ) into $\mathbf{Q}$,
 send ρ to P_s , and send $(\text{Leak}, \text{sid}, P_s, \rho, c, \phi)$ to $\mathcal{A}$.
- ◆ Upon receiving $(\text{Deliver}, \text{sid}, \rho)$ from $\mathcal{A}$:
 if $(\rho, P_s, c, \phi) \in \mathbf{Q}$ and $\rho \notin \mathbf{D}$, delete it from $\mathbf{Q}$, add ρ to $\mathbf{D}$,
 and deliver $(\text{Recv}, \text{sid}, P_s, c)$ to every $P \in \mathbb{P}$.

Fig. 2: Ideal authenticated public broadcast $\mathcal{F}_{\text{pub}}$ (adversary-visible, adversary-scheduled).

adversary controls P_i and may arbitrarily influence its future actions and state. We assume honest-party activations are atomic with respect to corruption, i.e., corruptions can occur only between activations. Consequently, temporary values created and erased within a single honest activation are never revealed by a later corruption.

2.2 Notation, conventions, and ideal channels

We call an algorithm efficient if, for input size λ , its running time is bounded by $\text{poly}(\lambda)$. Throughout the text, $\lambda \in \mathbb{N}$ denotes the security parameter with $\text{negl}(\lambda)$ denoting a negligible function on it; and $\approx_c$ represents computational indistinguishability. We write $x \leftarrow_{\mathbb{S}}$ to denote uniform sampling from a finite set $\mathbb{S}$. Throughout, for an elliptic curve $\mathbb{E}(\mathbb{F}_q)$, $\mathbb{G} \subseteq \mathbb{E}(\mathbb{F}_q)$ denotes a cyclic subgroup of prime order $p > 3$ written additively, with fixed generator $\mathcal{G}$. We fix an injective, self-delimiting encoding $\langle \cdot \rangle$ of mixed tuples into $\{0, 1\}^*$. All hash/oracle invocations are applied only to encodings of the form $\langle \cdot \rangle$, and all “equality of encoded tuples” statements are with respect to this encoding. Our $\tilde{O}(\cdot)$ bounds hide factors that are polynomial in the Fischlin parameters $(t(\lambda), b(\lambda), r(\lambda), S(\lambda))$, which are themselves at most $\text{polylog}(\lambda)$ in the asymptotic analysis; for any fixed concrete instantiation at a target security level $\lambda = \lambda_0$, these factors become constants.

The UC security parameter is λ . Our prime-order group is generated as a function of λ and has order $p = p(\lambda)$. Let $\kappa(\lambda) := \lceil \log_2 p(\lambda) \rceil$ denote the bitlength of the group order. We assume $\kappa(\lambda) = \Theta(\lambda)$; in particular, $\kappa(\lambda) = O(\lambda)$ and, for concrete instantiations, typically $\kappa(\lambda) \geq \lambda$ up to a constant factor; so that “PPT in λ ” and “PPT in κ ” are equivalent up to polynomial factors. Unless stated otherwise, all auxiliary protocol parameters are deterministic functions of λ (equivalently, of κ under $\kappa = \Theta(\lambda)$).

Note 2. Concrete realization for $\mathcal{F}_{\text{channel}}$ from [44]: IND-CCA KEM + IND-CPA and INT-CTXT AEAD [57]. To match the adaptive-corruption-with-secure-erasures semantics of $\mathcal{F}_{\text{channel}}$, per-session channel keys are treated as ephemeral and are securely erased after use (or kept inside a trusted boundary).

2.3 Admissible KeyBox profiles and key-opacity

Let $\mathcal{K}$ be a key space and let ξ be a distribution over $\mathcal{K}$. Let $\mathcal{F}_{\text{adm}}$ be a set of PPT stateful admissible operations, modeled as state-transition algorithms

$$f : \mathcal{K} \times \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^* \times \{0, 1\}^*,$$

where on input (k, st, m) , the operation outputs a new private state st' and a response y . Let $\text{PubMap} : \mathcal{K} \rightarrow \{0, 1\}^*$ be an efficiently computable public-information map. Let $\text{GetPub} \in \mathcal{F}_{\text{adm}}$ denote the stateless admissible operation

$$\text{GetPub}(k, \text{st}, m) := (\text{st}, \text{PubMap}(k)),$$

which ignores m and does not change state. Fix a PPT Interactive Turing Machine (ITM) simulator Sim and a PPT adversary $\mathcal{A}$. Define an experiment $\text{Exp}_{\mathcal{A}, \text{Sim}, \mathcal{F}_{\text{adm}}, \xi, \text{PubMap}}^{\text{opq}}(1^\lambda)$:

Some admissible interfaces are multi-stage (e.g., a **Start/Prove** pair) and therefore require that later calls read internal state written by earlier calls. To model this in a profile-centric way, we associate to each admissible operation a deterministic *state-family* identifier via a map

$$F : \mathcal{F}_{\text{adm}} \rightarrow \text{Fam},$$

where Fam is a finite identifier set fixed by the KeyBox API profile. The KeyBox maintains one internal state string per family, and all invocations of operations f with the same family id $F(f)$ share that state component. Unless stated otherwise, we take singleton families, i.e., $F(f) = f$.

1. Sample $k \leftarrow \xi$ and set $pk \leftarrow \text{PubMap}(k)$. Sample a hidden bit $\beta \leftarrow \{0, 1\}$.
2. Initialize a state table $\text{st}[\cdot]$ by setting $\text{st}[\varphi] \leftarrow \epsilon$ for every family identifier $\varphi \in \text{im}(F)$. Initialize a simulator oracle Sim_{pk} by running Sim on input $(1^\lambda, pk)$. Hence, Sim_{pk} may maintain state across oracle calls.
3. Run $\mathcal{A}(1^\lambda, pk)$ with oracle access to $\mathcal{O}_\beta(\cdot, \cdot)$ defined as:

$$\mathcal{O}_\beta(f, m) := \begin{cases} \text{let } \varphi \leftarrow F(f); \text{ let } (\text{st}', y) \leftarrow f(k, \text{st}[\varphi], m); \text{ st}[\varphi] \leftarrow \text{st}'; y & \text{if } \beta = 1 \text{ and } f \in \mathcal{F}_{\text{adm}}, \\ \text{Sim}_{pk}(f, m) & \text{if } \beta = 0 \text{ and } f \in \mathcal{F}_{\text{adm}}, \\ \perp & \text{if } f \notin \mathcal{F}_{\text{adm}}. \end{cases}$$

4. $\mathcal{A}$ outputs a bit $\beta' \in \{0, 1\}$. Output 1 iff $\beta' = \beta$.

Fig. 3 (p. 11) specifies a generic per-party KeyBox instance. We denote the instance owned by party P_i as $\mathcal{F}_{\text{KeyBox}}^{(i)}$, and sometimes write $\mathcal{F}_{\text{KeyBox}}$ when the owner is clear from context. Within an instance, keys are indexed by the local handle $\mu \in \{0, 1\}^*$; globally we refer to a slot as (P_i, μ) . The **SealToPeer/OpenFromPeer** API assumes that each party's sealing public key $\text{pk}_{\text{seal}}^{(P)}$ is authenticated and bound to that party's KeyBox instance (e.g., via attestation). We abstract the attestation/key-distribution mechanism by an authenticated sealing-key directory $\{\text{pk}_{\text{seal}}^{(P)}\}_{P \in \mathbb{P}}$ that is fixed and not adversary-influenceable. Hence, **SealToPeer** encrypts to the directory-defined $\text{pk}_{\text{seal}}^{(P_{\text{peer}})}$ for the designated peer, rather than accepting a raw recipient key as input. **Enc/Dec** denote encryption/decryption functions of an ideal public-key authenticated-encryption scheme with associated data, ad . We will use that (Enc, Dec) is probabilistic and IND-CCA secure, i.e., secure for polynomially many encryptions under a fixed public key. Thus, even when many **SealToPeer** calls encrypt different slot-resident plaintexts to the same recipient key $\text{pk}_{\text{seal}}^{(P_{\text{peer}})}$, the resulting ciphertexts are jointly simulatable as independent encryptions of a fixed dummy plaintext of the appropriate length under the same $\text{pk}_{\text{seal}}^{(P_{\text{peer}})}$, with fresh and independent randomness per call, and no other cross-call leakage at the API boundary.

To support multiple sessions on the same state-continuous KeyBox, we treat the mnemonic slot names as tags rather than literal constants. Concretely, in a session with identifier sid , we define the local slot handle used for **tag** as

$$\langle \text{sid}, \text{tag} \rangle \in \{0, 1\}^*,$$

where $\langle \cdot \rangle$ is the fixed injective tuple encoding used throughout.

Our KeyBox abstraction is a narrow, keystore-style resource tailored to NKK. For more general-purpose formal UC abstractions and discussion of subtle composability issues stemming from globally shared attestation keys, see [53, 51]. Making the attestation protocol explicit mandates extending our model, which can be done via a separate (sub-)protocol/functionality and instantiated independently, as in UC-style treatments of [51, 53, 12, 66]. Given that protecting against physically invasive or side-channel attacks necessitates specialized equipment [62], we consider such threats out of scope.

- ◆ **Parameters:** fixed owner $P_{\text{own}} \in \mathbb{P}$; authenticated sealing-key directory $\{\text{pk}_{\text{seal}}^{(P)}\}_{P \in \mathbb{P}}$ with local keypair $(\text{pk}_{\text{seal}}, \text{sk}_{\text{seal}})$ such that $\text{pk}_{\text{seal}} = \text{pk}_{\text{seal}}^{(P_{\text{own}})}$; and a set of public parameters pp .
- ◆ **State:** key table $\Lambda : \{0, 1\}^* \rightarrow \mathcal{K}$ (init $\emptyset$) for local slot $\mu \in \{0, 1\}^*$; operation state $\text{st}[\mu, \varphi] \in \{0, 1\}^*$ (init ϵ) for each family id $\varphi \in \text{im}(F)$; private buffer $\text{buf} : \{0, 1\}^\lambda \rightarrow (\{0, 1\}^* \times \{0, 1\}^*)$ (init $\emptyset$).
- ◆ **Admissible routines:** derivations χ_{adm} and operations $\mathcal{F}_{\text{adm}}$ with a designated key-independent subset $\mathcal{F}_{\text{KI}} \subseteq \mathcal{F}_{\text{adm}}$ (e.g., OpenFromPeer , $\text{USV.Cert} \in \mathcal{F}_{\text{KI}}$ and $\text{SealToPeer} \in \mathcal{F}_{\text{adm}} \setminus \mathcal{F}_{\text{KI}}$).
- ◆ **Procedure $\text{Resolve}(m)$:** parse m as $\langle m_1, \dots, m_t \rangle$; for each component that parses as a typed handle $\langle \text{hdl}, \tau \rangle$ with $\tau \in \text{dom}(\text{buf})$, let $(\text{ad}, s) := \text{buf}[\tau]$ and substitute $\langle \text{ad}, s \rangle$. If parsing fails or a referenced typed handle is missing, return $\perp$ and leave buf unchanged. Otherwise delete all used τ from buf and return the substituted tuple.
- ◆ **Upon receiving (Load, μ, g, m) from party P_{own} :**
 - If $\mu \in \text{dom}(\Lambda)$ or $g \notin \chi_{\text{adm}}$, return $\perp$. Let $m' \leftarrow \text{Resolve}(m)$; if $m' = \perp$ return $\perp$.
 - Compute $k \leftarrow g(1^\lambda, m')$. If $k = \perp$, return $\perp$. Set $\Lambda[\mu] \leftarrow k$ and return **ok**.
- ◆ **Upon receiving (Use, μ, f, m) from party P_{own} :**
 - If $f \notin \mathcal{F}_{\text{adm}}$, return $\perp$.
 - If $f \notin \mathcal{F}_{\text{KI}}$ and $\mu \notin \text{dom}(\Lambda)$, return $\perp$.
 - If $f = \text{SealToPeer}$: parse $m = \langle P_{\text{peer}}, \text{ad} \rangle$. If $P_{\text{peer}} \notin \mathbb{P}$, return $\perp$.
 - Let $\text{pk}_{\text{peer}} \leftarrow \text{pk}_{\text{seal}}^{(P_{\text{peer}})}$ and $s \leftarrow \Lambda[\mu]$. Return $c \leftarrow \text{Enc}_{\text{pk}_{\text{peer}}}(\text{ad}, s)$.
 - If $f = \text{OpenFromPeer}$: parse $m = \langle c, \text{ad} \rangle$. Compute $s \leftarrow \text{Dec}_{\text{sk}_{\text{seal}}}(\text{ad}, c)$; if decryption fails return $\perp$. Sample $\tau \leftarrow \{0, 1\}^\lambda$, set $\text{buf}[\tau] \leftarrow (\text{ad}, s)$, and return $\langle \text{hdl}, \tau \rangle$.
 - If $f = \text{USV.Cert}$: ignore μ . Sample $m_{\text{cert}} \leftarrow \mathbb{Z}_p^*$ and compute $\langle C, \zeta \rangle \leftarrow \text{Cert}(\text{pp}, m_{\text{cert}})$ (defined in Section 5 (p. 29)); erase m_{cert} and return $\langle C, \zeta \rangle$.
 - If $f \in \mathcal{F}_{\text{KI}}$ (general key-independent case): ignore μ . Let $\varphi \leftarrow F(f)$; compute $(\text{st}', y) \leftarrow f(\perp, \text{st}[\mu, \varphi], m)$, set $\text{st}[\mu, \varphi] \leftarrow \text{st}'$, and return y . (Key-independent operations receive $\perp$ in place of the resident key and must not depend on it.)
 - Otherwise (key-dependent): let $\varphi \leftarrow F(f)$; compute $(\text{st}', y) \leftarrow f(\Lambda[\mu], \text{st}[\mu, \varphi], m)$, set $\text{st}[\mu, \varphi] \leftarrow \text{st}'$, and return y .

Fig. 3: Per-party KeyBox functionality $\mathcal{F}_{\text{KeyBox}}^{(P_{\text{own}})}$ for an NKK KeyBox with KeyBox-to-KeyBox sealing.

Definition 2 (Key-opacity). Let $\mathcal{K}$ be a key space, let ξ be an efficiently sampleable distribution over $\mathcal{K}$, and let $\text{PubMap} : \mathcal{K} \rightarrow \{0, 1\}^*$ be an efficiently computable public-information map. For security parameter λ , $\mathcal{F}_{\text{adm}}$ is *key-opaque* with respect to (ξ, PubMap) if for every PPT adversary $\mathcal{A}$ there exists a PPT ITM simulator Sim such that

$$\text{Adv}_{\mathcal{A}}^{\text{opq}}(\lambda) := \left| \Pr \left[\text{Exp}_{\mathcal{A}, \text{Sim}, \mathcal{F}_{\text{adm}}, \xi, \text{PubMap}}^{\text{opq}}(1^\lambda) = 1 \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

When ξ is clear from context, we may omit it and simply say “key-opaque w.r.t. PubMap .”

Assumption 1 (Key-opacity (profile-level, multi-slot)) Fix an admissible KeyBox API profile $(\chi_{\text{adm}}, \mathcal{F}_{\text{adm}}, F)$ (Definition 3 (p. 12)) and a public-information map $\text{PubMap} : \mathcal{K} \rightarrow \{0, 1\}^*$. We assume slot-separable key-opacity for this profile:

- Single-slot opacity: $\mathcal{F}_{\text{adm}}$ is key-opaque w.r.t. PubMap in the sense of Definition 2 (p. 11).
- Slot separability / no cross-slot opacity couplings: In the multi-slot $\mathcal{F}_{\text{KeyBox}}$ functionality (Fig. 3 (p. 11)), every key-dependent admissible operation $f \in \mathcal{F}_{\text{adm}}$ is slot-local: on a call $\text{Use}(\mu, f, m)$ it may read the resident key $k_{i, \mu} := \Lambda[\mu]$ and the per-slot family state $\text{st}[\mu, F(f)]$, but it does not read or update any other slot’s key/state $(\Lambda[\mu'], \text{st}[\mu', \cdot])$ for $\mu' \neq \mu$. Moreover, the externally visible randomness/state used by distinct slots can be taken independent. Any randomized admissible routine (including SealToPeer) is modeled as using fresh, independent coins per invocation, and we do not model any additional cross-slot/cross-call leakage (e.g., shared-DRBG artifacts or timing channels) beyond the explicit transcript leakage of the ideal functionalities. Although SealToPeer encrypts different slot-resident values under the same recipient directory key $\text{pk}_{\text{seal}}^{(P_{\text{peer}})}$, IND-CCA security of (Enc, Dec) implies that the joint distribution of all such ciphertexts (across slots and across calls) is computationally indistinguishable from a product distribution of independently generated dummy ciphertexts under $\text{pk}_{\text{seal}}^{(P_{\text{peer}})}$. Hence, these outputs can be simulated either by independent per-slot simulators (each using fresh encryption randomness) or by a single

key-independent sealing simulator shared across slots. Key-independent interfaces $f \in \mathcal{F}_{\text{KI}}$ are assumed simulatable without knowing any resident key (and may be handled by separate key-independent wrapper state). For the scalar-share slots used in SDKG we instantiate $\text{PubMap}(k) := k\mathcal{G}$ (so GetPub returns $k\mathcal{G}$). The simulator is understood to run in the same ambient execution model as the surrounding protocol and it may use any simulator-only interface provided by that model.

Remark 1 (Key-opacity in multi-slot KeyBoxes). In the $\mathcal{F}_{\text{KeyBox}}$ -hybrid model each party's KeyBox instance stores multiple keys indexed by local slots $\mu \in \{0, 1\}^*$, with $k_{i,\mu} := \Lambda[\mu]$ and $\text{pk}_{i,\mu} := \text{PubMap}(k_{i,\mu})$. As implied by Assumption 1 (p. 11), in hybrids/proofs we may simulate different slots by running independent copies of the single-slot key-opacity simulator, one per slot (i, μ) , each maintaining its own state across queries to that slot. Key-independent calls such as OpenFromPeer are handled by separate key-independent simulation state when needed. When the profile includes SealToPeer , note that ciphertexts across different slots may be under the same recipient key $\text{pk}_{\text{seal}}^{(P_{\text{peer}})}$; nevertheless, by IND-CCA security the simulator may treat each SealToPeer output as an independently simulatable ciphertext, so running independent per-slot simulators remains sound at the level of the joint external transcript.

Remark 2 (Slot separability is an idealization (implementation caveat)). Assumption 1 (p. 11) treats different slots as independent black boxes: admissible operations are slot-local and the randomness used across slots and across calls can be taken independent, with no additional cross-slot/cross-call leakage at the KeyBox API boundary beyond the explicit leakage of our ideal functionalities. This is an idealization.

In concrete KeyBox/HSM implementations, cross-slot correlations can arise from shared entropy/DRBG state, nonce/-counter reuse, related-key derivation from a common root, or microarchitectural/timing side channels. Such correlations may invalidate key-opacity even when the underlying primitive is IND-CCA secure under the standard assumption of fresh, independent coins per encryption. The argument that many SealToPeer outputs are jointly simulatable as independent dummy encryptions relies on per-call independent randomness and the absence of extra correlated leakage. To that end, to instantiate Assumption 1 (p. 11) in practice, implementations should ensure: (i) domain-separated key derivation and per-purpose/per-slot RNG (or otherwise demonstrably independent coins) for randomized operations, including sealing; (ii) nonce/coin generation that is robust against reuse/correlation, or the use of misuse-resistant constructions where applicable; and (iii) a strict allowlist/profile adapter that prevents cross-mechanism compositions as in the PKCS#11 API-level attack literature [9].

Definition 3 (KeyBox API profile). A *KeyBox API profile* is a quadruple $(\chi_{\text{adm}}, \mathcal{F}_{\text{adm}}, \mathcal{F}_{\text{KI}}, F)$ where χ_{adm} is the set of admissible derivation routines that may be invoked via Load , $\mathcal{F}_{\text{adm}}$ is the set of admissible operations that may be invoked via Use , $\mathcal{F}_{\text{KI}} \subseteq \mathcal{F}_{\text{adm}}$ is the (profile-fixed) subset of *key-independent* admissible operations, and $F : \mathcal{F}_{\text{adm}} \rightarrow \text{Fam}$ is the state-family map. A KeyBox instance accepts only calls $\text{Load}(\mu, g, \cdot)$ with $g \in \chi_{\text{adm}}$ and $\text{Use}(\mu, f, \cdot)$ with $f \in \mathcal{F}_{\text{adm}}$; for operations $f \notin \mathcal{F}_{\text{KI}}$ the addressed slot μ must be populated (contain a resident key), while operations in $\mathcal{F}_{\text{KI}}$ may be invoked on empty slots and are interpreted as ignoring μ (and using only key-independent KeyBox state, if any).

Opaque buffer handles returned by OpenFromPeer are type-tagged in the global encoding: a handle is always represented as $\langle \text{hdl}, \tau \rangle = \langle \text{hdl}, \tau \rangle$ for $\tau \in \{0, 1\}^\lambda$, and Resolve substitutes only such tagged handles, thereby preventing accidental collisions with ordinary fields.

Remark 3 (Linear/one-shot handle consumption). In $\mathcal{F}_{\text{KeyBox}}$ (Fig. 3 (p. 11)), Resolve is *one-shot*: every typed handle $\langle \text{hdl}, \tau \rangle$ that is successfully substituted is deleted from buf before Resolve returns. Hence, handles returned by OpenFromPeer are linear resources: they cannot be reused across multiple Load/Use calls. Protocol steps that need the same sealed payload more than once must invoke OpenFromPeer again to obtain fresh handles; Algorithm 1 (p. 51) does this explicitly.

Remark 4 (Profiles must not hide extractor-relevant oracle logs). In the gRO-CRP model, straight-line extraction for our Fischlin-based UC-NIZK-AoKs requires the prover's oracle-log $\text{Log}_{\mathcal{P}^*}$ under the corresponding proof context (Definition 11 (p. 24), Remark 10 (p. 21)). Since KeyBox-internal oracle calls are not exposed at the host/API boundary, we require that any proof for which the UC argument invokes oracle-log-based extraction is generated by the host/party

ITM (outside the KeyBox). Concretely, an admissible KeyBox profile must not include any operation that produces the UC-context consistency AoKs used by the surrounding protocol, nor any equivalent proof-generation interface that would cause the relevant oracle queries to occur inside the KeyBox boundary.

This requirement is sometimes read as being in tension with the intuition that, in a deployment where the host OS/hypervisor is treated as adversarial, one would like all witness-bearing scalars to remain inside the trusted boundary. Our model does not claim (and does not require) this stronger property. In SDKG, the witnesses used for the UC-context consistency AoKs are treated as NXK-restricted material (Remark 7 (p. 16)): they may exist transiently in host RAM during an atomic local step and are then securely erased. The NXK guarantee enforced by $\mathcal{F}_{\text{KeyBox}}$ is the non-exportability of long-term KeyBox-resident shares, not protection against an “always-on” RAM adversary on an otherwise honest host. Achieving the latter would require either strengthening the model to expose an extractor-visible oracle log from within the trusted boundary, or replacing oracle-log-based straight-line extraction with a different UC proof mechanism.

2.4 State continuity and failure modes

Assumption 2 (State continuity) We assume that each $\mathcal{F}_{\text{KeyBox}}^{(i)}$ instance models a single hardware root whose internal sealed state is state-continuous, i.e., a PPT adversary $\mathcal{A}$ cannot (a) roll back $\mathcal{F}_{\text{KeyBox}}^{(i)}$ to a previous sealed snapshot, nor (b) fork/clone $\mathcal{F}_{\text{KeyBox}}^{(i)}$ into two independent instances that can be queried in parallel from the same prior state. Equivalently, $\mathcal{A}$ has no interface that resets $\mathcal{F}_{\text{KeyBox}}^{(i)}$ ’s private state to an earlier value.

Assumption 3 (Seed integrity invariant) Each $\mathcal{F}_{\text{KeyBox}}^{(i)}$ instance stores a static PRF seed $\text{seed}_i \in \{0, 1\}^\lambda$, provisioned at enrollment/manufacturing/secure setup, that is independent of every KeyBox-resident signing share k . We require:

1. Rollback-invariance: A rollback or reset of mutable device/host state (or even of the KeyBox’s mutable operation state $\text{st}[\cdot]$) does not alter seed_i . Concretely, seed_i resides in a write-once or append-only region of the KeyBox’s sealed state that is outside the scope of any rollback-vulnerable mutable-state snapshot.
2. Secrecy: seed_i is never exposed at the KeyBox API boundary: no admissible operation returns seed_i or any value from which seed_i can be efficiently recovered.
3. Independence from signing shares: seed_i is sampled independently of every KeyBox-resident signing share $k_{i,\mu}$ and of all other parties’ seeds seed_j for $j \neq i$.

When the KeyBox is realized by a hardware root of trust, seed_i is typically derived from the hardware’s unique device secret during secure provisioning via a domain-separated KDF, using a purpose tag disjoint from all other key-derivation purposes. If the seed integrity invariant is violated—e.g., seed_i is corrupted, rolled back to a stale value, or leaked at the API boundary—then deterministic nonce derivation can no longer be relied upon. In particular, if seed_i is revealed then LinOS prover nonces become predictable and (except with negligible probability over transcript generation) a single accepting LinOS transcript suffices to recover the resident share k . More generally, if seed_i is not rollback-invariant, rollback can again lead to nonce-reuse style failures that LinOS is designed to prevent. As a consequence, seed provisioning and protection must be treated as part of the state-continuity engineering budget.

Remark 5 (Why an independent seed). One might consider deriving the nonce seed from the resident signing key k itself via a KDF (as in EdDSA [39]), but this entangles the PRF security hypothesis with the discrete-log assumption on k : the reduction must argue that PRF outputs remain pseudorandom even when the adversary sees $K = k\mathcal{G}$ and interacts with k via the admissible KeyBox profile. While such a reduction can be carried out under a random-oracle/KDF assumption (cf. the analysis of EdDSA deterministic nonces), it tightens the bound and introduces an additional modeling assumption. Keeping seed_i independent of k yields a clean reduction: the PRF game is played entirely over seed_i , whose secrecy is a standalone KeyBox property (Assumption 3 (p. 13)), orthogonal to any DL-based argument. If a deployment must derive the seed from k (e.g., due to hardware constraints on independent secret provisioning), then the security argument additionally requires modeling the KDF as a random oracle (or a dual-PRF) and the resulting bound includes a KDF-security term. We flag this as a deployment caveat and recommend the independent-seed instantiation.

In our NXK setting, a corrupted party may delegate witness-bearing computation to a state-continuous KeyBox instance. By Assumption 2 (p. 13), such an instance cannot be rolled back or forked to answer the same commitment under two different challenges, and therefore extraction strategies that fundamentally depend on rewinding are not implementable at the hardware boundary. Note that state continuity is a strong assumption that real implementations may only approximate, and may require additional mechanisms to prevent rollback/forking-style state-continuity attacks [50]. In practice state continuity can be approximated using a monotonic freshness mechanism—e.g., hardware-backed monotonic counters in secure NVRAM/TPM, trusted time, or a server-maintained freshness oracle; details follow below.

Even under approximate state continuity, a single rollback of the KeyBox’s mutable operation state can—in a naïve design that samples and stores prover nonces—enable Schnorr special-soundness extraction and thus catastrophic disclosure of the resident share. To eliminate this specific failure mode, LinOS (Fig. 6 (p. 40)) derives all Fischlin prover nonces deterministically from a static, rollback-invariant seed seed stored inside the KeyBox (Assumption 3 (p. 13)) and session-bound inputs (sid, K, i) . Under this derivation, replaying the same session after rollback reproduces the same nonces and hence the same proof transcript; and distinct sessions yield pseudorandomly independent nonces under PRF security (Lemma 14 (p. 42)). This is a targeted hardening: it removes the “one rollback $\Rightarrow$ key disclosure” vector, but does not by itself close all state-continuity gaps (Remark 20 (p. 43)).

Definition 4 (State-continuity failure event and parameter). Let Bad_{sc} denote the event that, in a given execution, some KeyBox instance violates Assumption 2 (p. 13), i.e., its sealed state is rolled back to a prior value or forked/cloned into two independently queryable continuations from the same prior state. In a concrete deployment that approximates state continuity with an anti-rollback mechanism, let $\varepsilon_{\text{sc}}(\lambda)$ be any bound taken over all randomness, faults, and adversarial actions, on $\Pr[\text{Bad}_{\text{sc}}]$ for the lifetime of an execution at security parameter λ .

Lemma 1 (Additive degradation under approximate state continuity). *Consider any security statement in this paper proved in the $\mathcal{F}_{\text{KeyBox}}$ -hybrid model under Assumption 2 (p. 13), yielding an advantage bound of the form $\text{negl}(\lambda)$. In a concrete realization in which Assumption 2 (p. 13) holds except with probability at most $\varepsilon_{\text{sc}}(\lambda)$ (Definition 4 (p. 14)), the corresponding advantage bound becomes $\text{negl}(\lambda) + \varepsilon_{\text{sc}}(\lambda)$.*

Proof Sketch. Let $\mathcal{E}$ be the relevant distinguishing/forgery event. Then

$$\Pr[\mathcal{E}] \leq \Pr[\mathcal{E} \wedge \neg \text{Bad}_{\text{sc}}] + \Pr[\text{Bad}_{\text{sc}}] \leq \Pr[\mathcal{E} \mid \neg \text{Bad}_{\text{sc}}] + \varepsilon_{\text{sc}}(\lambda).$$

Conditioned on $\neg \text{Bad}_{\text{sc}}$, the execution matches the idealized model with state continuity, so $\Pr[\mathcal{E} \mid \neg \text{Bad}_{\text{sc}}] \leq \text{negl}(\lambda)$. ■

Definition 5 (Secure-erasure failure event and parameter). Let Bad_{er} denote the event that, in a given execution, the secure-erasure / atomic-activation semantics of Definition 1 (p. 8) is violated for some honest party’s host state. Concretely, Bad_{er} occurs if there exist an honest party P_i and a host-resident value v that the protocol designates as erased (or transient within one honest activation) such that v nevertheless becomes available to the adversary after the intended erasure point without a prior corruption of P_i . In a concrete deployment, let $\varepsilon_{\text{er}}(\lambda)$ be any bound taken over all randomness, faults, and adversarial actions, on $\Pr[\text{Bad}_{\text{er}}]$ for the lifetime of an execution at security parameter λ .

Lemma 2 (Additive degradation under approximate secure erasures). *Consider any security statement in this paper proved in the UC model with adaptive corruptions with secure erasures (Definition 1 (p. 8)), yielding an advantage bound of the form $\text{negl}(\lambda)$. In a concrete realization in which Definition 1 (p. 8) holds except with probability at most $\varepsilon_{\text{er}}(\lambda)$ (Definition 5 (p. 14)), the corresponding advantage bound becomes $\text{negl}(\lambda) + \varepsilon_{\text{er}}(\lambda)$.*

Proof Sketch. Let $\mathcal{E}$ be the relevant distinguishing/forgery event. Then

$$\Pr[\mathcal{E}] \leq \Pr[\mathcal{E} \wedge \neg \text{Bad}_{\text{er}}] + \Pr[\text{Bad}_{\text{er}}] \leq \Pr[\mathcal{E} \mid \neg \text{Bad}_{\text{er}}] + \varepsilon_{\text{er}}(\lambda).$$

Conditioned on $\neg \text{Bad}_{\text{er}}$, the execution matches the idealized model with secure erasures, so $\Pr[\mathcal{E} \mid \neg \text{Bad}_{\text{er}}] \leq \text{negl}(\lambda)$. ■

We do not restate $\varepsilon_{\text{sc}}(\lambda)$ or $\varepsilon_{\text{er}}(\lambda)$ in every theorem. By Lemmas 1 (p. 14) and 2 (p. 14), negligible security bounds in the remainder of the paper should be read as $\text{negl}(\lambda) + \varepsilon_{\text{sc}}(\lambda) + \varepsilon_{\text{er}}(\lambda)$ for the relevant deployment/execution.

Assumption 2 (p. 13) is a hard safety requirement for any KeyBox interface that is intended to be one-shot. If the anti-rollback mechanism enforcing state continuity fails even once—e.g., a monotonic counter wraps, sealed snapshots can be restored from backup, or a freshness oracle is bypassed—then rollback/forking becomes possible. In an unhardened design that samples and stores prover nonces in mutable state, the degradation is typically not graceful: the adversary may obtain multiple outputs from an interface intended to be one-shot and can often recover the resident share via special soundness from two responses under the same commitment. We therefore parameterize the failure of state continuity by the execution-level bad event Bad_{sc} (Definition 4 (p. 14)). In any deployment where $\Pr[\text{Bad}_{\text{sc}}] \leq \varepsilon_{\text{sc}}(\lambda)$, Lemma 1 (p. 14) implies that every advantage bound increases additively by $\varepsilon_{\text{sc}}(\lambda)$. Later, we employ a method to eliminate this specific catastrophic path by deriving all Fischlin prover nonces deterministically from a static, rollback-invariant seed while other rollback-affected state remains under the general ε_{sc} .

Concrete accounting for $\varepsilon_{\text{sc}}(\lambda)$ (non-normative). The cryptographic results in this paper do not attempt to bound $\varepsilon_{\text{sc}}(\lambda)$; it is a deployment/engineering parameter that upper-bounds the probability of any rollback/fork event Bad_{sc} over an execution (Definition 4 (p. 14)). We record two common approximation patterns to clarify what typically contributes to ε_{sc} :

(i) Monotonic counters: Suppose state continuity is approximated by a b -bit monotonic counter that is incremented once per state advance (e.g., per sealing epoch), and the implementation is engineered to *fail closed* (refuse to unseal/advance) before wrap-around and to require re-provisioning/rekeying before exhaustion. Then counter wrap-around contributes zero to $\varepsilon_{\text{sc}}(\lambda)$ for any deployment whose lifetime advance budget N_{max} satisfies $N_{\text{max}} < 2^b$ (ignoring physical faults). For scale, $2^{64} \approx 1.8 \times 10^{19}$ increments; even at 10^6 advances/day, exhaustion would occur only after $\approx 5 \times 10^{10}$ years. In practice, however, the effective budget is often dominated by write-endurance limits, rate limits, or administrative rotation, rather than bit-width; the same accounting applies by replacing 2^b with the enforced safe-advance cap.

(ii) Server-/time-based freshness: For oracle- or time-based approaches, the dominant contribution to ε_{sc} is typically policy, not cryptographic guessing: if the device ever continues to unseal/advance without a fresh token or a non-decreasing trusted-time reading (i.e., it fails open), we count that execution under Bad_{sc} . Conversely, if the mechanism is engineered to fail closed, outages impact availability but do not increase $\varepsilon_{\text{sc}}(\lambda)$. Appendix A (p. 74) discusses engineering trade-offs and re-provisioning strategies.

2.5 Hybrid execution model and NXK-restricted material

Definition 6 ($\mathcal{F}_{\text{KeyBox}}$ -hybrid model). Let $\mathcal{F}_{\text{KeyBox}}^{(i)}$ denote the ideal functionality for P_i 's KeyBox $_i$. The $\mathcal{F}_{\text{KeyBox}}$ -hybrid model is defined by a PPT environment $\mathcal{Z}$, a PPT adversary $\mathcal{A}$, a set of PPT parties $\{P_i\}_{i \in [n]}$, and a collection of ideal functionalities $\{\mathcal{F}_{\text{KeyBox}}^{(i)}\}_{i \in [n]}$ that are created as:

1. Instantiation: for every $i \in [n]$, generate a fresh functionality instance $\mathcal{F}_{\text{KeyBox}}^{(i)}$, initialized with an empty state.
2. Communication between parties is performed via $\mathcal{F}_{\text{channel}}$ for confidential authenticated point-to-point messages, and via $\mathcal{F}_{\text{pub}}$ for authenticated transcript-public dissemination.
3. Adaptive corruptions with secure erasures: parties are corrupted adaptively under Definition 1 (p. 8). Upon corruption of P_i , $\mathcal{A}$ learns only P_i 's current host state (outside the KeyBox boundary); any values explicitly erased by the protocol are not revealed. Thereafter, $\mathcal{A}$ controls P_i and may invoke $\mathcal{F}_{\text{KeyBox}}^{(i)}.\text{Load}$ and $\mathcal{F}_{\text{KeyBox}}^{(i)}.\text{Use}$, but KeyBox-resident secret state (in particular the map Λ of resident shares) is never revealed.

In the $\mathcal{F}_{\text{KeyBox}}$ -hybrid model, “ P_i invokes $\mathcal{F}_{\text{KeyBox}}^{(i)}.\text{Load/Use}(\dots)$ ” denotes a party-local call over the internal channel, executed immediately upon activation if P_i is honest. A corrupted P_i leaves the timing and admissible inputs to the adversary, i.e., the ideal functionality cannot directly write into $\mathcal{F}_{\text{KeyBox}}^{(i)}$ or force installation/registration for corrupted parties.

Remark 6 (KeyBox-driver wrapper for ideal-world KeyBox calls). In Fig. 3 (p. 11), $\mathcal{F}_{\text{KeyBox}}^{(P_{\text{own}})}$ accepts (Load, μ, g, m) and (Use, μ, f, m) only from its owner party P_{own} . Thus, under standard UC semantics an ideal functionality cannot directly issue Load/Use to $\mathcal{F}_{\text{KeyBox}}^{(i)}$. Throughout, whenever an ideal functionality description says “have P_i invoke $\mathcal{F}_{\text{KeyBox}}^{(i)}.\text{Load/Use}(\cdot)$,” this is shorthand for the following fixed mechanism: in the ideal execution, the party P_i is composed with a deterministic KeyBox-driver wrapper $\mathcal{W}_{\text{KB}}^{(i)}$ that behaves like the dummy party on all external ports, but additionally implements an internal command port from ideal functionalities. Upon receiving $(\text{KBcmd}, \text{sid}, \text{ops})$, where ops is a list of KeyBox calls of the form (Load, μ, g, m) and/or (Use, μ, f, m) , if P_i is honest then $\mathcal{W}_{\text{KB}}^{(i)}$ executes the listed calls sequentially against its local instance $\mathcal{F}_{\text{KeyBox}}^{(i)}$ and returns $(\text{KBret}, \text{sid}, \text{res})$ with the list of return values. If P_i is corrupted, $\mathcal{A}$ controls $\mathcal{W}_{\text{KB}}^{(i)}$ and may delay/modify/ignore these commands.

Remark 7 (NKK-restricted state). We call a (possibly structured) bitstring $v \in \{0, 1\}^*$ *share-deriving* only relative to a designated KeyBox share. Fix a party P_i and a local KeyBox slot $\mu \in \{0, 1\}^*$ in $\mathcal{F}_{\text{KeyBox}}^{(i)}$ that contains a scalar long-term share $k_{i,\mu} \in \mathbb{Z}_p$. We say that v is share-deriving for (i, μ) if there exist PPT-computable functions $a(\cdot), b(\cdot), y(\cdot)$ (fixed independently of the secret share $k_{i,\mu}$) such that on input v they output $a(v) \in \mathbb{Z}_p^*$, $b(v) \in \mathbb{Z}_p$, and $y(v) \in \mathbb{Z}_p$ with

$$y(v) = a(v) \cdot k_{i,\mu} + b(v) \pmod{p}.$$

Equivalently, from v alone one can compute a *caller-invertible* affine image $y(v) = L_{a(v), b(v)}(k_{i,\mu})$ together with map parameters $(a(v), b(v))$, where $L_{a,b}(x) := ax + b$, so the caller can recover $k_{i,\mu} = a(v)^{-1}(y(v) - b(v)) \pmod{p}$. When the target slot is clear from context, we omit (i, μ) and simply say that v is share-deriving. If (a, b) are fixed by the KeyBox profile or chosen by the caller, the functions $a(\cdot), b(\cdot)$ may hard-code those public parameters or parse them from v . Because we fix an injective, self-delimiting encoding $\langle \cdot \rangle$ of mixed tuples into $\{0, 1\}^*$, this definition applies equally to any finite collection of values $(v_1, \dots, v_t)$ by taking $v := \langle v_1, \dots, v_t \rangle$. We will sometimes abuse notation and refer to such a collection as *share-deriving material*.

We treat share-deriving material as *NKK-restricted*: it must not appear in the adversary-visible transcript beyond the explicit leakage modeled by our channels (e.g., $\mathcal{F}_{\text{channel}}$ ’s length leakage), and it must not be written to persistent storage outside a KeyBox. Honest parties may nevertheless handle share-deriving material transiently in host memory and may transmit it over authenticated, confidential channels (modeled by $\mathcal{F}_{\text{channel}}$) when required by the surrounding protocol, provided that (i) whenever it is used to install a long-term share in $\mathcal{F}_{\text{KeyBox}}^{(i)}$ it is delivered only via the internal secure channel between P_i and $\mathcal{F}_{\text{KeyBox}}^{(i)}$, and (ii) it is securely erased from host memory immediately after its last use. This transient handling is protocol-internal and is distinct from a KeyBox export interface: under an admissible KeyBox profile (Assumption 1 (p. 11)), $\mathcal{F}_{\text{KeyBox}}$ never returns resident shares (or caller-invertible affine images) in the clear. Consequently, an adversary can learn enough share-deriving material to recompute an honest device’s share only by corrupting the relevant endpoint(s) during the protocol before erasure, not from the public transcript alone. Share-deriving is intentionally a transcript-only notion: it captures values from which the caller can recover $k_{i,\mu}$ from v alone (i.e., without any additional secret inputs). An API can still leak a resident share indirectly by returning an output that is not share-deriving by itself but becomes share-deriving once combined with caller-held secrets permitted by the profile (e.g., a ciphertext under a caller-supplied wrapping key). Such caller-recoverable exports are ruled out by key-opacity, so we explicitly exclude caller-decryptable wrapping/export: if the caller knows the decryption key, it can decrypt to obtain share-deriving plaintext (or an invertible affine image), which is not simulatable from $\text{PubMap}(k_{i,\mu})$ alone and therefore violates key-opacity [9].

2.6 Real-world instantiations of admissible KeyBox profiles

Our KeyBox idealization is deliberately *profile-centric* (Definition 3 (p. 12)): the surrounding protocol fixes an admissible KeyBox profile $(\chi_{\text{adm}}, \mathcal{F}_{\text{adm}})$, and the KeyBox is assumed to accept only those derivations and operations. This is essential as API-level non-exportability alone does not suffice for our security arguments. Many deployed keystore/HSM APIs expose operations that either (a) directly leak share-deriving material, or (b) let a caller recover a resident share indirectly (e.g., via caller-decryptable wrapping/export under a caller-controlled key), even when

raw secret key bytes are nominally non-exportable. In our model this is captured by (i) pinning the profile and (ii) requiring key-opacity (Assumption 1 (p. 11)) for the induced external transcript, i.e., that everything observable outside the KeyBox boundary is simulatable given only the corresponding public key $\text{PubMap}(k)$ and the public query inputs. Further, Remark 7 (p. 16) treats all share-deriving material (including caller-invertible affine images of a stored share) as NXK-restricted, and separately excludes caller-decryptable wrapping/export via key-opacity even though the ciphertext alone need not be share-deriving under the transcript-only definition.

Implementation note (non-normative). As mentioned earlier, this paper’s security proofs assume a profile-centric KeyBox, satisfying key-opacity and state continuity (Assumption 2 (p. 13)). In practice, such a profile can be enforced by construction by interposing a narrow “profile adapter” between the protocol and a broader vendor API—e.g., a minimal TEE enclave used purely as a keystore, an attested enclave $\leftrightarrow$ KMS integration, or an HSM under a strict allowlist. The adapter must (i) forbid any vendor-API operation that returns share-deriving outputs of a resident key (i.e., caller-invertible affine images of an already-installed long-term share; cf. Remark 7 (p. 16)), while still permitting the protocol’s own key-independent operations (such as $\text{SDKG.LeafNit} \in \mathcal{F}_{\text{KI}}^{\text{SDKG}}$), whose outputs are share-deriving only prospectively and are handled as NXK-restricted transient host state with secure erasure; (ii) prohibit caller-decryptable wrapping/export, preserving key-opacity; (iii) pin sealing recipients to attested identities while engineering explicit freshness/anti-rollback to approximate state continuity. Appendix B (p. 75) summarizes candidate deployment families (Table 7 (p. 75)) and gives a profile-capture checklist. Throughout, when we say that some computation runs “inside the KeyBox boundary,” we include any minimal, pinned “profile adapter” that is itself part of the same attested, state-continuous trusted boundary as the KeyBox; and (iv) forbid any KeyBox API primitive that can generate the UC-extractable consistency AoKs used by the protocol (e.g., Fischlin-based UC-context proofs): these AoKs must be generated by the host/party ITM so that the UC simulator can record the prover’s gRO-CRP query log required for straight-line extraction (Remark 10 (p. 21)). It must also ensure that randomized operations (including sealing) use fresh, domain-separated randomness across slots and across calls (Remark 2 (p. 12)); otherwise the slot-separability component of key-opacity may fail in spite of IND-CCA security of the abstract scheme.

3 The Conflict: Verifiable Sharing vs. NXK

In this section, we isolate the structural reason that mandates UC-secure DKG protocols to enforce the core (R)VSS obligations: secrecy against unauthorized sets and uniqueness (a single well-defined shared secret) together with affine consistency of honest parties’ local outputs. In the literature, these obligations are ensured by a VSE layer, which is classically instantiated via (R)VSS and its variants (PVSS/AVSS) using polynomial sharing [60], commitments, and complaint/opening logic. In other UC(-style) DKGs, the same role is alternatively realized via commitment-and-proof / (N)IZK-based authenticated sharing that prevents equivocation and certifies transcript-defined affine relations (e.g., [46, 15]).

Definition 7 (DL experiment). Let $\mathbb{G}$ be a cyclic group of prime order $p = p(\lambda)$ with generator $\mathcal{G} \in \mathbb{G}$. For a PPT adversary $\mathcal{A}$, define

$$\text{Adv}_{\mathcal{A}}^{\text{dl}}(\lambda) := \Pr \left[x \leftarrow \mathbb{Z}_p^*; X := x\mathcal{G}; x' \leftarrow \mathcal{A}(\mathcal{G}, X) : x' = x \right].$$

Assumption 4 (DL) For all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{dl}}(\lambda) \leq \text{negl}(\lambda)$.

Definition 8 (DDLEQ game). Let $\mathbb{G}$ be a cyclic group of prime order $p = p(\lambda)$ with (public) generators $\mathcal{G}, \mathcal{H} \in \mathbb{G}$. Consider the experiment that samples $r, s \leftarrow \mathbb{Z}_p^*$ and a bit $b \leftarrow \{0, 1\}$. If $b = 1$ set $(A, B) := (r\mathcal{G}, r\mathcal{H})$; if $b = 0$ set $(A, B) := (r\mathcal{G}, s\mathcal{H})$. An adversary $\mathcal{A}$ is given $(\mathcal{G}, \mathcal{H}, A, B)$ and outputs a bit b' . Define

$$\text{Adv}_{\mathcal{A}}^{\text{ddleq}}(\lambda) := \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

Assumption 5 (DDLEQ) For all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{ddleq}}(\lambda) \leq \text{negl}(\lambda)$.

Remark 8 (Terminology: DDLEQ is DDH). Definition 8 (p. 17) is exactly the standard DDH distinguishing game on the tuple $(\mathcal{G}, \mathcal{H}, A, B)$, written in the “same-exponent across two bases” form matching DLEQ statements. We use the name DDLEQ only to align notation with the Chaum–Pedersen relation. (If $\log_{\mathcal{G}}(\mathcal{H})$ were known, the game would be trivial by checking $B = (\log_{\mathcal{G}}(\mathcal{H}) \cdot A)$.)

We use the standard UC framework with ITMs as in Canetti [13]. Let $\text{Exec}(\Psi, \mathcal{A}, \mathcal{Z}, \lambda)$ and $\text{Ideal}(\mathcal{F}, \text{Sim}, \mathcal{Z}, \lambda)$ denote the standard real and ideal execution ensembles (Exec/Ideal experiments).

Definition 9 (UC realization). A protocol Ψ UC-realizes an ideal functionality $\mathcal{F}$ if for every PPT adversary $\mathcal{A}$ there exists a PPT simulator Sim such that for every PPT environment $\mathcal{Z}$,

$$\text{Exec}(\Psi, \mathcal{A}, \mathcal{Z}, \lambda) \approx_c \text{Ideal}(\mathcal{F}, \text{Sim}, \mathcal{Z}, \lambda).$$

Note 3. Our only protocol security notion is UC realization. Other experiment-based definitions in the paper are standard assumptions or local properties of underlying resources/primitives used solely as hypotheses to establish UC realization of our ideal functionalities.

Informally, a DL-based DKG outputs: (i) a public key K to everyone and (ii) a secret sharing of its discrete log k among the parties according to the access structure Γ , so that authorized sets $A \in \Gamma$ can reconstruct k or jointly sign using their shares while unauthorized sets learn no (non-negligible) information about k beyond K . In a UC formulation, the (DL-based) ideal functionality $\mathcal{F}_{\text{DKG}}^{\Gamma}$ samples a fresh secret $k \leftarrow \mathbb{Z}_p$, distributes shares $(k_1, \dots, k_n)$ consistent with Γ , and outputs $K := k\mathcal{G}$. This implies three tightly coupled obligations that are classically associated with (R)VSS:

- Secrecy: for any corruption set $B \notin \Gamma$, the adversary learns no (non-negligible) information about k beyond K .
- Uniqueness (strong correctness): whenever honest parties accept completion, there exists a single value k such that every authorized set $A \in \Gamma$ can reconstruct that same k .
- Affine consistency: honest parties’ local outputs are consistent with one global sharing instance of k under Γ : the transcript cannot induce incompatible sharings across different honest subsets.

3.1 DKG subsumes dealerless (R)VSS

We capture the verifiable-sharing subtask by an ideal functionality for dealerless random (R)VSS that is exactly the UC-DKG functionality with its public-key output suppressed. Concretely, for a fixed access structure Γ , the functionality $\mathcal{F}_{\text{RVSS}}^{\Gamma}$ is defined as follows: on session identifier sid and upon receiving init from all parties, it samples $k \leftarrow \mathbb{Z}_p$ and distributes shares $(k_1, \dots, k_n)$ consistent with Γ , and outputs no additional public value. Equivalently, $\mathcal{F}_{\text{RVSS}}^{\Gamma}$ is obtained from $\mathcal{F}_{\text{DKG}}^{\Gamma}$ by locally dropping the public output $K := k\mathcal{G}^*$.

Theorem 1 (UC-DKG implies UC-RVSS). *Let Ψ be any protocol that UC-realizes $\mathcal{F}_{\text{DKG}}^{\Gamma}$ in some model $\mathcal{M}$. Then there exists a protocol Ψ' that UC-realizes $\mathcal{F}_{\text{RVSS}}^{\Gamma}$ in the same model $\mathcal{M}$.*

Proof Sketch. This is immediate from UC closure under efficient post-processing: Ψ' is obtained from Ψ by locally suppressing a public output, and $\mathcal{F}_{\text{RVSS}}^{\Gamma}$ is obtained from $\mathcal{F}_{\text{DKG}}^{\Gamma}$ by the same transformation. Formally, given any $\mathcal{A}$ for Ψ' , build $\mathcal{A}^*$ for Ψ that forwards messages unchanged and drops K , and apply the simulator for Ψ with the same post-processing. $\blacksquare$

Hence, any UC-secure DKG protocol must already satisfy the (R)VSS properties embodied by $\mathcal{F}_{\text{RVSS}}^{\Gamma}$: secrecy against unauthorized sets and a unique, well-defined shared secret underlying honest outputs. Thus, in the standard model (without trusted hardware), a DKG protocol needs a mechanism that enforces exactly these VSS-style guarantees. Whether it is realized via (R)VSS/PVSS/AVSS and/or via commitment-and-proof / ZK-based authenticated sharing, it plays the same conceptual role which is that of a VSE layer.

* An RVSS functionality that also outputs $K := k\mathcal{G}$ can be denoted by $\mathcal{F}_{\text{RVSS}}^{\Gamma, \text{pk}}$; we will not use it here.

Lemma 3 (Uniqueness is necessary for UC-DKG). *Let Ψ be a protocol intended to UC-realize $\mathcal{F}_{\text{DKG}}^\Gamma$. Suppose there exists a non-negligible probability event wherein an execution of Ψ terminates without honest abort, yet there exist authorized sets $A, B \in \Gamma$ whose respective reconstructions yield $k_A \neq k_B$, then Ψ does not UC-realize $\mathcal{F}_{\text{DKG}}^\Gamma$.*

Proof Sketch. Define a PPT environment $\mathcal{Z}$ that runs one execution and then adaptively corrupts all parties in $A \cup B$. From their revealed states, $\mathcal{Z}$ computes k_A and k_B using the prescribed reconstruction algorithm and outputs 1 iff $k_A \neq k_B$. By assumption, $\Pr[\mathcal{Z} \text{ outputs } 1]$ is non-negligible in the real world. In the ideal world, $\mathcal{F}_{\text{DKG}}^\Gamma$ samples a single secret k and distributes shares consistent with that k , so any authorized set must reconstruct the same k . Thus, $\Pr[\mathcal{Z} \text{ outputs } 1] = 0$ (up to negligible reconstruction error) in the ideal world, contradicting UC indistinguishability. ■

Lemma 3 (p. 19) is precisely the “verifiability” obligation that VSS packages: malicious parties must not be able to make different honest, authorized subsets accept incompatible sharings. Therefore, any UC-secure DKG must implement a mechanism that prevents (or detects and neutralizes) equivocation in the distribution of share material and/or in the public data that defines the sharing. For additional discussion of DKG security notions and constructions, see [43].

3.2 Exported-share enforcement vs. NXK

In the traditional, transcript-visible share (a.k.a. “exportable-share”) model, parties exchange explicit share-derived values over the network (so they become part of the adversary-visible transcript), and an adaptive adversary can later corrupt parties and inspect their local states.

1. Hiding of each contribution: parties must contribute randomness to the final key without revealing their secret contribution to unauthorized corruptions.
2. Binding/consistency of each contribution: a malicious party must not be able to send inconsistent information to different recipients in a way that makes honest parties accept incompatible sharings. Moreover, the transcript must support the simulation/extraction requirements demanded by UC.

This is exactly what classical VSS-based mechanisms provide: each party acts as a dealer, shares a random secret in a verifiable way, and the final secret is the sum of the non-disqualified contributions. In DL-based constructions, Feldman/Pedersen-style commitments [29, 54] additionally expose the public group element corresponding to each dealer’s secret, enabling computation of the public key. The same enforcement role can also be realized via “authenticated sharing” based on commitments and (N)IZK proofs: parties commit to contributions and prove knowledge/consistency of the relations that the transcript induces.

For the following Proposition, fix a prime field $\mathbb{F}_p$ and an integer $t \geq 1$. Let $f(X) \in \mathbb{F}_p[X]$ be uniformly random of degree at most t , and define $k_i := f(i)$ for $i \in [n]$. Let $x_{\text{new}} \in \mathbb{F}_p$ with $x_{\text{new}} \notin [n]$ denote the evaluation point for a joining device. Let pp denote the public parameters and any fixed transcript-public information. We work in the $\text{NXK}/\mathcal{F}_{\text{KeyBox}}$ setting from Section 2 (p. 7). Let τ_{ext} denote the external enrollment view/transcript outside all KeyBox instances. Assume $f(x_{\text{new}})$ is computationally unpredictable given pp : there exists a function $\varepsilon = \varepsilon(\lambda)$ such that for every PPT predictor $\mathcal{B}$ and every realization of pp in the support,

$$\Pr[\mathcal{B}(\text{pp}) = f(x_{\text{new}}) \mid \text{pp}] \leq \varepsilon(\lambda).$$

Proposition 1 (External fresh-share enrollment and NXK). *Let the setup be as above. Then for any PPT strategy that computes an output $\hat{k}_{\text{new}} \in \mathbb{F}_p$ from $(\text{pp}, \tau_{\text{ext}})$,*

$$\Pr[\hat{k}_{\text{new}} = f(x_{\text{new}}) \mid \text{pp}] \leq \varepsilon(\lambda) + \text{negl}(\lambda).$$

Proof. Define Hybrid $\mathcal{D}_0$ as the real enrollment execution and hybrid $\mathcal{D}_1$ by modifying $\mathcal{D}_0$ as follows: maintain a table of simulator instances indexed by KeyBox slots. Whenever a slot (P, μ) is first queried and has an installed key $k_{P, \mu} := \Lambda[P, \mu]$, set $\text{pk}_{P, \mu} \leftarrow \text{PubMap}(k_{P, \mu})$ and initialize an independent simulator instance $\text{Sim}_{P, \mu}$ by running Sim on input $(1^\lambda, \text{pk}_{P, \mu})$. Thereafter, for every call to $\mathcal{F}_{\text{KeyBox}}.\text{Use}(\mu, f, m)$ initiated by owner P with $f \in \mathcal{F}_{\text{adm}}$, respond as follows:

- Key-dependent interfaces (slot-bound): If $f \neq \text{OpenFromPeer}$, replace the real reply produced by $f(k_{P,\mu}, \text{st}[P, \mu, F(f)], m)$ with the output of $\text{Sim}_{P,\mu}(f, m)$.
- Key-independent interfaces (sealing-only): If $f = \text{OpenFromPeer}$, answer using a separate key-independent simulator state Sim_P^{KI} as allowed by Assumption 1 (p. 11). Concretely, Sim_P^{KI} is stateful across OpenFromPeer calls and maintains its own buffer state for typed handles so that subsequent uses of those handles (via Resolve) are answered consistently with the handles it issued.

The instance $\text{Sim}_{P,\mu}$ is reused across all queries to the same slot so it may maintain state, while different slots use independent instances (cf. Assumption 1 (p. 11) and Remark 1 (p. 12)).

By key-opacity of $\mathcal{F}_{\text{adm}}$ w.r.t. PubMap , the external transcript/view τ_{ext} in $\mathcal{D}_0$ and $\mathcal{D}_1$ are computationally indistinguishable; hence for any event $\mathcal{E}$,

$$|\Pr[\mathcal{E} \mid \mathcal{D}_0] - \Pr[\mathcal{E} \mid \mathcal{D}_1]| \leq \text{negl}(\lambda).$$

In $\mathcal{D}_1$, by construction, all externally visible key-dependent outputs are generated by the slotwise simulators from pp and the corresponding public values $\text{pk}_{P,\mu}$ only, while the key-independent sealing-only interface OpenFromPeer is handled by the separate simulator state Sim_P^{KI} (which, per Assumption 1 (p. 11), does not require any resident key).

Let $\mathcal{A}_{\text{out}}$ be the (PPT) procedure that outputs $\hat{k}_{\text{new}}$ from $(\text{pp}, \tau_{\text{ext}})$. Define a PPT predictor $\mathcal{B}$ that on input pp samples τ_{ext} according to $\mathcal{D}_1$ (using the same slotwise simulators $\{\text{Sim}_{P,\mu}\}$ and key-independent sealing simulator states $\{\text{Sim}_P^{\text{KI}}\}$ as above) and outputs $\mathcal{A}_{\text{out}}(\text{pp}, \tau_{\text{ext}})$. Then for every fixed pp in the support,

$$\Pr[\hat{k}_{\text{new}} = f(x_{\text{new}}) \mid \text{pp}, \mathcal{D}_1] = \Pr[\mathcal{B}(\text{pp}) = f(x_{\text{new}}) \mid \text{pp}] \leq \varepsilon(\lambda).$$

Transferring back to $\mathcal{D}_0$ via indistinguishability yields $\Pr[\hat{k}_{\text{new}} = f(x_{\text{new}}) \mid \text{pp}] \leq \varepsilon(\lambda) + \text{negl}(\lambda)$. ■

Remark 9 (Min-entropy vs. public group elements in pp). In a prime-order group with fixed generator $\mathcal{G}$, the map $a \mapsto a\mathcal{G}$ is a bijection on $\mathbb{Z}_p$. Thus, if pp contains a group element that is a deterministic one-to-one function of $f(x_{\text{new}})$, then the information-theoretic conditional min-entropy of $f(x_{\text{new}})$ given pp is 0, even though recovering $f(x_{\text{new}})$ from pp may be computationally hard under DL. Accordingly, Proposition 1 (p. 19) is stated in terms of computational unpredictability. As a special case, if pp is such that $\max_{a \in \mathbb{F}_p} \Pr[f(x_{\text{new}}) = a \mid \text{pp}] \leq 2^{-h}$ for some $h(\lambda)$ (e.g., when pp is statistically independent of $f(x_{\text{new}})$), then the unpredictability hypothesis holds with $\varepsilon(\lambda) = 2^{-h}$, recovering the corresponding information-theoretic bound.

Note that Proposition 1 (p. 19) is a statement about external derivation/export: it rules out computing a clear value $\hat{k}_{\text{new}}$ intended to equal $f(x_{\text{new}})$ using only $(\text{pp}, \tau_{\text{ext}})$, where τ_{ext} is the view outside all KeyBox instances. It does not rule out enrollment mechanisms in which share-derived material is transferred only via attested KeyBox-to-KeyBox sealing and is decrypted/consumed inside a KeyBox. Dynamic joins that assign a fresh independent Shamir share typically require some party outside KeyBoxes to compute, reveal, or otherwise export share-derived values. Proposition 1 (p. 19) captures this obstruction under key-opacity: no PPT strategy can externally compute such a fresh share from $(\text{pp}, \tau_{\text{ext}})$. Therefore, join protocols that require exporting share-derived values are incompatible with NXX. Therefore, in our SDKG protocol, we instead realize post-DKG enrollment via RDR, where additional devices are enrolled as redundant front-ends for an existing role/share using $\text{SealToPeer}/\text{OpenFromPeer}$, avoiding exported share-derived plaintexts and preserving the public key.

4 Cryptographic Primitives for State-Continuous KeyBoxes

Our end-to-end security claim is a UC realization theorem for SDKG. To keep the exposition modular, we state required properties of underlying proof/certificate mechanisms using standard game-based definitions in the gRO-CRP global-setup model, and we instantiate them in disjoint, domain-separated contexts so these guarantees apply within the surrounding UC execution. We analyze our protocols in the UC framework augmented with a global resource shared across all sessions: a random-oracle-like functionality H that is sampled once per UC execution and used by all ITMs. This is in the spirit of UC formulations with a global random oracle (e.g., [14, 17, 48, 27, 11]). Formally, H is joint state that persists across sessions and sub-protocols (cf. [18]).

- ◆ **Global setup:** A nonempty context set $\text{Ctx} \subseteq \{0, 1\}^*$ and a finite range $\mathcal{Y} := \{0, 1\}^\lambda$. Fix a partition $\text{Ctx} = \text{Ctx}_{\text{np}} \dot{\cup} \text{Ctx}_{\text{p}}$ into non-programmable contexts Ctx_{np} and restricted-programmable contexts Ctx_{p} .
- ◆ **State:** a lazy table $T : \text{Ctx} \times \{0, 1\}^* \rightarrow \mathcal{Y}$ (init empty).
- ◆ **Semantics (direct access):** All interfaces below are delivered locally to the invoking ITM (i.e., not as network messages). If the invoking ITM is adversary-controlled, then $\mathcal{A}$ is notified of the full query/answer transcript: for each invocation it learns (ctx, x, y) where y is the returned value.
- ◆ **Interfaces:**
 - **Query**(ctx, x): if $(\text{ctx}, x) \notin \text{dom}(T)$, sample $y \leftarrow \mathcal{Y}$ and set $T[\text{ctx}, x] \leftarrow y$. Return $T[\text{ctx}, x]$.
 - **SimProgram**(ctx, x, y) (simulator-only): If $\text{ctx} \notin \text{Ctx}_{\text{p}}$ or $(\text{ctx}, x) \in \text{dom}(T)$, return $\perp$. Set $T[\text{ctx}, x] \leftarrow y$ and return ok.

Fig. 4: Global setup functionality $\mathcal{G}_{\text{gRO-CRP}}$ implementing gRO-CRP.

Remark 10 (Oracle-tape convention). When we say that an extractor or simulator inspects $\text{Log}_{\mathcal{P}^*}$, we mean that it runs the ITM $\mathcal{P}^*$ with explicit oracle access to the global functionality $\mathcal{G}_{\text{gRO-CRP}}$ and records the transcript of its local calls to **Query**(ctx, x) together with the corresponding replies. When a set of contexts is relevant, $\text{Log}_{\mathcal{P}^*}$ is understood to be restricted to those contexts.

UC usage. In our UC proofs, whenever straight-line extraction is applied to a proof produced by an adversary-controlled host prover (i.e., outside any KeyBox boundary), the simulator obtains the required oracle transcript by recording the **Query** calls made by that adversary-controlled ITM (cf. Fig. 4 (p. 21)). We never assume access to **Query** traces issued inside an honest KeyBox instance; such KeyBox-internal oracle calls are not exposed at the host/API boundary under local-call semantics.

Definition 10 (gRO-CRP). The gRO-CRP-hybrid model is the UC model augmented with the single global setup functionality $\mathcal{G}_{\text{gRO-CRP}}$ (Fig. 4 (p. 21)). It implements an oracle $H : \text{Ctx} \times \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ with local-call semantics. The context set Ctx is partitioned as $\text{Ctx} = \text{Ctx}_{\text{np}} \dot{\cup} \text{Ctx}_{\text{p}}$. All ITMs may invoke **Query**(ctx, x). In programmable contexts $\text{ctx} \in \text{Ctx}_{\text{p}}$, the simulator additionally has access to **SimProgram**(ctx, x, y) as specified in Fig. 4 (p. 21); no other ITM can invoke **SimProgram**.

Thus, unlike existing approaches (e.g., [15]) wherein the UC proof is carried out in a strict GRO setting, we work in gRO-CRP: a single global oracle resource with local-call semantics and explicit context-based domain separation. Our gRO-CRP model can be characterized as a context-partitioned instance of the restricted-programmable GRO variants in the taxonomy of Camenisch et al. [11]: it coincides with strict GRO on the non-programmable contexts Ctx_{np} and adds only fresh-point, non-overwriting programmability for the simulator on the designated proof contexts Ctx_{p} . On non-programmable contexts Ctx_{np} (used for transcript digests/receipts), gRO-CRP coincides with the standard non-programmable GRO. For proof contexts Ctx_{p} , gRO-CRP additionally exposes a simulator-only programming hook **SimProgram** to realize the universal simulation interface required by our UC-NIZK(-AoK)s (Definition 11 (p. 24)) instantiated via the optimized Fischlin transform (Definition 16 (p. 25)). On proof contexts Ctx_{p} , gRO-CRP exposes only a *simulator-only* programming hook **SimProgram**. No protocol party (and hence no adversary) can program oracle outputs. Thus, in Ctx_{p} contexts, the simulator can fail only if an external ITM *pre-queries* an input that the simulator intends to program. Lemma 4 (p. 22) bounds this pre-query event and implies that, for our uses (Fischlin-based UC-NIZKs in contexts in Ctx_{p}), the simulator programs only fresh points except with negligible probability.

For any PPT machine making $\text{poly}(\lambda)$ **Query** calls, each fresh (ctx, x) returns an independent uniform $y \leftarrow \mathcal{Y}$. Moreover, for any (ctx^*, x^*) that was neither queried nor simulator-programmed, $H(\text{ctx}^*, x^*)$ is uniform conditioned on the machine's view. This aligns with the standard programmable-random-oracle abstraction used to express the universal simulation interface for NIZKs: the simulator may set oracle values at a (negligible) set of fresh inputs associated with simulated proofs, while all non-proof uses of hashing (contexts in Ctx_{np}) remain strictly non-programmable. The programming hook is a simulator interface in the idealized model; it has no concrete analogue for a fixed hash; it is included solely to realize the universal simulation interface for RO-based NIZKs. Accordingly, instantiating **Query** by a fixed domain-separated hash function should be read as the usual (global) RO heuristic for programmable-RO-based UC-NIZKs, with instantiation caveats as studied in the GRO literature (e.g., [11]) and in recent work on limitations for distributing RO-based proofs (e.g., [27]).

Because $\mathcal{G}_{\text{gRO-CRP}}$ has local-call semantics, the simulator can obtain an oracle query/answer log $\text{Log}_{\mathcal{P}^*}$ only for provers that are themselves adversary-controlled ITMs. Specifically, KeyBox ITM is never adversary-controlled (even when its owner party is corrupted), so any $\text{Query}(\cdot, \cdot)$ calls issued inside a KeyBox are transcript-private and are unavailable to the UC simulator. Therefore, whenever our UC proofs rely on straight-line extraction from $\text{Log}_{\mathcal{P}^*}$ (e.g., for Fischlin-based UC-context AoKs in contexts in Ctx_{p}), the corresponding prover must be the host/party ITM outside any KeyBox boundary. Equivalently, the admissible KeyBox API profile must not expose any operation that outputs those UC-context proofs (or any artifact that would verify as such a proof) in a way that keeps the relevant oracle queries inside the KeyBox. KeyBox-resident proof-generation primitives, if present, must be instantiated in disjoint contexts and are used only under ZK/simulation (no extraction).

Relation to standard models. It is useful to compare three increasingly strong global-oracle abstractions:

- Strict GRO: only $\text{Query}(\text{ctx}, x)$ is available in all contexts. In this model the universal simulation interface required by our Fischlin-based UC-NIZKs is unattainable (Proposition 2 (p. 27)).
- gRO-CRP strict GRO on Ctx_{np} plus a simulator-only, fresh-point programming hook on Ctx_{p} . Programming is *non-overwriting* (fails on pre-queried points) and is used only to realize the UC-NIZK simulation interface in proof contexts.
- Fully programmable global RO: a simulator can program arbitrary points (potentially even overwrite) in all contexts. We do not assume this: gRO-CRP disallows programming outside Ctx_{p} and disallows overwriting anywhere.

Thus, gRO-CRP is strictly stronger than strict GRO but strictly weaker than a fully programmable GRO. For a broader study of GRO formulations (strict, programmable, restricted programmable/observable), see [11].

Lemma 4 (Pre-query bound for gRO-CRP programming). *Fix any context $\text{ctx} \in \text{Ctx}_{\text{p}}$. Consider a UC execution (possibly involving many concurrently interleaved protocol sessions) in which the ideal-world simulator makes at most $m = m(\lambda)$ calls to $\text{SimProgram}(\text{ctx}, x_j, y_j)$ at (possibly adaptive) inputs $x_1, \dots, x_m \in \{0, 1\}^*$. Let Bad_{pre} denote the event that for some j , the j -th call to $\text{SimProgram}(\text{ctx}, x_j, y_j)$ returns $\perp$ (equivalently, $(\text{ctx}, x_j) \in \text{dom}(\mathbf{T})$ at the time of that call). Suppose that immediately before each x_j is fixed, conditioned on the complete external view view_j of all non-simulator ITMs, the point x_j has conditional min-entropy at least $h_j(\lambda)$ in the sense that*

$$\max_{u \in \{0,1\}^*} \Pr[x_j = u \mid \text{view}_j] \leq 2^{-h_j(\lambda)}.$$

If the total number of $\text{Query}(\text{ctx}, \cdot)$ calls made by all non-simulator ITMs before the j -th programming attempt is at most $Q_j(\lambda)$, then

$$\Pr[\text{Bad}_{\text{pre}}] \leq \sum_{j=1}^m Q_j(\lambda) \cdot 2^{-h_j(\lambda)}.$$

Proof Sketch. For a fixed j , let S_j be the set of inputs queried via $\text{Query}(\text{ctx}, \cdot)$ by non-simulator ITMs prior to the j -th call to SimProgram . By assumption $|S_j| \leq Q_j$. Conditioned on view_j , we have

$$\Pr[x_j \in S_j \mid \text{view}_j] \leq \sum_{u \in S_j} \Pr[x_j = u \mid \text{view}_j] \leq |S_j| \cdot 2^{-h_j} \leq Q_j \cdot 2^{-h_j}.$$

A union bound over $j \in [m]$ yields the claim. ■

Hence, if $m, Q_j = \text{poly}(\lambda)$ and $h_j(\lambda) = \omega(\log \lambda)$ for all j , then $\Pr[\text{Bad}_{\text{pre}}] = \text{negl}(\lambda)$.

Lemma 5 (No cross-context influence in gRO-CRP). *Fix any execution in the gRO-CRP-hybrid model. For every non-programmable context $\text{ctx} \in \text{Ctx}_{\text{np}}$, the joint distribution of all replies to calls $\text{Query}(\text{ctx}, \cdot)$ is identical to that of a standard (non-programmable) GRO for that context, even conditioned on an arbitrary sequence of simulator calls $\text{SimProgram}(\text{ctx}', \cdot, \cdot)$ in contexts $\text{ctx}' \in \text{Ctx}_{\text{p}}$.*

Programming event	Proof context	Fresh high-entropy component in programmed input
Simulating UC-context DL proofs	$\text{ctx}_{\text{UC}} \in \text{Ctx}_{\text{p}}$	Programmed inputs include a fresh response $z_i \in \mathbb{Z}_p$ inside $\langle x, \mathbf{a}, i, e_i, z_i \rangle$ (Definition 16 (p. 25)); conditioned on the full external view, z_i is uniform, so $H_\infty(x) \geq \log p - O(1)$.
Simulating UC-context DLEQ proofs	$\text{ctx}_{\text{DLEQ}} \in \text{Ctx}_{\text{p}}$	Programmed inputs contain a fresh simulator-chosen response component $z_i \in \mathbb{Z}_p$ inside the encoded Fischlin query input, giving $H_\infty(x) \geq \log p - O(1)$.
Simulating KeyBox-context DL proofs	$\text{ctx}_{\text{KeyBox}} \in \text{Ctx}_{\text{p}}$	Each programmed point includes a fresh $z_i \in \mathbb{Z}_p$ chosen by the simulator and embedded in the programmed Fischlin input. Under LinOS (Fig. 6 (p. 40)), this input is of the form $\langle \text{sid}, K, \mathbf{a}, i, e_i, z_i \rangle$, so conditioned on the external view, z_i is uniform and contributes $H_\infty = \Theta(\lambda)$ to that input.

Table 3: Concrete entropy sources: in all cases, a fresh $z_i \in \mathbb{Z}_p$ contributes $\Theta(\lambda)$ conditional min-entropy.

Proof Sketch. In Fig. 4 (p. 21), the oracle table T is indexed by pairs (ctx, x) . A call to $\text{SimProgram}(\text{ctx}', x', y')$ can write only the entry $\mathsf{T}[\text{ctx}', x']$ and only when $\text{ctx}' \in \text{Ctx}_{\text{p}}$. Therefore no entry with $\text{ctx} \in \text{Ctx}_{\text{np}}$ is ever written by SimProgram ; such entries are populated only by lazy sampling upon the first corresponding $\text{Query}(\text{ctx}, x)$ call. Thus, $H(\text{ctx}, \cdot)$ for $\text{ctx} \in \text{Ctx}_{\text{np}}$ is distributed exactly as in a standard global random oracle and is unaffected by programming in other contexts. ■

Remark 11 (Per-context domain separation in gRO-CRP). In $\mathcal{G}_{\text{gRO-CRP}}$ (Fig. 4 (p. 21)) the oracle table T is indexed by pairs (ctx, x) . Consequently, oracle activity in one context cannot affect any other context: a call $\text{Query}(\text{ctx}', \cdot)$ (resp. $\text{SimProgram}(\text{ctx}', \cdot, \cdot)$) reads/writes only entries of the form $\mathsf{T}[\text{ctx}', \cdot]$ and never touches $\mathsf{T}[\text{ctx}, \cdot]$ for $\text{ctx} \neq \text{ctx}'$. In particular:

- for $\text{ctx} \in \text{Ctx}_{\text{np}}$, the induced oracle $H(\text{ctx}, \cdot)$ is a strict (non-programmable) random oracle for that context even conditioned on arbitrary simulator programming in other contexts (cf. Lemma 5 (p. 22));
- for $\text{ctx} \in \text{Ctx}_{\text{p}}$, the induced oracle $H(\text{ctx}, \cdot)$ is a restricted-programmable random oracle for that context, where programming is non-overwriting and confined to that same ctx .

Thus, when logically distinct uses of hashing are assigned disjoint contexts (and inputs are injectively encoded), they behave as domain-separated uses of independent per-context oracles. This is the precise sense in which we apply ROM/gRO-CRP security arguments modularly inside an arbitrarily interleaved UC execution.

Remark 12 (Concrete entropy sources for gRO-CRP programming in this paper). Lemma 4 (p. 22) reduces simulator programming failure to the conditional min-entropy of the programmed inputs x_j . In all of our uses of Fischlin-based UC-NIZK simulation (DL and DLEQ), every SimProgram call is on an input that includes at least one fresh simulator-sampled prover response $z_i \in \mathbb{Z}_p$ (or the analogous response component for the underlying Σ -protocol (Definition 15 (p. 25))), embedded in the injective tuple encoding $\langle \cdot \rangle$ as in Definition 16 (p. 25). Table 3 (p. 23) summarizes the concrete entropy sources for the relevant inputs. Even under concurrency and adaptive corruptions, the “external view” view_j in Lemma 4 (p. 22) may include (a) the entire public transcript so far, (b) all corruption-revealed host state, subject to the protocol’s explicit erasures, and (c) all prior gRO-CRP replies to non-simulator ITMs; nevertheless, the simulator chooses each z_i after view_j is fixed, so z_i remains uniform conditioned on view_j . Since $\log p = \Theta(\lambda)$, this yields $h_j \geq \log p - O(1) = \Theta(\lambda)$.

Plugging $h_j = \Theta(\lambda)$ and $Q_j = \text{poly}(\lambda)$ into Lemma 4 (p. 22) yields

$$\Pr[\text{Bad}_{\text{pre}}] \leq \sum_{j=1}^m Q_j(\lambda) \cdot 2^{-\Theta(\lambda)} = \text{negl}(\lambda),$$

where $m = m(\lambda)$ is the total number of simulator programming attempts $\text{SimProgram}(\text{ctx}, \cdot, \cdot)$ in the given proof context $\text{ctx} \in \text{Ctx}_{\text{p}}$ over the entire UC execution (i.e., summed over all concurrently interleaved sessions). Since the simulator and all non-simulator ITMs are PPT, we have $m(\lambda) = \text{poly}(\lambda)$ and $Q_j(\lambda) = \text{poly}(\lambda)$ even under $\text{poly}(\lambda)$ concurrent sessions; hence a union bound over all programmed points (across all sessions) remains negligible.

In the UC simulations for USV and SDKG, the simulator never invokes $\text{SimProgram}(\text{ctx}_{\text{DLEQ}}, \cdot, \cdot)$. Intuitively, USV certificates are generated honestly (with witnesses) and are only verified/opened by the simulator in those UC hybrids; no UC hybrid ever requires simulating a new accepting USV/DLEQ proof without a witness.

UC-NIZKs in the adaptive-corruption setting have been studied since Groth et al. [37, 38]. For practical adaptive UC-NIZK-PoK constructions in (G)RO via straight-line compilation of Σ -protocols, see [49]. Our use requires the stronger AoK interface with straight-line extraction in gRO-CRP defined as:

Definition 11 (NIZK proofs/arguments and AoKs). Let $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ be an NP relation and let $\mathcal{L}_{\mathcal{R}} := \{x \in \mathcal{X} : \exists w \in \mathcal{W} \text{ s.t. } (x, w) \in \mathcal{R}\}$. A Non-Interactive Zero-Knowledge (NIZK) argument system for $\mathcal{R}$ is a triple of PPT algorithms $(\mathcal{K}, \mathcal{P}, \mathcal{V})$:

$$\text{Setup: } \mathcal{K}(1^\lambda) \rightarrow \text{pp}, \quad \text{Prove: } \mathcal{P}(\text{pp}, x, w) \rightarrow \pi \text{ for } (x, w) \in \mathcal{R}, \quad \text{Verify: } \mathcal{V}(\text{pp}, x, \pi) \rightarrow \{0, 1\},$$

satisfying:

- Completeness: for all $(x, w) \in \mathcal{R}$, $\Pr[\mathcal{V}(\text{pp}, x, \pi) = 1 \mid \text{pp} \leftarrow \mathcal{K}(1^\lambda), \pi \leftarrow \mathcal{P}(\text{pp}, x, w)] \geq 1 - \text{negl}(\lambda)$.
- (Computational) soundness: for all PPT $\mathcal{P}^*$ and all $x \notin \mathcal{L}_{\mathcal{R}}$,

$$\Pr[\mathcal{V}(\text{pp}, x, \pi) = 1 \mid \text{pp} \leftarrow \mathcal{K}(1^\lambda), \pi \leftarrow \mathcal{P}^*(\text{pp}, x)] \leq \text{negl}(\lambda).$$

- Zero-knowledge with universal simulation interface (in the gRO-CRP model): There exists a PPT simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ and (possibly empty) simulator-only state τ such that:
 - (i) $\{\text{pp} \leftarrow \mathcal{K}(1^\lambda)\} \approx_c \{\text{pp} : (\text{pp}, \tau) \leftarrow \text{Sim}_1(1^\lambda)\}$.
 - (ii) (Indistinguishability on true statements) For all $(x, w) \in \mathcal{R}$,

$$\{(\text{pp}, \pi) : \text{pp} \leftarrow \mathcal{K}(1^\lambda), \pi \leftarrow \mathcal{P}(\text{pp}, x, w)\} \approx_c \{(\text{pp}, \pi) : (\text{pp}, \tau) \leftarrow \text{Sim}_1(1^\lambda), \pi \leftarrow \text{Sim}_2(\text{pp}, \tau, x)\}.$$

- (iii) (Universal simulation interface) For every statement $x \in \mathcal{X}$, letting $\pi \leftarrow \text{Sim}_2(\text{pp}, \tau, x)$ we have

$$\Pr[\pi \neq \perp \wedge \mathcal{V}(\text{pp}, x, \pi) = 1] \geq 1 - \text{negl}(\lambda).$$

The simulator Sim_2 may invoke the simulator-only interface SimProgram in the proof's gRO-CRP context(s), which must lie in Ctx_{p} . If a required call to SimProgram returns $\perp$, then Sim_2 outputs $\perp^{**}$.

Additionally, $(\mathcal{K}, \mathcal{P}, \mathcal{V})$ is a NIZK-AoK for $\mathcal{R}$ if for every PPT prover $\mathcal{P}^*$ (with oracle access to H) there exists a PPT straight-line extractor $\text{Ext}_{\mathcal{P}^*}$ such that, in the experiment

$$\text{pp} \leftarrow \mathcal{K}(1^\lambda); \quad (x, \pi) \leftarrow \mathcal{P}^{*H}(\text{pp}); \quad w \leftarrow \text{Ext}_{\mathcal{P}^*}(\text{pp}, x, \pi; \text{Log}_{\mathcal{P}^*}),$$

we have

$$\Pr[\mathcal{V}(\text{pp}, x, \pi) = 1 \wedge (x, w) \notin \mathcal{R}] \leq \text{negl}(\lambda).$$

Definition 12 (Simulation soundness). Let $\Pi = (\mathcal{K}, \mathcal{P}, \mathcal{V})$ be a NIZK for relation $\mathcal{R}$ with simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$. We say Π is *simulation-sound* (for $\mathcal{R}$) if for every PPT adversary $\mathcal{A}$, the following experiment has negligible success probability: sample $(\text{pp}, \tau) \leftarrow \text{Sim}_1(1^\lambda)$, give pp to $\mathcal{A}$, and grant $\mathcal{A}$ oracle access to $\text{Sim}_2(\text{pp}, \tau, \cdot)$ producing simulated proofs for adaptively chosen statements. Let Q be the set of statements queried by $\mathcal{A}$ to this oracle. $\mathcal{A}$ outputs (x^*, π^*) and wins only if (i) $\mathcal{V}(\text{pp}, x^*, \pi^*) = 1$, (ii) $x^* \notin Q$, and (iii) $(x^*, w) \notin \mathcal{R}$ for all w .

Definition 13 (Simulation-extractability / simulation-sound AoK). Let $\Pi = (\mathcal{K}, \mathcal{P}, \mathcal{V})$ be a NIZK for relation $\mathcal{R}$ in the gRO-CRP model with simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$. We say Π is *simulation-extractable* (a.k.a. *simulation-sound AoK*) if there exists a PPT extractor Ext such that for every PPT adversary $\mathcal{A}$, the following holds with all but negligible probability: $(\text{pp}, \tau) \leftarrow \text{Sim}_1(1^\lambda)$. Run $\mathcal{A}$ on input pp with oracle access to (i) the proof-simulation oracle

^{**} Throughout this paper we use this universal notion. Thus, the real setup is transparent and no protocol party ever learns τ or can program H .

$\text{Sim}_2(\text{pp}, \tau, \cdot)$ and (ii) the global oracle interface $\text{Query}(\cdot, \cdot)$, while recording the oracle query/answer transcript $\text{Log}_{\mathcal{A}}$ (restricted to the gRO-CRP context(s) used by Π , cf. Remark 10 (p. 21)). Let Q be the set of statements queried by $\mathcal{A}$ to $\text{Sim}_2(\text{pp}, \tau, \cdot)$. When $\mathcal{A}$ outputs (x^*, π^*) with $\mathcal{V}(\text{pp}, x^*, \pi^*) = 1$ and $x^* \notin Q$, the extractor outputs

$$w^* \leftarrow \text{Ext}_{\mathcal{A}}(\text{pp}, \tau, x^*, \pi^*; \text{Log}_{\mathcal{A}})$$

such that $(x^*, w^*) \in \mathcal{R}$.

Definition 14 (Public-coin protocol). [2] A proof system $\Pi = (\mathcal{P}, \mathcal{V})$ is public-coin if each verifier message consists only of freshly sampled public randomness. Concretely, in round i , $\mathcal{V}$ samples $c_i \leftarrow_{\$} C_i$ uniformly from some finite set C_i and sends c_i . If $C_i = \{0, 1\}^t$ for all verifier rounds, we say that Π has t -bit challenges.

Next, we recall the standard folklore definition of Σ -protocol.

Definition 15 (Σ -protocol with t -bit challenge). Let $\mathcal{R}$ be an NP relation. A Σ -protocol for $\mathcal{R}$ with t -bit challenges is a three-move protocol $\Sigma = (\mathcal{P}_{\Sigma}, \mathcal{V}_{\Sigma})$: (i) $\mathcal{P}_{\Sigma}$ sends a ; (ii) $\mathcal{V}_{\Sigma}$ samples $e \leftarrow_{\$} \{0, 1\}^t$ and sends e ; (iii) $\mathcal{P}_{\Sigma}$ responds with z . The protocol satisfies the following properties:

- Completeness: for all $(x, w) \in \mathcal{R}$, an honest interaction accepts with probability 1.
- Special soundness: there exists a PPT extractor Ext such that from any two accepting transcripts (a, e, z) and (a, e', z') with $e \neq e'$ (one and the same first message a), Ext outputs w with $(x, w) \in \mathcal{R}$.
- (Computational) Special Honest-Verifier Zero-Knowledge: there exists a PPT simulator Sim such that for all $(x, w) \in \mathcal{R}$ and all $e \in \{0, 1\}^t$, the simulated transcript $\text{Sim}(x, e)$ is computationally indistinguishable from the verifier's view in an honest execution with challenge fixed to e .

We assume $2^{t(\lambda)} < p(\lambda)$ for every security parameter λ . Hence each t -bit challenge $e \in \{0, 1\}^t$ is interpreted as the corresponding integer $\bar{e} \in \{0, \dots, 2^t - 1\} \subset \mathbb{Z}_p$ via the natural injection. All prover responses and verifier checks treat $\bar{e}$ as an element of $\mathbb{Z}_p$.

Throughout, the public parameters pp and the gRO-CRP oracle H (together with the relevant context string) are fixed once per UC execution and are treated as implicit inputs to all algorithms. When this improves readability, we omit pp from signatures and write $\mathcal{P}(x, w)$ for $\mathcal{P}(\text{pp}, x, w)$ and $\mathcal{V}(x, \pi)$ for $\mathcal{V}(\text{pp}, x, \pi)$. Likewise, for a Σ -protocol we write $\mathcal{V}_{\Sigma}(x; a, e, z) = 1$ to denote acceptance of transcript (a, e, z) for statement x (with pp implicit in x).

The Fischlin transform [31] is a method to convert interactive public-coin proof systems into non-interactive ones. Unlike the Fiat–Shamir transform [30], which directly derives challenges from a random oracle, the Fischlin transform enforces an output structure criterion on the prover's transcript. Specifically, the prover must generate outputs that satisfy a rare structural condition (e.g., several trailing zeros), which is deliberately chosen to be rare, requiring the prover to perform multiple attempts to find a valid output. By observing the prover's queries to the random oracle, an extractor can identify at least two transcripts with the same initial commitment but different challenges. Then, *special soundness* property of the underlying Σ -protocol allows straight-line extraction of the witness. In gRO-CRP, “observing oracle queries” means emulating the adversarial prover ITM and recording its local Query calls under the proof context (cf. Remark 10 (p. 21)). Because witness-bearing computation may be delegated to a state-continuous KeyBox, we require proof systems with straight-line extractors (no rewinding). Hence, we employ Fischlin-based UC-NIZKs. See [49] for an explicit formalization of adaptive straight-line compilation of Σ -protocols and a proof that the randomized Fischlin transform satisfies it.

Definition 16 (Fischlin transform in gRO-CRP). Let $\Sigma = (\mathcal{P}_{\Sigma}, \mathcal{V}_{\Sigma})$ be a three-move protocol for an NP relation $\mathcal{R}$ with $t = O(\log \lambda)$ -bit challenges. For security parameter λ , fix parameter functions $b = b(\lambda)$, $r = r(\lambda)$, and $S = S(\lambda)$ satisfying $b, r = O(\log \lambda)$, $b \leq t$, $br = \omega(\log \lambda)$, $2^{t-b} = \omega(\log \lambda)$, and $S = \Theta(r)$. Let $H : \text{Ctx} \times \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda}$ be the global oracle provided by $\mathcal{G}_{\text{gRO-CRP}}$ in the gRO-CRP model. Assume $b(\lambda) \leq \lambda$. For $u \in \{0, 1\}^*$, define $H_b(\text{ctx}, u) := \text{lsb}_b(H(\text{ctx}, u)) \in \{0, \dots, 2^b - 1\}$. When the Fischlin transform $F[\Sigma]$ is used as a (UC-)NIZK (including when run inside an honest KeyBox), instantiate it with a proof context $\text{ctx} \in \text{Ctx}_p$. Let $\langle \cdot \rangle$ be any fixed injective encoding of tuples into $\{0, 1\}^*$. $F[\Sigma]$ produces a non-interactive proof system $(\mathcal{P}', \mathcal{V}')$ as:

Prover $\mathcal{P}'(x, w)$: Run r independent first moves of Σ on (x, w) to obtain commitments $\mathbf{a} := (a_1, \dots, a_r)$. For each $i \in [r]$ do:

1. Initialize $\widehat{s}_i \leftarrow 2^b - 1$ and $(\widehat{e}_i, \widehat{z}_i) \leftarrow (\perp, \perp)$.
2. For $e = 0, 1, \dots, 2^t - 1$ do:
 - Compute the Σ -response $z_{i,e}$ for challenge e (using w), and set $s_{i,e} := H_b(\text{ctx}, \langle x, \mathbf{a}, i, e, z_{i,e} \rangle)$.
 - If $s_{i,e} = 0$, set $(e_i, z_i) \leftarrow (e, z_{i,e})$ and break.
 - Else, if $s_{i,e} \leq \widehat{s}_i$, set $\widehat{s}_i \leftarrow s_{i,e}$ and $(\widehat{e}_i, \widehat{z}_i) \leftarrow (e, z_{i,e})$.
3. If the loop ended without $s_{i,e} = 0$, set $(e_i, z_i) \leftarrow (\widehat{e}_i, \widehat{z}_i)$.

Let $s_i := H_b(\text{ctx}, \langle x, \mathbf{a}, i, e_i, z_i \rangle)$. Output proof as $\pi := ((a_i, e_i, z_i))_{i=1}^r$.

Verifier $\mathcal{V}'(x, \pi)$: Parse $\pi = ((a_i, e_i, z_i))_{i=1}^r$, set $\mathbf{a} = (a_1, \dots, a_r)$, and accept iff:

$$(i) \ \forall i \in [r], \mathcal{V}_\Sigma(x; a_i, e_i, z_i) = 1 \text{ and } (ii) \ \sum_{i=1}^r H_b(\text{ctx}, \langle x, \mathbf{a}, i, e_i, z_i \rangle) \leq S.$$

Remark 13 (Asymptotic vs. concrete Fischlin parameters). Definition 16 (p. 25) fixes parameter functions $(t(\lambda), b(\lambda), r(\lambda), S(\lambda))$ and imposes asymptotic growth conditions (e.g., $2^{t-b} = \omega(\log \lambda)$ and $br = \omega(\log \lambda)$) that are used only to derive negligible completeness and soundness/extraction error as $\lambda \rightarrow \infty$ (Lemma 6 (p. 27)). When we later quote a concrete tuple (t, b, r, S) , this is shorthand for an instantiation at a fixed target security level $\lambda = \lambda_0$, i.e., $(t, b, r, S) = (t(\lambda_0), b(\lambda_0), r(\lambda_0), S(\lambda_0))$. In that concrete setting, we evaluate the corresponding explicit bounds from Fischlin’s analysis, rather than claiming that a constant tuple satisfies the asymptotic growth conditions for all λ .

Throughout this paper, every UC-NIZK(-AoK) we use is instantiated via the optimized Fischlin transform [20] in the gRO-CRP model (Definition 10 (p. 21)).

Early-break rarity search [20]. For each i , the prover evaluates $H_b(\text{ctx}, \langle x, \mathbf{a}, i, e, z_{i,e} \rangle)$ on successive challenges and stops at the first e with $s_{i,e} = 0$. Since H_b is uniform over $\{0, \dots, 2^b - 1\}$, the expected number of trials per i is 2^b . For a cap of 2^t , the per-repetition “no hit” probability is

$$p_{\text{miss}} := \Pr[\forall e \in \{0, 1\}^t : s_{i,e} \neq 0] = (1 - 2^{-b})^{2^t} \approx e^{-2^{t-b}}.$$

Under Definition 16 (p. 25), we require $2^{t-b} = \omega(\log \lambda)$; hence $p_{\text{miss}} = \text{negl}(\lambda)$. With fixed concrete parameters, p_{miss} is an explicit completeness probability. The honest-rejection probability of an r -fold proof is at most $r \cdot p_{\text{miss}}$ by a union bound. This event is an operational / liveness failure: an honest prover may abort and retry. We present the general slack- S formulation in Definition 16 (p. 25). By “optimized Fischlin”, we mean the optimized variant from [20], that fixes parameters and streamlines the prover/verification logic for efficiency.

We say that a Σ -protocol $\Sigma = (\mathcal{P}_\Sigma, \mathcal{V}_\Sigma)$ has *unique responses* if for every statement x , first message a , and challenge e , there exists at most one response z such that $\mathcal{V}_\Sigma(x; a, e, z) = 1$. If Σ has special soundness and unique responses, then in the random-oracle/gRO-CRP model $F[\Sigma]$ is a NIZK-AoK with a straight-line (online) extractor [31, Thm. 2]. For any PPT adversary making at most Q distinct queries to H under a context, the soundness / extraction-error is bounded by

$$\Pr[\mathcal{V}'(x, \pi) = 1 \text{ for } x \notin \mathcal{L}] \leq (Q + 1) \cdot \frac{N_{r,S}}{2^{br}} + \text{negl}(\lambda),$$

where

$$N_{r,S} := \sum_{T=0}^S \binom{T+r-1}{r-1} = \binom{S+r}{r}$$

is the number of r -tuples $(s_1, \dots, s_r) \in \{0, \dots, 2^b - 1\}^r$ with $\sum_i s_i \leq S$. In particular, for any $Q = \text{poly}(\lambda)$, the soundness/extraction error is negligible whenever

$$br - \log N_{r,S} = \omega(\log \lambda),$$

e.g., under $S = \Theta(r)$ this is implied by $br = \omega(\log \lambda)$ since $\log N_{r,S} = O(r)$.

Lemma 6 (Negligible Fischlin error for admissible parameters). *Let $(t(\lambda), b(\lambda), r(\lambda), S(\lambda))$ satisfy Definition 16 (p. 25). Then for every $Q = \text{poly}(\lambda)$, the soundness/knowledge-extraction error of $F[\Sigma]$ against any PPT prover that makes at most Q distinct queries under the transform context is negligible in λ . Moreover, the honest-rejection probability is $\text{negl}(\lambda)$.*

Proof Sketch. We know that the soundness error is at most $(Q + 1) \cdot N_{r,S}/2^{br} + \text{negl}(\lambda)$. Under Definition 16 (p. 25), $br - \log N_{r,S} = \omega(\log \lambda)$; hence the term is negligible for any $Q = \text{poly}(\lambda)$. The honest-rejection bound $r \cdot (1 - 2^{-b})^{2^t}$ is negligible since $2^{t-b} = \omega(\log \lambda)$. ■

For completeness, note that in an honest execution each repetition i yields $s_i = 0$ whenever the prover's rarity search finds some challenge e with $H_b(\cdot) = 0$. In that case the verifier's sum test holds with $\sum_{i=1}^r s_i = 0 \leq S$. Thus, an explicit (union-bound) upper bound on honest-rejection is

$$\Pr[\mathcal{V}' \text{ rejects an honest proof}] \leq r \cdot (1 - 2^{-b})^{2^t} + \text{negl}(\lambda) \approx r \cdot \exp(-2^{t-b}) + \text{negl}(\lambda).$$

Soundness/knowledge-extraction/security error of the Fischlin transform, for a prover making at most Q distinct oracle queries, under the proof context is bounded by

$$\varepsilon_{\text{sec}}(Q) := (Q + 1) \cdot \frac{N_{r,S}}{2^{br}} + \text{negl}(\lambda).$$

Consequently, choosing parameter functions $t(\lambda), b(\lambda), r(\lambda), S(\lambda)$ as in Definition 16 (p. 25) yields negligible soundness and completeness error in λ . When quoting concrete numbers, we always label which bound is liveness and which is security.

The next proposition is an immediate corollary of Fischlin soundness: in a strict GRO model the UC simulator has no programming capability and is therefore just another PPT prover for $F[\Sigma]$. We nevertheless state it explicitly because our UC-NIZK(-AoK) definition requires a universal simulation interface (Definition 11 (p. 24)) that must succeed even on off-language statements, and strict GRO rules this out. This motivates the move to gRO-CRP (Definition 10 (p. 21)) and clarifies the model separation relative to works proved in strict GRO (e.g., [15]).

Proposition 2 (Strict GRO is insufficient for Fischlin universal simulation). *Fix an NP relation $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ such that $\mathcal{X} \setminus \mathcal{L}_{\mathcal{R}} \neq \emptyset$, where $\mathcal{L}_{\mathcal{R}} := \{x \in \mathcal{X} : \exists w (x, w) \in \mathcal{R}\}$. Let $\Sigma = (\mathcal{P}_{\Sigma}, \mathcal{V}_{\Sigma})$ be a Σ -protocol for $\mathcal{R}$ with special soundness and unique responses. Let $F[\Sigma] = (\mathcal{K}, \mathcal{P}', \mathcal{V}')$ denote the (optimized) Fischlin transform (Definition 16 (p. 25)) instantiated with parameter functions $(t(\lambda), b(\lambda), r(\lambda), S(\lambda))$ satisfying Definition 16 (p. 25). Consider the UC model augmented with a strict global random oracle (GRO) that provides only $\text{Query}(\cdot, \cdot)$ and no simulator-only programming interface. Assume the public parameters $\text{pp} \leftarrow \mathcal{K}(1^\lambda)$ are generated by the honest/transparent setup of $F[\Sigma]$. Then $F[\Sigma]$ cannot satisfy the universal simulation interface of Definition 11 (p. 24) by any PPT simulator. Concretely, for every PPT simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ that has access only to Query , for every fixed statement $x^* \in \mathcal{X} \setminus \mathcal{L}_{\mathcal{R}}$, if $(\text{pp}, \tau) \leftarrow \text{Sim}_1(1^\lambda)$ and $\pi^* \leftarrow \text{Sim}_2(\text{pp}, \tau, x^*)$, then*

$$\Pr[\mathcal{V}'(\text{pp}, x^*, \pi^*) = 1] \leq \text{negl}(\lambda),$$

where the probability is over the coins of Sim and the strict GRO.

Proof Sketch. [Immediate from soundness (model-separation point)] Fix any $x^* \in \mathcal{X} \setminus \mathcal{L}_{\mathcal{R}}$. In the strict GRO model the simulator has access only to $\text{Query}(\cdot, \cdot)$ and has no programming interface. Consequently, $\text{Sim}_2(\text{pp}, \tau, \cdot)$ is simply a PPT prover for the non-interactive proof system $F[\Sigma]$, with oracle access to Query in the proof context used by $F[\Sigma]$.

Let $Q(\lambda)$ upper bound the number of distinct oracle queries that Sim_2 makes under that proof context; since Sim_2 is PPT, $Q(\lambda) = \text{poly}(\lambda)$. By Fischlin soundness for admissible parameters (Lemma 6 (p. 27)), any PPT prover making at most $Q(\lambda)$ distinct oracle queries outputs an accepting proof for an off-language statement with probability at most $\text{negl}(\lambda)$. Applying this to Sim_2 yields

$$\Pr[\mathcal{V}'(\text{pp}, x^*, \pi^*) = 1] \leq \text{negl}(\lambda), \quad \text{where } \pi^* \leftarrow \text{Sim}_2(\text{pp}, \tau, x^*).$$

However, the universal simulation interface in Definition 11 (p. 24) requires that for every statement $x \in \mathcal{X}$ (specifically, x^*), the simulator outputs an accepting proof with probability $1 - \text{negl}(\lambda)$. This contradiction shows that $F[\Sigma]$ cannot satisfy the universal simulation interface in strict GRO. ■

For DLEQ proofs we restrict the statement space to

$$\mathcal{X}_{\text{DLEQ}} := (\mathbb{G} \setminus \{0_{\mathbb{G}}\}) \times (\mathbb{G} \setminus \{0_{\mathbb{G}}\}).$$

Henceforth, we refer to Π_{DL} and Π_{DLEQ} for the Fischlin transforms of the Schnorr and Chaum–Pedersen Σ -protocols [59, 19] (for $\mathcal{R}_{\text{DL}}$ and $\mathcal{R}_{\text{DLEQ}}$ resp.), instantiated in disjoint gRO-CRP contexts; concrete details appear in Section 6 (p. 38).

Lemma 7 (Fischlin-based UC-NIZK-AoKs for DL and DLEQ in gRO-CRP). *Assume DL hardness in the prime-order group fixed by the security parameter λ . Fix Fischlin parameters $(t(\lambda), b(\lambda), r(\lambda), S(\lambda))$ satisfying Definition 16 (p. 25). Let Π_{DL} denote the (optimized) Fischlin instantiation described in Section 6 (p. 38) for relation $\mathcal{R}_{\text{DL}}$, instantiated in a gRO-CRP context in Ctx_{p} . If, in addition, DDLEQ hardness holds, let Π_{DLEQ} denote the corresponding (optimized) Fischlin instantiation for relation $\mathcal{R}_{\text{DLEQ}}$, instantiated in a disjoint gRO-CRP context in Ctx_{p} . Then the following hold for Π_{DL} ; and, under the additional DDLEQ assumption, the same items also hold for Π_{DLEQ} .*

- (i) *Zero-knowledge (universal simulation interface): The proof system satisfies computational zero-knowledge with the universal simulation interface of Definition 11 (p. 24) in its corresponding context in Ctx_{p} .*
- (ii) *NIZK-AoK with straight-line extraction: The proof system is a NIZK-AoK in the sense of Definition 11 (p. 24), with an online/straight-line extractor that may inspect the adversary’s gRO-CRP query/answer log under the corresponding proof context. For any $Q = \text{poly}(\lambda)$ distinct gRO-CRP queries under that proof context, the resulting knowledge/soundness error is negligible (Lemma 6 (p. 27)).*
- (iii) *Simulation-extractability: The proof system is simulation-extractable for fresh statements in the sense of Definition 13 (p. 24).*
- (iv) *Composability under domain separation in gRO-CRP: The guarantees in (i)–(iii) continue to hold when the proof system is invoked as a subroutine inside an arbitrarily interleaved UC execution in the gRO-CRP-hybrid model, provided that every use of the global oracle that serves a distinct protocol-level purpose is domain-separated by disjoint gRO-CRP contexts and injective encodings.*

Proof Sketch. Items (i) and (ii) follow by instantiating Fischlin’s analysis [31, Thm. 2] for Σ -protocols with special soundness and unique responses (here, Schnorr for $\mathcal{R}_{\text{DL}}$ and Chaum–Pedersen for $\mathcal{R}_{\text{DLEQ}}$), with negligible error by Lemma 6 (p. 27). The optimized prover/verifier organization of [20] affects efficiency but not the underlying transform security argument. Item (iii) follows from Fischlin’s simulation-soundness / simulation-extractability theorem [31, Thm. 3]. In our gRO-CRP formulation, the simulator’s universal simulation interface is realized via `SimProgram` in the corresponding contexts in Ctx_{p} . Since `SimProgram` fails on already-defined points, the simulator’s only failure mode is a pre-query collision on a programmed input; by Lemma 4 (p. 22), this occurs with negligible probability for the Fischlin-based proofs whose programmed inputs include fresh high-entropy material.

For item (iv), fix any UC execution in the gRO-CRP-hybrid model with arbitrarily many concurrently interleaved sessions and sub-protocols. By construction of $\mathcal{G}_{\text{gRO-CRP}}$, the oracle table is indexed by pairs (ctx, x) , so oracle activity (including simulator programming) in any context $\text{ctx}' \neq \text{ctx}_{\Pi}$ cannot affect the distribution of replies in ctx_{Π} (cf. Lemma 5 (p. 22) and Remark 11 (p. 23)). Injective encodings together with disjoint contexts ensure that logically distinct uses do not collide on the same (ctx, x) , so each invocation of Π is subject to the same per-context oracle behavior as in the standalone gRO-CRP analysis.

Under UC concurrency, the total number of oracle queries and simulator programming attempts in any fixed programmable context remains $\text{poly}(\lambda)$. The simulator’s only failure mode for the universal simulation interface is a pre-query collision on a point it intends to program; Lemma 4 (p. 22), together with a union bound over all simulator programming attempts in that context across the entire UC execution, bounds this event by $\text{negl}(\lambda)$. Conditioned on the complement, the proofs’ ZK, AoK, and simulation-extractability guarantees in (i)–(iii) apply unchanged inside the UC execution. ■

5 Enforcing Public Structure without Export: USV Certificates

Unique Structure Verification (USV) is a non-interactive, publicly verifiable certificate that lets anyone derive a unique public opening for a commitment to a hidden scalar without exporting that scalar^{***}. Extraction is public and straight-line: it is a deterministic function of the certificate and uses no trapdoor or rewinding.

Assumption 6 (Transparent generator derivation) Let $\mathcal{L}$ be a public randomness source (e.g., [42]) that is sampled outside the protocol and is not adversary-influenceable: conditioned on the adversary's view prior to publication, $\mathcal{L}$ has min-entropy at least λ . Fix a prime-order group $\mathbb{G}$ of size p with canonical generator $\mathcal{G}$. For a deterministic, publicly specified hash-to-group map, H2G [28], whose output distribution (over random $\mathcal{L}$) is computationally indistinguishable from uniform over $\mathbb{G} \setminus \{0_{\mathbb{G}}\}$, define $\mathcal{H} := \mathcal{H}_{c^*}$ where $\mathcal{H}_c := \text{H2G}(\text{USV.H} \parallel \mathcal{L} \parallel \text{enc}(c))$, $\text{enc}(c)$ is the 4-byte big-endian encoding of c , and c^* is the smallest $c \geq 0$ such that $\mathcal{H}_c \notin \{0_{\mathbb{G}}, \mathcal{G}\}$. Set $\text{pp} := (\mathbb{G}, p, \mathcal{G}, \mathcal{H})$, and define the deterministic public setup procedure

$$\text{Setup}(1^\lambda, \mathcal{L}) \rightarrow \text{pp}$$

Definitions 17 (p. 29)–19 (p. 30) describe the stand-alone primitive; our composable statement is the UC realization of $\mathcal{F}_{\text{USV}}$ (Theorem 2 (p. 36)).

Definition 17 (USV certificate scheme). A USV certificate scheme consists of the deterministic public setup procedure

$$\text{Setup}(1^\lambda, \mathcal{L}) \rightarrow \text{pp}$$

specified in Assumption 6 (p. 29), together with the following algorithms

$$\text{Cert}(\text{pp}, m) \rightarrow (C, \zeta), \quad \mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) \in \{0, 1\}, \quad \text{Derive}(\text{pp}, C, \zeta) \rightarrow (\Upsilon \text{ or } \perp),$$

and a deterministic *public-opening projection*

$$\text{PubOpen}(\text{pp}, \cdot) : \mathcal{O} \rightarrow \mathbb{G} \cup \{\perp\},$$

where $\mathcal{O}$ is the opening space of Derive (and we adopt the convention $\text{PubOpen}(\text{pp}, \perp) = \perp$). We define a *verified opening* algorithm as

$$\text{Open}(\text{pp}, C, \zeta) := \begin{cases} \text{Derive}(\text{pp}, C, \zeta) & \text{if } \mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) = 1, \\ \perp & \text{otherwise.} \end{cases}$$

We also define the associated *public opening* as

$$\text{Open}_M(\text{pp}, C, \zeta) := \text{PubOpen}(\text{pp}, \text{Open}(\text{pp}, C, \zeta)).$$

There exists a polynomial-time decidable relation $\mathcal{R}_{\text{pp}} \subseteq \mathcal{C} \times \mathcal{O}$ such that the following hold:

1. Completeness: For every $m \neq 0$,

$$\Pr \left[\mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) = 1 \wedge \Upsilon := \text{Open}(\text{pp}, C, \zeta) \neq \perp \wedge (C, \Upsilon) \in \mathcal{R}_{\text{pp}} \wedge \text{Open}_M(\text{pp}, C, \zeta) \neq \perp : (C, \zeta) \leftarrow \text{Cert}(\text{pp}, m) \right] \geq 1 - \text{negl}(\lambda).$$

2. Deterministic verified opening (uniqueness): For any fixed (C, ζ) , $\text{Derive}(\text{pp}, C, \zeta)$ is deterministic and returns either $\perp$ or a single value $\Upsilon \in \mathcal{O}$. Moreover, $\text{Open}(\text{pp}, C, \zeta) = \perp$ iff $\mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) = 0$, and whenever $\text{Open}(\text{pp}, C, \zeta) = \Upsilon \neq \perp$, we have $(C, \Upsilon) \in \mathcal{R}_{\text{pp}}$. Further, $\text{PubOpen}(\text{pp}, \Upsilon)$ is deterministic; hence $\text{Open}_M(\text{pp}, C, \zeta)$ is deterministic and returns either $\perp$ or a single $M \in \mathbb{G}$.
3. Opening-conditional tag simulatability: There exists a PPT simulator Sim_{cert} such that:

^{***} In UC, we will model handle binding as verifier-scoped; see Section 5.3 (p. 35)

- (a) Correctness of simulated tags: For every $(C, \Upsilon) \in \mathcal{R}_{\text{pp}}$, if $\tilde{\zeta} \leftarrow \text{Sim}_{\text{cert}}(\text{pp}, C, \Upsilon)$ then, except with probability $\text{negl}(\lambda)$,

$$\mathcal{V}_{\text{cert}}(\text{pp}, C, \tilde{\zeta}) = 1 \quad \text{and} \quad \text{Derive}(\text{pp}, C, \tilde{\zeta}) = \Upsilon.$$

Equivalently, except with probability $\text{negl}(\lambda)$, $\text{Open}(\text{pp}, C, \tilde{\zeta}) = \Upsilon$, and therefore $\text{Open}_M(\text{pp}, C, \tilde{\zeta}) = \text{PubOpen}(\text{pp}, \Upsilon)$.

- (b) Indistinguishability conditioned on the opening: For every m ,

$$\begin{aligned} \left\{ (C, \Upsilon, \zeta) : (C, \zeta) \leftarrow \text{Cert}(\text{pp}, m); \Upsilon \leftarrow \text{Open}(\text{pp}, C, \zeta) \right\} \approx_c \\ \left\{ (C, \Upsilon, \tilde{\zeta}) : (C, \zeta) \leftarrow \text{Cert}(\text{pp}, m); \Upsilon \leftarrow \text{Open}(\text{pp}, C, \zeta); \tilde{\zeta} \leftarrow \text{Sim}_{\text{cert}}(\text{pp}, C, \Upsilon) \right\}. \end{aligned}$$

For the UC realization of $\mathcal{F}_{\text{USV}}$ and for SDKG, we rely on completeness and deterministic verified openings (Items 1–2 of Definition 17 (p. 29)) together with equivocation resistance (Definition 19 (p. 30)). The stronger opening-conditional tag-simulatability interface (Item 3) is not required by any UC hybrid in this paper; we include it because it is useful in variants where the ideal world fixes an opening first and the simulator must later backfill a compatible accepting tag, and because it holds for our instantiation (Lemma 8 (p. 32)).

Definition 18 (Equivocation experiment). Work in the gRO-CRP-hybrid model of Definition 10 (p. 21) with global oracle $\mathcal{G}_{\text{gRO-CRP}}$ (Fig. 4 (p. 21)). The experiment and the adversary share access to the same oracle interface $\text{Query}(\cdot, \cdot)$. Fix public parameters $\text{pp} = (\mathbb{G}, p, \mathcal{G}, \mathcal{H})$ and the USV algorithms $(\text{Cert}, \text{Derive}, \mathcal{V}_{\text{cert}}, \text{Open}, \text{PubOpen})$ from Definition 17 (p. 29). Experiment $\text{Exp}_{\mathcal{A}}^{\text{eqv}}(1^\lambda)$ is defined as:

1. Sample $\mathcal{L}$ according to the public randomness source/beacon distribution (Assumption 6 (p. 29)), and set $\text{pp} \leftarrow \text{Setup}(1^\lambda, \mathcal{L})$.
2. Run $\mathcal{A}^{\text{Query}}(1^\lambda, \text{pp})$, i.e., $\mathcal{A}$ on input $(1^\lambda, \text{pp})$ with oracle access to $\text{Query}(\cdot, \cdot)$, and obtain (C, ζ, ζ') with $\zeta \neq \zeta'$.
3. Let $b \leftarrow 1$ iff $\mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) = 1 \wedge \mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta') = 1 \wedge \text{Open}_M(\text{pp}, C, \zeta) \neq \text{Open}_M(\text{pp}, C, \zeta')$, where $\mathcal{V}_{\text{cert}}$ (and the embedded verifier $\mathcal{V}_{\text{DLEQ}}$) evaluates any needed oracle calls as $\text{Query}(\text{ctx}_{\text{DLEQ}}, \cdot)$ in the dedicated USV/DLEQ proof context $\text{ctx}_{\text{DLEQ}} \in \text{Ctx}_p$ (disjoint from ctx_{UC} and $\text{ctx}_{\text{KeyBox}}$). Otherwise set $b \leftarrow 0$. Output b .

Define $\text{Adv}_{\mathcal{A}}^{\text{eqv}}(\lambda) := \Pr[\text{Exp}_{\mathcal{A}}^{\text{eqv}}(1^\lambda) = 1]$.

Definition 19 (Equivocation resistance). A USV certificate scheme is *equivocation resistant* (in the gRO-CRP-hybrid model) if for every PPT adversary $\mathcal{A}$ with oracle access to $\text{Query}(\cdot, \cdot)$, $\text{Adv}_{\mathcal{A}}^{\text{eqv}}(\lambda) \leq \text{negl}(\lambda)$.

In the NXK/KeyBox setting targeted by this paper, USV certificate generation is mediated by the KeyBox API. Concretely, we include a key-independent admissible operation $\text{USV.Cert} \in \mathcal{F}_{\text{adm}}$ that samples an internal witness scalar $m \leftarrow \mathbb{Z}_p^*$, computes $\langle C, \zeta \rangle \leftarrow \text{Cert}(\text{pp}, m)$, erases m , and returns only $\langle C, \zeta \rangle$ to the host. Formally, a party obtains a certificate by invoking

$$\langle C, \zeta \rangle \leftarrow \mathcal{F}_{\text{KeyBox}}^{(P)}.\text{Use}(\mu, \text{USV.Cert}, \langle \text{lbl} \rangle),$$

where the slot argument μ is ignored (as for the key-independent OpenFromPeer interface in Fig. 3 (p. 11)).

Relation to standard notions. USV can be viewed as a small “commit-and-certify” primitive whose interface differs from similar abstractions as:

- A standard commitment is opened by revealing the full witness (message and randomness). USV never reveals the scalar; instead it yields a deterministic public opening to the induced group element that is sufficient for the transcript-defined affine consistency checks used under NXK.
- Extractable commitments typically provide a privileged extractor that outputs the committed message (using trapdoors and/or rewinding). USV instead makes extraction public and deterministic, but extracts only the canonical group element (not the scalar), aligning with NXK.
- Equivocable commitments enable opening a fixed commitment to different messages. In contrast, USV is *opening-unique* and explicitly requires equivocation resistance. The simulator’s power is orthogonal: it can simulate tags conditioned on a chosen opening, which is exactly what we need in the UC hybrids.

- PVSS-style objects certify well-formed encrypted shares or allow recovery by a set of parties. USV provides neither ciphertexts nor recoverable secrets; it certifies only public structure needed to replace exported-share enforcement under NXK.
- The tag ζ can be interpreted as a compact NIZK-style certificate of well-formedness, but with the special feature that it induces a canonical and deterministic public opening that the transcript can reference.
- USV supports publishing commitment-shaped material (C, ζ) that deterministically defines the canonical public point

$$M := \text{Open}_M(\text{pp}, C, \zeta) = m\mathcal{G}.$$

The scalar m remains non-exportable and is hidden computationally (under DL): since M is public and $m \mapsto m\mathcal{G}$ is a bijection in a prime-order group, m is not information-theoretically hidden once a valid certificate is published. In the SDKG base run (Section 7 (p. 40)), the leaf transmits (C, ζ) in Round 1, so M is transcript-defined from the outset.

5.1 Why USV is needed under hardened NXK profiles (overview)

Several later checks (in SDKG verification and in the transcript-driven idealization) require certain group elements to be deterministic functions of the public transcript in straight-line. The principal example is a leaf-defined point $M = m\mathcal{G}$ (and derived auxiliaries): verifiers and the UC simulator must be able to compute these points from the transcript alone.

In our NXK setting, long-term shares (and any other KeyBox-resident secrets) are API-non-exportable (Assumption 1 (p. 11)). Any share-deriving material is NXK-restricted: it must remain transcript-private and must not be written to persistent storage outside a KeyBox, though it may be handled transiently in host RAM during an atomic local step and must then be securely erased (*Reader Note 2.1* (p. 8); Remark 7 (p. 16)). State continuity furthermore rules out rewinding/forking-based extraction at the hardware boundary (Assumption 2 (p. 13)). Finally, KeyBox-local oracle calls are not visible to the UC simulator (local-call semantics).

Under hardened/minimal profiles, the scalar m underlying $M = m\mathcal{G}$ is generated inside a state-continuous KeyBox and is not exported. A plain hiding commitment $C = \text{Commit}(m; r)$ therefore does not determine M unless one can extract m (or otherwise obtain $m\mathcal{G}$) in straight-line. This leaves three design options:

1. Publish $M = m\mathcal{G}$ directly: This removes the need for USV, but assumes the profile allows computing/exporting $m\mathcal{G}$ for fresh ephemeral scalars.
2. Commit to m and extract m from an opening proof: If the opening proof is generated inside the KeyBox (to avoid exporting m), then straight-line extraction fails in our model: there is no rewinding/forking (state continuity) and no simulator access to the KeyBox’s oracle-log. If the opening proof is generated outside the KeyBox, then the leaf must materialize (m, r) (or an equivalent caller-invertible image sufficient to derive M) in non-KeyBox state, which contradicts the hardened/minimal-profile design point.
3. Publish commitment-shaped material with a publicly verifiable certificate that deterministically yields M : USV implements exactly this: from (C, ζ) anyone can compute the unique public opening $M = \text{Open}_M(\text{pp}, C, \zeta)$, while m remains non-exportable.

Least-privilege motivation (why Option 1 may be disallowed). This restriction can arise in deployments where the protocol principal is authorized to use a non-exportable asymmetric key (e.g., sign/derive), but is explicitly denied the separate capability to retrieve its public key.[†] This is an interface/profile assumption, not a claim that KeyBoxes in general cannot export public points.

For the formal necessity statement for the commit-only alternative, see Section 8.2 (p. 57) (Lemma 18 (p. 58)).

[†] Concretely, several cloud KMS products gate “get public key” behind distinct permissions; e.g., AWS KMS `kms:GetPublicKey` [1], Google Cloud KMS `cloudkms.cryptoKeyVersions.viewPublicKey` [36], and Azure Key Vault `get` reads the public part of a key [52].

5.2 An Instantiation

Let $\mathcal{R}_{\text{pp}} := \{(C, (M, R)) \in \mathbb{G}^3 : C = M + R, M \neq 0_{\mathbb{G}}, R \neq 0_{\mathbb{G}}\}$, and define the DLEQ relation

$$\mathcal{R}_{\text{DLEQ}} := \{((\text{pp}, A, B), r) : (A, B) \in \mathcal{X}_{\text{DLEQ}} \wedge r \in \mathbb{Z}_p^* \wedge A = r\mathcal{G} \wedge B = r\mathcal{H}\}.$$

Let Π_{DLEQ} be the UC-NIZK-AoK (via optimized Fischlin in the gRO-CRP model; see Section 6 (p. 38)) for $\mathcal{R}_{\text{DLEQ}}$. We instantiate Π_{DLEQ} in the dedicated context $\text{ctx}_{\text{DLEQ}} \in \text{Ctx}_p$, which is disjoint from ctx_{UC} and $\text{ctx}_{\text{KeyBox}}$.

Concrete algorithms for an instantiation follow:

- **Cert**(pp, m): on input $m \leftarrow \mathbb{Z}_p^*$ sample $r \leftarrow \mathbb{Z}_p^* \setminus \{-m\}$. Set $M := m\mathcal{G}$, $R := r\mathcal{H}$, $C := M + R$, $\nu := mr^{-1} \bmod p$, and $v := (m + r)\mathcal{G}$. Define $A := v - M$ and $B := C - M$. Compute $\pi_{\text{DLEQ}} \leftarrow \mathcal{P}_{\text{DLEQ}}(\text{pp}, (A, B), r)$ and output $\zeta := (\nu, v, \pi_{\text{DLEQ}})$ together with C . Erase m, r .
- **Derive**(pp, C, ζ): parse $\zeta = (\nu, v, \pi_{\text{DLEQ}})$. If $\nu \in \{-1, 0\} \bmod p$, output $\perp$. Else set $M := \frac{\nu}{\nu+1}v$ and $R := C - M$ and output $\Upsilon := (M, R)$.
- **PubOpen**(pp, Υ): parse $\Upsilon = (M, R)$ and output M . If parsing fails, output $\perp$.
- $\mathcal{V}_{\text{cert}}$ (pp, C, ζ): parse $\zeta = (\nu, v, \pi_{\text{DLEQ}})$. Compute $\Upsilon \leftarrow \text{Derive}(\text{pp}, C, \zeta)$; if $\Upsilon = \perp$ output 0. Else parse $\Upsilon = (M, R)$, set $A := v - M$ and $B := C - M$. Output 1 iff $\mathcal{V}_{\text{DLEQ}}(\text{pp}, (A, B), \pi_{\text{DLEQ}}) = 1$.
- **Open**(pp, C, ζ): $\text{Open}(\text{pp}, C, \zeta) := \text{Derive}(\text{pp}, C, \zeta)$ if $\mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) = 1$, and $\perp$ otherwise.

Properties of the USV certificate scheme

- **Correctness**: For honest generation, $v = (m + r)\mathcal{G}$ and $\nu = mr^{-1}$. Hence,

$$\frac{\nu}{\nu+1}v = \frac{mr^{-1}}{mr^{-1}+1}(m + r)\mathcal{G} = m\mathcal{G}, \text{ and } R = C - M.$$

Therefore, $\mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) = 1$ and $\text{Open}(\text{pp}, C, \zeta) = (M, R) \in \mathcal{R}_{\text{pp}}$.

- **Unique verified opening**: For any fixed (C, ζ) , $\text{Derive}(\text{pp}, C, \zeta)$ is deterministic. Hence, there is at most one candidate opening Υ it can output. Moreover, whenever $\text{Open}(\text{pp}, C, \zeta) \neq \perp$, we have $\text{Open}(\text{pp}, C, \zeta) = \text{Derive}(\text{pp}, C, \zeta) = (M, R)$ with $M = \frac{\nu}{\nu+1}v$ uniquely determined (for $\nu \neq -1$), which leads to uniquely determined $R = C - M$.
- **Straight-line verified public extraction**: Extraction of the public opening is $\text{Open}(\text{pp}, C, \zeta)$. It is deterministic, uses no trapdoor, and is non-rewinding.
- **Opening-conditional tag simulatability**: Given (C, Υ) with $\Upsilon = (M, R)$ and $(C, \Upsilon) \in \mathcal{R}_{\text{pp}}$: sample $\bar{\nu} \leftarrow \mathbb{Z}_p^* \setminus \{-1\}$ and set $\bar{v} := (1 + \bar{\nu}^{-1})M$. Define $\bar{A} := \bar{v} - M$ and $\bar{B} := C - M$. Compute $\bar{\pi}_{\text{DLEQ}} \leftarrow \text{Sim}_{\text{DLEQ}}(\text{pp}, (\bar{A}, \bar{B}))$ using the UC-NIZK simulator, and output $\tilde{\zeta} := (\bar{\nu}, \bar{v}, \bar{\pi}_{\text{DLEQ}})$. Note that for a random $\bar{\nu}$, the derived instance $(\bar{A}, \bar{B})$ need not lie in $\mathcal{R}_{\text{DLEQ}}$. Accordingly, the simulator relies on the NIZK simulator to produce an accepting proof even for off-language statements, and indistinguishability holds under the DDLEQ assumption; see Lemma 8 (p. 32). By construction, $\text{Derive}(\text{pp}, C, \tilde{\zeta}) = (M, R)$, and thus $\text{Open}(\text{pp}, C, \tilde{\zeta}) = \Upsilon$ whenever $\mathcal{V}_{\text{cert}}(\text{pp}, C, \tilde{\zeta}) = 1$.

Lemma 8 (Opening-conditional tag simulatability). *Assume DDLEQ hardness and that the Fischlin-based UC-NIZK Π_{DLEQ} in the gRO-CRP model has the universal simulation interface and straight-line extraction guarantees of Lemma 7 (p. 28). Then the simulator Sim_{cert} satisfies Item 3 (opening-conditional tag simulatability) of Definition 17 (p. 29).*

Proof. Fix λ and let $\mathcal{D}$ be any PPT distinguisher. We compare the following two experiments, which both output a tuple $(\text{pp}, C, \Upsilon, \zeta)$ where $\Upsilon = (M, R)$. Note that **Cert**(pp, m) samples $m \in \mathbb{Z}_p^*$ and $r \in \mathbb{Z}_p^* \setminus \{-m\}$, so the resulting $M = m\mathcal{G} \neq 0_{\mathbb{G}}$ and $R = r\mathcal{H} \neq 0_{\mathbb{G}}$; hence $(C, \Upsilon) \in \mathcal{R}_{\text{pp}}$ and the simulator's derived instance $(\bar{A}, \bar{B}) \in \mathcal{X}_{\text{DLEQ}}$ in every invocation.

Experiment $\text{RealTag}(1^\lambda)$:

1. Sample $\text{pp} \leftarrow \text{Setup}(1^\lambda, \mathcal{L})$, $m \leftarrow \mathbb{Z}_p^*$, and $r \leftarrow \mathbb{Z}_p^* \setminus \{-m\}$.
2. Define $M := m\mathcal{G}$, $R := r\mathcal{H}$, $C := M + R$, $\nu := mr^{-1} \bmod p$, $v := (m + r)\mathcal{G}$, $A := v - M$ and $B := C - M$.

3. Compute $\pi_{\text{DLEQ}} \leftarrow \mathcal{P}_{\text{DLEQ}}(\text{pp}, (A, B), r)$ and set $\zeta := (\nu, v, \pi_{\text{DLEQ}})$. Output $(\text{pp}, C, \Upsilon, \zeta)$ where $\Upsilon := (M, R)$.

Experiment $\text{SimTag}(1^\lambda)$:

1. Sample $\text{pp} \leftarrow \text{Setup}(1^\lambda, \mathcal{L})$, $m \leftarrow \mathbb{Z}_p^*$, and $r \leftarrow \mathbb{Z}_p^* \setminus \{-m\}$.
2. Define $M := m\mathcal{G}$, $R := r\mathcal{H}$, $C := M + R$ and $\Upsilon := (M, R)$.
3. Sample $\bar{\nu} \leftarrow \mathbb{Z}_p^* \setminus \{-1\}$ and set $\bar{v} := (1 + \bar{\nu}^{-1})M$. Let $\bar{A} := \bar{v} - M$ and $\bar{B} := C - M$.
4. Compute $\bar{\pi}_{\text{DLEQ}} \leftarrow \text{Sim}_{\text{DLEQ}}(\text{pp}, (\bar{A}, \bar{B}))$ and set $\bar{\zeta} := (\bar{\nu}, \bar{v}, \bar{\pi}_{\text{DLEQ}})$. Output $(\text{pp}, C, \Upsilon, \bar{\zeta})$.

We prove that $\left| \Pr[\mathcal{D}(\text{RealTag}(1^\lambda)) = 1] - \Pr[\mathcal{D}(\text{SimTag}(1^\lambda)) = 1] \right| \leq \text{negl}(\lambda)$.

Hybrid $\mathcal{D}_0$: This is identical to RealTag .

Hybrid $\mathcal{D}_1$: Modify $\mathcal{D}_0$ by replacing the real proof $\pi_{\text{DLEQ}} \leftarrow \mathcal{P}_{\text{DLEQ}}(\text{pp}, (A, B), r)$ with a simulated proof $\bar{\pi}_{\text{DLEQ}} \leftarrow \text{Sim}_{\text{DLEQ}}(\text{pp}, (A, B))$ for the same statement (A, B) . All other values are unchanged. Since (A, B) is a true DLEQ statement in $\mathcal{D}_0$ (indeed $A = r\mathcal{G}$ and $B = r\mathcal{H}$), the zero-knowledge property of Π_{DLEQ} for true statements implies $\mathcal{D}_0 \approx_c \mathcal{D}_1$.

$\mathcal{D}_1$ vs. SimTag : Assume for contradiction that there exists a PPT distinguisher $\mathcal{D}$ and a non-negligible function $\epsilon(\lambda)$ such that

$$\left| \Pr[\mathcal{D}(\mathcal{D}_1(1^\lambda)) = 1] - \Pr[\mathcal{D}(\text{SimTag}(1^\lambda)) = 1] \right| \geq \epsilon(\lambda)$$

for infinitely many λ . We construct a PPT distinguisher $\mathcal{B}$ for DDLEQ.

Distinguisher $\mathcal{B}$ (for DDLEQ): $\mathcal{B}$ receives $\text{pp} = (\mathcal{G}, \mathcal{H})$ and a challenge pair $(A^*, B^*) \in \mathcal{G} \times \mathcal{G}$ sampled as: either $(A^*, B^*) = (r\mathcal{G}, r\mathcal{H})$ (DDLEQ-true) for uniform $r \leftarrow \mathbb{Z}_p^*$, or $(A^*, B^*) = (\rho\mathcal{G}, r\mathcal{H})$ (DDLEQ-false) for uniform independent $\rho, r \leftarrow \mathbb{Z}_p^*$. $\mathcal{B}$ performs:

1. Sample $\nu \leftarrow \mathbb{Z}_p^* \setminus \{-1\}$.
2. Define $M := \nu A^*$, $R := B^*$, $C := M + R$, and $v := M + A^*$. Compute $\pi_{\text{DLEQ}} \leftarrow \text{Sim}_{\text{DLEQ}}(\text{pp}, (A^*, B^*))$.
3. Output $\mathcal{B}$'s guess as $\mathcal{D}(\text{pp}, C, (M, R), (\nu, v, \pi_{\text{DLEQ}}))$.

We claim that the distribution fed to $\mathcal{D}$ by $\mathcal{B}$ matches $\mathcal{D}_1$ in the DDLEQ-true case, and matches SimTag up to negligible statistical distance in the DDLEQ-false case.

DDLEQ-true case: Here $(A^*, B^*) = (r\mathcal{G}, r\mathcal{H})$ for uniform $r \in \mathbb{Z}_p^*$. $\mathcal{B}$ samples $\nu \in \mathbb{Z}_p^* \setminus \{-1\}$ and sets $M = \nu r\mathcal{G}$, $R = r\mathcal{H}$, $C = M + R$, and $v = M + r\mathcal{G} = (\nu + 1)r\mathcal{G}$. Let $m := \nu r \bmod p$; then $M = m\mathcal{G}$ and $v = (m + r)\mathcal{G}$ and $\nu = mr^{-1}$. Moreover $r = -m$ would imply $\nu = -1$, which is excluded; thus the support condition $r \in \mathbb{Z}_p^* \setminus \{-m\}$ holds automatically. Finally, the proof is generated as $\text{Sim}_{\text{DLEQ}}(\text{pp}, (A^*, B^*)) = \text{Sim}_{\text{DLEQ}}(\text{pp}, (r\mathcal{G}, r\mathcal{H}))$, which is exactly the proof distribution in $\mathcal{D}_1$. Hence, the output distribution is identical to $\mathcal{D}_1$.

DDLEQ-false case: Here $(A^*, B^*) = (\rho\mathcal{G}, r\mathcal{H})$ for independent uniform $\rho, r \in \mathbb{Z}_p^*$. $\mathcal{B}$ samples $\nu \in \mathbb{Z}_p^* \setminus \{-1\}$ and defines $M = \nu\rho\mathcal{G}$ and $R = r\mathcal{H}$ and $v = M + \rho\mathcal{G}$. Let $m := \nu\rho \bmod p$. Then $M = m\mathcal{G}$, and also

$$A^* = \rho\mathcal{G} = \nu^{-1}M, \quad v = M + A^* = (1 + \nu^{-1})M,$$

so the tuple (ν, v) is distributed exactly as in Sim_{tag} , and the proof is generated by the same simulator Sim_{DLEQ} . The only difference from Sim_{tag} is that Sim_{tag} samples $r \leftarrow \mathbb{Z}_p^* \setminus \{-m\}$ whereas here $r \leftarrow \mathbb{Z}_p^*$ independently of m . These two r -distributions differ by statistical distance at most

$$\Pr[r = -m] \leq \frac{1}{p-1},$$

which is negligible in λ since p is exponential in λ . Therefore the DDLEQ-false output distribution is negligibly close to Sim_{tag} . Thus, $\mathcal{B}$ distinguishes DDLEQ-true from DDLEQ-false with advantage at least $\epsilon(\lambda) - \text{negl}(\lambda)$, contradicting DDLEQ. Hence, $\mathcal{D}_1 \approx_c \text{SimTag}$. Combining $\mathcal{D}_0 \approx_c \mathcal{D}_1$ (ZK) and $\mathcal{D}_1 \approx_c \text{SimTag}$ (DDLEQ) yields $\text{RealTag} \approx_c \text{SimTag}$, as required. $\blacksquare$

Remark 14 (Status of opening-conditional tag simulatability). Lemma 8 (p. 32) proves a stronger interface than what our UC hybrids need: no UC proof in this paper requires fabricating an accepting USV tag ζ for a fixed commitment C without knowing a witness. Consequently, the UC simulators constructed for Theorems 2 (p. 36) and 3 (p. 60) never call the DLEQ proof simulator and never program ctx_{DLEQ} . We retain the lemma because it is a natural property of USV as a standalone primitive and is useful for other compositions (e.g., “commit now, open-to- M later” designs under NXK).

Lemma 9 (Equivocation resistance of the USV instantiation). *Assume further that the embedded proof system Π_{DLEQ} used inside $\mathcal{V}_{\text{cert}}$ is a NIZK-AoK for $\mathcal{R}_{\text{DLEQ}}$ in the gRO-CRP model with an online extractor Ext_{DLEQ} (as in Section 6 (p. 38)). Then, under the DL assumption (Assumption 4 (p. 17)) and the transparent generator derivation assumption (Assumption 6 (p. 29)), the USV certificate scheme of Section 5.2 (p. 32) is equivocation resistant (Definition 19 (p. 30)).*

Proof. Fix any PPT adversary $\mathcal{A}$. We construct a PPT algorithm $\mathcal{B}$ that receives a DL challenge $(\mathcal{G}, X)$ with $X = x\mathcal{G}$ for $x \leftarrow \mathbb{Z}_p^*$ (Definition 7 (p. 17)) and outputs x with non-negligible probability whenever $\mathcal{A}$ wins the equivocation game. Specifically, $\mathcal{B}$ sets $\text{pp} := (\mathbb{G}, p, \mathcal{G}, X)$ —embedding the DL challenge X as the secondary generator $\mathcal{H}$ —and simulates the equivocation experiment for $\mathcal{A}$ on these parameters. By Assumption 6 (p. 29), the Setup-derived $\mathcal{H}$ is computationally indistinguishable from uniform over $\mathbb{G} \setminus \{0_{\mathbb{G}}\}$, and the DL challenge X is exactly uniform over $\mathbb{G} \setminus \{0_{\mathbb{G}}\}$ (since $x \leftarrow \mathbb{Z}_p^*$ and the map $z \mapsto z\mathcal{G}$ is a bijection on $\mathbb{Z}_p$). Hence $\mathcal{A}$ ’s view under $\text{pp} = (\mathbb{G}, p, \mathcal{G}, X)$ is computationally indistinguishable from its view in the real equivocation experiment, and so $\text{Adv}_{\mathcal{B}}^{\text{dl}}(\lambda) \geq \text{Adv}_{\mathcal{A}}^{\text{eqv}}(\lambda) - \text{negl}(\lambda)$. Since $\text{Adv}_{\mathcal{B}}^{\text{dl}}(\lambda) \leq \text{negl}(\lambda)$ by Assumption 4 (p. 17), it follows that $\text{Adv}_{\mathcal{A}}^{\text{eqv}}(\lambda) \leq \text{negl}(\lambda)$.

Algorithm $\mathcal{B}^{\mathcal{A}}(\mathcal{G}, X)$. $\mathcal{B}$ sets $\mathcal{H} := X$ and $\text{pp} := (\mathbb{G}, p, \mathcal{G}, \mathcal{H})$, then internally simulates the equivocation experiment for $\mathcal{A}$:

1. Give pp to $\mathcal{A}$ and answer $\mathcal{A}$ ’s gRO-CRP queries by lazy sampling, while recording the complete query/answer log $\text{Log}_{\mathcal{A}}$ under each DLEQ-proof context.
2. Receive (C, ζ, ζ') from $\mathcal{A}$, where $\zeta = (\nu, v, \pi_{\text{DLEQ}})$ and $\zeta' = (\nu', v', \pi'_{\text{DLEQ}})$.
3. Compute $\Upsilon \leftarrow \text{Derive}(\text{pp}, C, \zeta)$ and $\Upsilon' \leftarrow \text{Derive}(\text{pp}, C, \zeta')$. If either is $\perp$, output $\perp$. Otherwise parse $\Upsilon = (M, R)$ and $\Upsilon' = (M', R')$.
4. If $\mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) = 0$ or $\mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta') = 0$, output $\perp$. If $M = M'$, output $\perp$.
5. Define $A := v - M$, $B := C - M$, $A' := v' - M'$, and $B' := C - M'$.
6. Run the AoK extractor to obtain witnesses:

$$r \leftarrow \text{Ext}_{\text{DLEQ}}(\text{pp}, (A, B), \pi_{\text{DLEQ}}; \text{Log}_{\mathcal{A}}), \quad r' \leftarrow \text{Ext}_{\text{DLEQ}}(\text{pp}, (A', B'), \pi'_{\text{DLEQ}}; \text{Log}_{\mathcal{A}}).$$

If either extraction fails (outputs $\perp$) or the extracted values do not satisfy $A = r\mathcal{G}$, $B = r\mathcal{H}$ and $A' = r'\mathcal{G}$, $B' = r'\mathcal{H}$, output $\perp$.

7. Compute $m := \nu r \bmod p$ and $m' := \nu' r' \bmod p$.
8. Compute $\hat{r} := r' - r \bmod p$. If $\hat{r} = 0$, output $\perp$. Otherwise output

$$x := (m - m') \cdot (\hat{r})^{-1} \bmod p.$$

Assume $\mathcal{A}$ wins, i.e., $\mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) = \mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta') = 1$ and $M \neq M'$ where $\text{Derive}(\text{pp}, C, \zeta) = (M, R)$ and $\text{Derive}(\text{pp}, C, \zeta') = (M', R')$. Since $\mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) = 1$, it follows that $\nu \neq -1 \bmod p$. Moreover, by definition of $\mathcal{V}_{\text{cert}}$ and our setting of (A, B) , we have $\mathcal{V}_{\text{DLEQ}}(\text{pp}, (A, B), \pi_{\text{DLEQ}}) = 1$. By the NIZK-AoK property, except with negligible probability the extractor returns a witness $r \in \mathbb{Z}_p^*$ such that $A = r\mathcal{G}$ and $B = r\mathcal{H}$. Similarly, except with negligible probability, the extractor returns $r' \in \mathbb{Z}_p^*$ such that $A' = r'\mathcal{G}$ and $B' = r'\mathcal{H}$. Using Derive’s defining equations, for $\zeta = (\nu, v, \pi_{\text{DLEQ}})$ we have

$$M = \frac{\nu}{\nu + 1}v, \quad \text{hence} \quad A = v - M = \frac{1}{\nu + 1}v,$$

so $v = (\nu + 1)A$ and therefore $M = \frac{\nu}{\nu+1}v = \nu A = \nu r \mathcal{G} = m \mathcal{G}$ where $m := \nu r \bmod p$. Likewise $M' = m' \mathcal{G}$ where $m' := \nu' r' \bmod p$. Also, by definition, $B = C - M = r \mathcal{H}$, $B' = C - M' = r' \mathcal{H}$, so we obtain two decompositions of C :

$$C = M + r \mathcal{H} = m \mathcal{G} + r \mathcal{H} \quad \text{and} \quad C = M' + r' \mathcal{H} = m' \mathcal{G} + r' \mathcal{H}.$$

Subtracting yields $(m - m') \mathcal{G} = (r' - r) \mathcal{H}$. Since $M \neq M'$ and the map $z \mapsto z \mathcal{G}$ is a bijection on $\mathbb{Z}_p$ (prime-order group), we have $m \neq m'$. Therefore, the above equality implies $r' \neq r$ (otherwise $(m - m') \mathcal{G} = 0$ would force $m = m'$). Hence, $\hat{r} := r' - r \in \mathbb{Z}_p^*$ and is invertible, and the value

$$x := (m - m') \cdot (\hat{r})^{-1} \bmod p$$

satisfies $x \mathcal{G} = \mathcal{H} = X$. $\mathcal{B}$ outputs such an x whenever $\mathcal{A}$ wins the equivocation experiment and both extractor invocations succeed. The extractor failure probability is negligible by the AoK guarantee, and the hybrid loss from replacing the Setup-derived $\mathcal{H}$ with the DL challenge X is negligible by Assumption 6 (p. 29). Thus, $\text{Adv}_{\mathcal{B}}^{\text{dl}}(\lambda) \geq \text{Adv}_{\mathcal{A}}^{\text{eqv}}(\lambda) - \text{negl}(\lambda)$. Since $\text{Adv}_{\mathcal{B}}^{\text{dl}}(\lambda) \leq \text{negl}(\lambda)$ by Assumption 4 (p. 17), it follows that $\text{Adv}_{\mathcal{A}}^{\text{eqv}}(\lambda) \leq \text{negl}(\lambda)$. ■

5.3 UC Security

We model certificates with a handle-bound ideal functionality that stores the *verified* opening implied by (C, ζ) , namely the (unique) value returned by $\text{Open}(\text{pp}, C, \zeta)$, and binds a handle to (C, M) via a gRO-CRP receipt digest, where $M := \text{Open}_M(\text{pp}, C, \zeta)$ is the verified public opening implied by (C, ζ) .

In our applications, each USV handle is intended for a single committer-verifier pair. Accordingly, we define $\mathcal{F}_{\text{USV}}$ (Fig. 5 (p. 36)) so that each handle is scoped to both endpoints: $\mathcal{F}_{\text{USV}}$ maintains independent state keyed by $(\text{sid}, \text{cid}, P_s, P_r)$. This prevents a party $P'_s \neq P_s$ from invalidating another sender's handle by reusing the same (sid, cid) toward the same relying party P_r , while still keeping recipient-local state (so a corrupted committer may equivocate across different relying parties, as in the real protocol). In $\mathcal{F}_{\text{USV}}$, the leakage to Sim includes the public certificate (C, ζ) along with the receipt digest d : all three values are public (the certificate (C, ζ) is intended for delivery to P_r over $\mathcal{F}_{\text{channel}}$, and d is deterministic from the committed tuple and the global gRO-CRP oracle). This enables Sim to simulate the $\mathcal{F}_{\text{channel}}$ delivery of (C, ζ, d) to a corrupted recipient.

Remark 15 (Game-based equivocation vs. UC-level non-malleability). Lemma 9 (p. 34) is a standalone game-based statement about the USV certificate scheme itself: it rules out producing two verifying tags for the same commitment C that induce different public openings. In the UC setting, the relevant “no substitution under a fixed handle” guarantee is instead enforced by $\mathcal{F}_{\text{USV}}$'s receipt binding in the non-programmable context $\text{USV.rcpt} \in \text{Ctx}_{\text{np}}$ and captured by Lemma 10 (p. 35) (handle-bound non-malleability), which is what the UC proof of Theorem 2 (p. 36) relies on.

Lemma 10 (Handle-bound non-malleability). *Let $M := \text{Open}_M(\text{pp}, C, \zeta) \neq \perp$ be the USV opening for the honest commit and let $d := H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}, P_s, P_r, C, M \rangle)$. Then for every (C', ζ') such that $M' := \text{Open}_M(\text{pp}, C', \zeta') \neq \perp$ and $(C', M') \neq (C, M)$,*

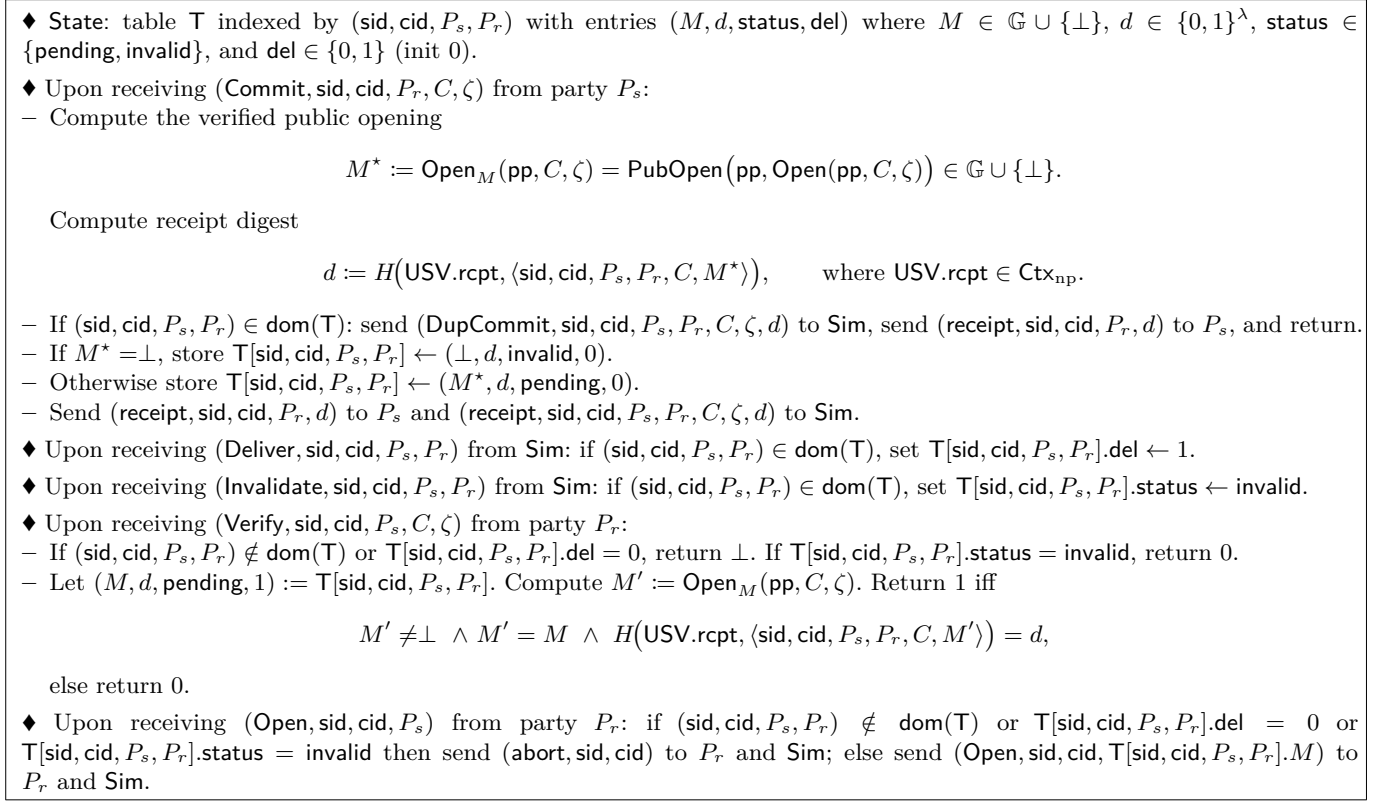
$$\Pr \left[\mathcal{F}_{\text{USV}}.\text{Verify}(\text{sid}, \text{cid}, P_s, C', \zeta') = 1 \right] \leq \text{negl}(\lambda).$$

Proof Sketch. Let $H_{\text{rcpt}}(u) := H(\text{USV.rcpt}, u)$. Fix the honest receipt input $u := \langle \text{sid}, \text{cid}, P_s, P_r, C, M \rangle$ and receipt $d := H_{\text{rcpt}}(u)$. For any distinct pair $(C', \zeta') \neq (C, \zeta)$, injectivity of $\langle \cdot \rangle$ implies that $u' := \langle \text{sid}, \text{cid}, P_s, P_r, C', M' \rangle \neq u$. A successful substitution (i.e., $\mathcal{F}_{\text{USV}}.\text{Verify}(\text{sid}, \text{cid}, C', \zeta') = 1$) therefore implies a second-preimage for the fixed target input u , namely $H_{\text{rcpt}}(u') = H_{\text{rcpt}}(u) = d$ with $u' \neq u$.

In the random-oracle/gRO-CRP model, suppose the adversary makes at most $Q = \text{poly}(\lambda)$ queries to H_{rcpt} before the Verify call. The Verify procedure itself evaluates $H_{\text{rcpt}}(u')$ once. Hence

$$\Pr \left[H_{\text{rcpt}}(u') = d \right] \leq \frac{Q+1}{|\mathcal{Y}|} \quad (\text{in particular } \leq (Q+1)/2^\lambda \text{ when } |\mathcal{Y}| = 2^\lambda),$$

which is negligible. ■

Fig. 5: Two-party USV certificate functionality $\mathcal{F}_{\text{USV}}$ with gRO-CRP receipt binding and verified openings.

The PPT simulator Sim only needs pp and the public leakage from $\mathcal{F}_{\text{USV}}$. In the ideal world, on receiving each $(\text{Commit}, \text{sid}, \text{cid}, C, \zeta)$, $\mathcal{F}_{\text{USV}}$ computes the receipt $d := H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}, P_s, P_r, C, M \rangle)$ and stores the public opening $M := \text{Open}_M(\text{pp}, C, \zeta)$ only if the certificate verifies, i.e., only if $\mathcal{V}_{\text{cert}}(\text{pp}, C, \zeta) = 1$. $\mathcal{F}_{\text{USV}}$ sends $(\text{receipt}, \text{sid}, \text{cid}, P_s, P_r, C, \zeta, d)$ to Sim; Sim uses (C, ζ, d) to reproduce the $\mathcal{F}_{\text{channel}}$ traffic: Sim sends $(\text{Send}, \text{sid}, (C, \zeta, d))$ to the $\mathcal{F}_{\text{channel}}$ instance for (P_s, P_r) , generating the same Leak ticket and length leakage as in the real world. When $\mathcal{A}$ issues $(\text{Deliver}, \text{sid}, \rho)$, $\mathcal{F}_{\text{channel}}$ delivers (C, ζ, d) to P_r , and Sim concurrently sends $(\text{Deliver}, \text{sid}, \text{cid}, P_s, P_r)$ to $\mathcal{F}_{\text{USV}}$ to activate the recipient-side entry. If P_r is corrupted, Sim additionally forwards (C, ζ, d) to the corrupted P_r . Upon receiving $(\text{DupCommit}, \text{sid}, \text{cid}, P_s, P_r, \dots)$ from $\mathcal{F}_{\text{USV}}$, Sim sends the duplicate over $\mathcal{F}_{\text{channel}}$ identically and, when $\mathcal{A}$ delivers the duplicate, sends $(\text{Invalidate}, \text{sid}, \text{cid}, P_s, P_r)$ to $\mathcal{F}_{\text{USV}}$. When a party is adaptively corrupted after a successful Commit, Sim reconstructs Υ deterministically as $\text{Open}(\text{pp}, C, \zeta)$. This matches the real world since the verified opening is a fixed deterministic function of (C, ζ) .

Theorem 2 (UC security of $\mathcal{F}_{\text{USV}}$). *Let Π_{USV} denote the concrete protocol that maintains a local table T^Π indexed by $(\text{sid}, \text{cid}, P_s, P_r)$ with entries (M, d, status) , tracking the (M, d, status) components of $\mathcal{F}_{\text{USV}}$'s state (the delivery flag del is implicit in Π_{USV} : the recipient's entry is created only upon $\mathcal{F}_{\text{channel}}$ delivery), and whose per-interface behavior is:*

- **Commit** $(\text{sid}, \text{cid}, P_r, C, \zeta)$: party P_s receives (C, ζ) (e.g., from a prior $\text{Cert}(\text{pp}, m)$ invocation). Compute $M := \text{Open}_M(\text{pp}, C, \zeta)$ and the receipt $d := H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}, P_s, P_r, C, M \rangle)$ (in context $\text{USV.rcpt} \in \text{Ctx}_{\text{np}}$). If $(\text{sid}, \text{cid}, P_s, P_r) \in \text{dom}(T^\Pi)$, mark $T^\Pi[\text{sid}, \text{cid}, P_s, P_r].\text{status} \leftarrow \text{invalid}$ and return $(\text{receipt}, \text{sid}, \text{cid}, P_r, d)$ to P_s . If $M = \perp$, store $T^\Pi[\text{sid}, \text{cid}, P_s, P_r] \leftarrow (\perp, d, \text{invalid})$; else store $T^\Pi[\text{sid}, \text{cid}, P_s, P_r] \leftarrow (M, d, \text{pending})$. Send (C, ζ, d) to P_r over $\mathcal{F}_{\text{channel}}$.
- **Receive** (at P_r): Upon receiving (C, ζ, d) from P_s via $\mathcal{F}_{\text{channel}}$, P_r computes $M' := \text{Open}_M(\text{pp}, C, \zeta)$ and $d' := H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}, P_s, P_r, C, M' \rangle)$. If $(\text{sid}, \text{cid}, P_s, P_r) \in \text{dom}(T^\Pi)$, set $T^\Pi[\text{sid}, \text{cid}, P_s, P_r].\text{status} \leftarrow \text{invalid}$ and return. If $M' = \perp$ or $d' \neq d$, store $T^\Pi[\text{sid}, \text{cid}, P_s, P_r] \leftarrow (\perp, d, \text{invalid})$; else store $T^\Pi[\text{sid}, \text{cid}, P_s, P_r] \leftarrow (M', d, \text{pending})$.

- **Verify**(sid, cid, P_s , C' , ζ'): recipient P_r looks up $T^\Pi[\text{sid}, \text{cid}, P_s, P_r]$. If the entry does not exist, return $\perp$. If $T^\Pi[\text{sid}, \text{cid}, P_s, P_r].\text{status}$ is invalid, return 0. Let $(M, d, \text{pending}) := T^\Pi[\text{sid}, \text{cid}, P_s, P_r]$. Locally compute $\mathcal{V}_{\text{cert}}(\text{pp}, C', \zeta')$; if it rejects, return 0. Compute $M' := \text{Open}_M(\text{pp}, C', \zeta')$ and $d' := H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}, P_s, P_r, C', M' \rangle)$. Return 1 iff $M' \neq \perp \wedge M' = M \wedge d' = d$.
- **Open**(sid, cid): P_r reads the stored M (or $\perp$ if none).

All verification/derivation is deterministic from (C, ζ) and the public algorithms $(\mathcal{V}_{\text{cert}}, \text{Derive}, \text{PubOpen})$. Then Π_{USV} UC-realizes $\mathcal{F}_{\text{USV}}$ in the $(\mathcal{F}_{\text{channel}}, \text{gRO-CRP})$ -hybrid model; i.e., for every PPT adversary $\mathcal{A}$ there exists a PPT simulator Sim such that for every PPT environment $\mathcal{Z}$,

$$\text{Exec}(\Pi_{\text{USV}}, \mathcal{A}, \mathcal{Z}, \lambda) \approx_c \text{Ideal}(\mathcal{F}_{\text{USV}}, \text{Sim}, \mathcal{Z}, \lambda).$$

Proof. Fix any PPT adversary $\mathcal{A}$ and PPT environment $\mathcal{Z}$. We construct a PPT simulator Sim and employ the following sequence of hybrids:

- Hybrid $\mathcal{D}_0$ (real execution): Each honest party P_s receives (C, ζ) (generated via $\text{Cert}(\text{pp}, m)$ by the calling protocol or locally), computes $M := \text{Open}_M(\text{pp}, C, \zeta)$ and the receipt

$$d = H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}, P_s, P_r, C, M \rangle),$$

maintains the handle table T^Π (including duplicate-handle invalidation), and sends (C, ζ, d) to P_r .

- Hybrid $\mathcal{D}_1$ (syntactic resampling / explicit Cert randomness): Proceed identically to $\mathcal{D}_0$, except that for each honest certificate we (re-)sample the full random tape of Cert and re-evaluate it. Concretely, we sample $r \leftarrow \mathbb{Z}_p^* \setminus \{-m\}$ together with the internal randomness used by the randomized prover $\mathcal{P}_{\text{DLEQ}}$ (and hence by the Fischlin transform), and then compute (C, ζ) by running $\text{Cert}(\text{pp}, m)$ with that sampled randomness (and the fixed global oracle H). All other external inputs and protocol randomness are unchanged. This is a purely syntactic refactoring of how Cert 's coins are sampled, so $\mathcal{D}_0 \stackrel{d}{=} \mathcal{D}_1$.
- Hybrid $\mathcal{D}_2$ (ideal execution), switch to $\mathcal{F}_{\text{USV}}$ and Sim : For each honest handle, $\mathcal{F}_{\text{USV}}$ stores the same receipt $d = H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}, P_s, P_r, C, M \rangle)$. Since honest certificates verify, $\mathcal{F}_{\text{USV}}$ stores $M := \text{Open}_M(\text{pp}, C, \zeta)$. By Lemma 10 (p. 35), any in-flight substitution under a fixed handle is detected in both worlds. For every Commit (regardless of P_r 's corruption status), Sim receives $(\text{receipt}, \text{sid}, \text{cid}, P_s, P_r, C, \zeta, d)$ from $\mathcal{F}_{\text{USV}}$ and sends $(\text{Send}, \text{sid}, (C, \zeta, d))$ to the $\mathcal{F}_{\text{channel}}$ instance for (P_s, P_r) , generating the same Leak ticket as in $\mathcal{D}_0$. When $\mathcal{A}$ issues Deliver for this ticket, $\mathcal{F}_{\text{channel}}$ delivers (C, ζ, d) to P_r and Sim concurrently sends $(\text{Deliver}, \text{sid}, \text{cid}, P_s, P_r)$ to $\mathcal{F}_{\text{USV}}$ to activate the recipient-side entry. If P_r is honest, P_r then processes (C, ζ, d) identically to $\mathcal{D}_0$ and any subsequent $\mathcal{F}_{\text{USV}}.\text{Verify}$ or $\mathcal{F}_{\text{USV}}.\text{Open}$ call returns the same result as the local computation in $\mathcal{D}_0$ (since $\text{del} = 1$ after delivery and all checks are deterministic). If P_r is corrupted, Sim additionally forwards (C, ζ, d) to the corrupted P_r . Upon receiving $(\text{DupCommit}, \text{sid}, \text{cid}, P_s, P_r, \dots)$ from $\mathcal{F}_{\text{USV}}$, Sim reproduces the duplicate over $\mathcal{F}_{\text{channel}}$ and, upon delivery, sends $(\text{Invalidate}, \text{sid}, \text{cid}, P_s, P_r)$ to $\mathcal{F}_{\text{USV}}$, matching the real-world recipient-side invalidation timing. The adversary may invoke $\mathcal{F}_{\text{USV}}.\text{Verify}$ or $\mathcal{F}_{\text{USV}}.\text{Open}$ directly; Sim simply forwards the deterministic public computations $\mathcal{V}_{\text{cert}}$ and Open_M that the adversary could perform itself from (C, ζ) . No USV/DLEQ proof simulation is needed because the adversary holds (C, ζ) (via the simulated $\mathcal{F}_{\text{channel}}$ delivery) and can verify using the public algorithms. Adaptive corruptions reveal identical internal states because the verified opening $\text{Open}(\text{pp}, C, \zeta)$ is public and deterministic. Hence, $\mathcal{D}_1 \approx_c \mathcal{D}_2$, and therefore $\mathcal{D}_0 \approx_c \mathcal{D}_2$. Interpreting $\mathcal{D}_0$ as the real execution $\text{Exec}(\Pi_{\text{USV}}, \mathcal{A}, \mathcal{Z}, \lambda)$ and $\mathcal{D}_2$ as the ideal execution $\text{Ideal}(\mathcal{F}_{\text{USV}}, \text{Sim}, \mathcal{Z}, \lambda)$, we conclude $\text{Exec}(\Pi_{\text{USV}}, \mathcal{A}, \mathcal{Z}, \lambda) \approx_c \text{Ideal}(\mathcal{F}_{\text{USV}}, \text{Sim}, \mathcal{Z}, \lambda)$. ■

The simulator in this UC proof never invokes SimProgram in ctx_{DLEQ} (indeed it never needs to simulate USV/DLEQ proofs); it only forwards verifier oracle queries via Query .

Broader applicability of USV Although USV is introduced here to make the SDKG transcript “structure-complete” under NXK, the primitive is not specific to DKG since it turns a KeyBox-resident, non-exportable scalar into a transcript-defined public group element via a publicly verifiable certificate. As an orthogonal illustration, Appendix C (p. 76) shows how to build NXK-compatible commit–reveal randomness beacons using USV.

6 Enforcing Consistency via Straight-Line Extraction

Since $\mathcal{G}$ generates $\mathbb{G}$, every $M \in \mathbb{G}$ can be written as $M = m\mathcal{G}$ for some $m \in \mathbb{Z}_p$. Accordingly, for DL-style statements the relevant security notion in our use-cases is *knowledge soundness* (extractability of the witness), rather than language non-membership soundness. We use standard Schnorr (DL) and Chaum–Pedersen (DLEQ) Σ -protocols, which have special soundness and unique responses in prime-order groups. Applying the optimized Fischlin transform in gRO-CRP yields UC-NIZK-AoKs with straight-line extractors for both relations. Define the NP relation

$$\mathcal{R}_{\text{DL}} := \{((\text{pp}, M), m) : m \in \mathbb{Z}_p \wedge M = m\mathcal{G}\}.$$

We intentionally allow $m = 0$ (and hence $M = 0_{\mathbb{G}}$), since protocol-derived witnesses (e.g., affine-relation witnesses and derived shares) can be 0 under adversarial choice. For the hardness assumption (Definition 7 (p. 17)) we sample $x \leftarrow \mathbb{Z}_p^*$ to avoid the trivial instance $X = 0_{\mathbb{G}}$; including $x = 0$ would change success probability by at most $1/p = \text{negl}(\lambda)$.

Tagged DL statements. To rule out replay/mauling of UC-context proofs across sessions and to justify the use of simulation-extractability for fresh statements in concurrent executions, we bind every UC-context DL statement to the session identifier sid and a fixed label $\ell \in \{0, 1\}^*$ describing the proof position. Concretely, we use the tagged relation

$$\mathcal{R}_{\text{DL}}^{\text{tag}} := \{((\text{pp}, \text{sid}, \ell, M), m) : m \in \mathbb{Z}_p \wedge M = m\mathcal{G}\}.$$

In SDKG, every invocation of $\Pi_{\text{DL}}^{\text{UC}}$ is on a tagged statement of the form $x := \langle \text{sid}, \ell, M \rangle$ (with pp implicit), so the Fischlin hash input in Definition 16 (p. 25) includes (sid, ℓ) along with M . We will slightly abuse notation and keep writing $\mathcal{P}_{\text{DL}}$ and $\mathcal{V}_{\text{DL}}$ for this tagged variant.

In this section, all proofs contexts are in Ctx_p . Given statement (pp, M) and witness m such that $M = m\mathcal{G}$, the standard Schnorr Σ -protocol for $\mathcal{R}_{\text{DL}}$ is:

1. Commit: Prover samples $j \leftarrow \mathbb{Z}_p$ and sends $J := j\mathcal{G}$.
2. Challenge: Verifier samples $e \leftarrow \{0, 1\}^t$, interprets it as an integer $\bar{e} \in [0, 2^t - 1]$, embeds it into $\mathbb{Z}_p$ (e.g., require $2^t < p$), and sends $\bar{e}$ to the prover.
3. Response: Prover sends $z := j + \bar{e} \cdot m \bmod p$.
4. Verify: accept iff $z\mathcal{G} = J + \bar{e} \cdot M$.

Special soundness holds since from two accepting transcripts $(J, \bar{e}, z)$ and $(J, \bar{e}', z')$ with $\bar{e} \neq \bar{e}'$ one extracts $m = (z - z')(\bar{e} - \bar{e}')^{-1} \bmod p$.

Lemma 11 (Unique responses for Schnorr). *The Schnorr Σ -protocol for $\mathcal{R}_{\text{DL}}$ has unique responses.*

Proof Sketch. For fixed (M, J, e) , verification requires $z\mathcal{G} = J + eM$ in a prime-order group. Since $\mathcal{G}$ generates $\mathbb{G}$, this equation has a unique solution $z \in \mathbb{Z}_p$. $\blacksquare$

Recall the public parameters $\text{pp} = (\mathbb{G}, p, \mathcal{G}, \mathcal{H})$ and the NP relation

$$\mathcal{R}_{\text{DLEQ}} := \left\{ ((\text{pp}, A, B), r) : (A, B) \in \mathcal{X}_{\text{DLEQ}} \wedge r \in \mathbb{Z}_p^* \wedge A = r\mathcal{G} \wedge B = r\mathcal{H} \right\}.$$

Chaum–Pedersen Σ -protocol for $\mathcal{R}_{\text{DLEQ}}$. Given statement (pp, A, B) and witness r such that $A = r\mathcal{G}$ and $B = r\mathcal{H}$, the Chaum–Pedersen Σ -protocol (with t -bit challenge) is:

1. Commit: Prover samples $j \leftarrow \mathbb{Z}_p$, and sends $J_1 := j\mathcal{G}$ and $J_2 := j\mathcal{H}$.
2. Challenge: Verifier samples $e \leftarrow \{0, 1\}^t$, interprets it as $\bar{e} \in [0, 2^t - 1]$, embeds it into $\mathbb{Z}_p$ (e.g., require $2^t < p$), and sends $\bar{e}$.
3. Response: Prover sends $z := j + \bar{e} \cdot r \bmod p$.
4. Verify: reject if $A = 0_{\mathbb{G}}$ or $B = 0_{\mathbb{G}}$; otherwise accept iff $z\mathcal{G} = J_1 + \bar{e} \cdot A$ and $z\mathcal{H} = J_2 + \bar{e} \cdot B$.

Special soundness for Chaum–Pedersen: From any two accepting transcripts $(J_1, J_2, \bar{e}, z)$ and $(J_1, J_2, \bar{e}', z')$ with $\bar{e} \neq \bar{e}'$, one can efficiently extract a witness

$$r = (z - z') \cdot (\bar{e} - \bar{e}')^{-1} \bmod p$$

satisfying $A = r\mathcal{G}$ and $B = r\mathcal{H}$.

Lemma 12 (Unique responses for Chaum–Pedersen). *The Chaum–Pedersen Σ -protocol for $\mathcal{R}_{\text{DLEQ}}$ has unique responses.*

Proof Sketch. Fix any statement (A, B) , first message (J_1, J_2) , and challenge $\bar{e}$. If an accepting response z exists, it must satisfy $z\mathcal{G} = J_1 + \bar{e} \cdot A$. Since $\mathcal{G}$ generates a prime-order group, the map $z \mapsto z\mathcal{G}$ is a bijection on $\mathbb{Z}_p$, so z is uniquely determined. Hence, there is at most one accepting response. ■

UC-NIZK-AoK for DLEQ from Fischlin in gRO-CRP. Applying the (optimized) Fischlin transform to the above Chaum–Pedersen Σ -protocol yields a UC-NIZK-AoK for $\mathcal{R}_{\text{DLEQ}}$ in the gRO-CRP model. We denote it by $\Pi_{\text{DLEQ}} = (K_{\text{DLEQ}}, \mathcal{P}_{\text{DLEQ}}, \mathcal{V}_{\text{DLEQ}})$, with a straight-line extractor Ext_{DLEQ} that may inspect the adversary’s gRO-CRP query/answer log under the DLEQ-proof context(s).

UC-NIZK-AoK for DL from Fischlin in gRO-CRP. Applying the (optimized) Fischlin transform to the above Schnorr Σ -protocol yields a UC-NIZK-AoK for $\mathcal{R}_{\text{DL}}$ in the gRO-CRP model. We denote this system by $\Pi_{\text{DL}} = (K_{\text{DL}}, \mathcal{P}_{\text{DL}}, \mathcal{V}_{\text{DL}})$ and summarize its properties (which also apply to Π_{DLEQ}):

- Completeness: honest proofs produced using witness m verify.
- (Computational) ZK: real proofs are computationally indistinguishable from simulated proofs.
- Simulation-extractability (fresh statement) in the sense of Definition 13 (p. 24) [31, Thm. 3].
- Knowledge soundness / extraction: there exists a straight-line PPT extractor Ext_{DL} such that for any PPT adversarial prover $\mathcal{P}^*$ that outputs (M, π) with $\mathcal{V}_{\text{DL}}(\text{pp}, M, \pi_{\text{DL}}) = 1$, the extractor outputs $m \leftarrow \text{Ext}_{\text{DL}}(\text{pp}, M, \pi; \text{Log}_{\mathcal{P}^*})$ satisfying $M = m\mathcal{G}$, except with negligible probability, where $\text{Log}_{\mathcal{P}^*}$ is the prover’s logged gRO-CRP query/answer transcript under the proof context.

Our use of Fischlin-style straight-line compilation of DL-based Σ -protocols into UC-NIZK-PoK/AoK is closely aligned with [49], which targets adaptive UC security in global random-oracle models; we further tailor the model/usage to gRO-CRP and KeyBox local-call semantics.

6.1 Affine DL relation

Let $\text{Open}_M(\text{pp}, C, \zeta)$ denote the public opening derived from a USV certificate as in Definition 17 (p. 29). Fix $\gamma \in \mathbb{Z}_p$ and public points $X, M, B, \Delta \in \mathbb{G}$. Define affine DL NP relation

$$\mathcal{R}_{\text{aff}} := \left\{ ((X, \gamma, M, B, \Delta), (\alpha, \delta)) : \alpha\mathcal{G} = M + \gamma B \wedge \delta\mathcal{G} = X - \Delta - (M + \gamma B) \right\}.$$

Equivalently, define $Y := M + \gamma B$ and $D := X - \Delta - Y$. Then (α, δ) is a witness iff $Y = \alpha\mathcal{G}$ and $D = \delta\mathcal{G}$, where Y and D are deterministically derived from the public tuple $(X, \gamma, M, B, \Delta)$.

Remark 16 (Realizing a UC-NIZK-AoK for $\mathcal{R}_{\text{aff}}$ from Π_{DL}). Let Π_{DL} be the UC-NIZK-AoK for $\mathcal{R}_{\text{DL}}$. A UC-NIZK-AoK for $\mathcal{R}_{\text{aff}}$ is obtained by parallel composition of two instances of Π_{DL} :

- prove knowledge of α such that $Y = M + \gamma B = \alpha\mathcal{G}$;
- prove knowledge of δ such that $D = X - \Delta - (M + \gamma B) = \delta\mathcal{G}$.

Concretely, the proof is $\pi_{\text{aff}} := (\pi_Y, \pi_D)$ where $\pi_Y \leftarrow \mathcal{P}_{\text{DL}}(\text{pp}, Y, \alpha)$ and $\pi_D \leftarrow \mathcal{P}_{\text{DL}}(\text{pp}, D, \delta)$. Verification of π_{aff} checks both π_Y and π_D . The corresponding extractor outputs (α, δ) by running the two Π_{DL} extractors.

As noted earlier, we instantiate UC-NIZK(-AoK)s via the Fischlin transform in the gRO-CRP model. For efficiency we assume the optimized prover/verifier organization and batch-verification techniques of [20]. Accordingly, we omit low-level optimization details and refer to [20] for the optimized algorithms, while relying on [31] for the core transform security proof.

◆ **Fixed Fischlin parameters:** Let $(t, b, r, S) := (t(\lambda), b(\lambda), r(\lambda), S(\lambda))$ be the canonical parameter tuple fixed for this KeyBox profile (Definition 16 (p. 25)). Callers do not supply (and cannot influence) these values.

◆ **Static seed:** Let $\text{seed} \in \{0, 1\}^\lambda$ be the KeyBox-resident static PRF seed (Assumption 3 (p. 13)), provisioned independently of any resident signing share. Let $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^{\lceil \log_2 p \rceil + \lambda}$ be a secure pseudorandom function.

◆ **FS.Start(sid, K):** Check $K = \text{PubMap}(k)$; allocate a fresh handle $\mu_{\text{FS}} \in \text{slot}_{\text{fs}}$, where $\text{slot}_{\text{fs}} \in \{0, 1\}^\lambda$. For each $i \in [r]$, derive the prover nonce deterministically:

$$j_i := \text{PRF}(\text{seed}, \langle \text{"LinOS/nonce/v1"}, \text{sid}, K, i \rangle) \bmod p.$$

Set $J_i := j_i \mathcal{G}$, $a_i := J_i$ for $i \in [r]$; bind the tuple $(\mu_{\text{FS}} \mapsto (\text{sid}, K, \{j_i\}_{i \in [r]}, \{a_i\}_{i \in [r]}, t, b, r, S))$ and return $(\mu_{\text{FS}}, \mathbf{a})$, where $\mathbf{a} := (a_1, \dots, a_r)$.

◆ **FS.Prove(μ_{FS}):** If μ_{FS} is sealed, return $\perp$. Else, for each $i \in [r]$, the KeyBox runs an early-break rarity search: initialize $\hat{s}_i \leftarrow 2^b - 1$ and $(\hat{e}_i, \hat{z}_i) \leftarrow (\perp, \perp)$. For $e = 0, 1, \dots, 2^t - 1$ it internally computes

$$z \leftarrow j_i + e k \bmod p, \quad s \leftarrow H_b(\text{ctx}_{\text{KeyBox}}, \langle \text{sid}, K, \mathbf{a}, i, e, z \rangle).$$

If $s = 0$, it sets $(e_i, z_i) \leftarrow (e, z)$ and breaks; otherwise, if $s \leq \hat{s}_i$ it updates $\hat{s}_i \leftarrow s$ and $(\hat{e}_i, \hat{z}_i) \leftarrow (e, z)$. If the loop ends without $s = 0$, it sets $(e_i, z_i) \leftarrow (\hat{e}_i, \hat{z}_i)$. It discards all per- e temporary values other than the selected (e_i, z_i) , outputs the optimized-Fischlin proof $\pi_{\text{DL}} = ((a_i, e_i, z_i))_{i=1}^r$, and seals μ_{FS} . Subsequent calls on μ_{FS} return $\perp$.

◆ **FS.Verify(sid, K, π_{DL}):** Accept iff (i) $\forall i, z_i \mathcal{G} = a_i + e_i K$, and (ii) $\sum_{i=1}^r H_b(\text{ctx}_{\text{KeyBox}}, \langle \text{sid}, K, \mathbf{a}, i, e_i, z_i \rangle) \leq S$.

Fig. 6: LinOS-Fischlin API with profile-fixed parameters.

7 Star DKG (SDKG)

Fix Fischlin parameter functions $t = t(\lambda), b = b(\lambda), r = r(\lambda), S = S(\lambda)$ as in Definition 16 (p. 25). Whenever the protocol refers to t, b, r, S , it means their values at the current security parameter λ . Let H_{s32} denote the gRO-CRP H under context $\text{SDKG.s32} \in \text{Ctx}_{\text{np}}$, i.e., $H_{\text{s32}}(u) := H(\text{SDKG.s32}, u)$. All invocations of H_{s32} are on an encoded tuple, i.e., $H_{\text{s32}}(\langle \cdot \rangle)$.

For the sake of readability, we begin by providing our protocol, $\Psi_{\text{SDKG}}^{(3)}$, for $\Gamma_0 = \{\{P_1, P_2\}, \{P_1, P_3\}\}$ and then extend it to a 1+1-out-of- n SDKG scheme. Hence, we first present $\Psi_{\text{SDKG}}^{(3)}$ in the $(\mathcal{F}_{\text{KeyBox}}, \mathcal{F}_{\text{USV}}, \mathcal{F}_{\text{channel}}, \mathcal{F}_{\text{pub}})$ -hybrid and gRO-CRP models. Accordingly, the protocol description contains explicit invocations of the ideal certificate functionality $\mathcal{F}_{\text{USV}}$. The corresponding real-world protocol $\hat{\Psi}_{\text{SDKG}}^{(3)}$ is obtained by replacing each call to $\mathcal{F}_{\text{USV}}$ with an execution of its concrete realization Π_{USV} (Corollary 2 (p. 67)). In Section 8 (p. 55), Theorem 3 (p. 60) establishes UC security of the hybrid protocol $\Psi_{\text{SDKG}}^{(3)}$; Corollary 2 (p. 67) then compiles out $\mathcal{F}_{\text{USV}}$ via UC composition to obtain UC security of $\hat{\Psi}_{\text{SDKG}}^{(3)}$ in the $(\mathcal{F}_{\text{KeyBox}}, \mathcal{F}_{\text{channel}}, \mathcal{F}_{\text{pub}})$ -hybrid (and gRO-CRP) model.

Remark 17. We use three proof contexts in Ctx_{p} (all mutually disjoint) for domain separation:

- $\Pi_{\text{DL}}^{\text{UC}}$: UC-NIZK-AoK for DL-based consistency statements, instantiated under $\text{ctx}_{\text{UC}} \in \text{Ctx}_{\text{p}}$. This is the only proof type from which the UC proof extracts witnesses.
- $\Pi_{\text{DL}}^{\text{KeyBox}}$: the sealed one-shot DL prover inside a KeyBox, instantiated under the disjoint context $\text{ctx}_{\text{KeyBox}} \in \text{Ctx}_{\text{p}}$. These proofs are treated under ZK/simulation only; the UC proof never extracts from them.
- Π_{DLEQ} : the DLEQ NIZK-AoK embedded in USV certificates, instantiated under the disjoint context $\text{ctx}_{\text{DLEQ}} \in \text{Ctx}_{\text{p}}$.

In the UC simulations for $\mathcal{F}_{\text{USV}}$ and SDKG, the simulator never programs ctx_{DLEQ} .

Definition 20 (PRF for nonce derivation). Let $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^{\lceil \log_2 p \rceil + \lambda}$ be a keyed function family. We say PRF is a secure pseudorandom function if for every PPT distinguisher $\mathcal{D}$ that makes at most $Q = \text{poly}(\lambda)$ adaptive queries,

$$\left| \Pr_{K \leftarrow \mathcal{K}\{0, 1\}^\lambda} [\mathcal{D}^{\text{PRF}(K, \cdot)}(1^\lambda) = 1] - \Pr_{F \leftarrow \mathcal{F}\text{Func}} [\mathcal{D}^{F(\cdot)}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

where Func is the set of all functions from $\{0, 1\}^*$ to $\{0, 1\}^{\lceil \log_2 p \rceil + \lambda}$.

Fig. 6 (p. 40) defines LinOS (Linear One-Shot), a sealed, handle-bound prover for Schnorr DL that runs the optimized Fischlin transform inside the KeyBox . In LinOS , the PRF is keyed by the static seed seed (Assumption 3 (p. 13)), and the derived nonce is reduced modulo p :

$$j_i := \text{PRF}(\text{seed}, \langle \text{"LinOS/nonce/v1"}, \text{sid}, K, i \rangle) \bmod p.$$

Remark 18 (Injective encoding and domain separation for LinOS nonces). The PRF input in Fig. 6 (p. 40) is the injective tuple encoding $\langle \text{"LinOS/nonce/v1"}, \text{sid}, K, i \rangle$ under the paper’s fixed self-delimiting encoding $\langle \cdot \rangle$ (Section 2.2 (p. 9)). Because $\langle \cdot \rangle$ is injective and self-delimiting, distinct tuples $(\text{sid}, K, i) \neq (\text{sid}', K', i')$ map to distinct PRF inputs. The domain-separation tag "LinOS/nonce/v1" prevents collisions with any other use of seed within the KeyBox profile.

Session-identifier requirement: The rollback-robustness argument (Lemma 14 (p. 42)) relies on the property that distinct signing/proving instances use distinct sid values. In the SDKG protocol of this paper, each base-run session has a unique sid by construction, and LinOS is invoked at most once per (sid, K) pair. In LinOS , the statement being proved is $K = \text{PubMap}(k)$ (the public key bound to the resident share), and K is already part of the PRF input. Therefore, for the uses in this paper, distinct signing instances always produce distinct PRF inputs, and no additional message binding beyond (sid, K) is needed.

If LinOS were adapted for a setting in which the same (sid, K) could be used to produce proofs for different external messages m (e.g., as part of a threshold signing protocol), then the PRF input must additionally include a commitment to m —for instance, $\langle \text{"LinOS/nonce/v1"}, \text{sid}, K, i, H(m) \rangle$ —to prevent nonce reuse across different messages. This extension is straightforward but is not required by our SDKG construction.

In contrast to a naïve design that samples prover nonces $j_i \leftarrow \mathbb{Z}_p$ and stores them in mutable KeyBox state, LinOS derives each nonce deterministically as $j_i := \text{PRF}(\text{seed}, \langle \text{"LinOS/nonce/v1"}, \text{sid}, K, i \rangle) \bmod p$ from a static KeyBox -resident seed seed (Assumption 3 (p. 13)) and session-bound inputs. This deterministic derivation follows the design principle of RFC 6979 [56] and EdDSA [39] (deterministic nonces for (EC)DSA/Schnorr signatures) and of resettable zero-knowledge [16], adapted to the Fischlin-transform setting. LinOS ensures: (i) all per-challenge linear evaluations $j_i + ek$ remain internal; (ii) at most one (e_i, z_i) ever leaves the KeyBox per commitment a_i , enforced by sealing and state continuity; and (iii) all rare-structure oracle queries are issued internally under a dedicated gRO-CRP context. In the UC proof, LinOS proofs are treated only as $\Pi_{\text{DL}}^{\text{KeyBox}}$ proofs, whereas all protocol-consistency proofs that require straight-line extraction use $\Pi_{\text{DL}}^{\text{UC}}$ under a disjoint context. For corrupted parties, the simulator mediates gRO-CRP access. We get the following security semantics:

- The statement K is bound at FS.Start to the KeyBox ’s resident key (share); the host cannot request a proof for any other statement.
- For each commitment a_i produced by FS.Start , at most one (e_i, z_i) ever leaves the KeyBox (enforced by sealing), preventing an external party from obtaining two responses to the same a_i .
- No intermediate $z_{i,e}$ is exposed beyond the selected (e_i, z_i) ; hence an adversary cannot solve for k by computing the difference of two responses with the same a_i .
- The “one-shot” sealing of μ_{FS} assumes state continuity (Assumption 2 (p. 13)). Even if the one-shot seal is circumvented by a rollback of mutable KeyBox state, the deterministic nonce derivation (Lemma 14 (p. 42)) ensures that replaying the same (sid, K) regenerates the same nonces j_i and hence the same commitments and (given the same gRO-CRP oracle answers) the same proof transcript—no new information is obtained. For distinct sessions with different sid , the derived nonces are pseudorandomly independent under PRF security, so the probability of a commitment collision is negligible. Consequently, the catastrophic “one rollback reveals k ” path via Schnorr special soundness is eliminated, and key-opacity is preserved even under approximate state continuity. This hardening follows the established principle of deterministic nonce generation [56, 39] and resettable ZK [16]. Note that this does not fully close the ε_{sc} gap for all rollback-affected state; see Remark 20 (p. 43) for the precise scope.
- All gRO-CRP oracle calls $H_b(\text{ctx}, \cdot)$ made during FS.Prove are local to the KeyBox ITM and are not visible at the host/API boundary. The host observes only $(\mathbf{a}, \pi_{\text{DL}})$ and the public verification outcome.

When LinOS is run on a KeyBox-resident share stored in a local KeyBox slot $\mu \in \{0,1\}^*$ (with public key $K = \text{PubMap}(k)$), the interfaces are invoked via the KeyBox API:

$$(\mu_{\text{FS}}, \mathbf{a}) \leftarrow \mathcal{F}_{\text{KeyBox}}^{(P)}.\text{Use}(\mu, \text{FS.Start}, \langle \text{sid}, K \rangle), \quad \pi_{\text{DL}} \leftarrow \mathcal{F}_{\text{KeyBox}}^{(P)}.\text{Use}(\mu, \text{FS.Prove}, \mu_{\text{FS}}).$$

The verifier-side predicate $\text{FS.Verify}(\text{sid}, K, \pi_{\text{DL}})$ is public and is evaluated outside the KeyBox.

Lemma 13 (LinOS preserves key-opacity). *Work in the gRO-CRP-hybrid model. Assume the baseline KeyBox profile is key-opaque w.r.t. PubMap (Definition 2 (p. 11)), and extend the admissible operations by adding $\text{FS.Start}/\text{FS.Prove}/\text{FS.Verify}$ as in Fig. 6 (p. 40). Instantiate LinOS under a dedicated Fischlin proof context $\text{ctx}_{\text{KeyBox}} \in \text{Ctx}_{\text{P}}$ that is disjoint from all other contexts. Then the extended profile remains key-opaque w.r.t. PubMap .*

Proof Sketch. Fix any KeyBox slot holding secret k with public key $K = \text{PubMap}(k)$. Extend the slot-wise key-opacity simulator Sim_K (Remark 1 (p. 12)) to answer LinOS queries as follows.

Simulating FS.Start : Maintain a cache T keyed by (sid, K) whose entries store an accepting Fischlin transcript π_{DL} and its first-message tuple $\mathbf{a}$. On input $\text{FS.Start}(\text{sid}, K')$, if $K' \neq K$ output $\perp$. Otherwise sample a fresh handle μ_{FS} . If $(\text{sid}, K) \in \mathsf{T}$, retrieve $(\mathbf{a}, \pi_{\text{DL}}) \leftarrow \mathsf{T}[\text{sid}, K]$. Else run the Fischlin-based UC-NIZK simulator for the DL statement “ $\exists k : K = \text{PubMap}(k)$ ” in context $\text{ctx}_{\text{KeyBox}}$ to obtain an accepting proof $\pi_{\text{DL}} = ((a_i, e_i, z_i))_{i=1}^r$ (with $\mathbf{a} := (a_1, \dots, a_r)$), and set $\mathsf{T}[\text{sid}, K] \leftarrow (\mathbf{a}, \pi_{\text{DL}})$. This simulation may invoke SimProgram only in $\text{ctx}_{\text{KeyBox}}$. Return $(\mu_{\text{FS}}, \mathbf{a})$ and store π_{DL} under μ_{FS} .

Simulating FS.Prove : On input $\text{FS.Prove}(\mu_{\text{FS}})$, if μ_{FS} is unknown or already sealed output $\perp$; otherwise output the stored π_{DL} and mark μ_{FS} sealed, matching LinOS one-shot semantics.

Simulating FS.Verify : On input $\text{FS.Verify}(\text{sid}, K', \pi)$, output the deterministic verification result, which is simulatable without k .

Indistinguishability: In the real LinOS execution the prover nonces $j_1, \dots, j_r$ are derived deterministically via $\text{PRF}(\text{seed}, \cdot)$ (Fig. 6 (p. 40)). In particular, for fixed (sid, K) the first message $\mathbf{a}$ (and thus the full transcript released by FS.Prove) repeats across repeated calls. We argue indistinguishability in two steps:

Step (a): Replace all $\text{PRF}(\text{seed}, \cdot)$ evaluations with outputs of a truly random function F . By PRF security (Definition 20 (p. 40)) and secrecy of seed (Assumption 3 (p. 13)), this introduces at most $\text{negl}(\lambda)$ distinguishing advantage. After this replacement, for each fixed (sid, K) the tuple $(j_1, \dots, j_r)$ is (up to the negligible statistical distance of mod- p reduction) uniform over $\mathbb{Z}_p^r$ and independent across distinct inputs $\langle \text{"LinOS/nonce/v1"}, \text{sid}, K, i \rangle$; repeated evaluations on the same input repeat the same nonce, matching the real correlation across repeated FS.Start queries.

Step (b): In the resulting hybrid, for each distinct pair (sid, K) queried, the externally visible output is a Fischlin-based UC-NIZK transcript for the DL statement $K = \text{PubMap}(k)$ under context $\text{ctx}_{\text{KeyBox}}$ with truly random prover nonces (and replay on repeats of (sid, K)). By the computational ZK guarantee of the Fischlin transform in gRO-CRP (Lemma 7 (p. 28)), we can replace each such real transcript with a simulated one generated by the Fischlin simulator, while preserving replay behavior by caching the simulator’s output per (sid, K) as above (up to the simulator’s programming failure event). Combining Steps (a) and (b) yields the claim.

Simulator failure event: The only simulator failure mode is that a $\text{SimProgram}(\text{ctx}_{\text{KeyBox}}, x, y)$ attempt returns $\perp$ because the host/adversary pre-queried $(\text{ctx}_{\text{KeyBox}}, x)$. By Lemma 4 (p. 22), this pre-query event is negligible because the programmed inputs x include fresh high-entropy components. Finally, by gRO-CRP local-call semantics (Fig. 4 (p. 21)), the host/API boundary never observes the KeyBox’s internal oracle-query trace; only the released transcript must be simulated. Hence, adding LinOS preserves key-opacity. $\blacksquare$

Lemma 14 (Rollback robustness of LinOS commitments). *Work in the gRO-CRP-hybrid model. Consider the LinOS interface (Fig. 6 (p. 40)) with deterministic nonce derivation via PRF keyed by seed (Definition 20 (p. 40)). Assume that PRF is a secure pseudorandom function (Definition 20 (p. 40)), and that seed satisfies the seed integrity invariant (Assumption 3 (p. 13)). Let $\mathcal{A}$ be a PPT adversary that may:*

1. *invoke $\text{FS.Start}(\text{sid}, K)$ and $\text{FS.Prove}(\mu_{\text{FS}})$ via the KeyBox API;*
2. *roll back or reset the mutable operation state $\text{st}[\cdot]$ of the KeyBox to any prior value (violating state continuity for mutable state, but not altering seed or the resident key k);*

3. *make arbitrary gRO-CRP queries.*

Then $\mathcal{A}$ cannot obtain two accepting Fischlin transcripts $\pi = (a_i, e_i, z_i)_{i=1}^r$ and $\pi' = (a'_i, e'_i, z'_i)_{i=1}^r$ for the same (sid, K) with $a_j = a'_j$ and $e_j \neq e'_j$ for some j , except with probability negligible in λ .

Proof Sketch. We argue via a two-step hybrid.

Step 1: Suppose $\mathcal{A}$ rolls back the KeyBox’s mutable state and re-invokes $\text{FS.Start}(\text{sid}, K)$ with the same (sid, K) . Because `seed` is rollback-invariant (Assumption 3 (p. 13)) and k is unchanged, the PRF derivation

$$j_i = \text{PRF}(\text{seed}, \langle \text{"LinOS/nonce/v1"}, \text{sid}, K, i \rangle) \bmod p$$

produces the same nonces $j_1, \dots, j_r$ and hence the same commitments $a_1, \dots, a_r$. In FS.Prove , the rarity-search queries $H_b(\text{ctx}_{\text{KeyBox}}, \langle \text{sid}, K, \mathbf{a}, i, e, z_{i,e} \rangle)$ are deterministic functions of (j_i, k, H) ; since j_i and k are unchanged, and H is a (lazy-sampled) global random oracle whose table is not affected by KeyBox rollback, the rarity search produces the same selected (e_i, z_i) for each i . Therefore, the replayed proof transcript is identical to the original—no new information is obtained.

Step 2: Suppose $\mathcal{A}$ invokes $\text{FS.Start}(\text{sid}', K)$ with $\text{sid}' \neq \text{sid}$. We show that the commitments $a'_i = j'_i \mathcal{G}$ are independent of $a_i = j_i \mathcal{G}$ by a PRF-to-random replacement. Consider the hybrid experiment $\mathcal{D}_1$ in which all PRF outputs $\text{PRF}(\text{seed}, \cdot)$ are replaced by outputs of a truly random function F . By PRF security (Definition 20 (p. 40)) and secrecy of `seed` (Assumption 3 (p. 13)), $\mathcal{D}_0 \approx_c \mathcal{D}_1$. In $\mathcal{D}_1$, because $\langle \cdot \rangle$ is injective, $\langle \text{"LinOS/nonce/v1"}, \text{sid}, K, i \rangle \neq \langle \text{"LinOS/nonce/v1"}, \text{sid}', K, i' \rangle$ whenever $(\text{sid}, i) \neq (\text{sid}', i')$, so the derived nonces are independent uniform values. Hence, the probability that $a_j = a'_j$ for any j (i.e., a commitment collision $j_j \mathcal{G} = j'_j \mathcal{G}$) is at most $r^2/p = \text{negl}(\lambda)$. Without a shared commitment, Schnorr special-soundness extraction cannot be applied, so the adversary cannot recover k .

In either case (same or different `sid`), $\mathcal{A}$ cannot obtain two transcripts sharing a commitment a_j with distinct challenges $e_j \neq e'_j$ except with negligible probability. Therefore, the Schnorr witness-extraction attack via nonce reuse is thwarted. ■

Lemma 14 (p. 42) rules out Schnorr witness extraction on the resident share k via rollback-induced nonce reuse, except with negligible probability.

Remark 19 (Fischlin-transform compatibility). Because the repetition index i is an explicit input to the PRF derivation, the nonces $j_1, \dots, j_r$ within a single session remain mutually independent (under PRF security). Therefore, the Fischlin rarity-search procedure is unchanged: each repetition i independently searches over challenges $e \in \{0, \dots, 2^t - 1\}$ using its own independently derived j_i . The completeness, soundness, and knowledge-extraction analyses of the Fischlin transform (Definition 16 (p. 25), Lemma 6 (p. 27)) apply without modification. Only the *source* of prover randomness changes—from “sample uniformly and store in mutable state” to “derive on demand from a static seed”—while the statistical properties required by the transform (near-uniform, mutually independent nonces) are preserved.

Remark 20 (Scope of the rollback-robustness hardening). Lemma 14 (p. 42) is a surgical hardening that removes one specific catastrophic failure mode: the path from “single KeyBox rollback” to “full secret-key disclosure” via Schnorr nonce reuse / special-soundness extraction. It does not, by itself, close the broader ε_{sc} gap (Definition 4 (p. 14)) or address all rollback/state-continuity issues for other protocol state (e.g., monotonic counters, sealed records, session metadata, or the one-shot sealing flag μ_{FS}). A rollback that restores the mutable one-shot seal could still allow the adversary to obtain a second proof transcript; however, by Lemma 14 (p. 42), this second transcript is identical to the first, and therefore reveals no new information about k . General state-continuity remains an engineering requirement for other protocol-level invariants (e.g., preventing replay of registration or re-execution of protocol rounds), and ε_{sc} continues to parameterize those residual failure modes as before.

Remark 21 (Scope of un-hardened randomness inside USV.Cert and SDKG.Leafnit). The PRF-based nonce derivation of LinOS (Fig. 6 (p. 40)) is applied specifically to the interface whose rollback failure mode is catastrophic: reuse of a Schnorr commitment across two distinct challenges enables special-soundness extraction of the long-term resident share k . Two other KeyBox-internal randomized operations are not hardened in this way and remain covered by the general ε_{sc} accounting (Definition 4 (p. 14)):

◆ **SDKG.LeafInit(sid) (key-independent)**: sample $m_2 \leftarrow \mathbb{Z}_p^*$ and $b_2 \leftarrow \mathbb{Z}_p^*$; define $f_2(x) := m_2 + b_2x$. Compute $B_2 := b_2\mathcal{G}$ and $\sigma_{2,1} := f_2(2)$, $\sigma_{2,2} := f_2(3)$, $\sigma_{2,3} := f_2(1)$ in $\mathbb{Z}_p$. Compute $(C_2, \zeta_2) \leftarrow \text{Cert}(\text{pp}, m_2)$. Erase (m_2, b_2, f_2) and return $(C_2, \zeta_2, B_2, \sigma_{2,1}, \sigma_{2,2}, \sigma_{2,3})$.

Fig. 7: Key-independent leaf routine for hardened/minimal NXK deployments of SDKG.

1. **DLEQ prover randomness inside USV.Cert** (Section 5.2 (p. 32)): The USV certificate generator **Cert** invokes the UC-NIZK prover $\mathcal{P}_{\text{DLEQ}}$ for the Chaum–Pedersen relation using fresh randomness sampled from the KeyBox DRBG (both for the witness r and for the internal Fischlin nonces of $\mathcal{P}_{\text{DLEQ}}$). A rollback that resets the DRBG can therefore cause reuse or replay of this internal randomness, resulting in repeated certificates and/or in multiple certificates being generated from the same pre-rollback state. In our Fischlin-based instantiation (Definition 16 (p. 25)), re-executing $\mathcal{P}_{\text{DLEQ}}$ with the same randomness yields an identical transcript. However, in any alternative deployment in which the DLEQ prover could output two accepting Chaum–Pedersen transcripts for the same statement and first message but with different challenges (e.g., via an interactive DLEQ interface under rollback), special soundness would recover the witness r ; combined with the public ν in ζ , this recovers $m = \nu r$. This does not by itself expose the long-term signing share k , but may matter for applications that require confidentiality of ephemeral USV witnesses.
2. **Ephemeral sampling in SDKG.LeafInit**: The key-independent leaf routine samples $m_2 \leftarrow \mathbb{Z}_p^*$ and $b_2 \leftarrow \mathbb{Z}_p^*$ using ordinary KeyBox DRBG randomness. A rollback that resets the DRBG and re-invokes **SDKG.LeafInit** can produce different (m_2, b_2) and hence a different Round 1 tuple, leading to inconsistent protocol state if the rest of the session proceeds using the original values. As above, this is subsumed by ε_{sc} and does not by itself disclose k .

In summary, the PRF-based hardening is targeted at the single interface whose rollback failure mode is catastrophic, while other KeyBox-internal randomized operations remain under the general ε_{sc} accounting as before.

For the star access structure $\Gamma_0 := \{\{P_1, P_2\}, \{P_1, P_3\}\}$, our end-to-end claim is UC realization of an *NXK-star DKG* interface for Γ_0 : applications should see only a public key (or abort) and the induced installation of non-exportable long-term shares inside the parties’ KeyBoxes.

Reader Note 7.1: Application-visible functionality

The functionality exposed to applications is $\mathcal{F}_{\text{DKG}}^{\text{NXK}}$ (Definition 23 (p. 55)), which is obtained by wrapping the proof-oriented transcript-driven functionality $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)) as $\mathcal{F}_{\text{DKG}}^{\text{NXK}} = \mathcal{W}_{\text{DKG}} \circ \mathcal{F}_{\text{SDKG}}$. We prove UC realization for $\mathcal{F}_{\text{SDKG}}$ and then lift it to $\mathcal{F}_{\text{DKG}}^{\text{NXK}}$ via interface restriction (Lemma 17 (p. 56)). $\mathcal{F}_{\text{DKG}}^{\text{NXK}}$ exposes only:

- **DKG output**: upon successful completion, make a public key $K \in \mathbb{G}$ available to all parties (P_1 and P_2 receive K directly at finalization; P_3 receives K via the real protocol’s $\mathcal{F}_{\text{pub}}$ publications—preserved under UC emulation—and/or the registration payload from P_1); as usual in UC-DKG, no output is provided on abort or selective-abort.
- **NXK share installation**: install long-term shares into the parties’ local KeyBoxes (via $\mathcal{F}_{\text{KeyBox}}$), with no export interface for the underlying scalars or caller-invertible affine images.
- **Optional post-finalization registration (RDR)**: if invoked, output the corresponding **registered** event(s) and install the recovery-role share in the joining device’s KeyBox.

All transcript bookkeeping and the simulator-only **Program** hook in $\mathcal{F}_{\text{SDKG}}$ are not part of the application interface and are hidden by the wrapper $\mathcal{W}_{\text{DKG}}$ (Definition 23 (p. 55)).

We prove UC realization first for the proof-oriented functionality $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)), and then obtain the application-visible interface $\mathcal{F}_{\text{DKG}}^{\text{NXK}}$ by local interface restriction (Lemma 17 (p. 56)). In the protocol and definition below, $\sigma_{i,j}$ denote the scalar polynomial evaluations for SDKG and KeyBox installation/registration, respectively.

Definition 21 (SDKG KeyBox derivation routines). Fix public parameters $\text{pp} = (\mathbb{G}, p, \mathcal{G}, \mathcal{H})$ and let $\langle \cdot \rangle$ be a tuple encoding. All arithmetic below is in $\mathbb{Z}_p$.

- $g_{1,2}(1^\lambda, l)$: parse $l = \langle \sigma_{1,1}, \sigma_{2,1}, \sigma_{3,1} \rangle$ with each $\sigma_{i,j} \in \mathbb{Z}_p$. If parsing fails, output $\perp$. Set $x_1 := \sigma_{1,1} + \sigma_{2,1} + \sigma_{3,1}$ and output $k_{1,2} := 3x_1$.

- $g_{1,3}(1^\lambda, l)$: parse $l = \langle \sigma_{1,1}, \sigma_{2,1}, \sigma_{3,1}, \sigma_{1,3} \rangle$ with each component in $\mathbb{Z}_p$. If parsing fails, output $\perp$. Set $x_1 := \sigma_{1,1} + \sigma_{2,1} + \sigma_{3,1}$ and output $k_{1,3} := 2\sigma_{1,3} - x_1$.
- $g_2(1^\lambda, l)$: parse $l = \langle \sigma_{1,2}, \sigma_{2,2}, \sigma_{3,2} \rangle$ with each component in $\mathbb{Z}_p$. If parsing fails, output $\perp$. Set $x_2 := \sigma_{1,2} + \sigma_{2,2} + \sigma_{3,2}$ and output $k_2 := -2x_2$.
- $g_3^{(P_r, P_{sp}, \text{sid})}(1^\lambda, l)$ (parameterized by joiner P_r , sponsor P_{sp} , and session sid):
For the base case, set $P_r = P_3$ and $P_{sp} = P_2$. For the n -party extension, $P_r = P_i$ and $P_{sp} = P_j$ for arbitrary joiner P_i ($i \geq 3$) and sponsor $P_j \in \{2\} \cup \text{Reg}$. Parse l as either
 - (a) $\langle \sigma_{2,3}, \sigma_{3,2}, \sigma_{3,1}, K, K_{1,3} \rangle$ with $\sigma_{2,3}, \sigma_{3,2}, \sigma_{3,1} \in \mathbb{Z}_p$ and $K, K_{1,3} \in \mathbb{G}$, or
 - (b) $\langle \langle \text{ad}_{23}, \sigma_{2,3} \rangle, \langle \text{ad}_{32}, \sigma_{3,2} \rangle, \langle \text{ad}_{31}, \sigma_{3,1} \rangle, K, K_{1,3} \rangle$ where each $\text{ad} \in \{0, 1\}^*$ and each $\sigma \in \mathbb{Z}_p$. In case (b), additionally parse each ad_{xy} as

$$\text{ad}_{xy} = \langle \text{SDKG.reg}, \text{sid}', P'_s, P'_r, \text{tag} \rangle$$

and require the associated-data fields are mutually consistent and match the closure parameters: all three sid' fields equal sid , all three P'_r fields equal P_r , and $P'_s = P_{sp}$ with $\text{tag} = \text{k23}$ for ad_{23} , $P'_s = P_{sp}$ with $\text{tag} = \text{k32}$ for ad_{32} , and $P'_s = P_1$ with $\text{tag} = \text{k31}$ for ad_{31} .

- If parsing fails, including the case-(b) associated-data checks, output $\perp$. Compute $k_3 := 2 \cdot (\sigma_{2,3} + 2\sigma_{3,1} - \sigma_{3,2}) \bmod p$. Let $K_3 := k_3 \mathcal{G}$. If $K_{1,3} + K_3 \neq K$, output $\perp$, else output k_3 .
- For each role pair $(a, b) \in \{(\text{c}, \text{sp}), (\text{sp}, \text{c2}), (\text{sp2}, \text{c3})\}$ and corresponding concrete base-case instantiation $(i, j) \in \{(3, 1), (3, 2), (2, 3)\}$, define $g_{i,j}^{\text{reg}, (P_r, P_{sp}, \text{sid})}(1^\lambda, l)$ (parameterized by joiner P_r , sponsor P_{sp} , session sid) as: parse l as either
 - (a) $\langle \sigma_{i,j} \rangle$ with $\sigma_{i,j} \in \mathbb{Z}_p$; in this case output $k_{i,j}^{\text{reg}} := \sigma_{i,j}$, or
 - (b) $\langle \langle \text{ad}, \sigma_{i,j} \rangle \rangle$ with $\text{ad} \in \{0, 1\}^*$ and $\sigma_{i,j} \in \mathbb{Z}_p$. In this case, parse $\text{ad} = \langle \text{SDKG.reg}, \text{sid}', P'_s, P'_r, \text{tag} \rangle$ and require the fields match the closure parameters: $\text{sid}' = \text{sid}$, $P'_r = P_r$, and $\text{tag} = \text{k31}$ with $P'_s = P_1$ if $(i, j) = (3, 1)$, $\text{tag} = \text{k32}$ with $P'_s = P_{sp}$ if $(i, j) = (3, 2)$, and $\text{tag} = \text{k23}$ with $P'_s = P_{sp}$ if $(i, j) = (2, 3)$.
If the ad parse or associated-data check fails, output $\perp$; else output $k_{i,j}^{\text{reg}} := \sigma_{i,j}$.
- If parsing fails, output $\perp$. For the base case (P_3 joining with sponsor P_2), we write g_3 for $g_3^{(P_3, P_2, \text{sid})}$ and $g_{i,j}^{\text{reg}}$ for $g_{i,j}^{\text{reg}, (P_3, P_2, \text{sid})}$ when the closure parameters are clear from context.

Remark 22 (Associated-data fields vs. g-routine checks). In SDKG registration we use AEAD associated data strings of the form $\text{ad} = \langle \text{SDKG.reg}, \text{sid}, P_s, P_r, \text{tag} \rangle$. The AEAD layer (via $\text{SealToPeer}/\text{OpenFromPeer}$ in $\mathcal{F}_{\text{KeyBox}}$) binds the entire tuple $(\text{sid}, P_s, P_r, \text{tag})$ to the ciphertext: in Fig. 3 (p. 11), OpenFromPeer computes $s \leftarrow \text{Dec}_{\text{sk}_{\text{seal}}}(\text{ad}, c)$ and returns a handle that later resolves to $\langle \text{ad}, s \rangle$ only if the ciphertext authenticates under that exact ad .

Importantly, the protocol does not accept ad strings from the network. In Algorithm 1 (p. 51), the receiver P_3 deterministically recomputes the intended $\text{ad}_{31}, \text{ad}_{32}, \text{ad}_{23}$ from $(\text{sid}, P_s, P_r, \text{tag})$ and supplies them to OpenFromPeer . Therefore, when P_3 is honest, any cross-session/cross-peer mix-and-match attempt would require opening a ciphertext under an ad that does not match its sealing ad , which causes OpenFromPeer to return $\perp$. Hence, our derivation routines g_3 and $g_{i,j}^{\text{reg}}$ verify that the parsed associated-data tuple $(\text{sid}', P'_s, P'_r, \text{tag})$ matches the intended session/peers/slot. The tag component enforces *slot disjointness*, while the extra sid, P_s, P_r checks provide interface-level robustness if a buggy/adversarial host supplies inconsistent associated data to OpenFromPeer .

If the sponsor is corrupted during registration, it may send arbitrary ciphertexts. Any in-transit modification of a ciphertext (or cross-session/slot substitution) is rejected by OpenFromPeer due to AEAD integrity under the receiver-supplied associated data (Fig. 3 (p. 11) and Algorithm 1 (p. 51)). Moreover, even if a corrupted sponsor generates fresh ciphertexts under the correct associated data but encrypting incorrect scalars, the joiner's recovery-share derivation g_3 uniquely determines k_3 via $K_{1,3} + K_3 = K$ where $K_3 = k_3 \mathcal{G}$ (Definition 21 (p. 44)). Because g_3 constrains only the linear combination $\sigma_{2,3} - \sigma_{3,2}$ (not the individual values), the sponsor may shift both slots by a common offset without affecting k_3 . Hence, the sponsor can at most force an abort (denial of registration) or shift the per-device sponsor-state slots, but cannot cause installation of a recovery share inconsistent with K .

Fix the KeyBox ideal functionality $\mathcal{F}_{\text{KeyBox}}$ (Fig. 3 (p. 11)) and the LinOS interface (Fig. 6 (p. 40)). We fix the admissible KeyBox profile $(\chi_{\text{adm}}^{\text{SDKG}}, \mathcal{F}_{\text{adm}}^{\text{SDKG}}, \mathcal{F}_{\text{KI}}^{\text{SDKG}}, F^{\text{SDKG}})$ (Definition 3 (p. 12)) as follows: the only derivation routines

invoked via **Load** are $\chi_{\text{adm}}^{\text{SDKG}} := \{g_{1,2}, g_{1,3}, g_2, g_3, g_{3,1}^{\text{reg}}, g_{3,2}^{\text{reg}}, g_{2,3}^{\text{reg}}\}$. The only operations invoked via **Use** are

$$\begin{aligned}\mathcal{F}_{\text{adm}}^{\text{SDKG}} &:= \{\text{GetPub}, \text{SDKG.LeafInit}, \text{USV.Cert}, \text{FS.Start}, \text{FS.Prove}, \text{SealToPeer}, \text{OpenFromPeer}\}, \\ \mathcal{F}_{\text{KI}}^{\text{SDKG}} &:= \{\text{OpenFromPeer}, \text{USV.Cert}, \text{SDKG.LeafInit}\},\end{aligned}$$

where FS.* are as in Fig. 6 (p. 40) and $\text{SealToPeer}/\text{OpenFromPeer}$ are the KeyBox-to-KeyBox sealing operations specified as part of $\mathcal{F}_{\text{KeyBox}}$ in Fig. 3 (p. 11). Recall that the predicate $\text{FS.Verify}(\text{sid}, K, \pi)$ is public/deterministic and is evaluated outside the KeyBox. We fix the profile’s family map F^{SDKG} by identifying the LinOS start/prove pair:

$$F^{\text{SDKG}}(\text{FS.Start}) = F^{\text{SDKG}}(\text{FS.Prove}) =: \text{FS}, \quad F^{\text{SDKG}}(f) = f \text{ for all other } f \in \mathcal{F}_{\text{adm}}^{\text{SDKG}}.$$

Note 4. The profile intentionally omits any operation for producing the UC-extractable consistency AoKs (e.g., $\Pi_{\text{DL}}^{\text{UC}} / \mathcal{R}_{\text{aff}}$) inside the KeyBox: those proofs are generated by the host/party ITM so that the simulator can log the prover’s gRO-CRP queries for straight-line extraction.

7.1 The Protocol

Our $\Psi_{\text{SDKG}}^{(3)}$ protocol is inspired by the DKG from [4]. Specifically, the high-level idea of mapping polynomial evaluations into compatible shares is shared by the two solutions. However, the settings and techniques differ: [4] constructs a $(2, 3)$ threshold ECDSA scheme with an offline party in a standalone, game-based model, and relies on Feldman VSS, Paillier-based MtA/MtAwc, and interactive ZK proofs over freely manipulable shares. By contrast, our scheme targets a star-shaped access structure in the NXK model. We obtain UC security in this setting by combining USV, NXK, and UC-NIZK-AoKs to replace the role of VSS and homomorphic share manipulations in [4].

Placement / profile convention. The protocol steps below are written at the level of algebraic relations (e.g., “sample m_2 and set $f_2(x) = m_2 + b_2x$ ”) and should not be read as fixing where a scalar is materialized. In the minimal/hardened NXK profile that motivates USV (Section 8.2 (p. 57)), the leaf’s Round 1 linear-polynomial setup is realized by the *key-independent* KeyBox operation $\text{SDKG.LeafInit} \in \mathcal{F}_{\text{KI}}^{\text{SDKG}}$ (Fig. 7 (p. 44)), which samples m_2 and b_2 inside the KeyBox boundary, outputs only $(C_2, \zeta_2, B_2, \sigma_{2,1}, \sigma_{2,2}, \sigma_{2,3})$, and erases (m_2, b_2) as named variables before returning. Note that m_2 remains trivially recoverable from the returned σ -values (e.g., $m_2 = 2\sigma_{2,3} - \sigma_{2,1}$); however, these values are NXK-restricted transient host state (Remark 7 (p. 16)), not adversary-visible, and are securely erased after their last use. No generic “export m_2 ” or “export $m_2\mathcal{G}$ ” interface is assumed. If, instead, one permits a transient-host-RAM deployment in which m_2 is sampled/held outside the KeyBox as NXK-restricted material with secure erasure, then one may simplify Round 1 by publishing $M_2 := m_2\mathcal{G}$ directly and omitting the USV instance; we keep USV so the same transcript format covers the hardened profile in which the adversary-visible transcript must still deterministically fix M_2 .

Even in the minimal (KeyBox-hardened long-term-share) profile, SDKG.LeafInit returns $\sigma_{2,1}, \sigma_{2,2}, \sigma_{2,3}$ to the host. These values are share-deriving and therefore NXK-restricted (Remark 7 (p. 16)); they are held only as transient host state needed to (i) generate the UC-context consistency AoKs and (ii) invoke the internal $\mathcal{F}_{\text{KeyBox}}.$ **Load** calls, and are then securely erased. This is intentional: the UC-context AoKs are host-generated so that the simulator can observe the prover’s gRO-CRP query log (Remark 4 (p. 12)).

Reconciliation with implementation guidance. The implementation note in Section 2.6 (p. 17) instructs the profile adapter to “forbid share-deriving outputs.” This refers to vendor-API operations that would export caller-invertible affine images of an *already-installed* resident share. It does not apply to SDKG.LeafInit , which is a key-independent operation ($\text{SDKG.LeafInit} \in \mathcal{F}_{\text{KI}}^{\text{SDKG}}$) that samples fresh scalars internally: its outputs become share-deriving only prospectively, after the subsequent **Load** calls install the corresponding long-term shares, and are erased immediately thereafter.

Next, we detail our $\Psi_{\text{SDKG}}^{(3)}$ protocol. Session setup designates P_2 as the initiating leaf (primary role). All computations are performed in $\mathbb{Z}_p$. Let $\text{pp} := (\mathbb{G}, p, \mathcal{G}, \mathcal{H})$ be the set of public parameters. Each step of the protocol is atomic for the local state. Detailed description follows:

Round 1 $P_2 \rightarrow P_1$:

- Leaf setup (hardened/minimal profile): P_2 obtains its Round 1 values by invoking the key-independent KeyBox operation SDKG.LeafInit :

$$(C_2, \zeta_2, B_2, \sigma_{2,1}, \sigma_{2,2}, \sigma_{2,3}) \leftarrow \mathcal{F}_{\text{KeyBox}}^{(2)}.\text{Use}(\langle \text{sid}, \text{ki} \rangle, \text{SDKG.LeafInit}, \langle \text{sid} \rangle).$$

Define the leaf's transcript-defined group element $M_2 := \text{Open}_M(\text{pp}, C_2, \zeta_2)$ (by USV correctness, $M_2 = m_2\mathcal{G}$ for the internal m_2 sampled inside SDKG.LeafInit). In the transient-host-RAM deployment, this step may equivalently be implemented by sampling (m_2, b_2) in host state, setting $f_2(x) = m_2 + b_2x$, and computing the same outputs, treating all σ -scalars as NXK-restricted and securely erased after their last use.

- For a fresh commitment identifier cid_2 , P_2 sends $(\text{Commit}, \text{sid}, \text{cid}_2, P_1, C_2, \zeta_2)$ to $\mathcal{F}_{\text{USV}}$ and receives receipt digest d from $\mathcal{F}_{\text{USV}}$.
- P_2 generates $\sigma_{3,2} \leftarrow \mathbb{Z}_p$; computes $h_{3,2} := H_{\text{s32}}(\langle \text{sid}, \text{cid}_2, \sigma_{3,2}\mathcal{G} \rangle)$ and sends $(C_2, \zeta_2, B_2, h_{3,2}, \sigma_{2,1}, \text{sid}, \text{cid}_2, d)$ to P_1 .

Round 2 $P_1 \rightarrow P_2$:

- P_1 verifies that sid and cid_2 are fresh, calls $\mathcal{F}_{\text{USV}}.\text{Verify}(\text{sid}, \text{cid}_2, P_2, C_2, \zeta_2)$, and requires it returns 1; else, P_1 aborts.
- P_1 computes $M_2 \leftarrow \text{Open}_M(\text{pp}, C_2, \zeta_2)$ (abort if $M_2 = \perp$) and checks that

$$d = H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}_2, P_2, P_1, C_2, M_2 \rangle).$$

It aborts if the check fails.

- P_1 parses $\zeta_2 = (\nu_2, v_2, \pi_{\text{DLEQ}_2})$, checks

$$(i) \nu_2 \neq -1 \bmod p \quad \text{and} \quad (ii) \sigma_{2,1}\mathcal{G} - 2B_2 = M_2.$$

It aborts if any verification fails.

- P_1 samples $m_1 \leftarrow \mathbb{Z}_p^*$ and $b_1, \sigma_{3,1} \leftarrow \mathbb{Z}_p$; sets $f_1(x) := m_1 + b_1x$, and computes

$$M_1 := m_1\mathcal{G}, B_1 := b_1\mathcal{G}, \sigma_{1,1} := f_1(2), \sigma_{1,2} := f_1(3), \sigma_{1,3} := f_1(1), X_1 := \sigma_{1,1}\mathcal{G} + \sigma_{2,1}\mathcal{G} + \sigma_{3,1}\mathcal{G}.$$

Equivalently, $X_1 = x_1\mathcal{G}$ for the conceptual scalar $x_1 := \sigma_{1,1} + \sigma_{2,1} + \sigma_{3,1}$, which is never materialized in host memory. Then, deletes m_1, b_1 , and f_1 .

- P_1 sends $(\text{sid}, X_1, M_1, B_1, \sigma_{1,2}, \pi_{\text{aff}_1})$ to P_2 , where π_{aff_1} is a UC-NIZK-AoK for the affine relation $\mathcal{R}_{\text{aff}}$ on statement

$$(X_1, \gamma_1, M_1, B_1, \Delta_1) \quad \text{with} \quad \gamma_1 := 2, \Delta_1 := \sigma_{2,1}\mathcal{G},$$

using witness[‡] $(\alpha_1, \delta_1) := (\sigma_{1,1}, \sigma_{3,1})$, i.e., P_1 generates UC-NIZK-AoKs for the DL relation $\mathcal{R}_{\text{DL}}$:

$$\pi_{Y_1} \leftarrow \mathcal{P}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff1.Y}, Y_1 \rangle, \alpha_1) \quad \text{with witness } \alpha_1 := \sigma_{1,1};$$

$$\pi_{D_1} \leftarrow \mathcal{P}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff1.D}, D_1 \rangle, \delta_1) \quad \text{with witness } \delta_1 := \sigma_{3,1},$$

then $\pi_{\text{aff}_1} = (\pi_{Y_1}, \pi_{D_1})$.

Round 3 $P_2 \rightarrow P_1$:

- P_2 verifies that $\sigma_{1,2}\mathcal{G} = M_1 + 3B_1$, and aborts if the check fails.
- P_2 computes $Y_1 = M_1 + 2B_1, D_1 = X_1 - \sigma_{2,1}\mathcal{G} - Y_1$, and accepts iff

$$\mathcal{V}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff1.Y}, Y_1 \rangle, \pi_{Y_1}) = 1 \quad \wedge \quad \mathcal{V}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff1.D}, D_1 \rangle, \pi_{D_1}) = 1$$

- P_2 computes the public group element $X_2 := \sigma_{1,2}\mathcal{G} + \sigma_{2,2}\mathcal{G} + \sigma_{3,2}\mathcal{G}$. Equivalently, $X_2 = x_2\mathcal{G}$ for the conceptual scalar $x_2 := \sigma_{1,2} + \sigma_{2,2} + \sigma_{3,2}$, which is never materialized in host memory. Compute $M_2 \leftarrow \text{Open}_M(\text{pp}, C_2, \zeta_2)$; define $Y_2 := M_2 + 3B_2$ and $D_2 := X_2 - \sigma_{1,2}\mathcal{G} - Y_2$.

[‡] The witnesses (α_i, δ_i) are σ -values and hence share-deriving material (Remark 7 (p. 16)). They are held only transiently for UC-proof generation and the subsequent KeyBox installation steps, then securely erased.

- P_2 generates two UC-NIZK-AoKs for the discrete-log relation $\mathcal{R}_{\text{DL}}$:

$$\begin{aligned}\pi_{Y_2} &\leftarrow \mathcal{P}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff2.Y}, Y_2 \rangle, \alpha_2) \text{ with witness } \alpha_2 := \sigma_{2,2}; \\ \pi_{D_2} &\leftarrow \mathcal{P}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff2.D}, D_2 \rangle, \delta_2) \text{ with witness } \delta_2 := \sigma_{3,2}.\end{aligned}$$

Equivalently, $\pi_{\text{aff}_2} := (\pi_{Y_2}, \pi_{D_2})$ is a UC-NIZK-AoK for the affine relation $\mathcal{R}_{\text{aff}}$ on statement $(X_2, \gamma_2, M_2, B_2, \Delta_2)$ with $\gamma_2 := 3$ and $\Delta_2 := \sigma_{1,2}\mathcal{G}$.

- P_2 computes the public key $K = 3X_1 - 2X_2$, and sends $(X_2, \pi_{\text{aff}_2}, K)$ to P_1 . Additionally, P_2 publishes K via the authenticated public broadcast functionality: it sends $(\text{Publish}, \text{sid}, K)$ to $\mathcal{F}_{\text{pub}}$.

Verification by P_1 : Parse $(X_2, \pi_{\text{aff}_2}, K)$ as $(X_2, (\pi_{Y_2}, \pi_{D_2}), K_{\text{rec}})$ and compute

$$M_2 \leftarrow \text{Open}_M(\text{pp}, C_2, \zeta_2), \quad Y_2 := M_2 + 3B_2, \quad D_2 := X_2 - \sigma_{1,2}\mathcal{G} - Y_2, \quad K = 3X_1 - 2X_2,$$

and accepts iff:

$$\begin{aligned}\mathcal{V}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff2.Y}, Y_2 \rangle, \pi_{Y_2}) &= 1 \wedge \\ \mathcal{V}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff2.D}, D_2 \rangle, \pi_{D_2}) &= 1 \wedge \\ h_{3,2} &= H_{\text{s32}}(\langle \text{sid}, \text{cid}_2, D_2 \rangle) \wedge \\ K &= K_{\text{rec}}.\end{aligned}$$

Post-accept public-key confirmation. If P_1 accepts the transcript above, P_1 also publishes the verified key via $\mathcal{F}_{\text{pub}}$: it sends $(\text{Publish}, \text{sid}, K)$ to $\mathcal{F}_{\text{pub}}$. This ensures that P_3 (and any other non-base-run party) can learn K from an honest source regardless of P_2 's behavior, binding the privately verified key to the public broadcast. In the protocol specification, P_3 obtains its reference K from the $\mathcal{F}_{\text{pub}}$ publications for sid as follows: P_3 requires that both P_1 's and P_2 's authenticated $\mathcal{F}_{\text{pub}}$ publications for sid agree; if P_3 receives $(\text{Recv}, \text{sid}, P_1, K^{(1)})$ and $(\text{Recv}, \text{sid}, P_2, K^{(2)})$ from $\mathcal{F}_{\text{pub}}$, it sets $K := K^{(1)}$ only if $K^{(1)} = K^{(2)}$, and aborts registration otherwise. This ensures that a corrupted sender alone cannot install a shifted key in the honest joiner's KeyBox; the check $K^{(1)} = K^{(2)}$ enforces consistency with the honest sender's publication.

Post-accept KeyBox installation. Each honest party finalizes by installing its long-term NXK shares and the retained registration scalars inside its local KeyBox via Load calls over the internal channel to $\mathcal{F}_{\text{KeyBox}}^{(i)}$, as soon as it has locally verified transcript consistency:

- P_2 installs immediately after completing its Round 3 verification and sending $(X_2, \pi_{\text{aff}_2}, K)$ to P_1 (within the same activation). At this point, P_2 has verified P_1 's affine AoKs and holds all required σ -scalars. P_2 invokes: $\mathcal{F}_{\text{KeyBox}}^{(2)}.\text{Load}(\langle \text{sid}, \text{k2} \rangle, g_2, \langle \sigma_{1,2}, \sigma_{2,2}, \sigma_{3,2} \rangle), \mathcal{F}_{\text{KeyBox}}^{(2)}.\text{Load}(\langle \text{sid}, \text{k32} \rangle, g_{3,2}^{\text{reg}}, \langle \sigma_{3,2} \rangle), \mathcal{F}_{\text{KeyBox}}^{(2)}.\text{Load}(\langle \text{sid}, \text{k23} \rangle, g_{2,3}^{\text{reg}}, \langle \sigma_{2,3} \rangle).$
- P_1 installs after receiving P_2 's Round 3 message and verifying the full transcript via Acc_{SDKG} (Algorithm 2 (p. 52)). P_1 invokes: $\mathcal{F}_{\text{KeyBox}}^{(1)}.\text{Load}(\langle \text{sid}, \text{k12} \rangle, g_{1,2}, \langle \sigma_{1,1}, \sigma_{2,1}, \sigma_{3,1} \rangle), \mathcal{F}_{\text{KeyBox}}^{(1)}.\text{Load}(\langle \text{sid}, \text{k13} \rangle, g_{1,3}, \langle \sigma_{1,1}, \sigma_{2,1}, \sigma_{3,1}, \sigma_{1,3} \rangle), \mathcal{F}_{\text{KeyBox}}^{(1)}.\text{Load}(\langle \text{sid}, \text{k31} \rangle, g_{3,1}^{\text{reg}}, \langle \sigma_{3,1} \rangle).$ P_1 additionally publishes the verified key via $\mathcal{F}_{\text{pub}}$ at this point.

The two parties install at different adversary-scheduled times; $\mathcal{F}_{\text{SDKG}}$'s atomic TryFinalize models the logical conjunction. If P_1 rejects, P_2 's installed shares are inert under NXK (unusable without P_1 's cooperation, since the access structure requires P_1 in every authorized pair).

On exposure of share-deriving material. The scalars $\sigma_{i,j}$ are protocol-internal and share-deriving in the sense of Remark 7 (p. 16): learning the appropriate tuple(s) of σ -values suffices to recompute a party's installed long-term share via the public derivation routines (Definition 21 (p. 44)). Accordingly, σ -values are never placed in the adversary-visible transcript: whenever they are transmitted, they are sent only over $\mathcal{F}_{\text{channel}}$, so the adversary learns at most the

explicit leakage modeled by $\mathcal{F}_{\text{channel}}$, unless it corrupts an endpoint. Beyond transport confidentiality, the protocol enforces an explicit erasure discipline compatible with adaptive corruptions with secure erasures (Definition 6 (p. 15)): each honest party keeps share-deriving scalars in host memory only until their last use, and then securely erases them. Concretely, the only times share-deriving material is intentionally fed into long-term state are the internal $\mathcal{F}_{\text{KeyBox}}.\text{Load}$ calls that install signing shares and the three registration scalars used for RDR; immediately after the corresponding Load calls return, the host erases the consumed values. Per party, (i) ephemerals erased immediately, (ii) the precise σ -tuples passed into $\mathcal{F}_{\text{KeyBox}}.\text{Load}$ and then erased, and (iii) the values retained only inside KeyBox slots $\langle \text{sid}, \text{k31} \rangle, \langle \text{sid}, \text{k32} \rangle, \langle \text{sid}, \text{k23} \rangle$ for later use via $\text{SealToPeer}/\text{OpenFromPeer}$ (Algorithm 1 (p. 51)). Optionally, to avoid any exposure of ephemeral scalars in host memory, the KeyBox can be extended with a small non-exporting interface that (i) samples ephemerals internally and (ii) performs the required group/field operations on opaque handles and, for RDR, seals payloads TEE-to-TEE via $\text{SealToPeer}/\text{OpenFromPeer}$; more details in Appendix A (p. 74).

Erasure discipline (adaptive corruptions). All local steps in the base run are atomic w.r.t. adaptive corruptions: whenever an honest party outputs an outgoing message (including one carrying a UC-context NIZK), it immediately performs the explicit secure erasures listed below before yielding control to the adversary/scheduler. Consequently, any corruption that occurs after the message is emitted reveals none of the erased values (Definition 6 (p. 15)).

- P_1 (Round 2): After computing $\pi_{\text{aff}_1} = (\pi_{Y_1}, \pi_{D_1})$ and sending $(\text{sid}, X_1, M_1, B_1, \sigma_{1,2}, \pi_{\text{aff}_1})$, P_1 securely erases all randomness and intermediate state used by the UC-NIZK prover(s). It retains only the σ -values needed for the later $\mathcal{F}_{\text{KeyBox}}^{(1)}.\text{Load}(\cdot)$ calls.
- P_2 (Round 3): After computing $\pi_{\text{aff}_2} = (\pi_{Y_2}, \pi_{D_2})$ and sending $(X_2, \pi_{\text{aff}_2}, K)$, P_2 securely erases all UC-NIZK prover randomness and Fischlin prover scratch state for π_{Y_2} and π_{D_2} . This includes per-trial rarity-search temporaries, which need not be retained beyond the current iteration under a streaming implementation, and retains only the σ -values needed for the later $\mathcal{F}_{\text{KeyBox}}^{(2)}.\text{Load}(\cdot)$ calls.
- All parties (post-install): Immediately after the final KeyBox installation step returns, each honest party securely erases all share-deriving scalars that were consumed by its Load calls (cf. Remark 7 (p. 16)). The only share-derived values retained after this point are the NXK-confined KeyBox slots themselves.

In particular, after an honest party has sent its protocol messages, an adaptive corruption reveals at most the retained σ -values (until installation) and never reveals any per-proof randomness beyond what is already encoded in the public proof strings.

Secure erasure in practice (non-normative). Our UC execution model uses the standard idealization of adaptive corruption with secure erasures: once the protocol explicitly erases a buffer, a later corruption reveals only the remaining (non-erased) local state (Definition 6 (p. 15)). We do not claim that commodity host platforms provide this property as a default. Rather, the intended interpretation is that NXK-restricted host-side material (including UC-context AoK witnesses and the prover’s ephemeral randomness/scratch state) is handled in a hardened process boundary and is engineered to (i) never reach persistent storage (swap/hibernation, crash dumps, logs), and (ii) be overwritten promptly with compiler-resistant zeroization primitives.

The optimized Fischlin prover need not store a large rarity-search “trace” in memory: for each repetition it can maintain only the current best candidate and overwrite per-challenge temporaries immediately, so the long-lived working set is $O(1)$ scalars/group elements per repetition. Operational mitigations commonly used to approximate the erasure assumption include locking sensitive pages to avoid paging, disabling core dumps, and avoiding runtimes that may transparently copy or intern sensitive buffers. If such controls cannot be ensured, then the host should be treated as effectively “always-on” adversarial (outside our corruption-with-erasures model), or the design should move witness-bearing computation into a hardened boundary with an explicit interface that supports the required simulation/extraction view.

Hardened deployment: what runs where (KeyBox vs. host). Our security arguments rely on a precise boundary between computations that may run inside a state-continuous KeyBox (or an enclave-resident profile adapter that is part of the same trusted boundary) and computations that must remain outside on the host. By “hardened” we mean

Component	Runs in	Why this placement is necessary
Long-term share generation/storage; Load derivations; GetPub ; SealToPeer / OpenFromPeer	KeyBox	Enforces NXX non-exportability and key-opacity: long-term shares and any caller-invertible affine images never cross the KeyBox boundary; only restricted API outputs are visible.
NXX-restricted witnesses for UC-context consistency AoKs	Host	Must remain outside to enable oracle-log-based straight-line extraction for Fischlin-based UC-NIZK-AoKs. These scalars are treated as NXX-restricted material and are securely erased after proof generation and Load .
USV certificate generation $\text{Cert}(\text{pp}, m)$	KeyBox	Cert is implemented inside the hardened boundary and only (C, ζ) is exported.
USV verification/opening $\mathcal{V}_{\text{cert}}, \text{Derive}, \text{Open}_M$ and receipt binding $H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}, P_s, P_r, C, M \rangle)$ where $M := \text{Open}_M(\text{pp}, C, \zeta)$	Host	These are public deterministic checks/derivations and must be transcript-defined for verifiers and the UC simulator; no secrets are required. Receipt binding is in a non-programmable context $\text{USV.rcpt} \in \text{Ctx}_{\text{np}}$.
UC-context consistency AoKs	Host	Must be outside to enable straight-line extraction from corrupted-party proofs by inspecting the adversary's gRO-CRP query log.
KeyBox-resident one-shot DL proofs	KeyBox	Used only under ZK/simulation; sealing targets one-shot responses (Assumption 2 (p. 13)), and deterministic nonce derivation ensures that even under rollback an attacker cannot obtain two different responses for the same commitment (Lemma 14 (p. 42)).
gRO-CRP programming SimProgram	Simulator only	Only the ideal-world simulator programs in contexts in Ctx_{p} ; parties and KeyBoxes never program H .

Table 4: Threat/placement summary for hardened SDKG deployments (host vs. KeyBox).

that long-term shares are confined to the KeyBox and remain API-non-exportable even if the host is later corrupted (Definition 6 (p. 15)). We still permit NXX-restricted ephemerals (including witnesses for UC-context AoKs) to exist transiently in host RAM during an atomic local step and to be protected only by secure erasures against adaptive corruptions; we do not aim to protect against an “always-on” RAM adversary on an otherwise honest host. The key distinction below is extractability: proofs that the UC simulator must extract from (via oracle-log inspection) cannot be generated inside a KeyBox, because gRO-CRP has local-call semantics and the simulator does not observe KeyBox-internal **Query** traces (Remark 10 (p. 21) and Definition 10 (p. 21)). Table 4 (p. 50) summarizes the intended placement in hardened deployments and the reason each placement is necessary. This placement is not merely an implementation preference: it is required by the combination of (i) state continuity (no rewinding/forking inside the KeyBox; Assumption 2 (p. 13)) and (ii) oracle-log-based straight-line extraction for Fischlin-based UC-NIZK-AoKs (Definition 11 (p. 24) and Remark 10 (p. 21)). If one wishes to move UC-extractable AoKs into a hardened boundary, the model must be strengthened to expose an extractable interface (e.g., a simulator-visible oracle-log/receipt interface) or one must switch to a proof mechanism whose UC extraction does not rely on observing the prover’s gRO-CRP queries.

Definition 22 (Transcript-derived $K_{1,3}$). In any accepted base-run transcript with $\mathsf{T}_2 = (X_1, M_1, B_1, \sigma_{1,2}, \pi_{\text{aff}_1})$, define the public metadata point

$$K_{1,3} := \text{Derive}_{K_{1,3}}(\mathsf{T}_2) := 2(M_1 + B_1) - X_1 \in \mathbb{G}.$$

Equivalently, $K_{1,3} = \text{PubMap}(k_{1,3})$ for $k_{1,3} := 2\sigma_{1,3} - x_1$, but the protocol uses only the public derivation above and never invokes **GetPub** on slot $\langle \text{sid}, k_{13} \rangle$.

The one-shot registration protocol in Algorithm 1 (p. 51) is modeled in $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)) by (i) leaking only the lengths of the two registration messages, denoted by $\ell_{\text{reg},1}$ for the $P_1 \rightarrow P_3$ message and $\ell_{\text{reg},2}$ for the sponsor-to- P_3 message, and (ii) installing the corresponding recovery-role share and sponsor-state slots inside $\mathcal{F}_{\text{KeyBox}}^{(3)}$ upon successful

Algorithm 1: *One-shot registration of recovery device P_3 .*

Input: sid , public K for this session (obtained by P_3 from the authenticated $\mathcal{F}_{\text{pub}}$ publications for sid : P_3 requires that both P_1 's and P_2 's $\mathcal{F}_{\text{pub}}$ publications agree, i.e., P_3 receives $(\text{Recv}, \text{sid}, P_1, K^{(1)})$ and $(\text{Recv}, \text{sid}, P_2, K^{(2)})$ from $\mathcal{F}_{\text{pub}}$ and sets $K := K^{(1)}$ only if $K^{(1)} = K^{(2)}$, aborting otherwise), and parties P_1, P_2 with sealed registration scalars installed inside their respective KeyBoxes—i.e., $\langle \text{sid}, \mathbf{k13} \rangle, \langle \text{sid}, \mathbf{k31} \rangle$ for P_1 and $\langle \text{sid}, \mathbf{k32} \rangle, \langle \text{sid}, \mathbf{k23} \rangle$ for P_2 . P_1 and P_2 (along with any already-registered parties/devices) additionally hold the public session metadata point $K_{1,3}$ derived from the base-run transcript (Definition 22 (p. 50)).

Associated data. P_3 ignores any associated-data strings received from the network. Else, it recomputes the following locally from $(\text{sid}, P_s, P_r, \text{tag})$ and supplies them to **OpenFromPeer**:

$$\text{ad}_{31} := \langle \text{SDKG.reg}, \text{sid}, P_1, P_3, \mathbf{k31} \rangle, \quad \text{ad}_{32} := \langle \text{SDKG.reg}, \text{sid}, P_2, P_3, \mathbf{k32} \rangle, \quad \text{ad}_{23} := \langle \text{SDKG.reg}, \text{sid}, P_2, P_3, \mathbf{k23} \rangle.$$

P_1 retrieves the session metadata $K_{1,3}$; computes $\varpi_1 \leftarrow \mathcal{F}_{\text{KeyBox}}^{(1)}.\text{Use}(\langle \text{sid}, \mathbf{k31} \rangle, \text{SealToPeer}, \langle P_3, \text{ad}_{31} \rangle)$, and sends $(\text{sid}, \varpi_1, K_{1,3}, K)$.

P_2 uses the session metadata $K_{1,3}$ and computes two ciphertexts:

$$\varpi_{2a} \leftarrow \mathcal{F}_{\text{KeyBox}}^{(2)}.\text{Use}(\langle \text{sid}, \mathbf{k32} \rangle, \text{SealToPeer}, \langle P_3, \text{ad}_{32} \rangle) \text{ and}$$

$$\varpi_{2b} \leftarrow \mathcal{F}_{\text{KeyBox}}^{(2)}.\text{Use}(\langle \text{sid}, \mathbf{k23} \rangle, \text{SealToPeer}, \langle P_3, \text{ad}_{23} \rangle), \text{ and sends } (\text{sid}, \varpi_{2a}, \varpi_{2b}, K_{1,3}).$$

Transport. The two registration messages ($P_1 \rightarrow P_3$ and $P_2 \rightarrow P_3$) are delivered over authenticated channels (modeled by $\mathcal{F}_{\text{channel}}$).

Input consistency. (P_3 buffers whichever registration message arrives first and applies the following checks once both have been received.) Upon receiving $(\text{sid}, \varpi_1, K_{1,3}^{(1)}, K^{\text{net}})$ from P_1 , require $K^{\text{net}} = K$ (otherwise abort). Upon receiving $(\text{sid}, \varpi_{2a}, \varpi_{2b}, K_{1,3}^{(2)})$ from P_2 , require $K_{1,3}^{(1)} = K_{1,3}^{(2)}$ (otherwise abort), and set $K_{1,3} \leftarrow K_{1,3}^{(1)}$.

P_3 forwards $(\varpi_1, \varpi_{2a}, \varpi_{2b}, \text{ad}_{31}, \text{ad}_{32}, \text{ad}_{23}, K, K_{1,3})$ into $\mathcal{F}_{\text{KeyBox}}^{(3)}$ which executes the following installation procedure (modeled as an atomic transaction):

- (1) **Install sponsor state.** Open ϖ_{2a} with ad_{32} and ϖ_{2b} with ad_{23} internally via **OpenFromPeer** to obtain opaque handles $\tau_{3,2}^{\text{reg}}$ and $\tau_{2,3}^{\text{reg}}$, then invoke **Load** $(\langle \text{sid}, \mathbf{k32} \rangle, g_{3,2}, \langle \tau_{3,2}^{\text{reg}} \rangle)$ and **Load** $(\langle \text{sid}, \mathbf{k23} \rangle, g_{2,3}, \langle \tau_{2,3}^{\text{reg}} \rangle)$.
- (2) **Install the recovery share.** (Re-)open ϖ_1 with ad_{31} , ϖ_{2a} with ad_{32} , and ϖ_{2b} with ad_{23} internally via **OpenFromPeer** to obtain fresh opaque handles $\tau_{3,1}, \tau_{3,2}, \tau_{2,3}$, and then invoke **Load** $(\langle \text{sid}, \mathbf{k3} \rangle, g_3, \langle \tau_{2,3}, \tau_{3,2}, \tau_{3,1}, K, K_{1,3} \rangle)$.

Accept iff all **Load** invocations above return ok.

completion. When the receiver P_3 is corrupted, an authenticated confidential-channel functionality reveals the entire delivered message to the adversary. Therefore, in $\mathcal{F}_{\text{SDKG}}$ we do not model registration transport by uniform dummy strings. Instead, the registration payloads are treated as syntactically valid encodings of the same tuple-structured messages as in Algorithm 1 (p. 51), with ciphertext components sampled from the same distribution as real **SealToPeer** outputs. Concretely, $\mathcal{F}_{\text{SDKG}}$ conceptually samples sealing ciphertexts

$$\varpi_1 \leftarrow \text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}_{31}, \sigma_{3,1}), \quad \varpi_{2a} \leftarrow \text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}_{32}, \sigma_{3,2}), \quad \varpi_{2b} \leftarrow \text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}_{23}, \sigma_{2,3}),$$

for the slot-bound associated data $(\text{ad}_{31}, \text{ad}_{32}, \text{ad}_{23})$ defined in Algorithm 1 (p. 51), and then forms the delivered network messages as the self-delimiting encodings

$$\langle \text{sid}, \varpi_1, K_{1,3}, K \rangle, \quad \langle \text{sid}, \varpi_{2a}, \varpi_{2b}, K_{1,3} \rangle.$$

This ensures that if P_3 is corrupted, the adversary observes well-formed messages and can invoke **OpenFromPeer** on the received ciphertexts. When P_3 is honest, the externally visible distribution of these sealed blobs remains simulatable from public information under our KeyBox key-opacity assumption (Assumption 1 (p. 11)), so the only explicit leakage retained by $\mathcal{F}_{\text{SDKG}}$ is message length. Registration is not modeled as an atomic two-message transaction w.r.t. corruption: if P_3 is corrupted after receiving only one of the two registration payloads, then the adversary learns the already-delivered payload(s) upon corruption and may complete or abort registration at will by (not) delivering and/or forwarding the remaining payload into $\mathcal{F}_{\text{KeyBox}}^{(3)}$.

Algorithm 2: *Acceptance predicate* $\text{Acc}_{\text{SDKG}}(\text{sid}, P_s, P_r, \mathcal{T})$

Input: session identifier sid ; USV committer P_s and relying party P_r ; transcript $\mathcal{T} = (\mathsf{T}_1, \mathsf{T}_2, \mathsf{T}_3)$,

where $\mathsf{T}_1 = (\text{cid}_2, C_2, \zeta_2, B_2, \sigma_{2,1}, h_{3,2}, d)$, $\mathsf{T}_2 = (X_1, M_1, B_1, \sigma_{1,2}, \pi_{\text{aff}_1})$, $\mathsf{T}_3 = (X_2, \pi_{\text{aff}_2}, K_{\text{rec}})$.

Output: 1 iff all checks below pass; otherwise 0.

Parse / derive. If parsing fails, return 0.

D1: Compute $M_2 \leftarrow \text{Open}_M(\text{pp}, C_2, \zeta_2)$ (if $M_2 = \perp$, return 0) and $d^* \leftarrow H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}_2, P_s, P_r, C_2, M_2 \rangle)$.

D2: Derive affine-check auxiliaries:

$$Y_1 \leftarrow M_1 + 2B_1; \quad D_1 \leftarrow X_1 - \sigma_{2,1}\mathcal{G} - Y_1; \quad Y_2 \leftarrow M_2 + 3B_2; \quad D_2 \leftarrow X_2 - \sigma_{1,2}\mathcal{G} - Y_2.$$

D3: Derive $K_{1,3} \leftarrow 2(M_1 + B_1) - X_1$ and key $\hat{K} \leftarrow 3X_1 - 2X_2$.

Checks. Accept iff all of the following hold:

C1: USV receipt consistency: $d = d^*$.

C2: USV certificate linkage: $\sigma_{2,1}\mathcal{G} - 2B_2 = M_2$.

C3: Affine AoK for X_1 : write $\pi_{\text{aff}_1} = (\pi_{Y_1}, \pi_{D_1})$ and require

$$\sigma_{1,2}\mathcal{G} = M_1 + 3B_1 \quad \wedge \quad \mathcal{V}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff1.Y}, Y_1 \rangle, \pi_{Y_1}) = 1 \quad \wedge \quad \mathcal{V}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff1.D}, D_1 \rangle, \pi_{D_1}) = 1.$$

C4: Affine AoK for X_2 : write $\pi_{\text{aff}_2} = (\pi_{Y_2}, \pi_{D_2})$ and require

$$\mathcal{V}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff2.Y}, Y_2 \rangle, \pi_{Y_2}) = 1 \wedge \mathcal{V}_{\text{DL}}(\text{pp}, \langle \text{sid}, \text{SDKG.aff2.D}, D_2 \rangle, \pi_{D_2}) = 1.$$

C5: Digest check for $h_{3,2}$: $h_{3,2} = H_{s32}(\langle \text{sid}, \text{cid}_2, D_2 \rangle)$.

C6: Key consistency: check $K_{\text{rec}} = \hat{K}$.

return 1.

Observation 1 (Lagrange weights) It follows from direct computation that for some scalars $\alpha, \beta \in \mathbb{Z}_p$, it holds that

$$\begin{aligned} K &= k\mathcal{G} = k_{1,2}\mathcal{G} + k_2\mathcal{G} = k_{1,3}\mathcal{G} + k_3\mathcal{G} = \alpha x_1\mathcal{G} - \beta x_2\mathcal{G} \\ &= (\alpha - \beta)(m_1 + m_2)\mathcal{G} + (2\alpha - 3\beta)(b_1 + b_2)\mathcal{G} + \alpha\sigma_{3,1}\mathcal{G} - \beta\sigma_{3,2}\mathcal{G}. \end{aligned}$$

For $u = 2$ and $v = 3$, the unique α, β satisfying $\alpha - \beta = 1$ and $\alpha u - \beta v = 0$ are $\alpha = 3$ and $\beta = 2$. Consequently, $\alpha x_1 - \beta x_2 = (m_1 + m_2) + \alpha\sigma_{3,1} - \beta\sigma_{3,2}$. More generally, for distinct $u, v \in \mathbb{Z}_p^*$, the unique choice is $\alpha = \frac{v}{v-u}$ and $\beta = \frac{u}{v-u}$.

For $\alpha = 3, \beta = 2$ (the Lagrange weights for $u = 2, v = 3$), in any accepting transcript $\mathcal{T}$, the UC-NIZK(-AoK)s and the USV opening uniquely determine (x_1, x_2) . Hence, the only $\mathcal{T}$ -consistent shares with $k = k_{1,2} + k_2$ are $(k_{1,2}, k_2) = (3x_1, -2x_2)$. Since f_1, f_2 are linear, $m_1 = f_1(0) = 2f_1(1) - f_1(2) = 2\sigma_{1,3} - \sigma_{1,1}$ and $m_2 = f_2(0) = 2f_2(1) - f_2(2) = 2\sigma_{2,3} - \sigma_{2,1}$. By Observation 1 (p. 52), $k = (m_1 + m_2) + 3\sigma_{3,1} - 2\sigma_{3,2}$. Substituting and regrouping yields $k = (2\sigma_{1,3} - x_1) + 2(\sigma_{2,3} - \sigma_{3,2} + 2\sigma_{3,1}) = k_{1,3} + k_3$.

Reader Note 7.2: $\mathcal{F}_{\text{pub}}$ agreement as a formal protocol requirement

In the base run, P_1 additionally verifies that $\hat{K}$ equals the value received from $\mathcal{F}_{\text{pub}}$ for this session, i.e., the $(\text{Recv}, \text{sid}, P_2, K_{\text{pub}})$ delivered by $\mathcal{F}_{\text{pub}}$ satisfies $K_{\text{pub}} = \hat{K}$. This supplementary check is a non-blocking local consistency guard evaluated by P_1 when the $\mathcal{F}_{\text{pub}}$ delivery arrives; it does not gate Acc_{SDKG} acceptance and does not introduce an additional protocol round. For registration, the $\mathcal{F}_{\text{pub}}$ -agreement check is a *formal protocol requirement*: any party P_i initiating registration must require that both P_1 's and P_2 's authenticated $\mathcal{F}_{\text{pub}}$ publications for sid agree (Algorithm 1 (p. 51)). This prevents a corrupted sender from installing a shifted key in an honest joiner's KeyBox by publishing $K_{\text{fake}} \neq K$ via $\mathcal{F}_{\text{pub}}$. For the n -device extension, all joiners check P_1 's and P_2 's base-run $\mathcal{F}_{\text{pub}}$

publications, regardless of the sponsor's identity. The immutability of $\mathcal{F}_{\text{pub}}$ (which provides no retract or re-publish interface) ensures that an honest base-run publication cannot be retroactively altered by a subsequent adaptive corruption. The base-run acceptance predicate Acc_{SDKG} itself operates on the transcript $\mathcal{T}$ alone (Theorem 3 (p. 60)).

Lemma 15 (Acc_{SDKG} need not call $\mathcal{F}_{\text{USV}}.\text{Verify}$). *In any real execution of $\Psi_{\text{SDKG}}^{(3)}$ in the $(\mathcal{F}_{\text{KeyBox}}, \mathcal{F}_{\text{USV}}, \mathcal{F}_{\text{channel}})$ -hybrid model, if P_1 is honest and the Round 2 message $\mathbf{T}_2 = (X_1, M_1, B_1, \sigma_{1,2}, \pi_{\text{aff}_1})$ is delivered to P_2 over $\mathcal{F}_{\text{channel}}$, then $\mathcal{F}_{\text{USV}}.\text{Verify}(\text{sid}, \text{cid}_2, P_2, C_2, \zeta_2) = 1$ and the receipt-digest consistency check holds:*

$$d = H(\text{USV.rcpt}, \langle \text{sid}, \text{cid}_2, P_2, P_1, C_2, M_2 \rangle) \quad \text{where } M_2 := \text{Open}_M(\text{pp}, C_2, \zeta_2).$$

Proof Sketch. An honest P_1 sends $\mathbf{T}_2$ only if $\mathcal{F}_{\text{USV}}.\text{Verify}(\text{sid}, \text{cid}_2, P_2, C_2, \zeta_2) = 1$ and the receipt digest check succeeds; otherwise it aborts before sending. Since $\mathcal{F}_{\text{channel}}$ is authenticated, an adversary cannot forge $\mathbf{T}_2$ on behalf of honest P_1 . ■

For any transcript that can arise with honest P_1 and authenticated channels, the receipt-digest check **C1** in Algorithm 2 (p. 52) already captures the effect of the explicit $\mathcal{F}_{\text{USV}}.\text{Verify}$ performed by honest P_1 in Round 2.

The KeyBox/Load step executes within a single honest-party activation (Definition 1 (p. 8)), which is atomic with respect to corruption. When both senders are honest, the derivation routines are deterministic on well-formed inputs and all Load calls succeed; if a sender is corrupted and a Load fails, registration is not recorded as complete and any partially installed sponsor-state slots are inert (NXK-restricted and unusable for key recovery without the recovery-share slot $\langle \text{sid}, \mathbf{k3} \rangle$). In practice, this can be implemented by delaying the sealed-storage write until after validation. Registering devices P_i for $i > 3$ follows the same pattern: P_1 always seals $\sigma_{3,1}$ from slot $\langle \text{sid}, \mathbf{k31} \rangle$, and the sponsor leaf seals the sponsor-state scalars from its local slots $\langle \text{sid}, \mathbf{k32} \rangle$ and $\langle \text{sid}, \mathbf{k23} \rangle$. The joining device installs (i) its long-term recovery share in slot $\langle \text{sid}, \mathbf{k3} \rangle$ via g_3 , and (ii) the sponsor-state slots $\langle \text{sid}, \mathbf{k32} \rangle$ and $\langle \text{sid}, \mathbf{k23} \rangle$ via $g_{3,2}^{\text{reg}}$ and $g_{2,3}^{\text{reg}}$. Consequently, every already-registered leaf can serve as sponsor for future registrations.

Parameterized registration (general case). For any joiner P_i ($i \geq 3$) with sponsor $P_j \in \{2\} \cup \text{Reg}$, the real-world registration executes Algorithm 1 (p. 51) with the following parameter substitutions: $P_3 \mapsto P_i$ and $P_2 \mapsto P_j$ (as sponsor), associated-data strings

$$\text{ad}_{1i} := \langle \text{SDKG.reg}, \text{sid}, P_1, P_i, \mathbf{k31} \rangle, \quad \text{ad}_{ji}^{32} := \langle \text{SDKG.reg}, \text{sid}, P_j, P_i, \mathbf{k32} \rangle, \quad \text{ad}_{ji}^{23} := \langle \text{SDKG.reg}, \text{sid}, P_j, P_i, \mathbf{k23} \rangle,$$

and derivation routines $g_3, g_{3,2}^{\text{reg}}, g_{2,3}^{\text{reg}}$ parameterized by (P_i, P_j, sid) per Definition 21 (p. 44). The $\mathcal{F}_{\text{pub}}$ -agreement check uses P_1 's and P_2 's base-run publications (not the sponsor's). Transport uses $\mathcal{F}_{\text{channel}}$. The KeyBox installation procedure within $\mathcal{F}_{\text{KeyBox}}^{(i)}$ is identical to Algorithm 1 (p. 51)'s steps (1)–(2).

In the setting of Proposition 1 (p. 19), any enrollment mechanism that requires a party to export or externally derive a fresh share-deriving material from the public/external view conflicts with NXK under key-opacity. Proposition 1 (p. 19) bounds the success probability of any PPT strategy that computes $\hat{k}_{\text{new}}$ from $(\text{pp}, \tau_{\text{ext}})$. SDKG's one-shot registration of P_3 does not attempt to compute $f(x_{\text{new}})$ (or any share-derived plaintext) from τ_{ext} . Instead, existing devices transfer the required registration scalars to P_3 only via (attested) KeyBox-to-KeyBox sealing using SealToPeer/OpenFromPeer, and P_3 's KeyBox derives and installs k_3 internally (via g_3). Externally, the transported ciphertexts are part of τ_{ext} and remain simulatable under key-opacity. Beyond $i = 3$, our scalable NXK-compatible RDR enrollment registers additional devices as redundant front-ends for the recovery role (with the same base-run-derived share guaranteed whenever at least one registration sender is honest).

Remark 23 ($\mathcal{F}_{\text{pub}}$ -agreement enforcement is simulator-mediated). In the real protocol, honest P_3 gates registration on receiving consistent $\mathcal{F}_{\text{pub}}$ publications from both P_1 and P_2 (Algorithm 1 (p. 51)). $\mathcal{F}_{\text{SDKG}}$ does not enforce this condition directly because $\mathcal{F}_{\text{pub}}$ is a shared hybrid functionality whose state $\mathcal{F}_{\text{SDKG}}$ cannot inspect. Instead, the UC proof of Theorem 3 (p. 60) shows that the simulator Sim conditions the RegGo message to $\mathcal{F}_{\text{SDKG}}$ on $\mathcal{F}_{\text{pub}}$ consistency: if Sim detects that a corrupted sender published a value on $\mathcal{F}_{\text{pub}}$ for sid different from the finalized K , then Sim does not forward RegGo and registration does not complete in the ideal world, matching the real-world abort behavior. This delegation to the simulator is not a specification gap but a standard UC modeling pattern for conditions involving shared hybrid resources.

♦ **State (per session sid):** corruption set $\text{Cor} \subseteq \{1, 2, 3\}$; flags $\text{finalized}, \text{RegPending}, \text{RegDone} \in \{0, 1\}$ (init 0); approvals $\text{Auth} \subseteq \{1, 2\}$ (init $\emptyset$); transcript slots $\text{T}_1, \text{T}_2, \text{T}_3$ (init $\perp$); optional programmed values $(x_1, x_2, \sigma_{3,1}, \sigma_{3,2}) \in (\mathbb{Z}_p \cup \{\perp\})^4$ (init $\perp$); retain the registration scalars $(\sigma_{3,1}, \sigma_{2,3}, \sigma_{3,2}) \in \mathbb{Z}_p^3$ and $K_{1,3} \in \mathbb{G}$ after finalization; registration-channel state multiset Q_{reg} of $(\rho, P_s, P_r, w, \phi)$, and delivered set D_{reg} (init empty); delivered registration payload buffers $w_{\text{reg},1}^{\text{del}}, w_{\text{reg},2}^{\text{del}} \in \{0, 1\}^* \cup \{\perp\}$ (init $\perp$).

♦ **Init:** Upon (init, 2, sid) from P_2 , send (init, sid) to P_1, P_2, P_3 and $\mathcal{A}$.

♦ **Transcript feed (ideal adversary/simulator interface; bookkeeping):** Upon receiving one of the following messages from $\mathcal{A}$, if the corresponding transcript slot is empty then store it and run **TryFinalize**:

- (Tin₁, sid, cid₂, C_2 , ζ_2 , B_2 , $\sigma_{2,1}$, $h_{3,2}$, d): if $\text{T}_1 = \perp$ then set $\text{T}_1 \leftarrow (\text{cid}_2, C_2, \zeta_2, B_2, \sigma_{2,1}, h_{3,2}, d)$.
- (Tin₂, sid, X_1 , M_1 , B_1 , $\sigma_{1,2}$, π_{aff_1}): if $\text{T}_2 = \perp$ then set $\text{T}_2 \leftarrow (X_1, M_1, B_1, \sigma_{1,2}, \pi_{\text{aff}_1})$.
- (Tin₃, sid, X_2 , π_{aff_2} , $\hat{K}$): if $\text{T}_3 = \perp$ then set $\text{T}_3 \leftarrow (X_2, \pi_{\text{aff}_2}, \hat{K})$.

♦ **TryFinalize:** If $\text{finalized} = 1$ or some $\text{T}_i = \perp$, do nothing. Otherwise let $\mathcal{T} := (\text{T}_1, \text{T}_2, \text{T}_3)$. If $\text{Acc}_{\text{SDKG}}(\text{sid}, P_2, P_1, \mathcal{T}) = 0$ (Algorithm 2 (p. 52)), do nothing. If $\text{Acc}_{\text{SDKG}}(\text{sid}, P_2, P_1, \mathcal{T}) = 1$, then:

1. If $x_1 = \perp$ or $x_2 = \perp$ or $\sigma_{3,1} = \perp$ or $\sigma_{3,2} = \perp$, do nothing and return. Let $\text{T}_1 = (\text{cid}_2, C_2, \zeta_2, B_2, \sigma_{2,1}, h_{3,2}, d)$, $\text{T}_2 = (X_1, M_1, B_1, \sigma_{1,2}, \pi_{\text{aff}_1})$, and $\text{T}_3 = (X_2, \pi_{\text{aff}_2}, K_{\text{rec}})$. Define $Y_1 := M_1 + 2B_1$ and $D_1 := X_1 - \sigma_{2,1}\mathcal{G} - Y_1$. Require $X_1 = x_1\mathcal{G}$, $X_2 = x_2\mathcal{G}$, $h_{3,2} = H_{\text{S32}}(\text{sid}, \text{cid}_2, \sigma_{3,2}\mathcal{G})$, $D_1 = \sigma_{3,1}\mathcal{G}$. Define $\sigma_{1,1} := x_1 - \sigma_{2,1} - \sigma_{3,1} \bmod p$, $\sigma_{1,3} := 2\sigma_{1,1} - \sigma_{1,2} \bmod p$, $k_2 := 3x_1$, $k_3 := -2x_2$, $k_{1,3} := 2\sigma_{1,3} - x_1 \bmod p$, $K_{1,3} := 2(M_1 + B_1) - X_1$, $k := k_{1,2} + k_2 \bmod p$, $\sigma_{2,2} := x_2 - \sigma_{1,2} - \sigma_{3,2} \bmod p$.
2. Define recovery-role and sponsor-state scalars: $k_3 := (k - k_{1,3}) \bmod p$, $\sigma_{2,3} := k_3 \cdot 2^{-1} - 2\sigma_{3,1} + \sigma_{3,2} \bmod p$. Output $K := k\mathcal{G}$ to P_1, P_2 and $\mathcal{A}$. Set $\text{finalized} \leftarrow 1$. (In the real protocol, P_3 obtains K by requiring agreement between both P_1 's and P_2 's base-run authenticated $\mathcal{F}_{\text{pub}}$ publications for sid before initiating registration; since $\mathcal{F}_{\text{pub}}$ publications are immutable once made, at least one honest base-run publication anchors K to the correct value. The registration payload from P_1 also carries K as a redundant consistency check.)
3. If $1 \notin \text{Cor}$, send to P_1 the internal KeyBox command $(\text{KBcmd}, \text{sid}, [(\text{Load}, \langle \text{sid}, k_{12} \rangle, g_{1,2}, \langle \sigma_{1,1}, \sigma_{2,1}, \sigma_{3,1} \rangle), (\text{Load}, \langle \text{sid}, k_{13} \rangle, g_{1,3}, \langle \sigma_{1,1}, \sigma_{2,1}, \sigma_{3,1}, \sigma_{1,3} \rangle), (\text{Load}, \langle \text{sid}, k_{31} \rangle, g_{3,1}^{\text{reg}}, \langle \sigma_{3,1} \rangle)])$.
4. If $2 \notin \text{Cor}$, send to P_2 the internal KeyBox command $(\text{KBcmd}, \text{sid}, [(\text{Load}, \langle \text{sid}, k_2 \rangle, g_2, \langle \sigma_{1,2}, \sigma_{2,2}, \sigma_{3,2} \rangle), (\text{Load}, \langle \text{sid}, k_{32} \rangle, g_{3,2}^{\text{reg}}, \langle \sigma_{3,2} \rangle), (\text{Load}, \langle \text{sid}, k_{23} \rangle, g_{2,3}^{\text{reg}}, \langle \sigma_{2,3} \rangle)])$.

♦ **Registration of P_3 :** Upon first (register, 3, sid) from P_3 : require $\text{finalized} = 1$ and $\text{RegDone} = 0$; set $\text{RegPending} \leftarrow 1$, $\text{Auth} \leftarrow \emptyset$; set $w_{\text{reg},1}^{\text{del}} \leftarrow \perp$ and $w_{\text{reg},2}^{\text{del}} \leftarrow \perp$; notify P_1, P_2 and $\mathcal{A}$ with (RegReq, 3, sid).

Upon (approve, 3, sid) from P_j with $j \in \{1, 2\}$ and $\text{RegPending} = 1$, set $\text{Auth} \leftarrow \text{Auth} \cup \{j\}$ and notify $\mathcal{A}$.

Upon (RegGo, 3, sid, w_1^*, w_2^*) from $\mathcal{A}$: if $\text{RegPending} = 1$, $\text{Auth} = \{1, 2\}$, $\text{RegDone} = 0$, and $\text{Q}_{\text{reg}} = \emptyset$, then: define slot-bound ad as in Algorithm 1 (p. 51): $\text{ad}_{31} := \langle \text{SDKG.reg}, \text{sid}, P_1, P_3, k_{31} \rangle$, $\text{ad}_{32} := \langle \text{SDKG.reg}, \text{sid}, P_2, P_3, k_{32} \rangle$, $\text{ad}_{23} := \langle \text{SDKG.reg}, \text{sid}, P_2, P_3, k_{23} \rangle$. Sample $\rho_1, \rho_2 \leftarrow \{0, 1\}^\lambda$.

- If $1 \notin \text{Cor}$, sample $\varpi_1 \leftarrow \text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}_{31}, \sigma_{3,1})$ and set $w_{\text{reg},1} := \langle \text{sid}, \varpi_1, K_{1,3}, K \rangle$. If $1 \in \text{Cor}$, set $w_{\text{reg},1} := w_1^*$.
- If $2 \notin \text{Cor}$, sample $\varpi_{2a} \leftarrow \text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}_{32}, \sigma_{3,2})$ and $\varpi_{2b} \leftarrow \text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}_{23}, \sigma_{2,3})$. Set $w_{\text{reg},2} := \langle \text{sid}, \varpi_{2a}, \varpi_{2b}, K_{1,3} \rangle$. If $2 \in \text{Cor}$, set $w_{\text{reg},2} := w_2^*$.

 For each $j \in \{1, 2\}$ with $w_{\text{reg},j} \neq \perp$: let $\phi_j := |w_{\text{reg},j}|$; insert $(\rho_j, P_j, P_3, w_{\text{reg},j}, \phi_j)$ into Q_{reg} ; send (Leak, sid, P_j, P_3, ρ_j, ϕ_j) to $\mathcal{A}$. If $j \in \text{Cor}$, additionally reveal $w_{\text{reg},j}$ to $\mathcal{A}$.

Upon receiving (Deliver, sid, ρ) from $\mathcal{A}$: if $(\rho, P_s, P_3, w, \phi) \in \text{Q}_{\text{reg}}$ and $\rho \notin \text{D}_{\text{reg}}$, delete it from Q_{reg} , add ρ to D_{reg} , and deliver (Recv, sid, P_s, w) to P_3 . If $3 \in \text{Cor}$, reveal w to $\mathcal{A}$ at delivery time. If $P_s = P_1$, set $w_{\text{reg},1}^{\text{del}} \leftarrow w$. If $P_s = P_2$, set $w_{\text{reg},2}^{\text{del}} \leftarrow w$.

If $\text{RegPending} = 1$, $\text{Auth} = \{1, 2\}$, $\text{RegDone} = 0$, $\text{Q}_{\text{reg}} = \emptyset$, and $w_{\text{reg},1}^{\text{del}} \neq \perp$ and $w_{\text{reg},2}^{\text{del}} \neq \perp$, then:

- If $3 \in \text{Cor}$: do nothing further as after corruption, completion/abort is controlled by $\mathcal{A}$.
- If $3 \notin \text{Cor}$: parse $w_{\text{reg},1}^{\text{del}} = \langle \text{sid}, \varpi_1, K_{1,3}^{(1)}, K^* \rangle$ and $w_{\text{reg},2}^{\text{del}} = \langle \text{sid}, \varpi_{2a}, \varpi_{2b}, K_{1,3}^{(2)} \rangle$. Require $K_{1,3}^{(1)} = K_{1,3}^{(2)}$ and set $K_{1,3}^* \leftarrow K_{1,3}^{(1)}$. If parsing fails then do nothing further. Require $K^* = K$ (the stored finalized public key). In the real protocol, the honest joiner additionally requires agreement of both senders' $\mathcal{F}_{\text{pub}}$ publications before initiating registration; the simulator matches this by conditioning RegGo on $\mathcal{F}_{\text{pub}}$ consistency (see proof of Theorem 3 (p. 60)). The $K^* = K$ check here is a secondary payload-level guard; otherwise do nothing further. Otherwise, have P_3 attempt the KeyBox installation procedure from Algorithm 1 (p. 51) using $(\varpi_1, \varpi_{2a}, \varpi_{2b}, \text{ad}_{31}, \text{ad}_{32}, \text{ad}_{23}, K^*, K_{1,3}^*)$, i.e., by invoking **OpenFromPeer** and **Load**($\cdot$) exactly as in Algorithm 1 (p. 51). If all invoked **Load** calls return ok, then set $\text{RegDone} \leftarrow 1$ and $\text{RegPending} \leftarrow 0$ and output (registered, 3, sid) to P_3 and $\mathcal{A}$. Otherwise do nothing further.

♦ **Corruptions:** Upon (Corrupt, i) from $\mathcal{A}$, add i to Cor and reveal P_i 's local state. Thereafter, $\mathcal{A}$ controls P_i and may invoke $\mathcal{F}_{\text{KeyBox}}^{(i)}$. Load/Use directly; $\mathcal{F}_{\text{SDKG}}$ does not mediate these calls.

♦ **Programming (simulator-only):** Before finalization, Sim may once send (Program, sid, $x_1^*, x_2^*, \sigma_{3,1}^*, \sigma_{3,2}^*$); set $x_1 \leftarrow x_1^*$, $x_2 \leftarrow x_2^*$, $\sigma_{3,1} \leftarrow \sigma_{3,1}^*$, $\sigma_{3,2} \leftarrow \sigma_{3,2}^*$. Then run **TryFinalize**. Finalization occurs only if the transcript slots are filled and the programmed values satisfy the **TryFinalize** consistency checks.

Fig. 8: Transcript-driven ideal functionality $\mathcal{F}_{\text{SDKG}}$.

8 UC Security

In the ideal functionality $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)), to avoid distributional mismatches under adaptive corruption, we use a transcript-driven finalization rule. In any accepting transcript, the UC-extractable affine AoKs and the $s32$ -digest check pin down unique values (x_1, x_2) , the value $\sigma_{3,2}$ bound into $h_{3,2}$, and (via the affine AoK for X_1) the value $\sigma_{3,1}$ bound by $D_1 = X_1 - \sigma_{2,1}\mathcal{G} - (M_1 + 2B_1) = \sigma_{3,1}\mathcal{G}$. Accordingly, $\mathcal{F}_{\text{SDKG}}$ does not sample $(x_1, x_2, \sigma_{3,1}, \sigma_{3,2})$ at finalization time. Instead, finalization is gated on a single simulator-only **Program** message supplying these transcript-consistent values; $\mathcal{F}_{\text{SDKG}}$ checks consistency against the stored transcript before outputting K and installing KeyBox shares. In $\mathcal{F}_{\text{SDKG}}$, the base-run record $\mathcal{T} = (\mathsf{T}_1, \mathsf{T}_2, \mathsf{T}_3)$ is supplied via the ideal functionality's adversary interface (i.e., by the ideal-world simulator) through the $\mathsf{Tin}_1/\mathsf{Tin}_2/\mathsf{Tin}_3$ messages. This is purely bookkeeping used to couple the ideal execution to the simulated real execution; it should not be interpreted as adversary-visible leakage. The slot tuples T_i may include fields that are transmitted only over $\mathcal{F}_{\text{channel}}$ in honest executions (e.g., σ -scalars), even though the adversary learns at most the explicit $\mathcal{F}_{\text{channel}}$ leakage unless it corrupts an endpoint.

KeyBox installation effects in $\mathcal{F}_{\text{SDKG}}$ are realized via the KeyBox-driver wrapper mechanism of Remark 6 (p. 16).

Lemma 16 (Latent (marginal) uniformity of the SDKG key). *Fix any session identifier sid of $\hat{\Psi}_{\text{SDKG}}^{(3)}$. Let $x_1 := \sigma_{1,1} + \sigma_{2,1} + \sigma_{3,1}$ and $x_2 := \sigma_{1,2} + \sigma_{2,2} + \sigma_{3,2}$ denote the (conceptual) scalars defined by the parties' sampled σ -values in the base run, and define the candidate key scalar and group element $\hat{k} := 3x_1 - 2x_2 \bmod p$, $\hat{K} := \hat{k}\mathcal{G} \in \mathbb{G}$. If at least one of $\sigma_{3,1}$ or $\sigma_{3,2}$ is sampled uniformly, then $\hat{K}$ is uniform in $\mathbb{G}$ marginally over that honest pad randomness.*

Proof. For the Lagrange weights $u = 2$ and $v = 3$, Observation 1 (p. 52) gives

$$\hat{k} = (m_1 + m_2) + 3\sigma_{3,1} - 2\sigma_{3,2} \bmod p,$$

and hence $\hat{K} = \hat{k}\mathcal{G}$. If $\sigma_{3,1}$ is sampled honestly, then $3\sigma_{3,1}$ is uniform in $\mathbb{Z}_p$ and therefore $\hat{k} = ((m_1 + m_2) - 2\sigma_{3,2}) + 3\sigma_{3,1}$ is uniform in $\mathbb{Z}_p$. If instead $\sigma_{3,2}$ is sampled honestly, then $-2\sigma_{3,2}$ is uniform in $\mathbb{Z}_p$ and therefore

$$\hat{k} = ((m_1 + m_2) + 3\sigma_{3,1}) - 2\sigma_{3,2}$$

is uniform in $\mathbb{Z}_p$. In either case $\hat{k}$ is uniform in $\mathbb{Z}_p$, and since $z \mapsto z\mathcal{G}$ is a bijection, $\hat{K} = \hat{k}\mathcal{G}$ is uniform in $\mathbb{G}$. Since we make no fairness/guaranteed-output-delivery claim, the distribution of the delivered key conditioned on completion may be biased by selective abort (cf. Cleve [22]). $\blacksquare$

8.1 From $\mathcal{F}_{\text{SDKG}}$ to the standard NXK-DKG interface

Fix the 1+1-out-of-3 star access structure

$$\Gamma_0 := \{\{P_1, P_2\}, \{P_1, P_3\}\}.$$

The functionality $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)) is deliberately transcript-driven: besides producing the public key K and installing non-exportable KeyBox shares, it (i) records the public transcript items that determine acceptance and (ii) offers a simulator-only one-shot **Program** hook used exclusively to couple real and ideal executions under adaptive corruptions and straight-line extraction.

Definition 23 (Standard NXK-star DKG functionality $\mathcal{F}_{\text{DKG}}^{\star, \text{NXK}}$). Fix the following deterministic polynomial-time wrapper ITM $\mathcal{W}_{\text{DKG}}$ that sits between the environment and an instance of $\mathcal{F}_{\text{SDKG}}$ and exposes only the interface that we regard as the standard NXK-DKG API for Γ_0 , namely:

1. the public key output K for each accepting session, with no output on abort;
2. the induced KeyBox installation effects (via the surrounding $\mathcal{F}_{\text{KeyBox}}$ instances); and
3. if invoked, the post-finalization registered events for device registration (RDR).

The wrapper suppresses $\mathcal{F}_{\text{SDKG}}$'s internal transcript bookkeeping (e.g., the transcript slots $\mathsf{T}_1, \mathsf{T}_2, \mathsf{T}_3$) and does not expose the simulator-only **Program** port to the environment. Define

$$\mathcal{F}_{\text{DKG}}^{\star, \text{NKK}} := \mathcal{W}_{\text{DKG}} \circ \mathcal{F}_{\text{SDKG}}.$$

As usual, $\mathcal{F}_{\text{DKG}}^{\star, \text{NKK}}$ makes no fairness guarantee: a corrupted party and/or the adversary-controlled scheduler may delay or prevent completion (selective abort), and may even condition its decision to abort on partial information about the (would-be) key.[§] Consequently, the uniformity guarantee we target is the conventional UC-DKG latent (marginal) uniformity (prior to conditioning on completion), not uniformity conditioned on completion.

Lemma 17 (Closure under interface restriction). *Let $\mathcal{W}_{\text{DKG}}$ be the fixed wrapper from Definition 23 (p. 55) and let $\mathcal{F}_{\text{DKG}}^{\star, \text{NKK}} := \mathcal{W}_{\text{DKG}} \circ \mathcal{F}_{\text{SDKG}}$. If a protocol Ψ UC-realizes $\mathcal{F}_{\text{SDKG}}$ in some model $\mathcal{M}$, then Ψ UC-realizes $\mathcal{F}_{\text{DKG}}^{\star, \text{NKK}}$ in the same model $\mathcal{M}$.*

Proof Sketch. This is UC closure under efficient local post-processing. Fix any PPT adversary $\mathcal{A}$ for Ψ when the ideal functionality is $\mathcal{F}_{\text{DKG}}^{\star, \text{NKK}}$. Define an adversary $\mathcal{A}'$ for Ψ when the ideal functionality is $\mathcal{F}_{\text{SDKG}}$ that runs $\mathcal{A}$ and locally applies the same forwarding/output-filtering that $\mathcal{W}_{\text{DKG}}$ applies between the environment and $\mathcal{F}_{\text{SDKG}}$ (i.e., it suppresses exactly the bookkeeping outputs/ports hidden by $\mathcal{W}_{\text{DKG}}$). By the hypothesis that Ψ UC-realizes $\mathcal{F}_{\text{SDKG}}$, there exists a PPT simulator Sim' for $\mathcal{A}'$ in the $\mathcal{F}_{\text{SDKG}}$ -ideal execution. Composing Sim' with the same local filtering yields a simulator for $\mathcal{A}$ in the $\mathcal{F}_{\text{DKG}}^{\star, \text{NKK}}$ -ideal execution. ■

Observation 2 (DKG properties captured by $\mathcal{F}_{\text{DKG}}^{\star, \text{NKK}}$) For $\Gamma_0 := \{\{P_1, P_2\}, \{P_1, P_3\}\}$ and any session sid of $\mathcal{F}_{\text{DKG}}^{\star, \text{NKK}}$, the following properties hold:

- (i) NKK: No non-corruption interface ever outputs the secret scalar k or any share-deriving plaintext (the pre-install adaptive-corruption window, in which retained σ -scalars may be revealed, is addressed in the proof of Theorem 3 (p. 60)). All long-term shares remain confined to the parties' $\mathcal{F}_{\text{KeyBox}}$ instances, and the adversary has only black-box access via the fixed admissible KeyBox profile.
- (ii) Uniqueness / consistency of the induced key: If the session completes (outputs K), then there exists a unique scalar $k \in \mathbb{Z}_p$ such that $K = k\mathcal{G}$, and the KeyBox-installed shares are consistent with a single global key under Γ_0 (i.e., the two authorized sets induce the same k). Concretely, the accepting transcript uniquely determines x_1, x_2 and hence $k = (3x_1 - 2x_2) \bmod p$ (Lemma 19 (p. 60)).
- (iii) Secrecy against unauthorized corruption sets: For any corruption set $B \subseteq \{P_1, P_2, P_3\}$ with $B \notin \Gamma$, the functionality does not enable recovery of k : the adversary never obtains enough long-term shares to reconstruct k , and any leakage from interacting with corrupted parties' KeyBoxes is limited to the admissible KeyBox profile and hence is simulatable from public information under key-opacity (Assumption 1 (p. 11)).
- (iv) Latent (marginal) uniformity (no fairness): As usual for UC-DKG, no fairness is guaranteed; however, if at least one of the designated pad scalars is honestly sampled, then the resulting public key K is uniform in $\mathbb{G}$ marginally (prior to conditioning on completion) (Lemma 16 (p. 55)).

Proof Sketch. Item (i) follows from the NKK/ $\mathcal{F}_{\text{KeyBox}}$ model (Definition 6 (p. 15) and Remark 7 (p. 16)). Item (ii) is Lemma 19 (p. 60) together with the deterministic KeyBox-installation logic in $\mathcal{F}_{\text{SDKG}} / \mathcal{F}_{\text{DKG}}^{\star, \text{NKK}}$. Item (iii) is exactly the intended DKG secrecy statement under Γ in the NKK model: unauthorized sets lack a reconstructing share set, and admissible KeyBox interactions do not reveal share-deriving plaintexts under key-opacity. Item (iv) is Lemma 16 (p. 55). ■

For the star access structure $\Gamma_0 = \{\{P_1, P_2\}, \{P_1, P_3\}\}$, in the $(\mathcal{F}_{\text{KeyBox}}, \mathcal{F}_{\text{channel}}, \mathcal{F}_{\text{pub}})$ -hybrid model, $\mathcal{W}_{\text{DKG}} \circ \mathcal{F}_{\text{SDKG}}$ matches the standard NKK-DKG semantics for Γ_0 in the following concrete sense:

1. Latent (marginal) uniformity under one honest pad: Define the transcript-derived candidate key $\hat{K} := 3X_1 - 2X_2$ whenever X_1 and X_2 are defined (cf. Algorithm 2 (p. 52), Step **D4**). If at least one of $\sigma_{3,1}$ or $\sigma_{3,2}$ is sampled by an

[§] Accordingly, while we can prove latent/marginal uniformity of the key prior to conditioning on completion, the distribution of the output key conditioned on completion may be biased via selective abort. This limitation is inherent in DKG/coin-flipping style tasks without guaranteed output delivery; see, e.g., Cleve [22].

honest party, then $\widehat{K}$ is uniform in $\mathbb{G}$, marginally over that honest pad randomness (i.e., prior to conditioning on session completion) (Lemma 16 (p. 55)).

2. Consistency with a single key: Acceptance implies there exist unique transcript-defined $x_1, x_2 \in \mathbb{Z}_p$ and thus a unique $k \in \mathbb{Z}_p$ such that $K = k\mathcal{G}$ and the KeyBox-installed shares are consistent with k under Γ_0 (Lemma 19 (p. 60) and the definition of the installed slots in Fig. 8 (p. 54)).
3. Non-exportability: All long-term shares remain confined to $\mathcal{F}_{\text{KeyBox}}$ by construction, and share-deriving material are NXK-restricted (Remark 7 (p. 16)).

As usual, no fairness guarantee is made: a corrupted party may selectively abort after learning enough to decide whether to proceed.

Remark 24 (On transcript-driven finalization). The simulator-only **Program** interface in $\mathcal{F}_{\text{SDKG}}$ is a proof device used to handle adaptive corruptions with straight-line extraction: it delays committing to certain internal scalars until the simulator can derive them from the public transcript. **Program** does not give the ideal world extra freedom to choose outputs: finalization is gated by deterministic transcript checks, and in any accepting transcript the induced values are uniquely determined (Lemma 19 (p. 60)). The wrapper $\mathcal{W}_{\text{DKG}}$ hides **Program** and all transcript slots from applications, yielding the standard NXK-DKG interface $\mathcal{F}_{\text{DKG}}^{*,\text{NXK}}$.

8.2 Formal necessity of USV for commit-only alternatives under hardened profiles

Section 5.1 (p. 31) gave the short design-space motivation; this section provides the formal obstruction and proof in the NXK/ $\mathcal{F}_{\text{KeyBox}}$ + gRO-CRP model.

Scope. The concrete **SDKG.LeafInit** of Section 7 (p. 40) (Fig. 7 (p. 44)) returns σ -values from which m_2 is host-recoverable (cf. the profile convention in Section 7 (p. 40)). In that profile, the host could in principle compute and publish $M_2 = m_2\mathcal{G}$ directly (Option 1 below). The necessity argument below addresses a design-space question: whether USV can be replaced by a plain hiding commitment whose adversary-visible footprint is only $C_2 = \text{Commit}(m_2; r_2)$, without any additional public material that determines M_2 . This is relevant both to the hypothetical hardened profile in which the KeyBox does not return share-deriving σ -values at all, and—independently—to any deployment where the host-recovered m_2 is erased before the transcript is finalized and no equivalent opening-to- $\mathbb{G}$ material has been published.

The concrete SDKG base run already transmits a full USV instance (C_2, ζ_2) , and hence M_2 is deterministically computable from the transcript as $M_2 = \text{Open}_M(\text{pp}, C_2, \zeta_2)$. The purpose of this section is to address a natural alternative design in which one attempts to replace USV by a generic hiding commitment to m_2 —that is, a commit-only transcript that contains C_2 but no public material that fixes $m_2\mathcal{G}$ —and then tries to recover M_2 by straight-line extraction of m_2 under NXK/ $\mathcal{F}_{\text{KeyBox}}$. We show this fails unless the model is strengthened or hiding is broken. Equivalently, in the NXK setting one must provide either explicit M_2 or USV-style public opening material that deterministically fixes M_2 without exporting m_2 .

What must be transcript-defined (and why). In the SDKG base run, both honest verification and the UC simulator must be able to compute certain group elements as deterministic functions of the public transcript, in straight-line. Concretely, verification (Algorithm 2 (p. 52)) and transcript-driven programming of $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)) require forming the leaf-dependent auxiliary point

$$Y_2 := M_2 + 3B_2 \quad \text{where} \quad M_2 := m_2\mathcal{G},$$

for a leaf scalar $m_2 \in \mathbb{Z}_p$ whose group image $M_2 = m_2\mathcal{G}$ does not appear in the adversary-visible transcript τ_{pub} . Equivalently, the transcript must determine M_2 via a deterministic PPT map (an “opening-to- $\mathbb{G}$ ” map), so that Y_2 and the subsequent affine auxiliaries are well-defined from the transcript alone.

Design options under $NXK/\mathcal{F}_{\text{KeyBox}}$. Under our KeyBox/gRO-CRP model, there are essentially three ways to make M_2 transcript-defined:

1. Publish M_2 directly: If the leaf can compute $M_2 = m_2\mathcal{G}$ outside the KeyBox, then it can simply transmit M_2 . This trivially makes Y_2 transcript-defined and removes the need for USV. We treat this as a different design point: it bypasses the commit-shaped transcript setting and assumes the leaf has a way to compute and publish $m_2\mathcal{G}$ without relying on straight-line extraction from a KeyBox-internal proof. Our focus is the hardened profile-centric setting in which m_2 is generated inside the KeyBox/profile-adapter and the admissible profile does not expose a generic “export $m\mathcal{G}$ ” interface for fresh ephemeral scalars. For example, when the admissible KeyBox profile corresponds to a cloud KMS role that permits signing/derivation but denies public-key retrieval [1, 36, 52]. In that setting, an alternative attempt using only a hiding commitment (commit-only transcript) would fail. (In the concrete SDKG.LeafInit profile of Section 7 (p. 40), the returned σ -values do allow host-side recovery of m_2 and hence of $m_2\mathcal{G}$; USV is retained so the transcript format is self-certifying without relying on any host-side scalar-to-group computation.)
2. Commit to m_2 and extract via an opening-AoK: One could have the leaf publish $C_2 \leftarrow \text{Commit}(m_2; r_2)$ and an AoK π_{open} of an opening, then let the verifier/simulator extract m_2 and set $M_2 = m_2\mathcal{G}$. In our setting this fails in straight-line: witness-bearing computation may be delegated to a state-continuous KeyBox (Assumption 2 (p. 13)), so rewinding/forking is unavailable; and straight-line extraction for our Fischlin-style UC-NIZK-AoKs relies on access to the prover’s gRO-CRP query log in the proof context (Remark 10 (p. 21)), which is hidden by local-call semantics (Definition 10 (p. 21)). Forcing π_{open} outside the KeyBox restores observability but requires the leaf to materialize (m_2, r_2) , or an equivalent caller-invertible affine image sufficient to derive M_2 , in its non-KeyBox state, which contradicts the hardened/minimal-profile design point of this section: m_2 is not available outside the KeyBox/profile-adapter and the admissible profile exposes no `export- $m_2$` or `export- $(m_2\mathcal{G})$` interface[¶]. Lemma 18 (p. 58) formalizes this obstruction.
3. Add public opening material that deterministically fixes M_2 without exporting m_2 : in the hardened profile-centric setting, the leaf runs $(C_2, \zeta_2) \leftarrow \text{Cert}(\text{pp}, m_2)$ inside the KeyBox boundary and releases only (C_2, ζ_2) ; everyone derives $M_2 = \text{Open}_M(\text{pp}, C_2, \zeta_2)$ outside. USV provides this and makes Y_2 (and the induced transcript-defined auxiliaries and shares) well-defined from the transcript, enabling both honest verification and straight-line UC simulation/programming.

Hence, the SDKG verification predicate and the UC proof need M_2 (hence Y_2) to be a deterministic function of the public transcript, while neither m_2 nor $m_2\mathcal{G}$ appears in τ_{pub} (the σ -values from which m_2 is host-recoverable travel only over $\mathcal{F}_{\text{channel}}$ and are therefore transcript-private; cf. the profile convention in Section 7 (p. 40)), and the KeyBox/local-call semantics prevent straight-line extraction of m_2 from a KeyBox-internal opening-AoK. USV resolves this by providing a public “opening-to- $\mathbb{G}$ ” map $M_2 = \text{Open}_M(\text{pp}, C_2, \zeta_2)$ that is transcript-defined yet does not violate KeyBox’s confidentiality assumptions. Our subsequent transcript analysis is stated in terms of $\text{Open}_M(\text{pp}, C_2, \zeta_2)$ rather than in terms of extracted witnesses.

In our SDKG verification predicate (Algorithm 2 (p. 52)) and in the transcript-driven idealization (Fig. 8 (p. 54)), the value

$$Y_2 := M_2 + 3B_2$$

must be computable in straight-line from the transcript-visible view, where B_2 is public and $M_2 := m_2\mathcal{G}$ is induced by the leaf’s hidden scalar. Accordingly, any design that replaces USV with a commit-only transcript must still enable a PPT straight-line derivation of Y_2 from the public transcript.

Lemma 18 (Commit-only transcripts cannot define M_2 under $NXK/\mathcal{F}_{\text{KeyBox}}$). *Fix the $NXK/\mathcal{F}_{\text{KeyBox}}$ setting in the gRO-CRP model. Let $\text{Com} = (\text{Commit}, \text{Open})$ be a computationally hiding commitment scheme for messages in $\mathbb{Z}_p$. Consider any protocol variant in which the leaf’s transcript-visible contribution contains only a commitment $C_2 := \text{Commit}(m_2; r_2)$ to $m_2 \in \mathbb{Z}_p$ and a public point $B_2 \in \mathbb{G}$, and the transcript contains no additional public material from which $M_2 := m_2\mathcal{G}$ is deterministically computable. Let τ_{pub} denote the transcript-visible view.*

[¶] If one allows such materialization, one can instead publish $M_2 = m_2\mathcal{G}$ directly as mentioned previously.

Assume further that the protocol variant satisfies transcript sampleability: there exists a PPT algorithm $\text{Sample}(\text{pp}, C_2)$ that, given only public parameters and a commitment C_2 , outputs $\tau_{\text{pub}}^{\setminus C_2}$ with a distribution computationally indistinguishable from the real conditional distribution of $\tau_{\text{pub}}^{\setminus C_2}$ given C_2 in an honest execution^{||}.

Suppose there exists a PPT straight-line algorithm Derive_Y such that

$$\Pr [\text{Derive}_Y(\tau_{\text{pub}}) = m_2\mathcal{G} + 3B_2]$$

is non-negligible (over the protocol coins and Derive_Y 's coins), where Derive_Y is given only τ_{pub} together with black-box access to corrupted KeyBoxes via admissible profiles. Then the hiding property of Com is violated.

Proof. Assume for contradiction that there exists a PPT straight-line simulator Sim that outputs the correct Y_2 with non-negligible probability.

Let τ_{pub} denote the *transcript-visible* portion of an execution (*Reader Note 2.1* (p. 8)), i.e., everything outside honest KeyBoxes and outside authenticated confidential channels. In the *commit-only* design point of this lemma, the leaf's only τ_{pub} -dependence on m_2 is through $C_2 = \text{Commit}(m_2; r_2)$ (by hypothesis: the transcript contains no additional public material that deterministically fixes $M_2 = m_2\mathcal{G}$). By the transcript-sampleability hypothesis, the remaining public fields $\tau_{\text{pub}}^{\setminus C_2}$ admit a PPT sampler $\text{Sample}(\text{pp}, C_2)$ that outputs $\tau_{\text{pub}}^{\setminus C_2}$ distributed as in a real execution conditioned on the given C_2 .

Case 1 (transcript-only derivation): Assume Sim 's output Y_2 is determined by τ_{pub} alone (i.e., it does not rely on extracting m_2 from a KeyBox-internal interaction). Formally, fix the induced PPT map $\text{Derive}_Y(\tau_{\text{pub}})$ that outputs the same Y_2 value that Sim outputs on public view τ_{pub} . Since B_2 is public, define

$$\text{Derive}_M(\tau_{\text{pub}}) := \text{Derive}_Y(\tau_{\text{pub}}) - 3B_2 \in \mathbb{G}.$$

By the non-negligible-success hypothesis, $\text{Derive}_Y(\tau_{\text{pub}}) = Y_2 = M_2 + 3B_2$, and thus $\text{Derive}_M(\tau_{\text{pub}}) = M_2 = m_2\mathcal{G}$, with non-negligible probability.

We now build a hiding adversary $\mathcal{B}$ against Com . On input a hiding challenge $C^* = \text{Commit}(m_b; r)$ for $b \in \{0, 1\}$ (for chosen distinct messages $m_0, m_1 \in \mathbb{Z}_p$), $\mathcal{B}$ samples $\tau_{\text{pub}}^{\setminus C_2} \leftarrow \text{Sample}(\text{pp}, C^*)$ and sets $\tau_{\text{pub}} := (C_2 := C^*, \tau_{\text{pub}}^{\setminus C_2})$. It computes $M^* := \text{Derive}_M(\tau_{\text{pub}})$ and outputs $b' = 0$ iff $M^* = m_0\mathcal{G}$. Because τ_{pub} is distributed as a real public transcript conditioned on $C_2 = \text{Commit}(m_b; r)$, the above correctness implies $\Pr[b' = b] \geq \frac{1}{2} + \epsilon(\lambda)$ for some non-negligible ϵ , contradicting computational hiding of Com .

Case 2, Sim obtains m_2 (or M_2) by extraction from an opening-AoK: To avoid Case 1, the only generic route is to require an AoK π_{open} of an opening (m_2, r_2) and extract m_2 to set $M_2 = m_2\mathcal{G}$. If π_{open} is generated outside the KeyBox so that standard extraction applies, then the leaf must first obtain (m_2, r_2) , or any caller-invertible affine image of m_2 sufficient to compute M_2 , in its *non-KeyBox* state. This contradicts the hardened/minimal-profile premise of this subsection, in which m_2 is generated inside the KeyBox/profile-adaptor and no admissible API exports m_2 (or $m_2\mathcal{G}$) in the clear^{**}. If instead π_{open} is generated inside the KeyBox to preserve NXK, then straight-line extraction for our Fischlin-based UC-NIZK(-AoK) mechanisms requires access to the prover's gRO-CRP query log in the relevant proof context (Remark 10 (p. 21)), which is hidden by local-call semantics (Definition 10 (p. 21)); and Assumption 2 (p. 13) rules out rewinding/forking the KeyBox. Thus Sim cannot extract in straight-line in the NXK/ $\mathcal{F}_{\text{KeyBox}}$ model. Combining the cases, Case 2 is ruled out in the NXK/ $\mathcal{F}_{\text{KeyBox}}$ model, so any successful straight-line derivation of Y_2 reduces to Case 1 and yields a non-negligible break of computational hiding for Com . Therefore, an additional public-opening mechanism that deterministically maps commitment material to M_2 (e.g., USV or an explicit M_2) is necessary. ■

The above lemma is specific to the NXK/ $\mathcal{F}_{\text{KeyBox}}$ execution model, where local-call semantics hide the KeyBox's gRO-CRP oracle-query tape (Remark 10 (p. 21)) and Assumption 2 (p. 13) forbids rewinding/forking a KeyBox. If one strengthens the model to permit such extraction capabilities, then the commit-only design may instead recover M_2 via the opening-AoK route discussed in Case 2 of the proof, and Lemma 18 (p. 58) no longer applies.

^{||} This condition holds for commit-only variants where the non- C_2 transcript fields are generated independently of m_2 , or where all m_2 -dependence passes through zero-knowledge proofs or UC-NIZK instances for which a universal simulation interface is available.

^{**} Again, if such materialization is allowed, the protocol can instead publish $M_2 = m_2\mathcal{G}$ directly.

8.3 Main Theorem

In $\mathcal{F}_{\text{SDKG}}$, the state $\mathbf{Q}_{\text{reg}}, \mathbf{D}_{\text{reg}}$ and the (Leak/Deliver/Recv) scheduling are an inlined instance of $\mathcal{F}_{\text{channel}}$ (Fig. 1 (p. 9)), specialized to the two registration payloads with leakage $\ell_{\text{reg},1}, \ell_{\text{reg},2}$.

Lemma 19 (Transcript uniquely determines the key). *Let $\mathcal{T}$ be a base-run (1+1-out-of-3) transcript such that $\text{Acc}_{\text{SDKG}}(\text{sid}, P_2, P_1, \mathcal{T}) = 1$, and let $M_2, Y_1, D_1, Y_2, D_2, \hat{K}$ be the values derived by Acc_{SDKG} from $\mathcal{T}$. Then:*

- (i) *There exist unique $x_1, x_2 \in \mathbb{Z}_p$ such that $X_1 = x_1\mathcal{G}$ and $X_2 = x_2\mathcal{G}$.*
- (ii) *Except with negligible probability in λ , the (straight-line) AoK extractors applied to the verifying DL subproofs in $\pi_{\text{aff}_1} = (\pi_{Y_1}, \pi_{D_1})$ and $\pi_{\text{aff}_2} = (\pi_{Y_2}, \pi_{D_2})$ recover witnesses $\alpha_i, \delta_i \in \mathbb{Z}_p$ such that $Y_i = \alpha_i\mathcal{G}$ and $D_i = \delta_i\mathcal{G}$ for $i \in \{1, 2\}$, and in particular determine*

$$x_1 = \sigma_{2,1} + \alpha_1 + \delta_1 \bmod p, \quad x_2 = \sigma_{1,2} + \alpha_2 + \delta_2 \bmod p.$$

- (iii) *Except with negligible probability in λ , $\text{Acc}_{\text{SDKG}}(\text{sid}, P_2, P_1, \mathcal{T}) = 1$ implies that D_2 equals the unique group element committed via $h_{3,2}$ in the non-programmable digest context SDKG.s32 .*

Proof Sketch. Item (i) is unconditional because $\mathbb{G}$ is cyclic of prime order with generator $\mathcal{G}$, making $z \mapsto z\mathcal{G}$ a bijection on $\mathbb{Z}_p$. Because $\text{Acc}_{\text{SDKG}}(\text{sid}, P_2, P_1, \mathcal{T}) = 1$ (Algorithm 2 (p. 52)), the DL verifications for Y_1, D_1, Y_2, D_2 all accept. By the AoK property of Π_{DL} , except with negligible extraction error the straight-line extractor outputs α_i, δ_i such that $Y_i = \alpha_i\mathcal{G}$ and $D_i = \delta_i\mathcal{G}$ for $i \in \{1, 2\}$. From the definitions $Y_1 := M_1 + 2B_1$ and $D_1 := X_1 - \sigma_{2,1}\mathcal{G} - Y_1$, we get

$$X_1 = \sigma_{2,1}\mathcal{G} + Y_1 + D_1 = (\sigma_{2,1} + \alpha_1 + \delta_1)\mathcal{G},$$

and similarly

$$X_2 = (\sigma_{1,2} + \alpha_2 + \delta_2)\mathcal{G},$$

yielding item (ii).

For item (iii), consider the Round 1 value $h_{3,2}$ and the later derived point D_2 used in Check **C5**. If the sender is honest, it computes $h_{3,2} = H_{\text{s32}}(\langle \text{sid}, \text{cid}_2, \sigma_{3,2}\mathcal{G} \rangle)$ and thus fixes the point $U := \sigma_{3,2}\mathcal{G}$ before D_2 is formed. If the sender is corrupted, there are two possibilities: (i) it queried $H_{\text{s32}}(\langle \text{sid}, \text{cid}_2, U \rangle)$ for some point $U \in \mathbb{G}$ and set $h_{3,2}$ to the reply, or (ii) it outputs a fresh λ -bit guess for $h_{3,2}$ without querying. In case (ii), since $H_{\text{s32}}(\langle \text{sid}, \text{cid}_2, D_2 \rangle)$ is uniform conditioned on the adversary's view, Check **C5** holds with probability at most $2^{-\lambda}$. In case (i), Check **C5** implies $H_{\text{s32}}(\langle \text{sid}, \text{cid}_2, D_2 \rangle) = H_{\text{s32}}(\langle \text{sid}, \text{cid}_2, U \rangle)$. Thus, unless a collision/second-preimage occurs for H_{s32} , we must have $D_2 = U$. (In honest executions this specializes to $D_2 = \sigma_{3,2}\mathcal{G}$.) Finally, since $\hat{K} = 3X_1 - 2X_2$, we obtain $\hat{K} = (3x_1 - 2x_2)\mathcal{G}$ by algebra. $\blacksquare$

Theorem 3 (UC realization of $\mathcal{F}_{\text{SDKG}}$ by $\Psi_{\text{SDKG}}^{(3)}$). *Fix Fischlin parameter functions $(t(\lambda), b(\lambda), r(\lambda), S(\lambda))$ satisfying Definition 16 (p. 25). Then assuming hardness of DL, PRF security of the nonce-derivation function (Definition 20 (p. 40)), and the seed integrity invariant (Assumption 3 (p. 13)), protocol $\Psi_{\text{SDKG}}^{(3)}$ UC-realizes the functionality $\mathcal{F}_{\text{SDKG}}$ in the $(\mathcal{F}_{\text{KeyBox}}, \mathcal{F}_{\text{USV}}, \mathcal{F}_{\text{channel}}, \mathcal{F}_{\text{pub}})$ -hybrid and gRO-CRP models against arbitrary adaptive corruptions of any subset of parties in $\{P_i\}_{i \in [3]}$.*

Proof. Fix any PPT real-world adversary $\mathcal{A}$ and PPT environment $\mathcal{Z}$. We construct a PPT simulator Sim such that

$$\text{Exec}\left(\Psi_{\text{SDKG}}^{(3)}, \mathcal{A}, \mathcal{Z}\right) \approx_c \text{Ideal}(\mathcal{F}_{\text{SDKG}}, \text{Sim}, \mathcal{Z})$$

in the $(\mathcal{F}_{\text{KeyBox}}, \mathcal{F}_{\text{USV}}, \mathcal{F}_{\text{channel}}, \mathcal{F}_{\text{pub}})$ -hybrid and gRO-CRP models, against adaptive corruptions with secure erasures.

Acceptance predicate. A session is accepting iff the deterministic checks in the protocol description verify (USV validity/digest, affine consistency equations, and all UC-NIZK verifications). The simulator applies the same checks and mirrors aborts. By Lemma 15 (p. 53), when P_1 is honest through Round 2 the $\mathcal{F}_{\text{USV}}$.Verify gate is implied by the presence of $\mathbf{T}_2$, hence Algorithm 2 (p. 52) captures acceptance. If P_1 is corrupted, no such implication is required.

UC-NIZK interfaces. All UC-NIZK proofs that matter for extraction use the dedicated UC context(s). By the AoK property of the Fischlin-based UC-NIZK, there exists a straight-line PPT extractor Ext_{DL} that, given a verifying proof for a DL statement and the prover’s gRO-CRP query/answer log under the UC proof context, outputs the corresponding witness except with negligible probability. For an affine proof $\pi_{\text{aff}_i} = (\pi_{Y_i}, \pi_{D_i})$, define $\text{Ext}_{\text{aff}}(\pi_{\text{aff}_i}) := (\text{Ext}_{\text{DL}}(\pi_{Y_i}), \text{Ext}_{\text{DL}}(\pi_{D_i}))$. Moreover, by the ZK property of the same UC-NIZK in the (programmable) gRO-CRP UC contexts, there exists a PPT simulator Sim_{UC} that can produce accepting proofs without witnesses in those contexts.

Simulator Sim. The simulator Sim runs $\mathcal{A}$ as a subroutine and maintains, for each session identifier sid , a session record containing the *functionality transcript slots* $\mathcal{T}_{\text{sid}} := (\mathsf{T}_1, \mathsf{T}_2, \mathsf{T}_3)$ (Fig. 8 (p. 54)), a stage variable, and (when defined) values $x_1(\text{sid}), x_2(\text{sid}), \sigma_{3,1}(\text{sid}), \sigma_{3,2}(\text{sid})$. It forwards all of $\mathcal{A}$ ’s gRO-CRP queries to the global oracle H and logs all $(\text{ctx}, x, H(\text{ctx}, x))$ triples for the UC proof contexts used by $\Pi_{\text{DL}}^{\text{UC}}$. For the non-programmable digest context SDKG.s32 , Sim only forwards queries (no programming). Whenever Sim ’s real-world emulation fixes the value that will populate one of the transcript slots $\mathsf{T}_1, \mathsf{T}_2, \mathsf{T}_3$ for session sid , Sim supplies it to $\mathcal{F}_{\text{SDKG}}$ by sending the corresponding $(\text{Tin}_i, \text{sid}, \dots)$ message on the ideal adversary/simulator interface. These transcript-slot values are used only to drive $\mathcal{F}_{\text{SDKG}}$ ’s deterministic acceptance/finalization logic and may include fields that are not adversary-visible in honest executions (they can be carried only over $\mathcal{F}_{\text{channel}}$); they are never output by $\mathcal{F}_{\text{SDKG}}$ and are hidden from $\mathcal{Z}$ by the wrapper $\mathcal{W}_{\text{DKG}}$. $\mathcal{F}_{\text{SDKG}}$ ’s stored transcript slots track the simulated real transcript under adversarial scheduling.

$\mathcal{F}_{\text{pub}}$ publications. Since $\mathcal{F}_{\text{pub}}$ is a hybrid functionality present in both real and ideal worlds, the simulator must reproduce the $\mathcal{F}_{\text{pub}}$ publications that honest parties make in the real protocol. Concretely, when Sim emulates an honest P_2 completing Round 3 (publishing K via $\mathcal{F}_{\text{pub}}$), Sim sends $(\text{Publish}, \text{sid}, K)$ to $\mathcal{F}_{\text{pub}}$ on behalf of the simulated honest P_2 . Likewise, when Sim emulates an honest P_1 accepting the transcript, Sim sends $(\text{Publish}, \text{sid}, K)$ to $\mathcal{F}_{\text{pub}}$ on behalf of the simulated honest P_1 . These calls are to the real $\mathcal{F}_{\text{pub}}$ instance in the hybrid model, ensuring that P_3 (and any environment $\mathcal{Z}$ listening on $\mathcal{F}_{\text{pub}}$ delivery ports) observes identical $\mathcal{F}_{\text{pub}}$ publications in both real and ideal executions.

$\mathcal{F}_{\text{channel}}$ messages. Since $\mathcal{F}_{\text{channel}}$ is a hybrid functionality present in both real and ideal worlds, Sim must reproduce the $\mathcal{F}_{\text{channel}}$ interactions that honest parties perform in the real protocol. Concretely, when Sim ’s honest-party emulation produces a Round 1 message from an honest P_2 , Sim sends $(\text{Send}, \text{sid}, c_1)$ to the $\mathcal{F}_{\text{channel}}$ instance for the (P_2, P_1) endpoint, where c_1 is the simulated Round 1 payload; this generates the corresponding Leak ticket that $\mathcal{A}$ observes. Likewise for Round 2 (honest $P_1 \rightarrow P_2$) and Round 3 (honest $P_2 \rightarrow P_1$). When $\mathcal{A}$ issues Deliver commands, $\mathcal{F}_{\text{channel}}$ delivers the original honest-party messages, matching the real-world delivery semantics. Since Sim sends the same payloads at the same simulation points, $\mathcal{A}$ (and $\mathcal{Z}$) observes identical $\mathcal{F}_{\text{channel}}$ leak tickets, scheduling metadata, and delivery patterns in both worlds.

$\mathcal{F}_{\text{USV}}$ interactions. Since $\mathcal{F}_{\text{USV}}$ is a hybrid functionality present in both real and ideal worlds, Sim must issue the corresponding $\mathcal{F}_{\text{USV}}$ calls as part of its honest-party emulation. When Sim emulates an honest P_2 in Round 1, Sim sends $(\text{Commit}, \text{sid}, \text{cid}_2, P_1, C_2, \zeta_2)$ to $\mathcal{F}_{\text{USV}}$ on behalf of the simulated honest P_2 and records the receipt d . When $\mathcal{A}$ ’s Deliver command causes $\mathcal{F}_{\text{channel}}$ to deliver P_2 ’s Round 1 message to the honest P_1 , Sim concurrently sends $(\text{Deliver}, \text{sid}, \text{cid}_2, P_2, P_1)$ to $\mathcal{F}_{\text{USV}}$ to set $\text{del} \leftarrow 1$ for the corresponding entry, so that the subsequent $\mathcal{F}_{\text{USV}}$.Verify call does not return $\perp$. Sim then emulates honest P_1 in Round 2 by calling $\mathcal{F}_{\text{USV}}$.Verify($\text{sid}, \text{cid}_2, P_2, C_2, \zeta_2$) and checks the return value, aborting if it returns 0 or $\perp$ (matching the real-world P_1 ’s behavior). These calls ensure that $\mathcal{F}_{\text{USV}}$ ’s internal table T is populated and delivery-activated identically in both worlds, so any subsequent query by a corrupted counterparty to $\mathcal{F}_{\text{USV}}$.Verify or $\mathcal{F}_{\text{USV}}$.Open returns the same result as in the real execution. Note that Lemma 15 (p. 53) concerns only the acceptance predicate Acc_{SDKG} : it says that Acc_{SDKG} need not separately check the $\mathcal{F}_{\text{USV}}$.Verify return value (because its success is implied by the existence of T_2). It does not mean Sim omits the $\mathcal{F}_{\text{USV}}$.Verify call; the call is part of Sim ’s honest-party emulation and its result is guaranteed to match by construction.

Hybrid argument. We define hybrids over the joint execution (potentially many concurrent sessions); Sim keeps a separate record per sid . Since the environment $\mathcal{Z}$, adversary $\mathcal{A}$, and all parties are PPT, the total number of sessions initiated and the total number of UC-context proofs simulated/verified in the execution are bounded by $\text{poly}(\lambda)$.

Specifically, for each programmable proof context $\text{ctx} \in \text{Ctx}_p$, the simulator makes at most $m_{\text{ctx}}(\lambda) = \text{poly}(\lambda)$ total calls to $\text{SimProgram}(\text{ctx}, \cdot, \cdot)$ across all sessions. Therefore, Lemma 4 (p. 22) applies with this global $m_{\text{ctx}}(\lambda)$, and the resulting union bound over all programming attempts (across all sessions) remains negligible. Indistinguishability is shown by a standard per-session hybrid argument together with this global union bound.

PRF assumption for LinOS nonces: In the real execution, each honest KeyBox instance derives LinOS nonces via $\text{PRF}(\text{seed}, \cdot)$ (Fig. 6 (p. 40)). By PRF security (Definition 20 (p. 40)) and the seed integrity invariant (Assumption 3 (p. 13)), these derived nonces are computationally indistinguishable from independently sampled uniform values in $\mathbb{Z}_p$. Concretely, a standard PRF-to-random hybrid replaces all $\text{PRF}(\text{seed}, \cdot)$ evaluations with outputs of a truly random function; the distinguishing advantage of this step is bounded by the PRF advantage $\text{negl}(\lambda)$. After this replacement, the nonces have the same joint distribution as in the original (sample-and-store) construction, so all subsequent hybrids proceed identically. This step introduces an additive $\text{negl}(\lambda)$ term from the PRF assumption, which is already absorbed into the asymptotic bound. By Lemma 14 (p. 42), this deterministic derivation additionally ensures that even under a rollback of the KeyBox’s mutable state (violating state continuity for the one-shot seal), the adversary cannot obtain two transcripts with the same commitment and different challenges, and therefore cannot extract the resident share via Schnorr special soundness.

Hybrid $\mathcal{D}_0$ (real execution): This is $\Psi_{\text{SDKG}}^{(3)}$ with honest parties, in the $(\mathcal{F}_{\text{KeyBox}}, \mathcal{F}_{\text{USV}}, \mathcal{F}_{\text{channel}}, \mathcal{F}_{\text{pub}})$ -hybrid and gRO-CRP models.

Hybrid $\mathcal{D}_1$ (simulate honest UC-NIZKs): Modify $\mathcal{D}_0$ as follows: whenever an honest party would output a UC-context UC-NIZK proof, replace it with a proof generated by Sim_{UC} in the corresponding UC context. All other values/messages are unchanged. By the ZK property of the Fischlin-based UC-NIZK in the UC gRO-CRP contexts, we have $\mathcal{D}_1 \approx_c \mathcal{D}_0$. Concretely, Sim uses SimProgram to realize the universal simulation interface; because SimProgram fails on already-defined points, the only divergence is the pre-query bad event of Lemma 4 (p. 22), which occurs with negligible probability.

Bookkeeping. In $\mathcal{D}_1$, Sim still samples and records (per session sid) the honest-party local scalars that determine the public points X_1, X_2 and the digest check (e.g., the values used to form $X_1 = x_1\mathcal{G}$, $X_2 = x_2\mathcal{G}$, $\sigma_{3,1}\mathcal{G}$, and $\sigma_{3,2}\mathcal{G}$), exactly as in $\mathcal{D}_0$; only the sent UC-context proof strings are replaced by simulated proofs via Sim_{UC} . Importantly, Sim will never invoke the Fischlin/AoK extractor on any proof output by Sim_{UC} , which may have been produced using SimProgram .

Lemma 20 (Fresh tagged statements for SDKG extraction). *Let Q be the set of UC-context DL statements (across all sessions in the execution) for which the simulator invoked the UC-proof simulator Sim_{UC} . In any session sid , SDKG verifies UC-context DL proofs only on tagged statements of the form $\langle \text{sid}, \ell, M \rangle$ where the label ℓ is fixed by the proof position (one of SDKG.aff1.Y , SDKG.aff1.D , SDKG.aff2.Y , SDKG.aff2.D). Then any UC-context DL proof contributed by a corrupted party in session sid and verified by an honest party is on a tagged statement x satisfying $x \notin Q$.*

Proof Sketch. Fix any session identifier sid . By construction of Hybrid $\mathcal{D}_1$, the simulator invokes the UC-proof simulator Sim_{UC} only for UC-context DL proofs that are generated by honest parties in that session (i.e., in honest proof positions). Let

$$Q_{\text{sid}} := \{x \in Q : x \text{ is of the form } \langle \text{sid}, \ell, M \rangle\}.$$

Then Q_{sid} contains exactly the tagged statements for which Sim_{UC} produced simulated UC-context proofs in session sid . Consider any UC-context DL proof that is verified by an honest party and whose originating party is *currently* corrupted at verification time. Two sub-cases arise:

Case 1: the party was corrupted at proof-generation time. Then Sim did not invoke Sim_{UC} for that proof position, so the corresponding tagged statement $x \notin Q_{\text{sid}}$ and thus $x \notin Q$. Simulation-extractability applies and Sim extracts a valid witness.

Case 2: the party was honest at proof-generation time but was corrupted after the proof-bearing message was queued in $\mathcal{F}_{\text{channel}}$ and before delivery. In this case Sim *did* invoke Sim_{UC} for the proof (since the party was honest at generation

time), so the statement x is in Q . However, Sim does not need to extract from this proof: it already holds the bookkeeping values $(x_1, x_2, \sigma_{3,1}, \sigma_{3,2})$ that it committed during honest simulation, and these remain valid regardless of the party's later corruption status. $\mathcal{F}_{\text{channel}}$ delivers the original message unchanged, so the transcript-defined group elements match the bookkeeping. Therefore the simulator uses the bookkeeping path (not extraction) for such proofs, and freshness relative to Q is not required.

Since the tag sid is part of the statement, statements from different sessions cannot collide; injectivity of $\langle \cdot \rangle$ rules out any other collisions. $\blacksquare$

Hybrid $\mathcal{D}_2$ (extract from adversarial UC-NIZKs; abort on failure): Modify $\mathcal{D}_1$ as follows: whenever $\mathcal{A}$ delivers a verifying UC-context affine proof $\pi_{\text{aff}_i} = (\pi_{Y_i}, \pi_{D_i})$ attributed to a party that was corrupted *at proof-generation time* (proofs generated by a then-honest party that was later adaptively corrupted are handled via the simulator's bookkeeping values without extraction; see Lemma 20 (p. 62), Case 2), Sim runs $\text{Ext}_{\text{aff}}(\pi_{\text{aff}_i})$ using $\text{Log}_{\mathcal{A}}$ in that UC context to obtain (α_i, δ_i) . If extraction fails or the extracted witnesses do not satisfy the relation, abort the session.

Because $\mathcal{D}_1$ may expose $\mathcal{A}$ to simulated proofs produced using SimProgram , the appropriate guarantee here is *simulation-extractability* (Definition 13 (p. 24)), not merely plain AoK soundness. Concretely, let Q be the set of UC-context statements for which Sim invoked the simulator Sim_{UC} . By Lemma 20 (p. 62), every corrupted-party UC-context DL statement on which Sim runs extraction is fresh relative to the simulator-produced statement set Q . Therefore, the simulation-extractability guarantee of Lemma 7 (p. 28)(iii) applies to these proofs and we get the bad event

$$\text{Bad}_{\text{ext/forge}} := \{\text{a corrupted party's UC-context proof verifies but no valid witness is extracted}\}$$

occurs with probability at most $\text{negl}(\lambda)$, and thus $\mathcal{D}_2 \approx_c \mathcal{D}_1$.

Hybrid $\mathcal{D}_3$ (switch to the ideal functionality via transcript-defined programming): Modify $\mathcal{D}_2$ by replacing the real-world key-derivation effect with interaction with $\mathcal{F}_{\text{SDKG}}$ as follows. Sim continues to simulate the network transcript towards $\mathcal{A}$ and $\mathcal{Z}$ and mirrors the same accept/abort predicate. Whenever $\mathcal{D}_2$ reaches an accepting transcript for some session sid , Sim defines $(x_1(\text{sid}), x_2(\text{sid}), \sigma_{3,1}(\text{sid}), \sigma_{3,2}(\text{sid}))$ as follows, without extracting from any simulated proof:

Fix a session sid and suppose the simulator's emulated transcript reaches a point where the stored transcript slots $\mathcal{T}_{\text{sid}} = (\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3)$ satisfy $\text{Acc}_{\text{SDKG}}(\text{sid}, P_2, P_1, \mathcal{T}_{\text{sid}}) = 1$ (Algorithm 2 (p. 52)). Let Y_1, D_1, Y_2, D_2 be the derived points from $\mathcal{T}_{\text{sid}}$, and write $\pi_{\text{aff}_i} = (\pi_{Y_i}, \pi_{D_i})$.

Define $\text{Bad}_{\text{ext}}(\text{sid})$ to be the event that, for some DL subproof in this session that is attributed to a corrupted party and verifies under $\mathcal{V}_{\text{DL}}$, straight-line extraction fails or yields a value not satisfying the DL relation. Let $\text{Bad}_{\text{s32}}(\text{sid})$ be the event that the s32-digest check in Acc_{SDKG} holds by a fresh guess without a prior query or by an oracle collision/second-preimage. In Hybrid $\mathcal{D}_2$, the simulator aborts the session on $\text{Bad}_{\text{ext}}(\text{sid})$, so below we condition on $\neg \text{Bad}_{\text{ext}}(\text{sid})$; and by standard RO reasoning, $\Pr[\text{Bad}_{\text{s32}}(\text{sid})]$ is negligible.

Conditioned on $\neg(\text{Bad}_{\text{ext}}(\text{sid}) \vee \text{Bad}_{\text{s32}}(\text{sid}))$, the simulator defines $(x_1, x_2, \sigma_{3,1}, \sigma_{3,2})$ by the following corruption-pattern analysis and then issues the one-shot $\mathcal{F}_{\text{SDKG}}$ programming command $(\text{Program}, \text{sid}, x_1, x_2, \sigma_{3,1}, \sigma_{3,2})$.

Hence, for corrupted parties, the simulator never needs to argue that the extracted witnesses equal some hidden internal σ -variables of the adversary. It only needs them to satisfy $Y_i = \alpha_i \mathcal{G}$ and $D_i = \delta_i \mathcal{G}$ for the transcript-defined points Y_i, D_i . Because $z \mapsto z\mathcal{G}$ is a bijection, these witnesses (when extracted) are uniquely determined by Y_i and D_i , and therefore the programmed values are canonical given the accepting transcript. Finally, by Lemma 19 (p. 60), the resulting programmed key is $K = (3x_1 - 2x_2)\mathcal{G}$, matching the real transcript-derived key $\hat{K}$ whenever the session accepts.

Then Sim invokes the one-shot programming interface of $\mathcal{F}_{\text{SDKG}}$:

$$(\text{Program}, \text{sid}, x_1(\text{sid}), x_2(\text{sid}), \sigma_{3,1}(\text{sid}), \sigma_{3,2}(\text{sid})).$$

By transcript mirroring, at the point Sim invokes $(\text{Program}, \text{sid}, \dots)$ the ideal $\mathcal{F}_{\text{SDKG}}$ has already received the same public transcript items via $\text{Tin}_1/\text{Tin}_2/\text{Tin}_3$ (up to adversarial scheduling), and since $\mathcal{F}_{\text{SDKG}}$ runs TryFinalize on both transcript stores and Program , the ideal output event is triggered exactly when the simulated transcript becomes accepting.

Pattern	How Sim fixes $(x_1, \sigma_{3,1})$	How Sim fixes $(x_2, \sigma_{3,2})$
P_1 honest, P_2 honest	From the honest emulation record: $X_1 = x_1\mathcal{G}$ and $D_1 = \sigma_{3,1}\mathcal{G}$ were formed using sampled scalars; Sim sets $(x_1, \sigma_{3,1})$ to those recorded values.	From the honest emulation record: $X_2 = x_2\mathcal{G}$ and $h_{3,2} = H_{s32}(\langle \text{sid}, \text{cid}_2, \sigma_{3,2}\mathcal{G} \rangle)$ were formed using sampled $\sigma_{3,2}$; Sim sets $(x_2, \sigma_{3,2})$ to those recorded values.
P_1 corrupted, P_2 honest	Extract (α_1, δ_1) from $\pi_{\text{aff}_1} = (\pi_{Y_1}, \pi_{D_1})$ and set $\sigma_{3,1} := \delta_1$, $x_1 := \sigma_{2,1} + \alpha_1 + \delta_1$.	As in the all-honest row (recorded from honest P_2 emulation).
P_1 honest, P_2 corrupted	As in the all-honest row (recorded from honest P_1 emulation).	Extract (α_2, δ_2) from $\pi_{\text{aff}_2} = (\pi_{Y_2}, \pi_{D_2})$ and set $\sigma_{3,2} := \delta_2$, $x_2 := \sigma_{1,2} + \alpha_2 + \delta_2$.
P_1 corrupted, P_2 corrupted	Extract (α_1, δ_1) and (α_2, δ_2) and define $\sigma_{3,1} := \delta_1$, $x_1 := \sigma_{2,1} + \alpha_1 + \delta_1$ and $\sigma_{3,2} := \delta_2$, $x_2 := \sigma_{1,2} + \alpha_2 + \delta_2$.	(same as left cell)

Table 5: Explicit determination of the programmed values in Theorem 3 (p. 60).

Ordering and slot consistency for registration. In Fig. 8 (p. 54), the registration handler is gated on `finalized = 1`. By construction, `finalized` is set only inside `TryFinalize`, and `TryFinalize` returns early unless the simulator has already supplied the one-shot `Program` message fixing $(x_1, x_2, \sigma_{3,1}, \sigma_{3,2})$. Therefore, every registration attempt occurs only after the above programming step has fixed these values.

Moreover, at the same point where `TryFinalize` sets `finalized` $\leftarrow 1$, it deterministically defines the registration scalars $(\sigma_{3,1}, \sigma_{3,2}, \sigma_{2,3})$ (Fig. 8 (p. 54), Steps (1)–(2)) and, for each honest sender, issues the corresponding KeyBox installation commands that load these scalars into the dedicated slots $\langle \text{sid}, \text{k31} \rangle$ (for P_1) and $\langle \text{sid}, \text{k32} \rangle, \langle \text{sid}, \text{k23} \rangle$ (for P_2) (Fig. 8 (p. 54)). Consequently, in the real protocol the subsequent sealing calls in Algorithm 1 (p. 51) encrypt exactly these same resident values, i.e.,

$$\varpi_1 \stackrel{d}{=} \text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}_{31}, \sigma_{3,1}), \quad \varpi_{2a} \stackrel{d}{=} \text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}_{32}, \sigma_{3,2}), \quad \varpi_{2b} \stackrel{d}{=} \text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}_{23}, \sigma_{2,3}),$$

which matches the ciphertext sampling performed by $\mathcal{F}_{\text{SDKG}}$ on honest senders in the ideal world. If a sender is corrupted, both the real protocol and $\mathcal{F}_{\text{SDKG}}$ allow the adversary to supply the delivered payload directly (via w_1^*, w_2^*), so no additional slot-consistency condition is required in that case.

By the gRO-CRP assumption for the non-programmable context SDKG.s32 , $\Pr[\text{Bad}_{s32}] \leq \text{negl}(\lambda)$. Conditioned on $\neg(\text{Bad}_{\text{ext/forge}} \vee \text{Bad}_{s32})$, the extracted scalars satisfy the relations in Lemma 19 (p. 60), and in particular the real-world output satisfies $K_{\text{real}} = (3x_1(\text{sid}) - 2x_2(\text{sid}))\mathcal{G}$. After programming, $\mathcal{F}_{\text{SDKG}}$ outputs

$$K(\text{sid}) = (3x_1(\text{sid}) - 2x_2(\text{sid}))\mathcal{G} = K_{\text{real}}.$$

Thus, conditioned on no bad event, the public key output seen by $\mathcal{Z}$ is identical in $\mathcal{D}_3$ and $\mathcal{D}_2$, and the simulated transcript/abort behavior is unchanged. Hence $\mathcal{D}_3 \approx_c \mathcal{D}_2$.

Registration of P_3 . In $\mathcal{D}_3$, when $\mathcal{F}_{\text{SDKG}}$ receives `(register, 3, sid)` and `finalized = 1`, the resulting interaction with $\mathcal{F}_{\text{KeyBox}}$ is exactly as specified by $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)). The only observable outcome is whether the host obtains $K_3 = \text{PubMap}(k_3)$ from $\mathcal{F}_{\text{KeyBox}}^{(3)}.\text{Use}$. Since $\mathcal{F}_{\text{KeyBox}}$ is an ideal functionality whose internal state is never revealed, and since Sim forwards $\mathcal{F}_{\text{KeyBox}}$ calls on behalf of corrupted parties exactly as in the real execution, $\mathcal{Z}$'s observable view of registration matches the real execution. Additionally, $\mathcal{F}_{\text{SDKG}}$ emits two adversary-scheduled, length-leaking channel messages (via `Leak/Deliver/Recv`) that model the sealed-payload transport in Algorithm 1 (p. 51). Concretely, $\mathcal{F}_{\text{SDKG}}$ sets the delivered payloads to $w_{\text{reg},1} := \langle \text{sid}, \varpi_1, K_{1,3}, K \rangle$ and $w_{\text{reg},2} := \langle \text{sid}, \varpi_{2a}, \varpi_{2b}, K_{1,3} \rangle$, where $\varpi_1, \varpi_{2a}, \varpi_{2b}$ are fresh sealing ciphertexts sampled as $\text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}, \sigma)$. Lemma 21 (p. 65) formalizes that: if P_3 is corrupted, $\mathcal{A}$ observes well-formed ciphertexts and can invoke `OpenFromPeer` on them exactly as in the real protocol; and if P_3 is honest, the adversary learns only the explicit length leakage $\ell_{\text{reg},1}, \ell_{\text{reg},2}$.

$\mathcal{F}_{\text{pub}}$ -disagreement abort. If Sim detects that $\mathcal{A}$ caused a corrupted sender to publish a value on $\mathcal{F}_{\text{pub}}$ for `sid` different from the finalized K , then Sim does not forward `(RegGo, ...)` to $\mathcal{F}_{\text{SDKG}}$. This prevents registration from completing in

the ideal world, matching the real-world abort behavior of honest P_3 under $\mathcal{F}_{\text{pub}}$ disagreement (Algorithm 1 (p. 51)). More precisely, in the real protocol the honest joiner gates registration on receiving consistent $\mathcal{F}_{\text{pub}}$ publications from both senders; when the simulated $\mathcal{F}_{\text{pub}}$ publications disagree, Sim withholds RegGo so that $\mathcal{F}_{\text{SDKG}}$ never reaches its completion check. Since $\mathcal{F}_{\text{pub}}$ is a shared hybrid functionality, the environment observes identical $\mathcal{F}_{\text{pub}}$ publications in both worlds, and in both worlds registration does not complete.

Lemma 21 (Registration/RDR simulation). *Fix a session sid after base finalization. Under $\mathcal{F}_{\text{channel}}$ and the KeyBox sealing interfaces SealToPeer/OpenFromPeer (Fig. 3 (p. 11)), the registration phase of Algorithm 1 (p. 51) is indistinguishable from the registration subroutine implemented by $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)), for any adaptive corruption pattern with secure erasures.*

More precisely: (i) if the receiver P_3 is honest, the adversary learns only the explicit length leakage $\ell_{\text{reg},1}, \ell_{\text{reg},2}$, and $\mathcal{F}_{\text{SDKG}}$ produces exactly the same leakage and (ii) if P_3 is corrupted, then the delivered payloads in $\mathcal{F}_{\text{SDKG}}$ contain ciphertext components sampled as $\text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}, \sigma)$, which matches exactly the distribution of real SealToPeer outputs, so the adversary's view (including subsequent OpenFromPeer calls) is identically distributed.

Proof Sketch. By definition of $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)), registration is enabled only once $\text{finalized} = 1$, which implies that $(\sigma_{3,1}, \sigma_{3,2}, \sigma_{2,3})$ have been fixed by TryFinalize (after the simulator's one-shot Program) and, for honest senders, loaded into the corresponding KeyBox slots used by SealToPeer.

If a sender is corrupted, both the real protocol and $\mathcal{F}_{\text{SDKG}}$ allow the adversary to supply arbitrary payloads (via w_1^*, w_2^*), yielding identical distributions. If the receiver is honest, $\mathcal{F}_{\text{channel}}$ reveals only message lengths; $\mathcal{F}_{\text{SDKG}}$ matches this via $(\text{Leak}, \cdot, \ell_{\text{reg},1}/\ell_{\text{reg},2})$. If the receiver is corrupted, then in the real protocol each ciphertext is produced by SealToPeer as $c \leftarrow \text{Enc}_{\text{pk}_{\text{seal}}^{(P_3)}}(\text{ad}, \sigma)$; $\mathcal{F}_{\text{SDKG}}$ samples the same distribution explicitly, so the delivered messages are identically distributed.

If the sponsor (either P_2 or an already-registered leaf) is corrupted, it may try to disrupt registration by sending malformed or altered ciphertext components in $(\varpi_{2a}, \varpi_{2b})$. However, P_3 recomputes the intended associated-data strings $\text{ad}_{32}, \text{ad}_{23}$ locally (Algorithm 1 (p. 51)) and supplies them to OpenFromPeer; by AEAD integrity in $\mathcal{F}_{\text{KeyBox}}$ (Fig. 3 (p. 11)), any tampering or mix-and-match across sessions/slots causes OpenFromPeer to return $\perp$ and the KeyBox-side installation transaction aborts. If instead the corrupted sponsor crafts fresh ciphertexts under the correct associated data but encrypting arbitrary scalars, the recovery-share derivation $\text{Load}(\langle \text{sid}, \text{k3} \rangle, g_3, \cdot)$ still uniquely determines the installed share k_3 via $K_{1,3} + k_3 \mathcal{G} = K$ (Definition 21 (p. 44)). Because g_3 constrains only the linear combination $\sigma_{2,3} - \sigma_{3,2}$ (not the individual values), a corrupted sponsor may shift both sponsor-state slots by a common offset Δ without affecting k_3 (cf. Theorem 4 (p. 69), cases (b)–(c)). Thus, a malicious sponsor can at most cause denial of registration or shift the per-device sponsor-state slots, but cannot make an honest joiner install a recovery share inconsistent with the established public key K .

Finally, when P_3 is honest, both worlds attempt the same KeyBox-internal sequence of OpenFromPeer and Load calls, so the observable success/failure outcome (and any subsequent public GetPub output) matches. ■

Installation timing and TryFinalize activation semantics. In the real protocol, P_2 installs its KeyBox shares during its Round 3 activation (before P_1 receives the message), while P_1 installs after receiving and verifying P_2 's Round 3 message. The simulator feeds $(\text{Tin}_3, \text{sid}, \dots)$ when the emulated honest P_2 produces its Round 3 message. This triggers TryFinalize, which issues KBcmd to both parties' KeyBox-driver wrappers (Remark 6 (p. 16)). Each wrapper processes the command upon the respective party's activation: P_2 's wrapper executes within P_2 's current activation (matching P_2 's immediate real-world installation), and P_1 's wrapper executes when P_1 is activated to process the $\mathcal{F}_{\text{channel}}$ delivery of Round 3 (matching P_1 's real-world installation timing). Thus, the $\mathcal{F}_{\text{KeyBox}}$ state at each party matches in both worlds at every corruption point.

Adaptive corruptions with secure erasures: explicit state consistency. We make explicit how the simulator answers adaptive corruptions under Definition 1 (p. 8). For each session sid and each honest party P_i , the simulator maintains a shadow host state st_i^{sid} consisting of exactly those host variables that the real protocol retains (i.e., does not erase) after P_i 's most recent honest activation in that session. Because honest activations are atomic w.r.t. corruption (Definition 1 (p. 8)), a real corruption reveals precisely the current st_i^{sid} and nothing erased within past activations. The simulator

	Pre-install shadow state	Post-install shadow state
Honest P_1	$\{\sigma_{2,1}, \sigma_{1,1}, \sigma_{1,3}, \sigma_{3,1}\}$ (plus public transcript items)	$\emptyset$ (all share-deriving scalars erased immediately after the Load calls return)
Honest P_2	$\{\sigma_{1,2}, \sigma_{2,2}, \sigma_{2,3}, \sigma_{3,2}\}$ (plus public transcript items)	$\emptyset$
Honest P_3	$\emptyset$	$\emptyset$

Table 6: NXK-restricted host variables retained across activations in $\Psi_{\text{SDKG}}^{(3)}$ under the stated erasure discipline.

updates st_i^{sid} in lockstep with its honest-party emulation and, upon corruption of P_i , returns st_i^{sid} (together with the already transcript-visible messages) as the revealed host state.

Crucially, in the base run the only NXK-restricted values that remain in host RAM across activations are the σ -scalars needed for the deferred post-accept $\mathcal{F}_{\text{KeyBox}}.\text{Load}$ calls; all UC-NIZK/Fischlin prover randomness and rarity-search scratch state is erased in the same activation that emits the proof-bearing message (as stated in the protocol’s erasure discipline), and therefore never appears in st_i^{sid} and is never revealed by later corruptions.

Consistency with $\mathcal{F}_{\text{SDKG}}$. Whenever Sim supplies $\text{Tin}_1/\text{Tin}_2/\text{Tin}_3$ to $\mathcal{F}_{\text{SDKG}}$ for session sid , any scalar fields not transcript-visible in honest executions (e.g., σ -values carried over $\mathcal{F}_{\text{channel}}$) are taken from the same shadow states st_i^{sid} . Thus, if an honest party is corrupted before post-accept installation, the corruption reveals exactly the values that (in both the real and ideal worlds) would be consumed by the pending **Load** calls; if it is corrupted after installation, these values have been erased and both worlds expose only black-box access to already-installed KeyBox slots (Definition 6 (p. 15)).

Let $\text{Bad} := \text{Bad}_{\text{ext/forge}} \cup \text{Bad}_{\text{s32}}$. We have $\Pr[\text{Bad}] \leq \text{negl}(\lambda)$, and conditioned on $\neg \text{Bad}$, the $\mathcal{D}_2$ -to- $\mathcal{D}_3$ transition produces identical transcripts, accept/abort behavior, public key outputs, and identically distributed corruption views. Together with the computational losses absorbed in the earlier $\approx_c$ transitions—the PRF-to-random hybrid step (introducing $\text{Adv}_{\text{PRF}}(\lambda)$), the ZK simulation in $\mathcal{D}_0 \rightarrow \mathcal{D}_1$ including the pre-query bound of Lemma 4 (p. 22), and the simulation-extractability step in $\mathcal{D}_1 \rightarrow \mathcal{D}_2$ —the total simulation error is bounded by $\text{Adv}_{\text{PRF}}(\lambda) + \Pr[\text{Bad}_{\text{pre}}] + \Pr[\text{Bad}_{\text{ext/forge}}] + \Pr[\text{Bad}_{\text{s32}}]$, each term being $\text{negl}(\lambda)$. The registration portion of the simulation follows from Lemma 21 (p. 65). Therefore,

$$\text{Exec}\left(\Psi_{\text{SDKG}}^{(3)}, \mathcal{A}, \mathcal{Z}\right) \approx_c \text{Ideal}(\mathcal{F}_{\text{SDKG}}, \text{Sim}, \mathcal{Z}).$$

This completes the proof for Theorem 3 (p. 60). ■

Remark 25 (Dominating bad events / failure terms). In the proof of Theorem 3 (p. 60), the simulation error is dominated by the union of the following explicit bad events:

- Bad_{sc} : a violation of state continuity (rollback/fork) for any KeyBox instance relied upon by the protocol (Assumption 2 (p. 13)). In a concrete realization that approximates state continuity via a freshness mechanism that can fail (e.g., counter exhaustion or oracle unavailability), let $\varepsilon_{\text{sc}}(\lambda)$ bound $\Pr[\text{Bad}_{\text{sc}}]$; then all distinguishing/forgery bounds in the paper should be read as $\text{negl}(\lambda) + \varepsilon_{\text{sc}}(\lambda)$.
- Bad_{prf} : the PRF security of the nonce-derivation function keyed by **seed** (Definition 20 (p. 40)) is broken, enabling an adversary to predict or correlate LinOS nonces across sessions. Under PRF security and the seed integrity invariant (Assumption 3 (p. 13)), $\Pr[\text{Bad}_{\text{prf}}] \leq \text{negl}(\lambda)$. Notably, by Lemma 14 (p. 42), even if Bad_{sc} occurs (a rollback of mutable KeyBox state), the deterministic nonce derivation prevents the catastrophic “rollback $\Rightarrow$ key disclosure” path: replaying the same (sid, K) reproduces the same transcript and reveals no new information about k .
- Bad_{rcpt} : a second-preimage/collision for a receipt-binding digest in a non-programmable context (e.g., for $H(\text{USV}.\text{rcpt}, \cdot)$ in $\mathcal{F}_{\text{USV}}$).
- Bad_{s32} : either (a) a successful λ -bit guess of $h_{3,2} = H_{\text{s32}}(\langle \text{sid}, \text{cid}_2, D_2 \rangle)$ without having queried that point, or (b) a collision/second-preimage under H_{s32} that allows an accepting transcript with $D_2 \neq U$, where U is the point committed by $h_{3,2}$.
- Bad_{pre} : a pre-query collision that causes a simulator programming attempt $\text{SimProgram}(\text{ctx}, x, y)$ with $\text{ctx} \in \text{Ctx}_p$ to return $\perp$ (Lemma 4 (p. 22)).

- Bad_{fs} : a Fischlin knowledge/soundness failure for any verifying UC-context NIZK, i.e., acceptance of a proof for which straight-line extraction fails or yields no valid witness (Lemma 6 (p. 27)).
- Bad_{hr} : honest rejection of an honest Fischlin proof due to a capped rarity-search miss. This is a liveness failure only (abort/retry) and is not a security break.
- Bad_{hdl} : an adversary guess of a live KeyBox opaque handle (e.g., in buf), allowing unintended resolution/use; bounded by a union bound over polynomially many guesses as $\Pr[\text{Bad}_{\text{hdl}}] \leq \text{poly}(\lambda) \cdot 2^{-\lambda}$.

By a union bound, the security distinguishing advantage is upper-bounded by

$$\Pr[\text{Bad}_{\text{rcpt}}] + \Pr[\text{Bad}_{\text{s32}}] + \Pr[\text{Bad}_{\text{pre}}] + \Pr[\text{Bad}_{\text{fs}}] + \Pr[\text{Bad}_{\text{hdl}}],$$

and each term is negligible under the stated hypotheses of Theorem 3 (p. 60). When compiling out $\mathcal{F}_{\text{USV}}$ to the concrete USV protocol (Corollary 2 (p. 67)), additional negligible terms stemming from the USV/DLEQ-based instantiation are bounded under DDLEQ as in Theorem 2 (p. 36) and Lemmas 8 (p. 32) and 9 (p. 34).

If it holds for the corrupted set that $B \in \Gamma$, no secrecy claims can be made on the conceptual signing key k . Even then, the functionality and the real protocol guarantee non-exportability: long-term secrets remain confined to the respective $\mathcal{F}_{\text{KeyBox}}^{(i)}$ instances and the adversary only obtains black-box access via **Use**. In any session wherein at least one of P_1 or P_2 honestly samples its auxiliary scalar $\sigma_{3,1}$ (resp. $\sigma_{3,2}$), the resulting key K is uniformly distributed in $\mathbb{G}$ (see Lemma 16 (p. 55)).

Corollary 1 (Standard NXK-DKG interface). *Under the hypotheses of Theorem 3 (p. 60), protocol $\Psi_{\text{SDKG}}^{(3)}$ UC-realizes $\mathcal{F}_{\text{DKG}}^{\star, \text{NXK}}$ in the same model.*

Proof. By Theorem 3 (p. 60), $\Psi_{\text{SDKG}}^{(3)}$ UC-realizes $\mathcal{F}_{\text{SDKG}}$ in the stated model. The claim follows by Lemma 17 (p. 56). ■

Corollary 2 (Compiling out $\mathcal{F}_{\text{USV}}$). *Let Π_{USV} be the concrete protocol of Section 5.2 (p. 32) implementing $\mathcal{F}_{\text{USV}}$, and define $\hat{\Psi}_{\text{SDKG}}^{(3)} := \Psi_{\text{SDKG}}^{(3)}[\Pi_{\text{USV}}/\mathcal{F}_{\text{USV}}]$. Under the hypotheses of Theorem 2 (p. 36) and Theorem 3 (p. 60), $\hat{\Psi}_{\text{SDKG}}^{(3)}$ UC-realizes $\mathcal{F}_{\text{SDKG}}$ in the $(\mathcal{F}_{\text{KeyBox}}, \mathcal{F}_{\text{channel}}, \mathcal{F}_{\text{pub}})$ -hybrid and gRO-CRP models.*

Proof. Immediate from UC composition: Π_{USV} UC-realizes $\mathcal{F}_{\text{USV}}$ (Theorem 2 (p. 36)) and $\Psi_{\text{SDKG}}^{(3)}$ UC-realizes $\mathcal{F}_{\text{SDKG}}$ in the $(\mathcal{F}_{\text{KeyBox}}, \mathcal{F}_{\text{USV}}, \mathcal{F}_{\text{channel}}, \mathcal{F}_{\text{pub}})$ -hybrid (Theorem 3 (p. 60)), with shared gRO-CRP access via domain-separated contexts. ■

9 UC Security of the 1+1-out-of- n SDKG Extension

To enroll a new recovery device P_i ($i \geq 3$), run the one-shot registration sub-protocol used for P_3 , sponsored by P_2 or any already registered leaf. When at least one of P_1 and the sponsor is honest, the $\mathcal{F}_{\text{pub}}$ -agreement check (Algorithm 1 (p. 51)) ensures that the honest joiner's reference K equals the stored finalized key, and g_3 then installs the replicated recovery-role share $k_i = k_3$ satisfying $K_{1,3} + k_3\mathcal{G} = K$. When both registration senders are corrupted, g_3 (Definition 21 (p. 44)) guarantees only that the installed share satisfies $K_{1,3} + K_3 = K$ for the adversary-supplied $K_{1,3}$; since the corrupted senders already know k , this confers no additional adversarial advantage, but the installed $(k_{1,3}, k_3)$ decomposition need not equal the transcript-derived one. $\mathcal{F}_{\text{SDKG}}^{(n)}$ (Fig. 9 (p. 68)) lifts $\mathcal{F}_{\text{SDKG}}$ to 1+1-out-of- n star access structure (for $n \geq 3$) by keeping the same transcript-driven base run for (P_1, P_2) and allowing a polynomial number of post-finalization RDRs for leaves P_i with $i \in \{3, \dots, n\}$, installing the same recovery-role share in $\mathcal{F}_{\text{KeyBox}}^{(i)}$ whenever at least one registration sender is honest.

◆ **State** (per session sid): run the base-run state of $\mathcal{F}_{\text{SDKG}}$ to completion. Maintain $\text{Reg} \subseteq \{3, \dots, n\}$ (init $\emptyset$), a pending table $\text{Pend}[i] = (\text{sponsor}, \text{ok}_1, \text{ok}_s, \rho_1, \rho_s, w_1^{\text{del}}, w_s^{\text{del}})$ (init undefined, with $\rho_1 = \rho_s = w_1^{\text{del}} = w_s^{\text{del}} = \perp$), registration-channel state multiset Q_{reg} of $(\rho, i, P_s, P_r, w, \phi)$ and delivered set D_{reg} (init empty). When an honest sponsor P_j constructs a registration payload for a new joiner, $\mathcal{F}_{\text{SDKG}}^{(n)}$ obtains the ciphertexts by commanding P_j 's KeyBox-driver wrapper (Remark 6 (p. 16)) to invoke SealToPeer on $\mathcal{F}_{\text{KeyBox}}^{(j)}$'s resident slots $\langle \text{sid}, \text{k32} \rangle$ and $\langle \text{sid}, \text{k23} \rangle$, rather than maintaining an explicit scalar table.

◆ **Base run**: Identical to $\mathcal{F}_{\text{SDKG}}$ up to and including TryFinalize .

◆ **Register**: Upon $(\text{register}, i, \text{sid}, j)$ from P_i with $i \in \{3, \dots, n\}$: require $\text{finalized} = 1$, $i \notin \text{Reg}$, and $j \in \{2\} \cup \text{Reg}$. If $\text{Pend}[i]$ is undefined, set $\text{Pend}[i] \leftarrow (j, 0, 0, \perp, \perp, \perp, \perp)$ and notify $\mathcal{A}$ with $(\text{RegReq}, i, \text{sid}, j)$.

Upon $(\text{approve}, i, \text{sid})$ from P_1 (resp. from sponsor P_j): if $\text{Pend}[i]$ is defined then set $\text{Pend}[i].\text{ok}_1 \leftarrow 1$ (resp. $\text{Pend}[i].\text{ok}_s \leftarrow 1$); notify $\mathcal{A}$.

◆ **RegGo** (sender-controlled on corruption): Upon $(\text{RegGo}, i, \text{sid}, w_1^*, w_s^*)$ from $\mathcal{A}$: if $\text{Pend}[i]$ is defined, $\text{Pend}[i].\text{ok}_1 = \text{Pend}[i].\text{ok}_s = 1$, and $(\text{Pend}[i].\rho_1, \text{Pend}[i].\rho_s) = (\perp, \perp)$, then:

- Let $j := \text{Pend}[i].\text{sponsor}$. Define slot-bound associated data strings:
$$\text{ad}_{1i} := \langle \text{SDKG.reg}, \text{sid}, P_1, P_i, \text{k31} \rangle, \quad \text{ad}_{ji}^{32} := \langle \text{SDKG.reg}, \text{sid}, P_j, P_i, \text{k32} \rangle, \quad \text{ad}_{ji}^{23} := \langle \text{SDKG.reg}, \text{sid}, P_j, P_i, \text{k23} \rangle.$$
- Reset delivered-payload buffers: set $\text{Pend}[i].w_1^{\text{del}} \leftarrow \perp$ and $\text{Pend}[i].w_s^{\text{del}} \leftarrow \perp$.
- Sample $\rho_1, \rho_s \leftarrow \mathcal{S}\{0, 1\}^\lambda$.
- If $1 \notin \text{Cor}$, sample $\varpi_1 \leftarrow \text{Enc}_{\text{pk}(P_i)}^{\text{seal}}(\text{ad}_{1i}, \sigma_{3,1})$ and set $w_{\text{reg},1} := \langle \text{sid}, \varpi_1, K_{1,3}, K \rangle$ and $\phi_1 := \ell_{\text{reg},1}$. If $1 \in \text{Cor}$, set $w_{\text{reg},1} := w_1^*$ and $\phi_1 := |w_{\text{reg},1}|$. If $w_{\text{reg},1} \neq \perp$, then: set $\text{Pend}[i].\rho_1 \leftarrow \rho_1$; insert $(\rho_1, i, P_1, P_i, w_{\text{reg},1}, \phi_1)$ into Q_{reg} ; send $(\text{Leak}, \text{sid}, P_1, P_i, \rho_1, \phi_1)$ to $\mathcal{A}$; and if $1 \in \text{Cor}$ additionally reveal $w_{\text{reg},1}$ to $\mathcal{A}$.
- If $j \notin \text{Cor}$, have P_j invoke $\varpi_{sa} \leftarrow \mathcal{F}_{\text{KeyBox}}^{(j)}.\text{Use}(\langle \text{sid}, \text{k32} \rangle, \text{SealToPeer}, \langle P_i, \text{ad}_{ji}^{32} \rangle)$ and $\varpi_{sb} \leftarrow \mathcal{F}_{\text{KeyBox}}^{(j)}.\text{Use}(\langle \text{sid}, \text{k23} \rangle, \text{SealToPeer}, \langle P_i, \text{ad}_{ji}^{23} \rangle)$ via the KeyBox-driver wrapper (Remark 6 (p. 16)). Set $w_{\text{reg},s} := \langle \text{sid}, \varpi_{sa}, \varpi_{sb}, K_{1,3} \rangle$ and $\phi_s := \ell_{\text{reg},2}$. If $j \in \text{Cor}$, set $w_{\text{reg},s} := w_s^*$ and $\phi_s := |w_{\text{reg},s}|$. If $w_{\text{reg},s} \neq \perp$, then: set $\text{Pend}[i].\rho_s \leftarrow \rho_s$; insert $(\rho_s, i, P_j, P_i, w_{\text{reg},s}, \phi_s)$ into Q_{reg} ; send $(\text{Leak}, \text{sid}, P_j, P_i, \rho_s, \phi_s)$ to $\mathcal{A}$; and if $j \in \text{Cor}$ additionally reveal $w_{\text{reg},s}$ to $\mathcal{A}$.

◆ **Deliver**: Upon $(\text{Deliver}, \text{sid}, \rho)$ from $\mathcal{A}$: if $(\rho, i, P_s, P_i, w, \phi) \in \text{Q}_{\text{reg}}$ and $\rho \notin \text{D}_{\text{reg}}$, delete it from Q_{reg} , add ρ to D_{reg} , and deliver $(\text{Recv}, \text{sid}, P_s, w)$ to P_i . If $i \in \text{Cor}$ reveal w to $\mathcal{A}$ at delivery time. If $\text{Pend}[i]$ is defined then:

- If $\rho = \text{Pend}[i].\rho_1$, set $\text{Pend}[i].\rho_1 \leftarrow \perp$ and $\text{Pend}[i].w_1^{\text{del}} \leftarrow w$. If $\rho = \text{Pend}[i].\rho_s$, set $\text{Pend}[i].\rho_s \leftarrow \perp$ and $\text{Pend}[i].w_s^{\text{del}} \leftarrow w$.
- If $\text{Pend}[i].\text{ok}_1 = \text{Pend}[i].\text{ok}_s = 1$, $(\text{Pend}[i].\rho_1, \text{Pend}[i].\rho_s) = (\perp, \perp)$, and $\text{Pend}[i].w_1^{\text{del}} \neq \perp$ and $\text{Pend}[i].w_s^{\text{del}} \neq \perp$, then:
 - If $i \in \text{Cor}$: set $\text{Reg} \leftarrow \text{Reg} \cup \{i\}$, delete $\text{Pend}[i]$, and output $(\text{registered}, i, \text{sid})$ to P_i and $\mathcal{A}$.
 - If $i \notin \text{Cor}$: let $j := \text{Pend}[i].\text{sponsor}$ and define the same $\text{ad}_{1i}, \text{ad}_{ji}^{32}, \text{ad}_{ji}^{23}$ as above. Parse $\text{Pend}[i].w_1^{\text{del}} = \langle \text{sid}, \varpi_1, K_{1,3}^{(1)}, K^* \rangle$ and $\text{Pend}[i].w_s^{\text{del}} = \langle \text{sid}, \varpi_{sa}, \varpi_{sb}, K_{1,3}^{(2)} \rangle$. If parsing fails, do nothing further. Require $K_{1,3}^{(1)} = K_{1,3}^{(2)}$; otherwise do nothing further. Set $K_{1,3}^* \leftarrow K_{1,3}^{(1)}$. Require $K^* = K$ (the stored finalized public key). In the real protocol, the honest joiner additionally requires agreement of P_1 's and P_2 's base-run $\mathcal{F}_{\text{pub}}$ publications; the simulator conditions RegGo on $\mathcal{F}_{\text{pub}}$ consistency. Otherwise do nothing further. Otherwise, have P_i forward $(\varpi_1, \varpi_{sa}, \varpi_{sb}, \text{ad}_{1i}, \text{ad}_{ji}^{32}, \text{ad}_{ji}^{23}, K^*, K_{1,3}^*)$ into $\mathcal{F}_{\text{KeyBox}}^{(i)}$, which executes the same KeyBox-side installation procedure as Algorithm 1 (p. 51) (with sponsor P_j): open the ciphertexts under the corresponding ad strings and invoke the corresponding Load calls to install $\langle \text{sid}, \text{k32} \rangle, \langle \text{sid}, \text{k23} \rangle, \langle \text{sid}, \text{k3} \rangle$. If all invoked Load calls return ok , then set $\text{Reg} \leftarrow \text{Reg} \cup \{i\}$; delete $\text{Pend}[i]$, and output $(\text{registered}, i, \text{sid})$ to P_i and $\mathcal{A}$. (Future honest-sponsor payloads from P_i are generated by invoking SealToPeer on $\mathcal{F}_{\text{KeyBox}}^{(i)}$'s resident slots $\langle \text{sid}, \text{k32} \rangle$ and $\langle \text{sid}, \text{k23} \rangle$ directly via the KeyBox-driver wrapper, avoiding the need to record the installed scalar values explicitly.) Otherwise do nothing further.

◆ **Corruptions/Programming**: As in $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)). Before base finalization, Sim may once send $(\text{Program}, \text{sid}, x_1^*, x_2^*, \sigma_{3,1}^*, \sigma_{3,2}^*)$; set $x_1 \leftarrow x_1^*, x_2 \leftarrow x_2^*, \sigma_{3,1} \leftarrow \sigma_{3,1}^*,$ and $\sigma_{3,2} \leftarrow \sigma_{3,2}^*$.

Fig. 9: Transcript-driven ideal functionality $\mathcal{F}_{\text{SDKG}}^{(n)}$.

Reader Note 9.1: Corrupted-joiner handling: $\mathcal{F}_{\text{SDKG}}$ vs. $\mathcal{F}_{\text{SDKG}}^{(n)}$

In the base $\mathcal{F}_{\text{SDKG}}$ (Fig. 8 (p. 54)), if the joiner P_3 is corrupted after both payloads are delivered, the functionality does nothing further and leaves completion to $\mathcal{A}$. This suffices because the base case has a single joiner with no subsequent sponsor chain. In $\mathcal{F}_{\text{SDKG}}^{(n)}$, a corrupted joiner i is immediately added to Reg and $(\text{registered}, i, \text{sid})$ is output, because in the real world the adversary (controlling $\mathcal{F}_{\text{KeyBox}}^{(i)}$) can signal completion and i may subsequently serve as sponsor; the ideal functionality must reflect this by granting sponsor eligibility. This is a deliberate semantic upgrade, not an inconsistency.

Theorem 4 (UC security of $\mathcal{F}_{\text{SDKG}}^{(n)}$). Fix Fischlin parameter functions $(t(\lambda), b(\lambda), r(\lambda), S(\lambda))$ satisfying Definition 16 (p. 25). Assume hardness of DL and DDLEQ, and the hypotheses used for Theorems 3 (p. 60) and 2 (p. 36). Then the compiled n -party protocol $\widehat{\Psi}_{\text{SDKG}}^{(n)}$ UC-realizes $\mathcal{F}_{\text{SDKG}}^{(n)}$ in the $(\mathcal{F}_{\text{KeyBox}}, \mathcal{F}_{\text{channel}}, \mathcal{F}_{\text{pub}})$ -hybrid and gRO-CRP models against adaptive corruptions with secure erasures.

Proof Sketch. Let $\mathcal{A}$ be any PPT real-world adversary and $\mathcal{Z}$ any PPT environment. We build a PPT simulator $\text{Sim}^{(n)}$ for the ideal execution with $\mathcal{F}_{\text{SDKG}}^{(n)}$.

Base run. $\text{Sim}^{(n)}$ simulates the base transcript exactly as in the proof of Theorem 3 (p. 60): it mediates $\mathcal{A}$'s gRO-CRP queries under the UC-proof contexts, extracts witnesses from verifying UC-NIZK-AoKs, computes the unique transcript-defined (x_1, x_2) in accepting sessions, computes the corresponding $\sigma_{3,1}$ and $\sigma_{3,2}$ as in Theorem 3 (p. 60), and programs $\mathcal{F}_{\text{SDKG}}^{(n)}$ once via $(\text{Program}, \text{sid}, x_1, x_2, \sigma_{3,1}, \sigma_{3,2})$ prior to finalization. This ensures that the public key output K and the base-installed KeyBox shares for P_1 and P_2 match the real execution distribution, up to negligible Bad events as in Theorem 3 (p. 60).

Registrations. After finalization, $\mathcal{F}_{\text{SDKG}}^{(n)}$ may receive any number of registration requests $(\text{register}, i, \text{sid})$ for distinct $i \in \{3, \dots, n\}$. In the real execution, each registration of an honest joiner affects only: (i) the transcript/messages of that registration session and (ii) the internal state of $\mathcal{F}_{\text{KeyBox}}^{(i)}$ via three Load calls that install the two sponsor-state slots $(\langle \text{sid}, \text{k32} \rangle$ and $\langle \text{sid}, \text{k23} \rangle$ via $g_{3,2}^{\text{reg}}$ and $g_{2,3}^{\text{reg}}$) and the replicated recovery-role share $(\langle \text{sid}, \text{k3} \rangle$ via $g_3)$, exactly as in Algorithm 1 (p. 51) with the appropriate joiner and sponsor parameters (Definition 21 (p. 44)). For a corrupted joiner i , the adversary controls $\mathcal{F}_{\text{KeyBox}}^{(i)}$ directly and may or may not invoke Load; correspondingly, $\mathcal{F}_{\text{SDKG}}^{(n)}$ adds i to Reg and emits $(\text{registered}, i, \text{sid})$ without KeyBox installation, matching the real-world behavior where $\mathcal{A}$ controls the completion signaling. Since $\mathcal{F}_{\text{KeyBox}}^{(i)}$ state is never revealed upon corruption (only black-box access via Use), $\text{Sim}^{(n)}$ can simulate each device registration independently, while ensuring that $\mathcal{F}_{\text{KeyBox}}^{(i)}$ ends up storing the correct shares and hence returns the same $\text{PubMap}(k_3)$ under Use as in the real protocol. Lemma 21 (p. 65) is stated for the P_3 base case; its proof extends to arbitrary joiner P_i with sponsor P_j because the registration procedure, the associated-data binding, and the g_3/g^{reg} routines are parameterized by $(P_r, P_{\text{sp}}, \text{sid})$ (Definition 21 (p. 44)). We distinguish four cases for the installed scalars:

- (a) *Both registration senders honest.* All installed scalars—including the individual sponsor-state slots $(\sigma_{3,2}, \sigma_{2,3})$ —are deterministic functions of the already-fixed values $(\sigma_{3,1}, \sigma_{3,2}, \sigma_{2,3}, K, K_{1,3})$.
- (b) *P_1 honest, sponsor corrupted.* The recovery-role share k_3 is still uniquely determined by g_3 (Definition 21 (p. 44)), which enforces $K_{1,3} + k_3\mathcal{G} = K$; however, because g_3 constrains only the linear combination $\sigma_{2,3} - \sigma_{3,2}$ (not the individual values), a corrupted sponsor may shift both slots by a common offset Δ without affecting k_3 . The actually-installed values reside in $\mathcal{F}_{\text{KeyBox}}^{(i)}$'s slots $\langle \text{sid}, \text{k32} \rangle$ and $\langle \text{sid}, \text{k23} \rangle$; the adversary supplies the sponsor payload w_s^* directly, so $\mathcal{F}_{\text{SDKG}}^{(n)}$ correctly mirrors the real-world state. When P_i later serves as an honest sponsor, $\mathcal{F}_{\text{SDKG}}^{(n)}$ invokes SealToPeer on these resident slots via the KeyBox-driver wrapper, producing ciphertexts with the same distribution as in the real protocol.
- (c) *P_1 corrupted, sponsor honest.* The corrupted P_1 controls $\mathcal{F}_{\text{KeyBox}}^{(1)}$ and can seal an arbitrary scalar $\sigma'_{3,1}$ via SealToPeer ; it also controls its own $\mathcal{F}_{\text{pub}}$ publication. However, the honest joiner P_i 's reference K is obtained by requiring agreement between P_1 's and P_2 's base-run authenticated $\mathcal{F}_{\text{pub}}$ publications for sid (Algorithm 1 (p. 51)):

P_i sets $K := K^{(1)}$ only if $K^{(1)} = K^{(2)}$, aborting otherwise. Since case (c) presupposes that at least one of P_1 or P_2 was honest at the time of its base-run $\mathcal{F}_{\text{pub}}$ publication—if both were corrupted during the base run, we are in case (d)—and $\mathcal{F}_{\text{pub}}$ publications are immutable once made (the functionality provides no retract interface), the honest publication anchors K to the correct value. Concretely, a corrupted P_1 that published $K_{\text{fake}} \neq K$ during the base run implies P_2 was honest and published the correct K , so $K^{(1)} \neq K^{(2)}$ and P_i aborts. If P_1 was honest during the base run and corrupted later, $K^{(1)} = K$ is already committed. In either sub-case, if P_1 publishes the correct K , then g_3 enforces $K_{1,3} + k_3\mathcal{G} = K$ with the correct K , and the recovery-role share is uniquely determined as in case (a). As in case (b), the sponsor-state slots may shift by a common offset without affecting k_3 ; the actually-installed values reside in $\mathcal{F}_{\text{KeyBox}}^{(i)}$'s slots and are accessed via **SealToPeer** when needed.

The adversary supplies w_1^* directly, so $\mathcal{F}_{\text{SDKG}}^{(n)}$ correctly mirrors the real-world state.

- (d) *Both senders corrupted* (requires $P_1 \in \text{Cor}$). The actually-installed values reside in $\mathcal{F}_{\text{KeyBox}}^{(i)}$'s resident slots, and $\mathcal{F}_{\text{SDKG}}^{(n)}$ invokes **SealToPeer** on these slots (via the KeyBox-driver wrapper) in subsequent honest-sponsor payloads, so real and ideal sponsor ciphertexts match.

In every case, $\mathcal{F}_{\text{SDKG}}^{(n)}$ performs the same $\mathcal{F}_{\text{KeyBox}}$ installation procedure as Algorithm 1 (p. 51), so the installed $\mathcal{F}_{\text{KeyBox}}^{(i)}$ state is identical in real and ideal executions. Consequently, when a newly registered device later serves as sponsor, $\mathcal{F}_{\text{SDKG}}^{(n)}$ invokes **SealToPeer** on the same resident slots that the real-world protocol would use, preserving the hybrid step's locality.

Composition over many registrations. Registrations for different i touch disjoint KeyBox instances $\mathcal{F}_{\text{KeyBox}}^{(i)}$ and do not modify (x_1, x_2, K) . When a newly registered device later sponsors another device, $\mathcal{F}_{\text{SDKG}}^{(n)}$ invokes **SealToPeer** on the sponsor's $\mathcal{F}_{\text{KeyBox}}$ resident slots via the KeyBox-driver wrapper; this dependency does not break hybrid locality because in every hybrid, $\mathcal{F}_{\text{SDKG}}^{(n)}$ performs the same $\mathcal{F}_{\text{KeyBox}}$ -mediated installation as the real protocol, producing identical installed slot values. Therefore, switching registration i from real to ideal leaves all state visible to subsequent registrations unchanged. We argue via a standard hybrid over $q = q(\lambda) \leq \text{poly}(\lambda)$ completed registrations: define hybrids $\mathcal{D}^{(j)}$ for $j = 0, \dots, q$ that replace the first j real registrations by the $\mathcal{F}_{\text{SDKG}}^{(n)}$ -generated ones. Consecutive hybrids differ in only one registration instance, so Lemma 21 (p. 65) gives a negligible per-step change, and a union bound over q steps keeps the total distinguishing advantage negligible. Thus, it follows from Theorem 3 (p. 60) and Corollary 2 (p. 67) that

$$\text{Exec}(\widehat{\Psi}_{\text{SDKG}}^{(n)}, \mathcal{A}, \mathcal{Z}) \approx_c \text{Ideal}(\mathcal{F}_{\text{SDKG}}^{(n)}, \text{Sim}^{(n)}, \mathcal{Z}).$$

■

Remark 26 (Why the two-party scoping is necessary). Under UC scheduling, a corrupted committer can equivocate by delivering distinct first commits to different relying parties. The compiled protocol Π_{USV} necessarily yields recipient-local state in that case, so it can only UC-realize an ideal functionality with the same relying-party scoped semantics.

To support scalable RDR with an arbitrary already-registered leaf as sponsor, SDKG designates a small subset of σ -values as *registration scalars* that are retained inside KeyBoxes in dedicated slots (Definition 21 (p. 44)). These retained values remain NXK-restricted and are used only via **SealToPeer**. If an instantiation exposes remote-attestation transcripts in the clear, this can introduce extra visible structure; modeling such leakage is orthogonal to our NXK consistency enforcement.

Optional. If external verifiability is desired: after KeyBox_3 installs k_3 , the host obtains $K_3 := \text{PubMap}(k_3)$ by invoking $\mathcal{F}_{\text{KeyBox}}^{(3)}.\text{Use}(\langle \text{sid}, k_3 \rangle, \text{GetPub}, \cdot)$ and obtains a proof π_{DL_3} by running the in-KeyBox optimized Fischlin prover:

$$(\mu_{\text{FS}}, \mathbf{a}) \leftarrow \mathcal{F}_{\text{KeyBox}}^{(3)}.\text{Use}(\langle \text{sid}, k_3 \rangle, \text{FS.Start}, \langle \text{sid}, K_3 \rangle), \quad \pi_{\text{DL}_3} \leftarrow \mathcal{F}_{\text{KeyBox}}^{(3)}.\text{Use}(\langle \text{sid}, k_3 \rangle, \text{FS.Prove}, \mu_{\text{FS}}).$$

The proof $\pi_{\text{DL}_3} = ((a_i, e_i, z_i))_{i=1}^r$ is a UC-NIZK-AoK for the DL relation $\mathcal{R}_{\text{DL}}$ on statement (pp, K_3) in the gRO-CRP model under context $\text{ctx}_{\text{KeyBox}}$. By construction, FS.Prove releases at most one distinct response per commitment—since even under rollback, deterministic nonce derivation prevents two different responses for the same commitment—and issues all rare-structure hash queries internally under $\text{ctx}_{\text{KeyBox}}$. Verification is public via $\text{FS.Verify}(K_3, \pi_{\text{DL}_3})$.

10 Complexity and Overhead

We summarize dominant computation and communication costs of SDKG (base 1+1-out-of-3 run) and its RDR extension to 1+1-out-of- n . Let $\mathbb{G}$ be a prime-order group of size p and write $\kappa := \log p$ (e.g., $\kappa \approx 256$ for standard 128-bit ECC instantiations). All NIZK(-AoK)s are instantiated via the optimized Fischlin transform in the gRO-CRP model with parameters (t, b, r, S) satisfying Definition 16 (p. 25). For our concrete estimates, we fix a target security level $\lambda = \lambda_0$ and instantiate

$$(t, b, r, S) := (t(\lambda_0), b(\lambda_0), r(\lambda_0), S(\lambda_0)) = (13, 8, 32, 32).$$

Hence, for any prover making at most Q distinct gRO-CRP queries under the relevant proof context, the resulting Fischlin knowledge/soundness error is $\leq (Q + 1) \cdot 2^{-195} + \text{negl}(\lambda)$, which constitutes a security bound. Separately, the early-break rarity search induces a per-proof honest-rejection probability bounded by $p_{\text{rej}} \leq r \cdot (1 - 2^{-b})^{2^t} \approx 2^{-41}$ for these parameters (liveness/completeness). This p_{rej} is an operational failure probability of the proof-generation subroutine, not an adversarial success probability; the prover retries proof generation upon rejection, with expected attempts $1/(1 - p_{\text{rej}}) \approx 1$. On rejection, the prover aborts and retries by choosing a fresh proof-session identifier $\text{sid}' \neq \text{sid}$ and invoking $\text{FS.Start}(\text{sid}', K)$ to obtain a fresh handle and commitments because reusing the same (sid, K) deterministically regenerates the same commitments. Using standard compressed encodings, the dominant objects are:

$$|\pi_{\text{DL}}| \approx 2.1 \text{ KiB}, \quad |\pi_{\text{aff}}| \approx 4.1 \text{ KiB}, \quad |(C, \zeta)| \approx 3.1\text{--}3.3 \text{ KiB}.$$

The base SDKG transcript contains one USV certificate and two affine AoKs, yielding a total transcript size of $\approx 11\text{--}13$ KiB (excluding small constant headers). Verification of a Schnorr-DL Fischlin proof performs $\Theta(r)$ Schnorr checks / $\Theta(r)$ scalar multiplications: one fixed-base by $\mathcal{G}$, one variable-base by the statement element K , plus additions, up to standard multi-scalar optimizations. The SDKG base run verifies a constant number of proof objects—one DLEQ proof inside USV and two affine proofs implemented by two DL proofs—hence the verification and proving costs are $\tilde{O}(\kappa^{2.585})$ bit-operations in the Karatsuba model (dominated by a constant number of scalar multiplications). Registration of an additional recovery device adds a constant-size exchange and an optional DL proof for external verifiability.

SDKG uses a constant number of proof objects in the base run and performs no resharing. Therefore, $\text{Comm}_{\text{SDKG}} = \tilde{O}(\kappa)$ bits and $\text{Comp}_{\text{SDKG}} = \tilde{O}(\kappa^{2.585})$ bit-ops. For the 1+1-out-of- n extension, RDR registers each additional device with $\text{Comm}_{\text{RDR, per device}} = \tilde{O}(\kappa)$ bits and $\text{Comp}_{\text{RDR, per device}} = \tilde{O}(\kappa^{2.585})$ bit-ops, so in total: $\text{Comm}(n) = \tilde{O}(n\kappa)$ and $\text{Comp}(n) = \tilde{O}(n\kappa^{2.585})$.

11 Conclusion

We studied UC-secure Distributed Key Generation (DKG) in the Non-eXportable Key (NXK) setting wherein long-term signing shares are confined to state-continuous trusted hardware (e.g., TEEs). In this NXK setting, the protocol cannot rely on classical VSE (VSS/PVSS/AVSS or commitment-and-proof analogues) that exports and manipulates shares (or affine images of shares), and state-continuity rule out rewinding/forking-style extraction once witness-bearing computation is delegated to the hardware boundary. Our starting point is therefore to decouple confidentiality from consistency: confidentiality can be delegated to the NXK/TEE boundary, while the protocol must still enforce transcript-defined affine consistency and uniqueness of the induced sharing without any share opening or resharing. We introduced Unique Structure Verification (USV), a publicly verifiable certificate mechanism that yields a deterministic public opening to a committed group element while keeping the underlying scalar hidden under DL and DDLEQ hardness, providing canonical public structure without trapdoors or programmable setup. We combined USV with UC-extractable NIZK arguments of knowledge (via an optimized Fischlin transform in our gRO-CRP-hybrid model) to obtain straight-line extraction without rewinding. Building on these tools, we constructed Star DKG (SDKG), a constant-round UC-secure DKG for a 1+1-out-of- n star access structure motivated by threshold cryptocurrency wallets where a designated service must co-sign yet cannot reconstruct the signing key alone (under key-opacity of the KeyBox profile). More precisely, for the base authorized pairs $\{P_1, P_2\}$ and $\{P_1, P_3\}$, the transcript-derived shares always

sum to k (Observation 2 (p. 56)); for each subsequently registered device P_i , the same holds whenever at least one registration sender (P_1 or the sponsoring leaf) is honest (Section 9 (p. 67)). When both senders are corrupted—which entails a corrupted P_1 and hence a fully corrupted authorized pair—the installed decomposition satisfies $K_{1,3} + K_3 = K$ but need not coincide with the transcript-derived split; because the adversary already controls an authorized pair, this confers no additional advantage. In all corruption scenarios, (ii) no unauthorized set—including the center P_1 alone—can recover k from the KeyBox-confined shares under key-opacity (Observation 2 (p. 56)). The operational signing completeness guarantee—that authorized pairs can *use* their NXK-confined shares to produce valid signatures—depends on composing SDKG with a suitable threshold signing protocol compatible with the KeyBox API and is left to future work. SDKG supports post-DKG device enrollment by registering additional recovery devices as replicated leaves, preserving the public key without resharing. Under DL and DDLEQ assumptions, PRF security for deterministic nonce derivation, the seed integrity invariant (Assumption 3 (p. 13)), and assuming KeyBox (TEE-resident NXK keystore) opacity, state continuity, and secure erasures, we prove that SDKG UC-realizes a transcript-driven ideal functionality. The protocol achieves $\tilde{O}(n \log p)$ communication and $\tilde{O}(n \log^{2.585} p)$ bit-operation cost in a prime-order group of size p , while each additional device registration costs $\tilde{O}(\log p)$ communication and $\tilde{O}(\log^{2.585} p)$ computation (with a base transcript of $\approx 11\text{--}13$ KiB in a 128-bit instantiation).

Acknowledgments

I thank Nolan Miranda for helpful discussions and feedback on early versions of the SDKG idea and protocol sketch.

References

1. Amazon Web Services: AWS Key Management Service API Reference: GetPublicKey. https://docs.aws.amazon.com/kms/latest/APIReference/API_GetPublicKey.html, accessed 2026-02-17
2. Babai, L., Moran, S.: Arthur-Merlin games: A randomized proof system, and a hierarchy of complexity classes. *Journal of Computer and System Sciences* **36**(2), 254–276 (1988)
3. Bacho, R., Kavousi, A.: SoK: Dlog-Based Distributed Key Generation. In: *IEEE S&P*. pp. 614–632 (2025)
4. Battagliola, M., Longo, R., Meneghetti, A., Sala, M.: Threshold ECDSA with an offline recovery party. *Mediterranean Journal of Mathematics* **19**(4) (2022)
5. Bellare, M., Neven, G.: Multi-signatures in the plain public-key model and a general forking lemma. In: *ACM CCS*. pp. 390–399 (2006)
6. Blum, M.: Coin flipping by telephone a protocol for solving impossible problems. *ACM SIGACT News* **15**(1), 23–27 (1983)
7. Boneh, D., Ding, X., Tsudik, G.: Fine-grained control of security capabilities. *ACM Transactions on Internet Technology* **4**(1), 60–82 (2004)
8. Boneh, D., Ding, X., Tsudik, G., Wong, C.M.: A method for fast revocation of public key certificates and security capabilities. In: *USENIX Security*. pp. 297–308 (2001)
9. Bortolozzo, M., Centenaro, M., Focardi, R., Steel, G.: Attacking and fixing PKCS#11 security tokens. In: *ACM CCS*. pp. 260–269 (2010)
10. Cachin, C., Kursawe, K., Lysyanskaya, A., Stroh, R.: Asynchronous verifiable secret sharing and proactive cryptosystems. In: *ACM CCS*. pp. 88–97 (2002)
11. Camenisch, J., Drijvers, M., Gagliardoni, T., Lehmann, A., Neven, G.: The wonderful world of global random oracles. In: *EUROCRYPT*. pp. 280–312 (2018)
12. Camenisch, J., Drijvers, M., Lehmann, A.: Universally composable direct anonymous attestation. In: *PKC*. pp. 234–264 (2016)
13. Canetti, R.: Universally composable security: a new paradigm for cryptographic protocols. In: *FOCS*. pp. 136–145 (2001)
14. Canetti, R., Dodis, Y., Pass, R., Walfish, S.: Universally composable security with global setup. In: *TCC*. pp. 61–85 (2007)
15. Canetti, R., Gennaro, R., Goldfeder, S., Makriyannis, N., Peled, U.: UC non-interactive, proactive, threshold ECDSA with identifiable aborts. *Cryptology ePrint Archive*, Report 2021/060 (2021), full version of the CCS’20 paper
16. Canetti, R., Goldreich, O., Goldwasser, S., Micali, S.: Resettable zero-knowledge. In: *STOC*. pp. 235–244 (2000)
17. Canetti, R., Jain, A., Scafuro, A.: Practical UC security with a global random oracle. In: *ACM CCS*. pp. 597–608 (2014)
18. Canetti, R., Rabin, T.: Universal composition with joint state. In: *CRYPTO*. pp. 265–281 (2003)
19. Chaum, D., Pedersen, T.P.: Wallet databases with observers. In: *CRYPTO*. pp. 89–105 (1992)

20. Chen, Y.H., Lindell, Y.: Optimizing and implementing Fischlin’s transform for UC-secure zero-knowledge. *IACR CiC* **1**(2) (2024)
21. Chor, B., Goldwasser, S., Micali, S., Awerbuch, B.: Verifiable secret sharing and achieving simultaneity in the presence of faults. In: *FOCS*. pp. 383–395 (1985)
22. Cleve, R.: Limits on the security of coin flips when half the processors are faulty. In: *STOC*. pp. 364–369 (1986)
23. Desmedt, Y.: Society and group oriented cryptography: A new concept. In: *CRYPTO*. pp. 120–127 (1987)
24. Desmedt, Y., Frankel, Y.: Threshold cryptosystems. In: *CRYPTO*. pp. 307–315 (1989)
25. Ding, X., Mazzocchi, D., Tsudik, G.: Experimenting with server-aided signatures. In: *NDSS* (2002)
26. Doerner, J., Kondi, Y., Lee, E., Shelat, A.: Threshold ECDSA in Three Rounds. In: *IEEE S&P*. pp. 3053–3071 (2024)
27. Doerner, J., Kondi, Y., Rosenbloom, L.N.: Sometimes you can’t distribute random-oracle-based proofs. In: *CRYPTO*. pp. 323–358 (2024)
28. Faz-Hernandez, A., Scott, S., Sullivan, N., Wahby, R.S., Wood, C.A.: RFC 9380: Hashing to elliptic curves (August 2023), <https://www.rfc-editor.org/rfc/rfc9380.html>, IRTF, Informational
29. Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: *FOCS*. pp. 427–438 (1987)
30. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: *CRYPTO*. pp. 186–194 (1986)
31. Fischlin, M.: Communication-efficient non-interactive proofs of knowledge with online extractors. In: *CRYPTO*. pp. 152–168 (2005)
32. Friedman, O., Marmor, A., Mutzari, D., Sadika, O., Scaly, Y.C., Spiizer, Y., Yanai, A.: 2PC-MPC: Emulating two party ECDSA in large-scale MPC. *Cryptology ePrint Archive*, Paper 2024/253 (2024)
33. Friedman, O., Marmor, A., Mutzari, D., Scaly, Y.C., Spiizer, Y.: Practical zero-trust threshold signatures in large-scale dynamic asynchronous networks. *Cryptology ePrint Archive*, Report 2025/297 (2025)
34. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology* **20**, 51–83 (2007)
35. Goldreich, O., Micali, S., Wigderson, A.: How to play ANY mental game. In: *STOC*. pp. 218–229 (1987)
36. Google Cloud: Cloud Key Management Service roles and permissions. <https://docs.cloud.google.com/iam/docs/roles-permissions/cloudkms>, accessed 2026-02-17
37. Groth, J., Ostrovsky, R., Sahai, A.: Perfect non-interactive zero knowledge for NP. In: *EUROCRYPT*. pp. 339–358 (2006)
38. Groth, J., Ostrovsky, R., Sahai, A.: New techniques for noninteractive zero-knowledge. *Journal of ACM* **59**(3), 11:1–11:35 (2012)
39. Josefsson, S., Liusvaara, I.: Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032 (Jan 2017), <https://www.rfc-editor.org/rfc/rfc8032>
40. Karatsuba, A., Ofman, Y.: Multiplication of multidigit numbers on automata. *Soviet Physics - Doklady* **7**, 595–596 (1963)
41. Katz, J.: Round-optimal, fully secure distributed key generation. In: *CRYPTO*. pp. 285–316 (2024)
42. Kelsey, J., ao, L.T.A.N.B., Peralta, R., Booth, H.: A reference for randomness beacons. Tech. Rep. NISTIR 8213 (2019)
43. Komlo, C., Goldberg, I., Stebila, D.: A formal treatment of distributed key generation, and new constructions. *Cryptology ePrint Archive*, Report 2023/292 (2023)
44. Küsters, R., Tuengerthal, M.: Universally composable symmetric encryption. In: *IEEE Computer Security Foundations Symposium*. pp. 293–307 (2009)
45. Lindell, Y.: Simple three-round multiparty Schnorr signing with full simulatability. *Communications in Cryptology* **1**(1), 1–49 (2024)
46. Lindell, Y., Nof, A.: Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody. In: *ACM CCS*. pp. 1837–1854 (2018)
47. Lueks, W., Hampiholi, B., Alpár, G., Troncoso, C.: Tandem: Securing keys by using a central server while preserving privacy. In: *PETs*. pp. 327–355 (2020)
48. Lysyanskaya, A., Rosenbloom, L.N.: Universally composable σ -protocols in the global random-oracle model. In: *TCC*. pp. 203–233 (2022)
49. Lysyanskaya, A., Rosenbloom, L.N.: Adaptive UC NIZK for practical applications. In: *LATINCRYPT*. pp. 76–108 (2025)
50. Martinico, L.: Trusted Execution for Private and Secure Computation: a Composable Approach. Ph.D. thesis, University of Edinburgh (2025)
51. Martinico, L., Kohlweiss, M.: AGATE: Augmented Global Attested Trusted Execution in the Universal Composability Framework. In: *IEEE Computer Security Foundations Symposium (CSF)*. pp. 49–64 (2025)
52. Microsoft: Key types, algorithms, and operations — Azure Key Vault. <https://learn.microsoft.com/en-us/azure/key-vault/keys/about-keys-details>, accessed 2026-02-17
53. Pass, R., Shi, E., Tramèr, F.: Formal abstractions for attested execution secure processors. In: *EUROCRYPT*. pp. 260–289 (2017)

54. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: CRYPTO. pp. 129–140 (1991)
55. Pointcheval, D., Stern, J.: Security proofs for signature schemes. In: EUROCRYPT. pp. 387–398 (1996)
56. Pornin, T.: Deterministic usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA). RFC 6979 (Aug 2013), <https://www.rfc-editor.org/rfc/rfc6979>
57. Rogaway, P.: Authenticated-encryption with associated-data. In: ACM CCS. pp. 98–107 (2002)
58. Schnorr, C.P.: Efficient identification and signatures for smart cards. In: CRYPTO. pp. 239–252 (1989)
59. Schnorr, C.P.: Efficient signature generation by smart cards. Journal of Cryptology **4**, 161–174 (1991)
60. Shamir, A.: How to share a secret. Communications of the ACM **22**, 612–613 (Nov 1979)
61. Shepherd, C., Markantonakis, K.: Trusted Execution Environments. Springer Cham (2024)
62. Shepherd, C., Markantonakis, K., v. Heijningen, N., Aboukassimi, D., Gaine, C., Heckmann, T., Naccache, D.: Physical fault injection and side-channel attacks on mobile devices: A comprehensive analysis. Computers & Security **111**, 102471 (2021)
63. Snetkov, N., Vakarjuk, J., Laud, P.: Universally composable server-supported signatures for smartphones. Cryptology ePrint Archive, Report 2024/1941 (2024)
64. Stadler, M.: Publicly verifiable secret sharing. In: International Conference on the Theory and Applications of Cryptographic Techniques. pp. 190–199 (1996)
65. Yao, A.C.: How to generate and exchange secrets. In: FOCS. pp. 162–167 (1986)
66. Zhang, X., Qin, K., Qu, S., Wang, T., Zhang, C., Gu, D.: Teamwork Makes TEE Work: Open and Resilient Remote Attestation on Decentralized Trust. IEEE Transactions on Dependable and Secure Computing **23**(2), 3798–3813 (2026). <https://doi.org/10.1109/tdsc.2025.3636884>
67. Zhang, Y., Qian, Y.: Randao: A DAO working as RNG of Ethereum (2019), <https://github.com/randao/randao/>

A Programmable Secure Hardware Integration

The main construction already assumes that USV certificate generation $\text{Cert}(\text{pp}, \cdot)$ is executed inside the KeyBox boundary and exports only (C, ζ) . This appendix describes an optional additional hardening that further reduces exposure of other ephemeral scalars and group/field arithmetic in host memory by shifting (i) sampling of ephemerals and (ii) scalar multiplication with public points into the KeyBox/TEE. At a high level, the host is then restricted to routing public group elements, protocol transcripts, and ciphertexts, while the KeyBox performs ephemeral sampling and arithmetic and (optionally) authenticated encryption/decryption so plaintexts never leave the KeyBox boundary.

Although this hardening is mostly written for TEEs, because they are a convenient programmable substrate, the profile applies to any KeyBox realization that can expose the same non-exporting ephemeral-handle operations without violating key-opacity/state-continuity.

Scope / proof compatibility. This profile is an implementation hardening and does not change the protocol transcript or acceptance predicate. Our UC analysis continues to apply to the baseline profile used in the main construction. UC-extractable consistency AoKs (whose straight-line extraction relies on observing the prover’s gRO-CRP query log) remain generated outside the KeyBox as in the main protocol; moving those AoKs inside the KeyBox would require a different extraction mechanism/model.

Illustrative hardened interface (informal). Model this by augmenting the admissible KeyBox operation set $\mathcal{F}_{\text{adm}}$ (Definition 3 (p. 12)) with ephemeral-handle arithmetic that outputs only public group elements. Let τ denote a type-tagged opaque handle to an ephemeral scalar $s \in \mathbb{Z}_p$ stored inside the KeyBox, similar in spirit to internal `buf` handles in $\mathcal{F}_{\text{KeyBox}}$ (Fig. 3 (p. 11)). Handles refer only to KeyBox-internal ephemeral state and are not interchangeable with long-term key slots.

- $\text{GenScalar}() \rightarrow \tau$: sample $s \leftarrow \mathbb{Z}_p$ internally and return only the handle τ .
- $\text{Mul}(\tau, P) \rightarrow Q$: for public $P \in \mathbb{G}$, output $Q := sP$ (and optionally consume τ).
- $\text{MulGen}(\tau) \rightarrow J$: output $J := s\mathcal{G}$ (a wrapper for $\text{Mul}(\tau, \mathcal{G})$).
- (Optional convenience) $\text{LinComb}(\{(\tau_i, P_i)\}_{i=1}^t, \{\alpha_i\}_{i=1}^t) \rightarrow Q$: for public $\alpha_i \in \mathbb{Z}_p$ and $P_i \in \mathbb{G}$, output $Q := \sum_{i=1}^t \alpha_i \cdot s_i P_i$.
- (Optional; needed for RDR / any KeyBox-to-KeyBox transport of share-derived payloads) retain the sealing interfaces `SealToPeer` and `OpenFromPeer` so the host supplies only peer identity and associated data and plaintexts never leave the KeyBox boundary.

Family	What matches $\mathcal{F}_{\text{KeyBox}}$	Profile adapter must forbid	Freshness / anti-rollback
TEE as restricted keystore	Minimal enclave exports only Load/Use ; sealing to attestation-bound peer keys	Wrap/export under host-chosen keys; APIs returning share-deriving outputs of resident keys [†] ; high-entropy failure channels	TPM/TEE counters, trusted time, or server freshness oracle
Attested enclave $\leftrightarrow$ KMS	KMS authorizes use/sealing conditioned on enclave measurement/attestation	KMS export/wrap features; host-selected recipients; policy gaps	KMS-side monotonic state + server oracle
HSM (PKCS#11 allowlist)	Non-extractable objects; sealing/wrap only to trusted/pinned recipients	Caller-decryptable wrapping and key-deriving ops	HSM counters or external freshness
Endpoint key-store	Hardware-bound NXK keys + restricted built-in ops; optional attestation	Limited programmability; operation menu may not support custom sub-routines	Platform counters/time + server oracle
TPM-assisted freshness	Provides monotonic primitives to bind sealed state to freshness	Integration complexity; counter exhaustion/availability	TPM NV counters / PCR-bound state

Table 7: Non-normative summary: common substrates for enforcing a KeyBox API profile.

[†]This refers to vendor-API operations that export caller-invertible affine images of an already-installed long-term share. Key-independent admissible operations (e.g., $\text{SDKG.LeafNit} \in \mathcal{F}_{\text{KI}}$) that output fresh share-deriving material for transient protocol use are part of the protocol’s own admissible profile and are not subject to this restriction; their outputs are NXK-restricted transient host state subject to secure erasure (Remark 7 (p. 16)).

Key-opacity intuition. Since the hardened interface returns only public group elements derived from fresh ephemerals and public inputs, its externally visible outputs remain simulatable given public information, aligning with the key-opacity assumption used throughout the paper.

B Candidate KeyBox Implementations and Profile-Capture Checklist

This appendix provides (i) concrete implementation classes that plausibly realize the KeyBox abstraction under a pinned profile, and (ii) an operational checklist for deriving/enforcing such a profile from vendor APIs. Table 7 (p. 75) summarizes candidate deployment families.

B.1 Candidate classes of implementations

The following implementation patterns are plausible realizations of the abstraction in Fig. 3 (p. 11), provided they are configured and constrained to enforce the intended KeyBox profile (Definition 3 (p. 12)) and do not silently expose mechanisms that violate key-opacity (Assumption 1 (p. 11)) or state continuity (Assumption 2 (p. 13)).

1. Dedicated TEEs as restricted keystores: Implement the KeyBox as a minimal enclave exposing only **Load/Use** endpoints. Approximate **SealToPeer** by pinning recipient keys to attested identities/measurements (e.g., HPKE-style sealing), and treat rollback/fork protection as an explicit subsystem (counters / trusted time / server freshness).
2. Attested enclave $\leftrightarrow$ KMS / KMS-backed enclaves: Split “KeyBox logic” (enclave) from “policy enforcement” (KMS) so that key usage and sealing are authorized by attestation evidence and measurement-based policy, rather than host-supplied keys.
3. HSMs under a strict allowlist profile (PKCS#11-style): Use HSM non-exportability, but restrict mechanisms to a small allowlist that rules out share-deriving outputs and caller-decryptable wrap/export; if wrapping exists, constrain it to trusted/pinned recipients only (to preserve key-opacity).
4. Endpoint hardware-backed keystores: Platform keystores often match the “NXK + restricted operations” model, but typically expose a fixed operation menu; they therefore realize only those profiles reducible to built-in operations plus pinned sealing.
5. TPM-assisted freshness/anti-rollback: Even when the KeyBox is a TEE/enclave, TPM-backed monotonic primitives can supply the freshness mechanism needed to approximate state continuity in hostile host environments.

C Illustrative application: N XK-compatible commit–reveal randomness beacons

Many practical protocols—such as lotteries, leader election, and randomness beacons [22, 6, 67]—require a commit–reveal discipline and/or VSS/VDF to prevent adaptive choice/grinding: if parties were to publish their contribution $M_i := m_i \mathcal{G}$ immediately, then a last-moving adversary can sample many candidate scalars m_i after seeing the honest M_j ’s and select one that biases a downstream predicate of the beacon (e.g., a target prefix), where the beacon is defined as $\mathcal{L} := \sum_i M_i$ (and typically $\rho := H(\text{beacon}, \langle \text{sid}, \mathcal{L} \rangle)$). In a classical commit–reveal design one would commit to m_i and later open by revealing m_i , but in the N XK setting the scalar is non-exportable. USV provides an alternative compatible under the N XK model in which the opening is to the induced group element rather than to the scalar. Concretely, each party samples $m_i \leftarrow \mathbb{Z}_p$ inside its KeyBox and computes a USV certificate $(C_i, \zeta_i) \leftarrow \text{Cert}(\text{pp}, m_i)$.

- Commit: broadcast only C_i (keep ζ_i private).
- Reveal: broadcast ζ_i . Anyone checks $\mathcal{V}_{\text{cert}}(\text{pp}, C_i, \zeta_i) = 1$ and derives the canonical public contribution

$$M_i := \text{Open}_M(\text{pp}, C_i, \zeta_i) = m_i \mathcal{G}.$$

- Output: set $\mathcal{L} := \sum_i M_i$ and $\rho := H(\text{beacon}, \langle \text{sid}, \mathcal{L} \rangle)$.

Hence, each M_i is a deterministic function of the public transcript via (C_i, ζ_i) , so a verifier (and, in UC, the simulator/extractor) can compute the exact statement points used by the protocol in straight-line without ever extracting or exporting m_i . At the same time, withholding ζ_i until the reveal phase restores the usual commit–reveal discipline, since the adversary cannot choose m_i as a function of others’ revealed contributions, while remaining fully compatible with non-exportable scalars.